MET's Institute of Engineering

Bhujbal Knowledge City, Adgaon, Nashik.

Department of Computer Engineering

"GRAPH & TREE"

Prepared By

Prof. Anand N. Gharu

(Assistant Professor)

Computer Dept.

CLASS : SE COMPUTER 2019

SUBJECT : DSA (SEM-II)

UNIT : III

03 February 2025

Note: The material to prepare this presentation has been taken from internet and are generated only for students reference and not for commercial use.

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)



WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b



Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

SYLLABUS

Graphs: Basic Concepts, Storage representation, Adjacency matrix, adjacency list, Traversals-depth first and breadth first, Minimum spanning Tree, Greedy algorithms for computing minimum spanning tree- Prims and Kruskal Algorithms

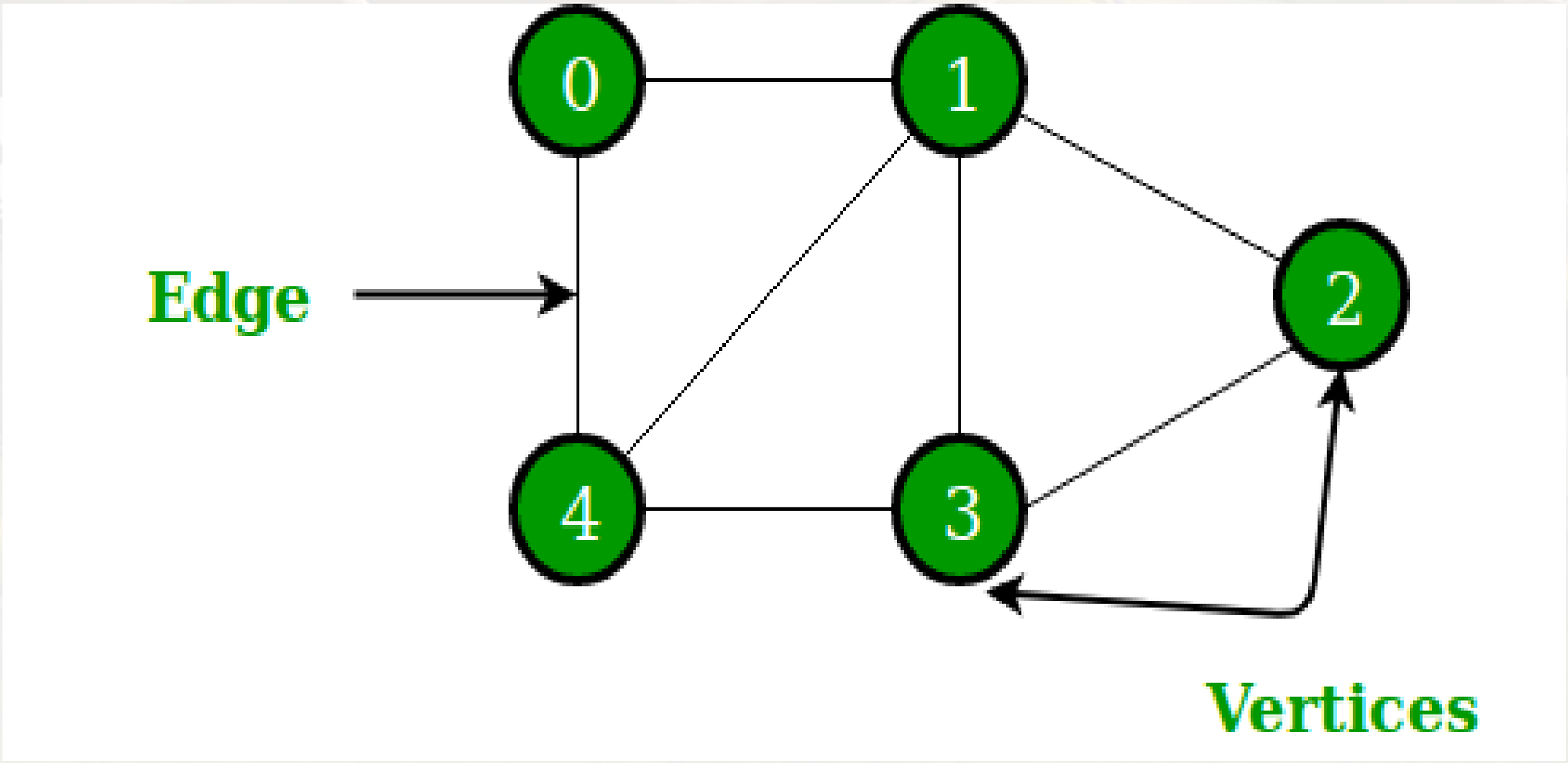
Trees: General tree and its representation: sequential and linked organization, Binary tree- properties, converting tree to binary tree, binary tree traversals (recursive and non-recursive) - inorder,preorder, post order, Operations on binary tree. Binary Search Tree (BST) and its operation

Case study: GPS/Navigation system that models a city map as a weighted graph and applies core graph algorithms ZIP/GZIP file compression using frequency-based encoding. using Huffman tree

GRAPH

Introduction of Graph

A graph is a non-linear data structure, which consists of vertices(or nodes) connected by edges(or arcs) where edges may be directed or undirected.



Introduction of Graph

A graph G can be defined as an ordered set G(V, E) where V(G) represents the set of vertices and E(G) represents the set of edges which are used to connect these vertices.

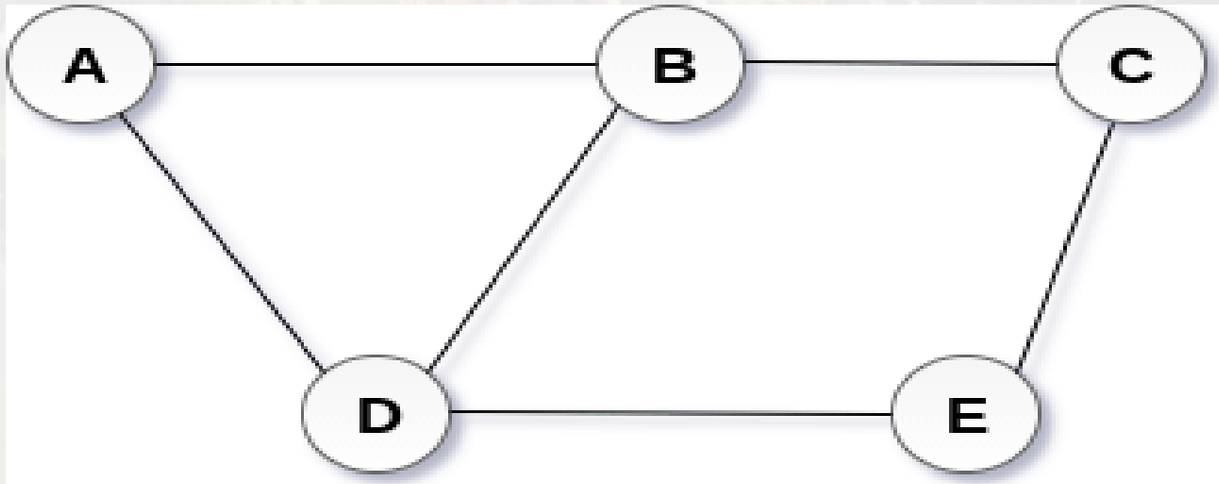
Types of Graph :

1. Directed Graph
2. Undirected Graph
3. Weighted Graph

Undirecred Graph

A graph G can be defined as an ordered set G(V, E) where V(G) represents the set of vertices and E(G) represents the set of edges which are used to connect these vertices.

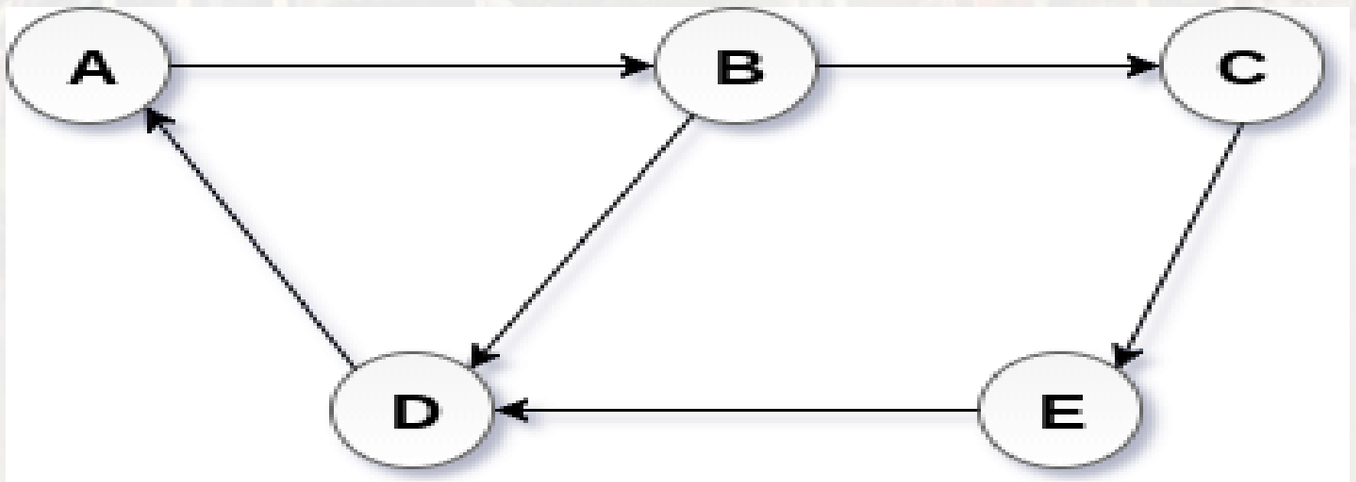
A Graph G(V, E) with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.



Undirected Graph

Directed Graph

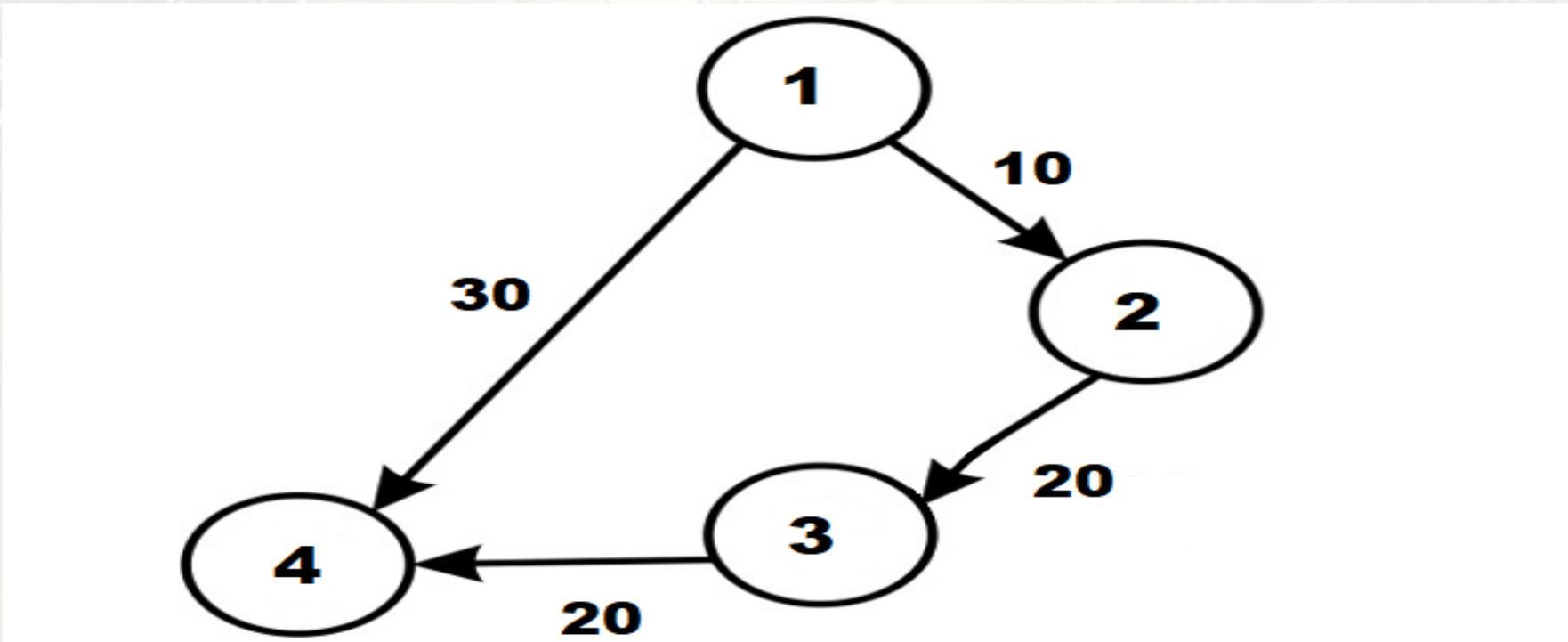
A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.



Directed Graph

Weighted Graph

A graph that has a value associated with every edge. The values corresponding to the edges are called weights. A value in a weighted graph can represent quantities such as cost, distance, and time, depending on the graph. Weighted graphs are typically used in modeling computer networks.



Weighted Graph

Terminologies of Graph

Path

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.

Closed Path

A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if Vo=VN.

Simple Path

If all the nodes of the graph are distinct with an exception Vo=VN, then such path P is called as closed simple path.

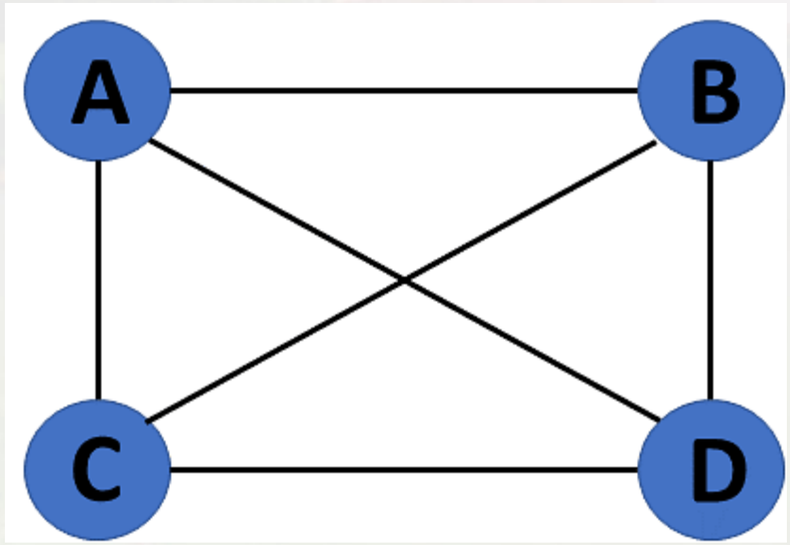
Terminologies of Graph

Cycle

A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Connected Graph

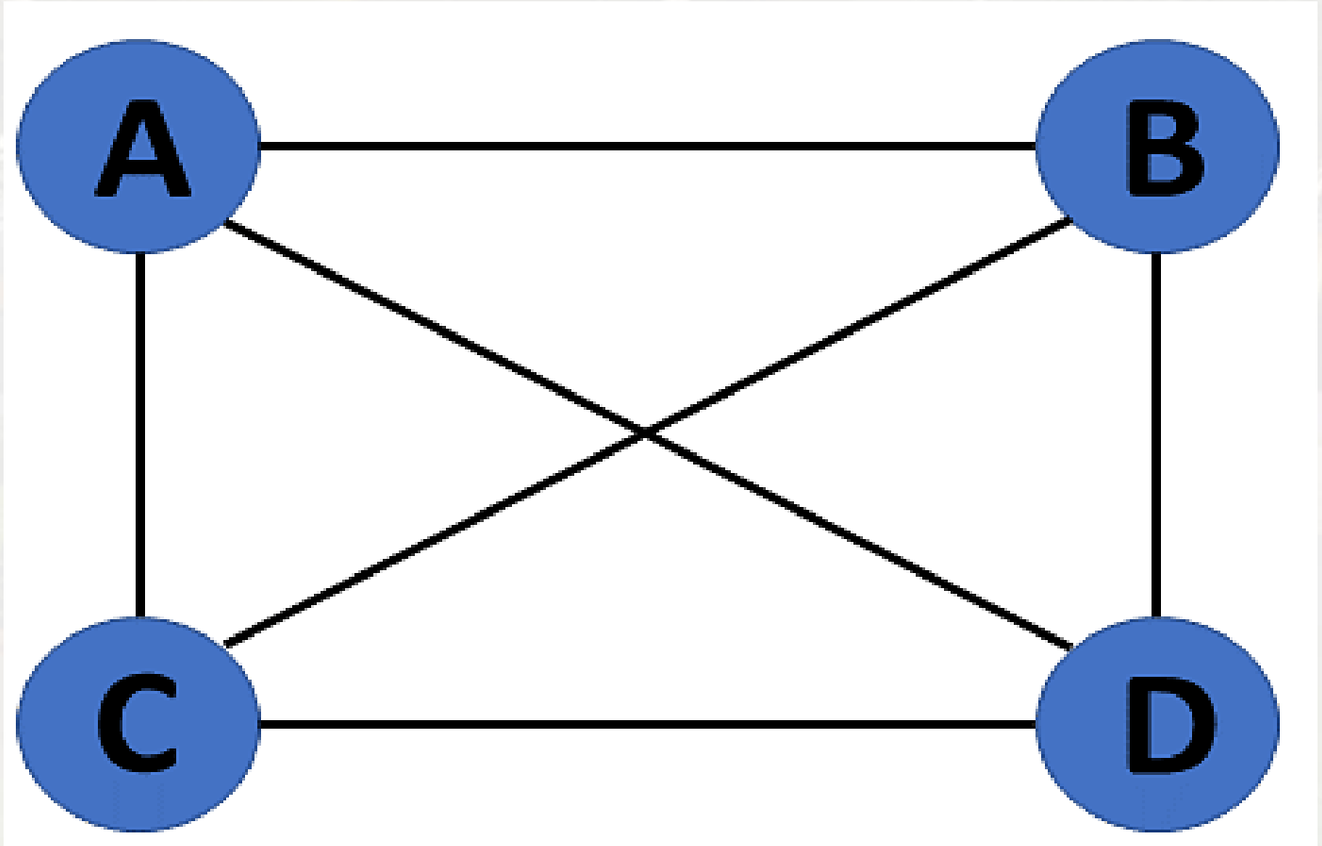
A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.



Terminologies of Graph

Complete Graph

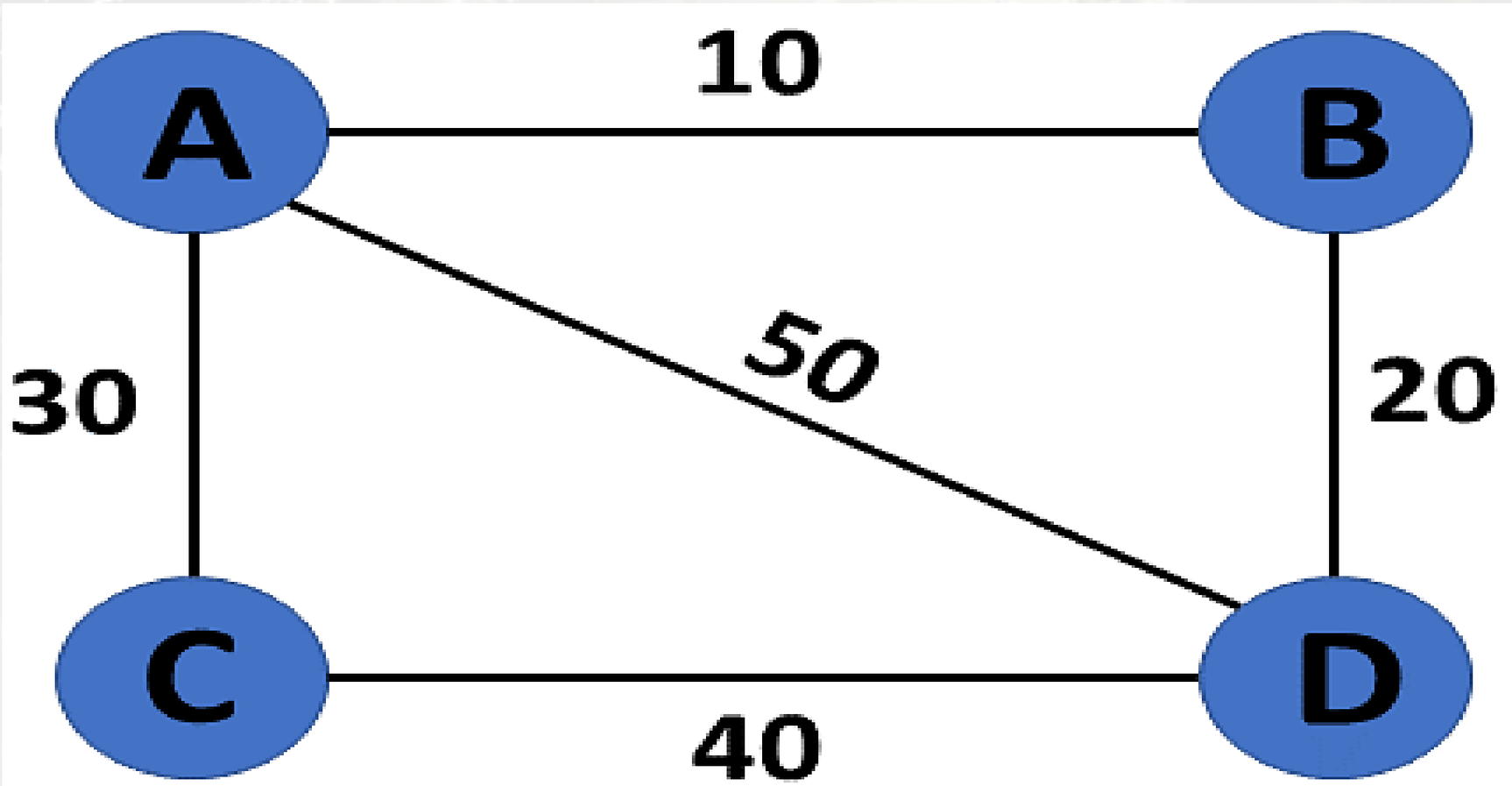
A complete graph is the one in which every node is connected with all other nodes. A complete graph contain n(n-1)/2 edges where n is the number of nodes in the graph.



Terminologies of Graph

Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as w(e) which must be a positive (+) value indicating the cost of traversing the edge.



Terminologies of Graph

Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as w(e) which must be a positive (+) value indicating the cost of traversing the edge.

Digraph

A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.

Terminologies of Graph

Loop

An edge that is associated with the similar end points can be called as Loop.

Adjacent Nodes

If two nodes u and v are connected via an edge e, then the nodes u and v are called as neighbours or adjacent nodes.

Degree of the Node

A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

Terminologies of Graph

- **Outgoing edges of a vertex** are directed edges that the vertex is the origin.
- **Incoming edges of a vertex** are directed edges that the vertex is the destination.
- **The degree of a vertex** in a graph is the total number of edges incident to it.
- In a directed graph, **the out-degree of a vertex** is the total number of outgoing edges
- the **in-degree** is the total number of incoming edges.

Terminologies of Graph

Pendant Vertex

A vertex with degree one is called a pendant vertex.

Isolated Vertex

A vertex with degree zero is called an isolated vertex.

Terminologies of Graph

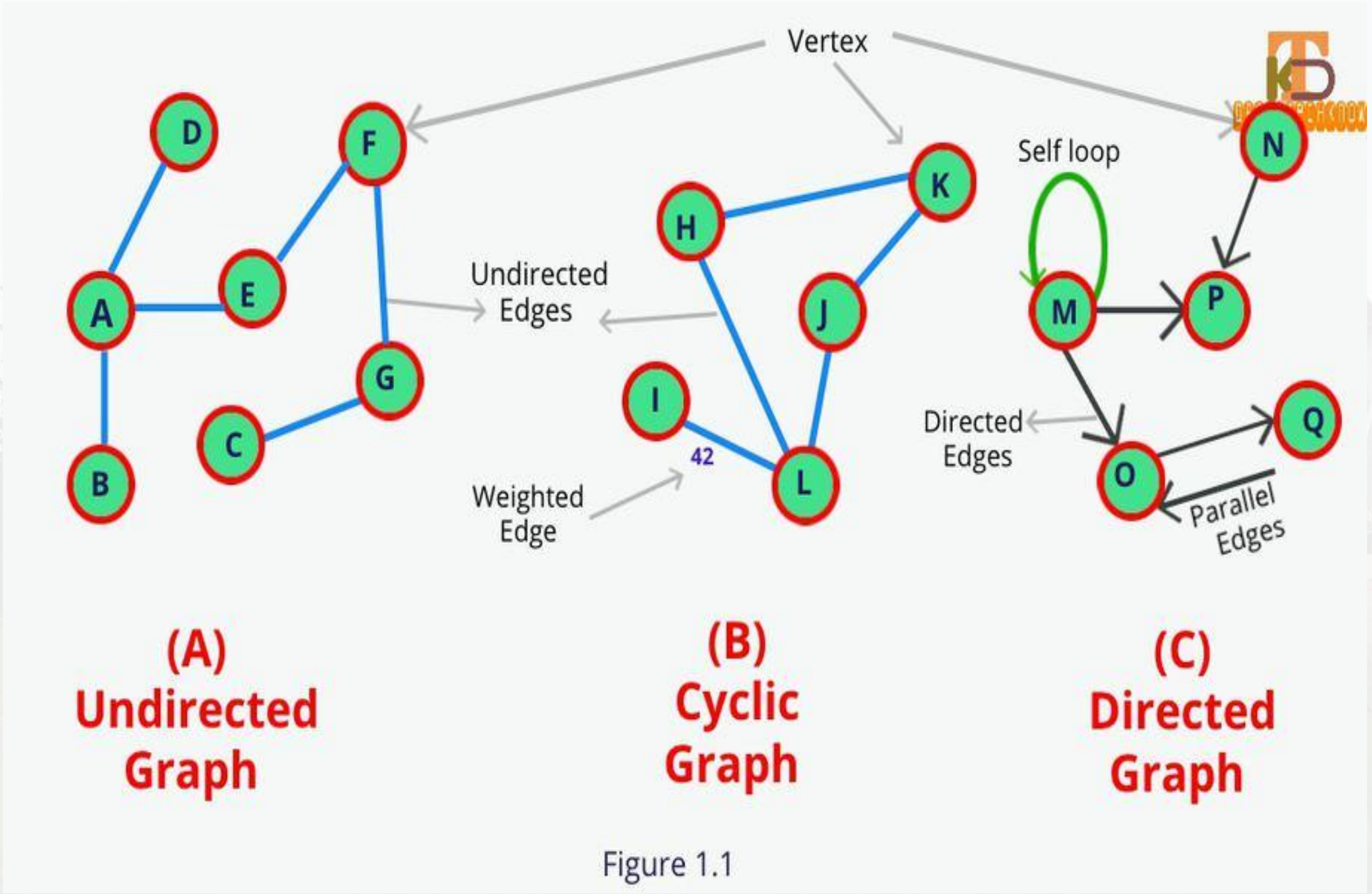


Figure 1.1

Graph Abstract Data Type

1. ***create() :Graph***
2. ***insert vertex(Graph, v) :Graph***
3. ***delete vertex(Graph, v) :Graph***
4. ***insert edge(Graph, u, v) :Graph***
5. ***delete edge(Graph, u, v) :Graph***
6. ***is empty(Graph) :Boolean;***
7. ***end graph***

Graph Representation

Adjacency Matrix Representation of Graph

We can easily represent the graphs using the following ways,

1. Adjacency matrix (sequential representation)
2. Adjacency list (linked representation)
3. Adjacency Multilist
4. Inverse Adjacency List

Graph Representation

- By Graph representation, we simply mean the technique which is to be used in order to store some graph into the computer's memory.
- There are two ways to store Graph into the computer's memory. In this part of this tutorial, we discuss each one of them in detail.

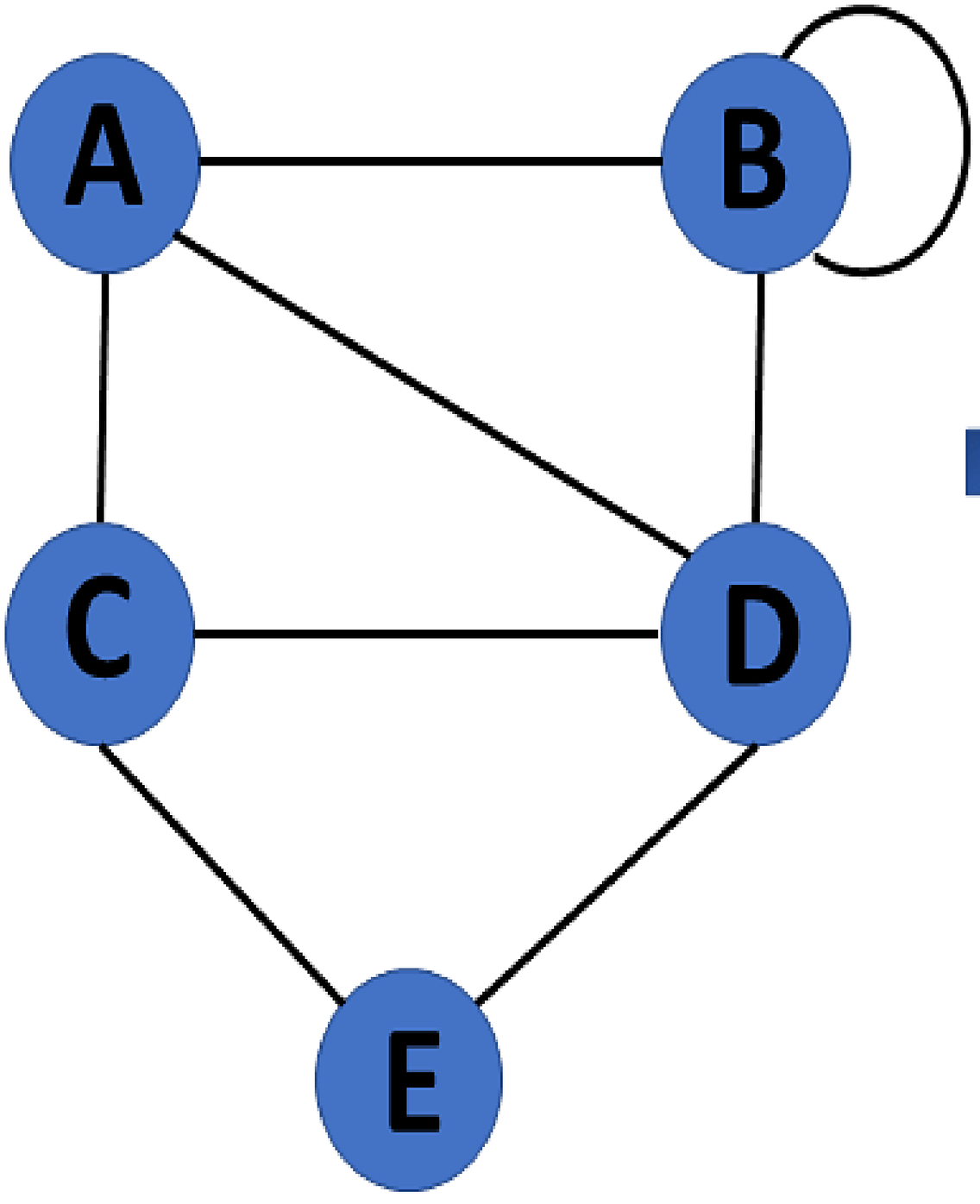
Adjacency Matrix

1. A sequential representation is an adjacency matrix.
2. It's used to show which nodes are next to one another.

   I.e., is there any connection between nodes in a graph?
3. You create an MXM matrix G for this representation. If an edge exists between vertex a and vertex b, the corresponding element of G, gi,j = 1, otherwise gi,j = 0.
4. If there is a weighted graph, you can record the edge's weight instead of 1s and 0s.

Adjacency Matrix

Undirected Graph Representation :

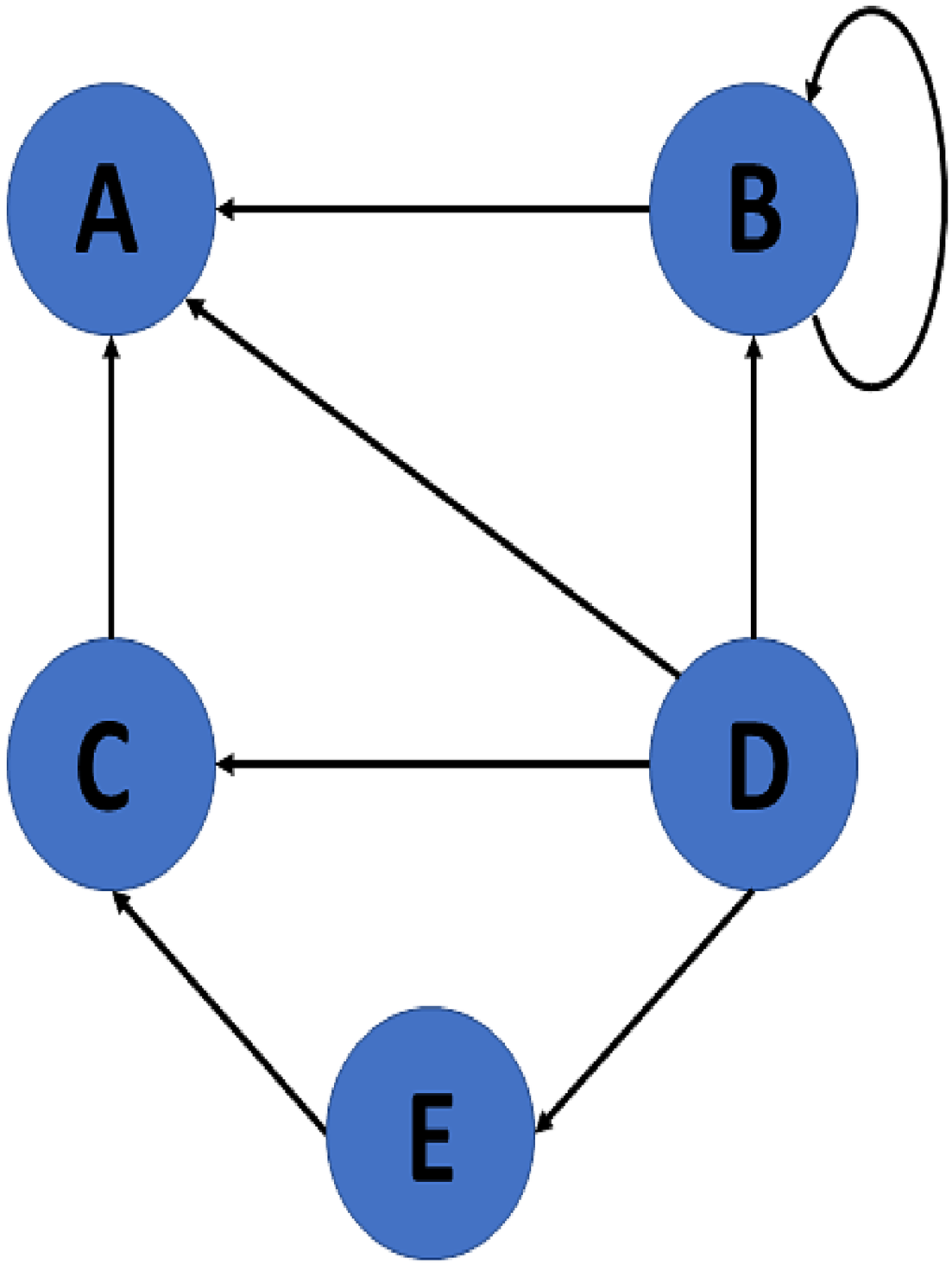


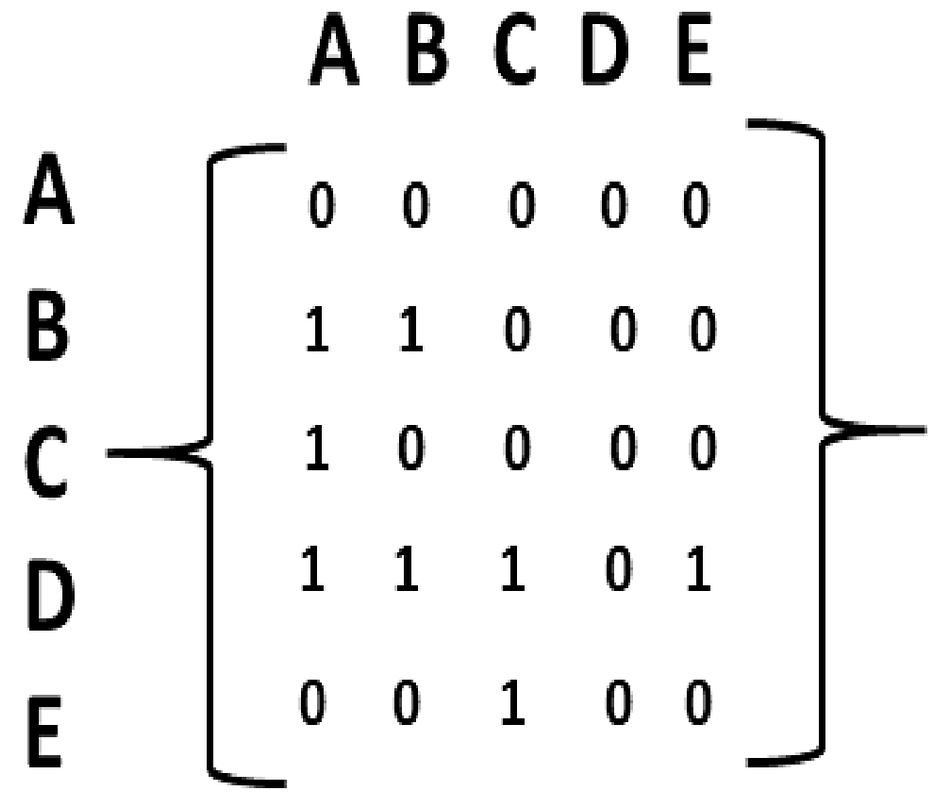


	A	B	C	D	E
A	0	1	1	1	0
B	1	1	0	1	0
C	1	0	0	1	1
D	1	1	1	0	1
E	0	0	1	1	0

Adjacency Matrix

Directed Graph Representation :



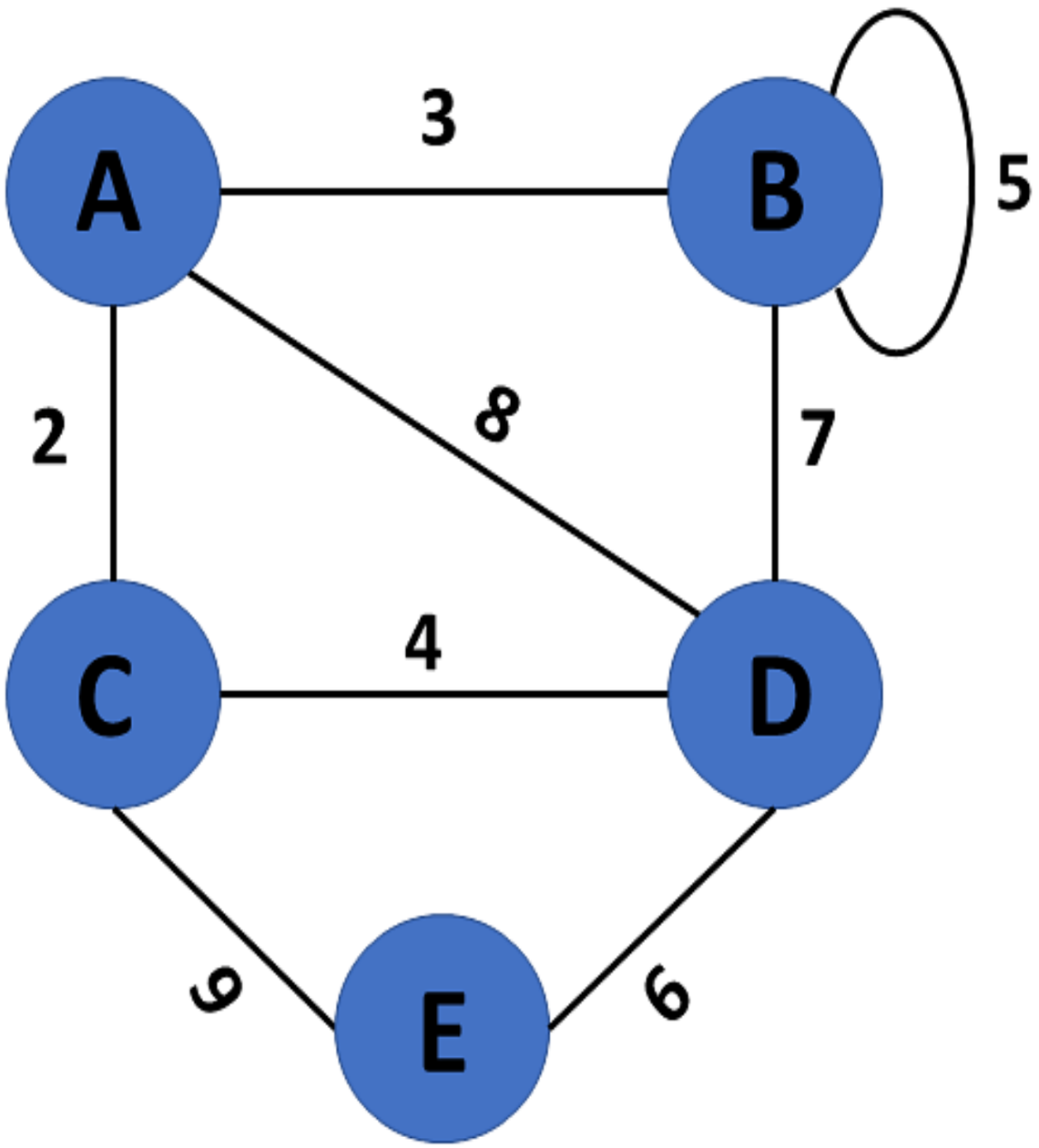


	A	B	C	D	E
A	0	0	0	0	0
B	1	1	0	0	0
C	1	0	0	0	0
D	1	1	1	0	1
E	0	0	1	0	0

Adjacency Matrix

Weighted Undirected Graph Representation

Weight or cost is indicated at the graph's edge, a weighted graph representing these values in the matrix



	A	B	C	D	E
A	0	3	2	8	0
B	3	5	0	7	0
C	2	0	0	4	9
D	8	7	4	0	6
E	0	0	9	6	0

Adjacency List

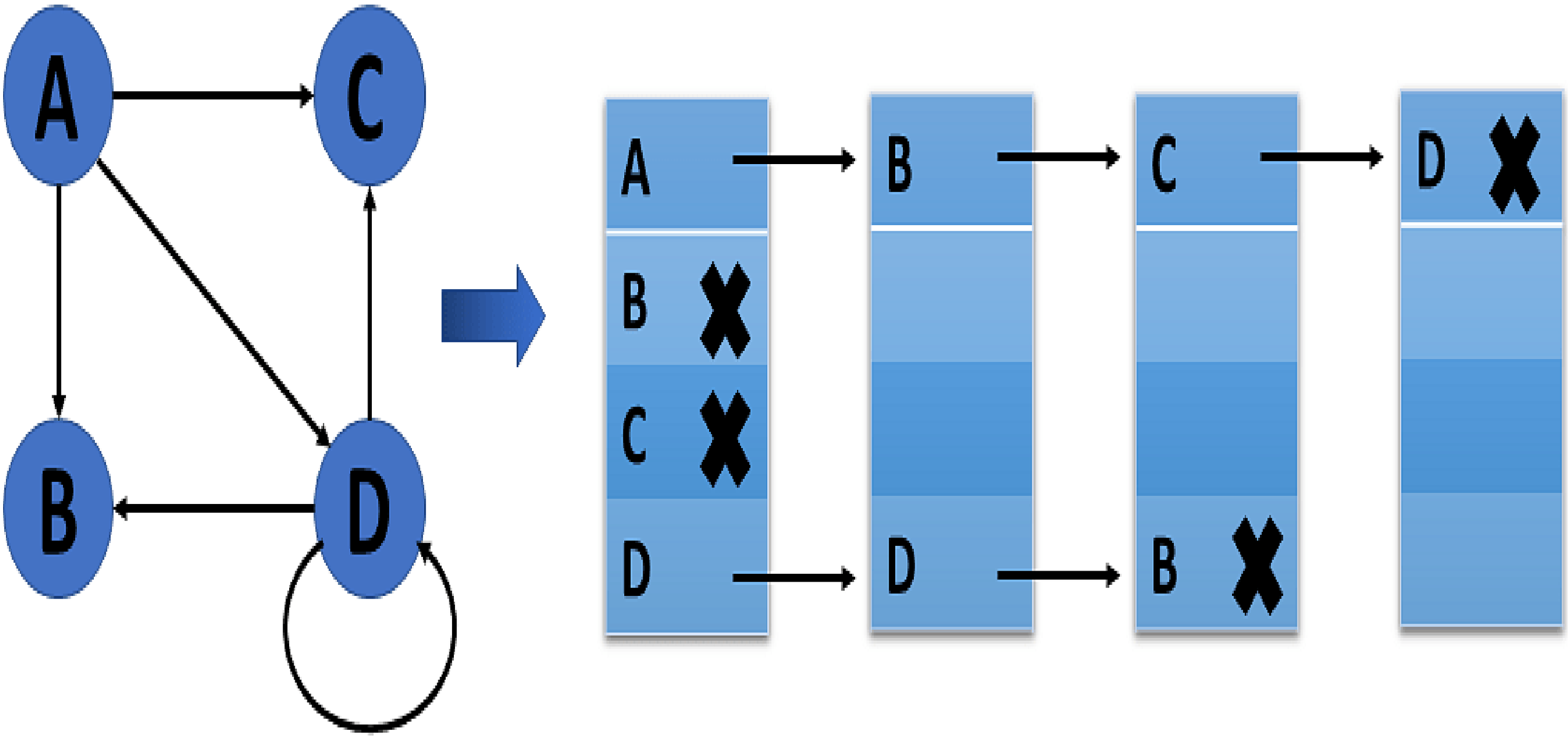
1. A linked representation is an adjacency list.

2. You keep a list of neighbors for each vertex in the graph in this representation. It means that each vertex in the graph has a list of its neighboring vertices.

3. You have an array of vertices indexed by the vertex number, and the corresponding array member for each vertex x points to a singly linked list of x's neighbors

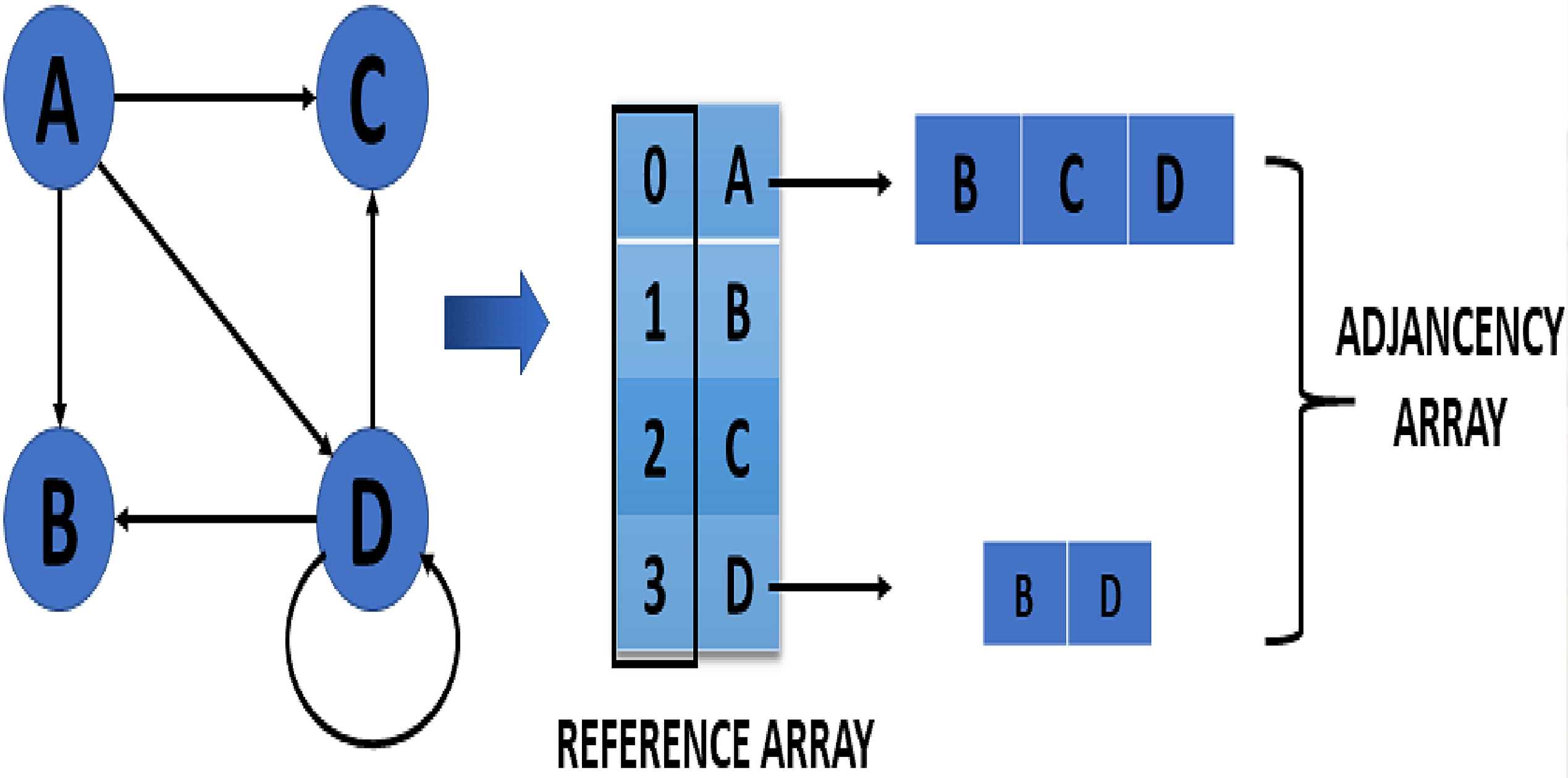
Adjacency List

Weighted Undirected Graph Representation Using Linked-List



Adjacency List

Weighted Undirected Graph Representation Using an Array



Graph Representation

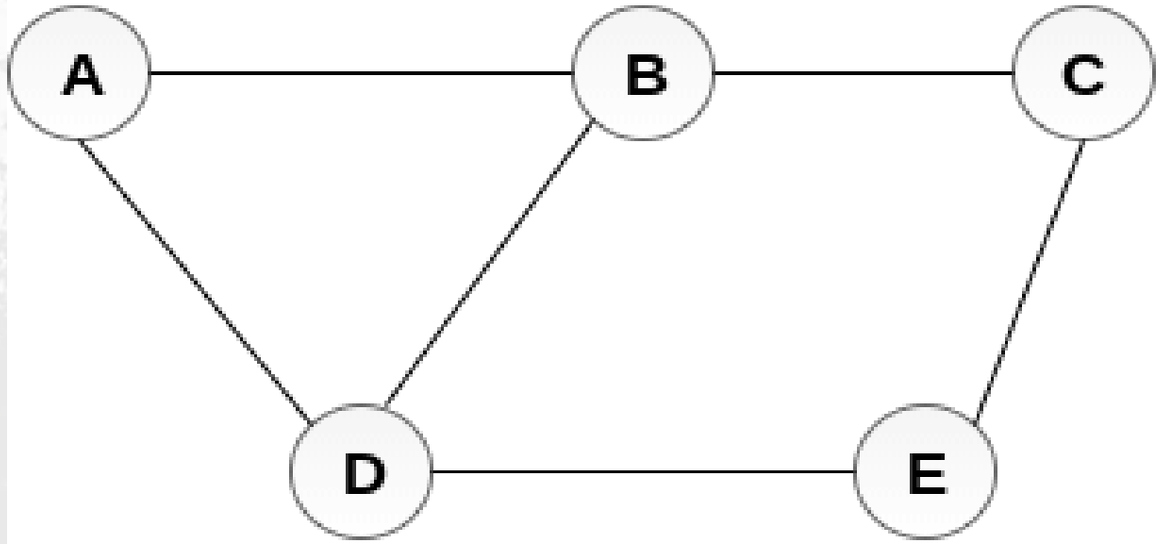
1. Sequential Representation

- In sequential representation, we use adjacency matrix to store the mapping represented by vertices and edges. In adjacency matrix, the rows and columns are represented by the graph vertices. A graph having n vertices, will have a dimension n x n.

- An entry Mij in the adjacency matrix representation of an undirected graph G will be 1 if there exists an edge between Vi and Vj.

Graph Representation

An undirected graph and its adjacency matrix representation is shown in the following figure.



Undirected Graph

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

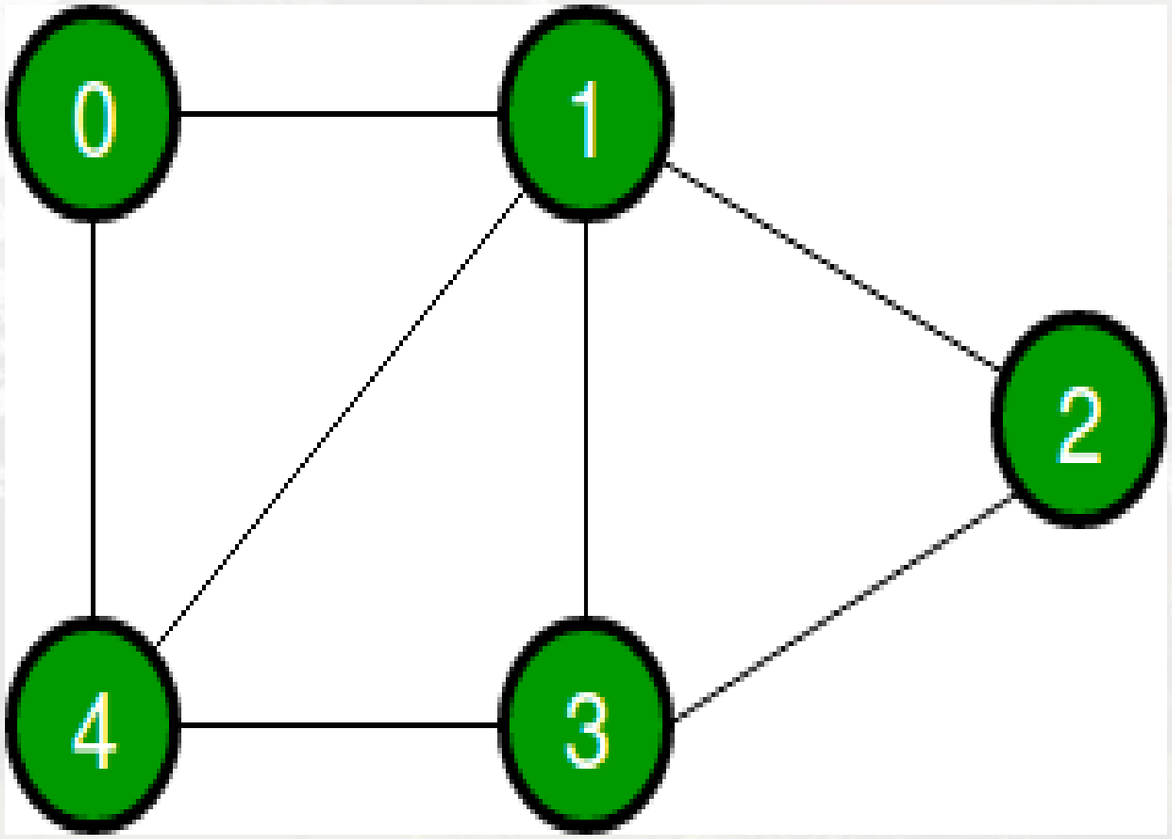
Adjacency Matrix

in the above figure, we can see the mapping among the vertices (A, B, C, D, E) is represented by using the adjacency matrix which is also shown in the figure.

There exists different adjacency matrices for the directed and undirected graph. In directed graph, an entry A_{ij} will be 1 only when there is an edge directed from V_i to V_j.

Graph Representation

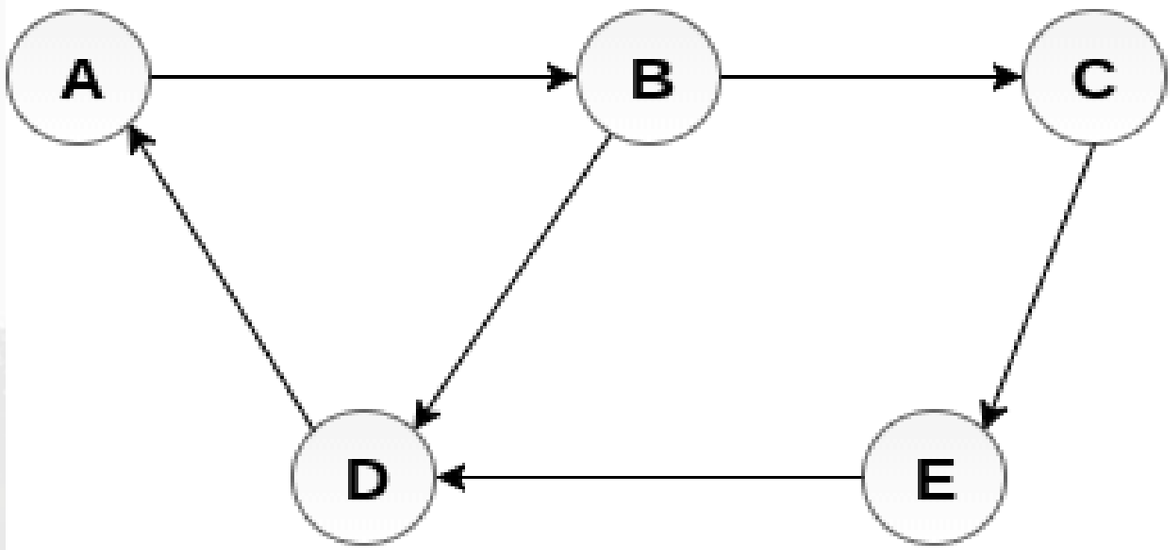
An undirected graph and its adjacency matrix representation is shown in the following figure.



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

Graph Representation

A directed graph and its adjacency matrix representation is shown in the following figure.



Directed Graph

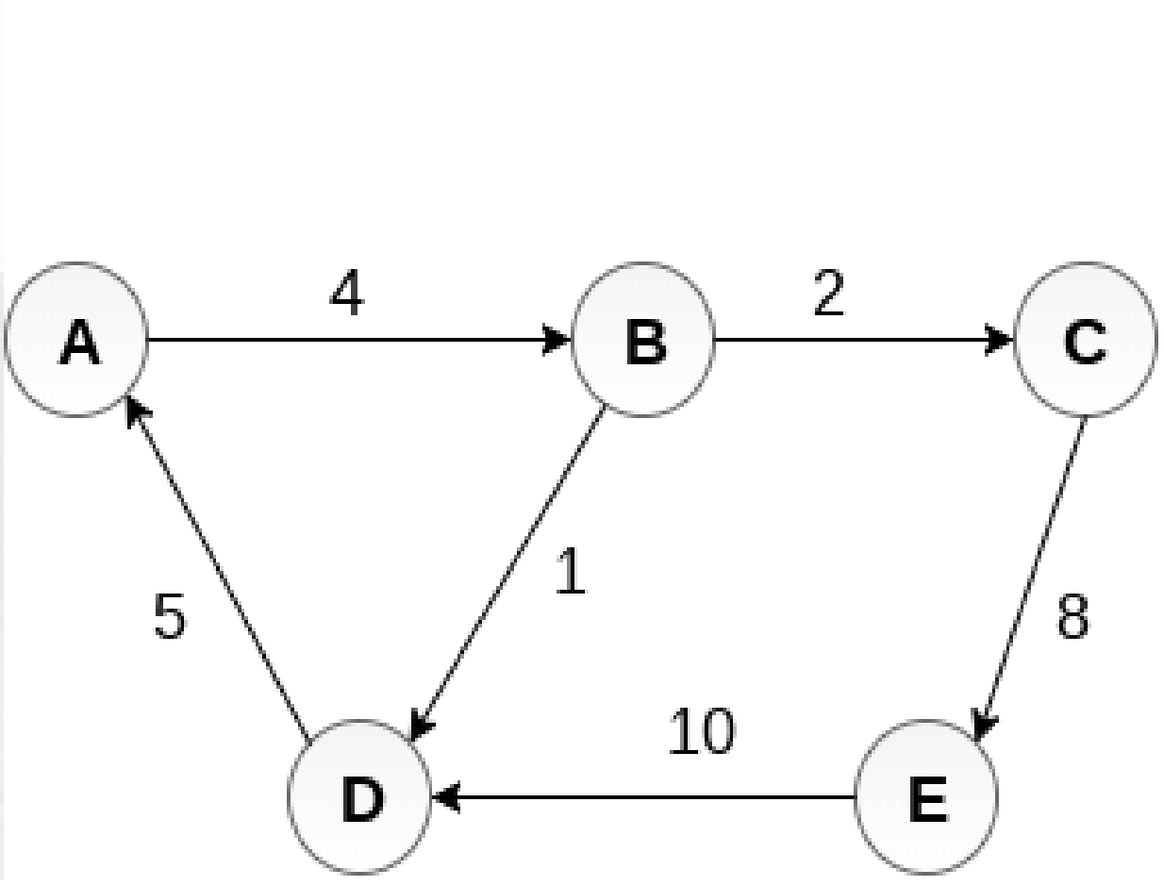
	A	B	C	D	E
A	0	1	0	0	0
B	0	0	1	1	0
C	0	0	0	0	1
D	1	0	0	0	0
E	0	0	0	1	0

Adjacency Matrix

Representation of weighted directed graph is different. Instead of filling the entry by 1, the Non- zero entries of the adjacency matrix are represented by the weight of respective edges.

Graph Representation

The weighted directed graph along with the adjacency matrix representation is shown in the following figure.



Weighted Directed Graph

	A	B	C	D	E
A	0	4	0	0	0
B	0	0	2	1	0
C	0	0	0	0	8
D	5	0	0	0	0
E	0	0	0	10	0

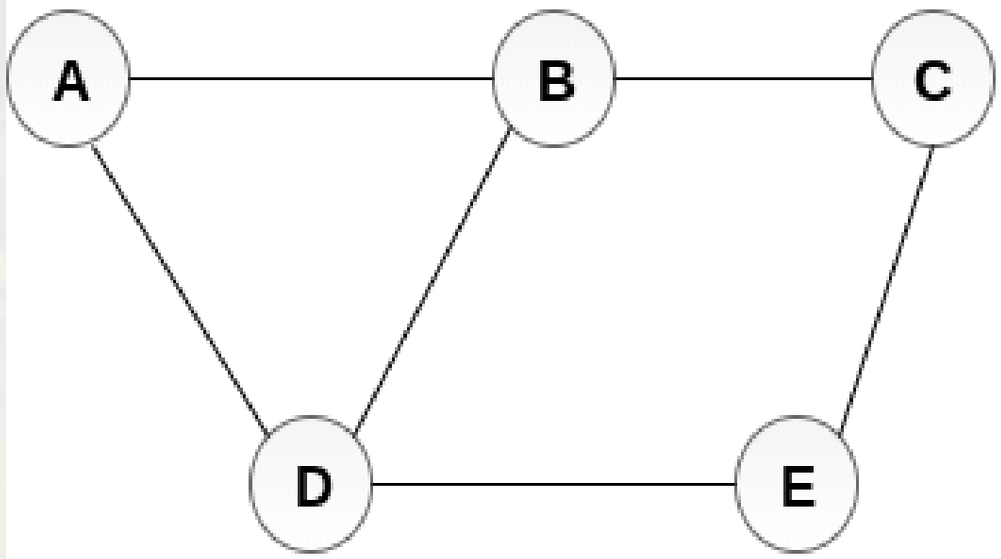
Adjacency Matrix

Graph Representation

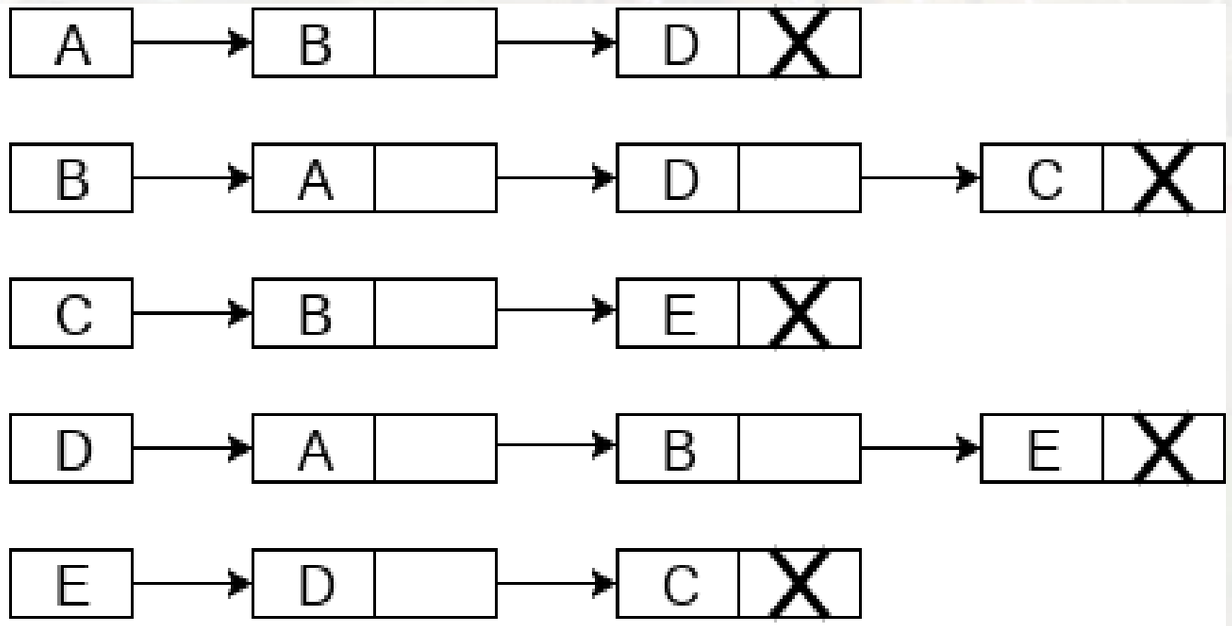
2. Linked Representation :

In the linked representation, an adjacency list is used to store the Graph into the computer's memory.

Consider the undirected graph shown in the following figure and check the adjacency list representation.



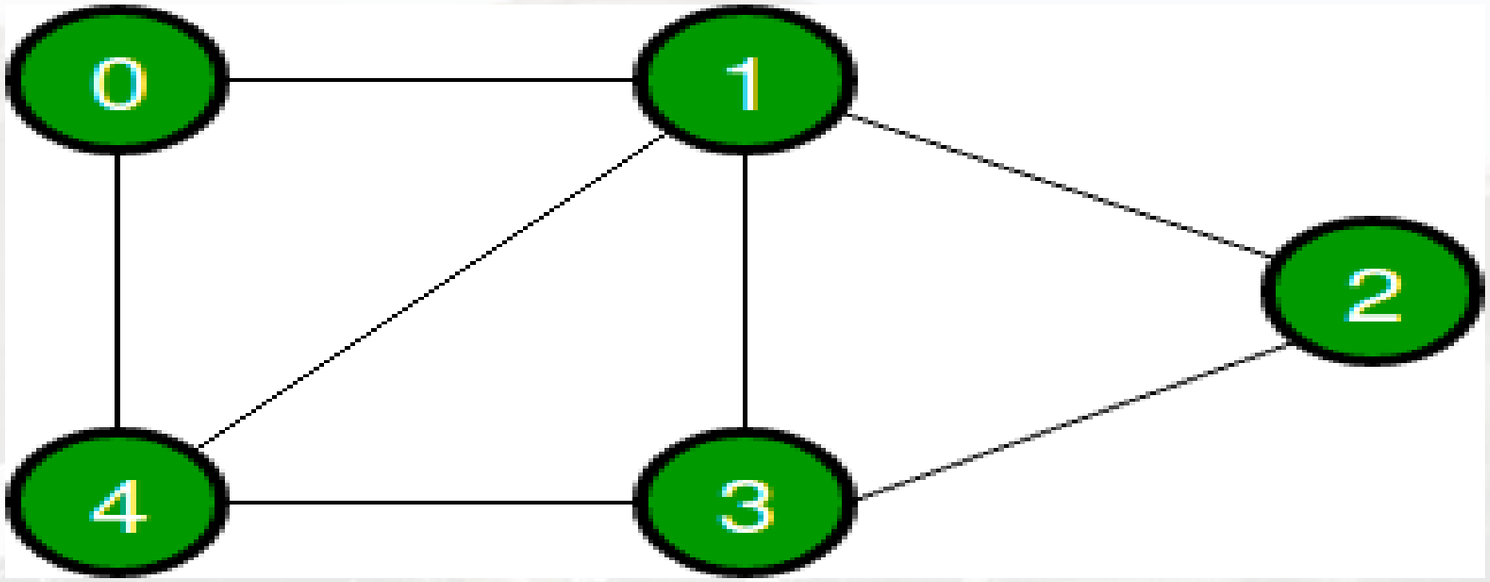
Undirected Graph

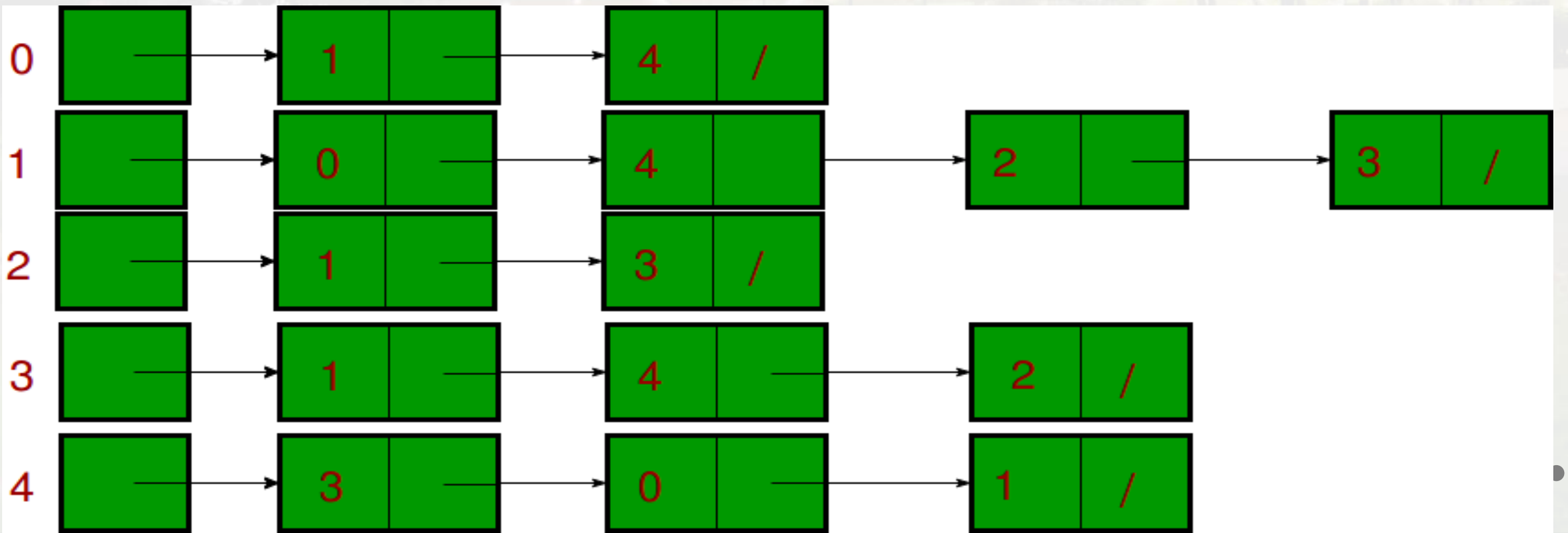


Adjacency List

Graph Representation

2. Linked Representation :



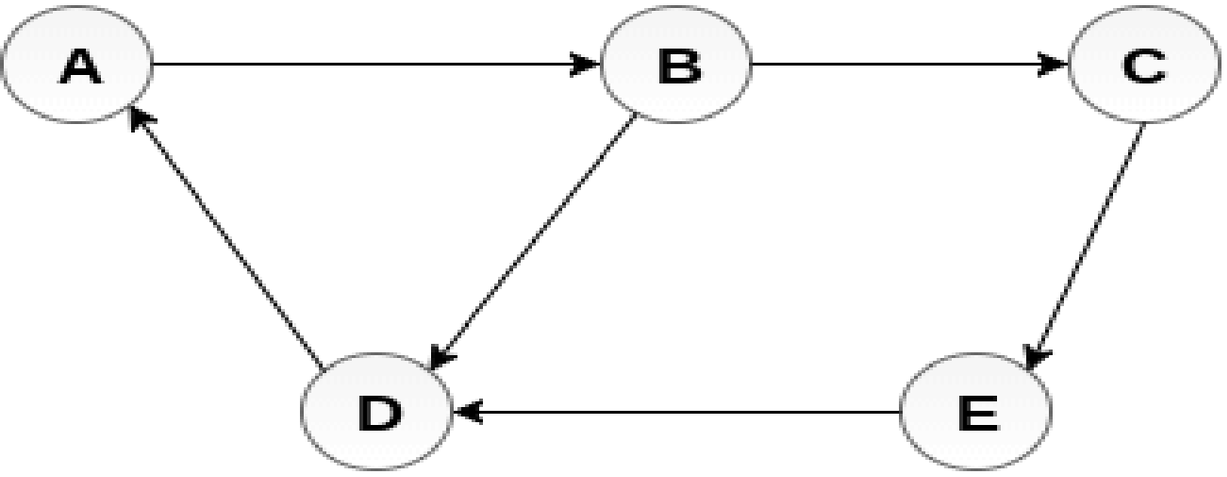


Graph Representation

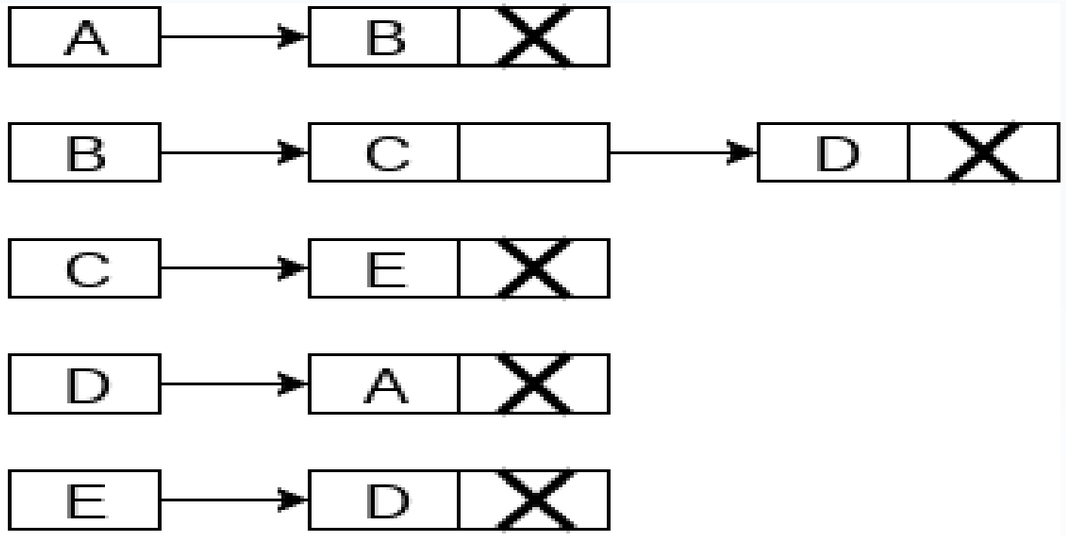
An adjacency list is maintained for each node present in the graph which stores the node value and a pointer to the next adjacent node to the respective node. If all the adjacent nodes are traversed then store the NULL in the pointer field of last node of the list. The sum of the lengths of adjacency lists is equal to the twice of the number of edges present in an undirected graph.

Consider the directed graph shown in the following figure and check the adjacency list representation of the graph.

Graph Representation



Directed Graph

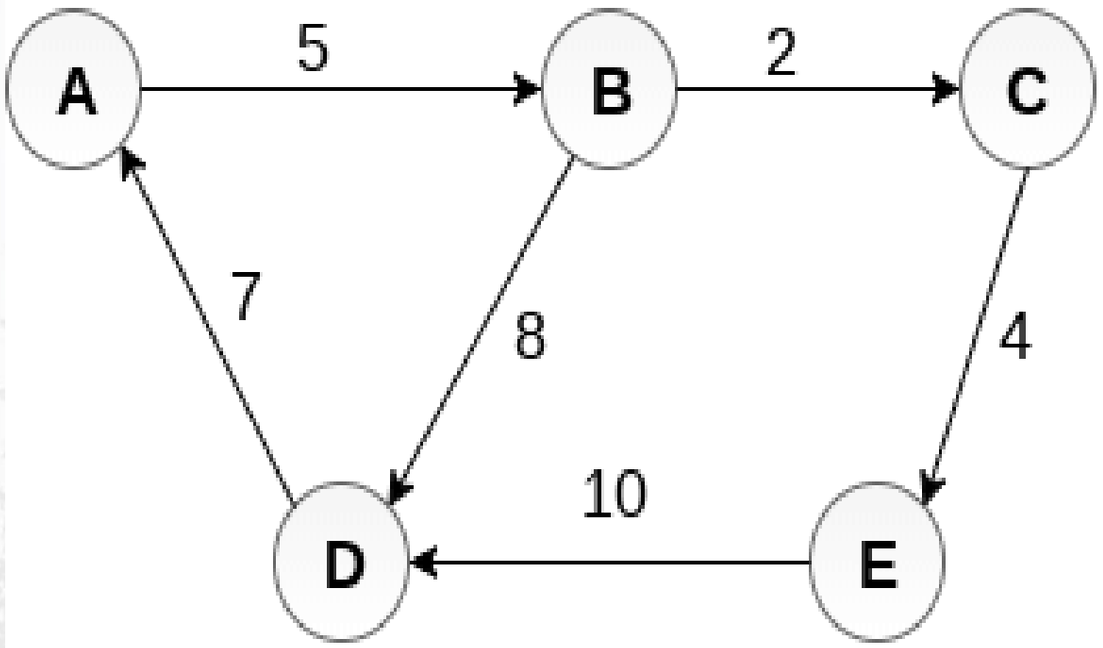


Adjacency List

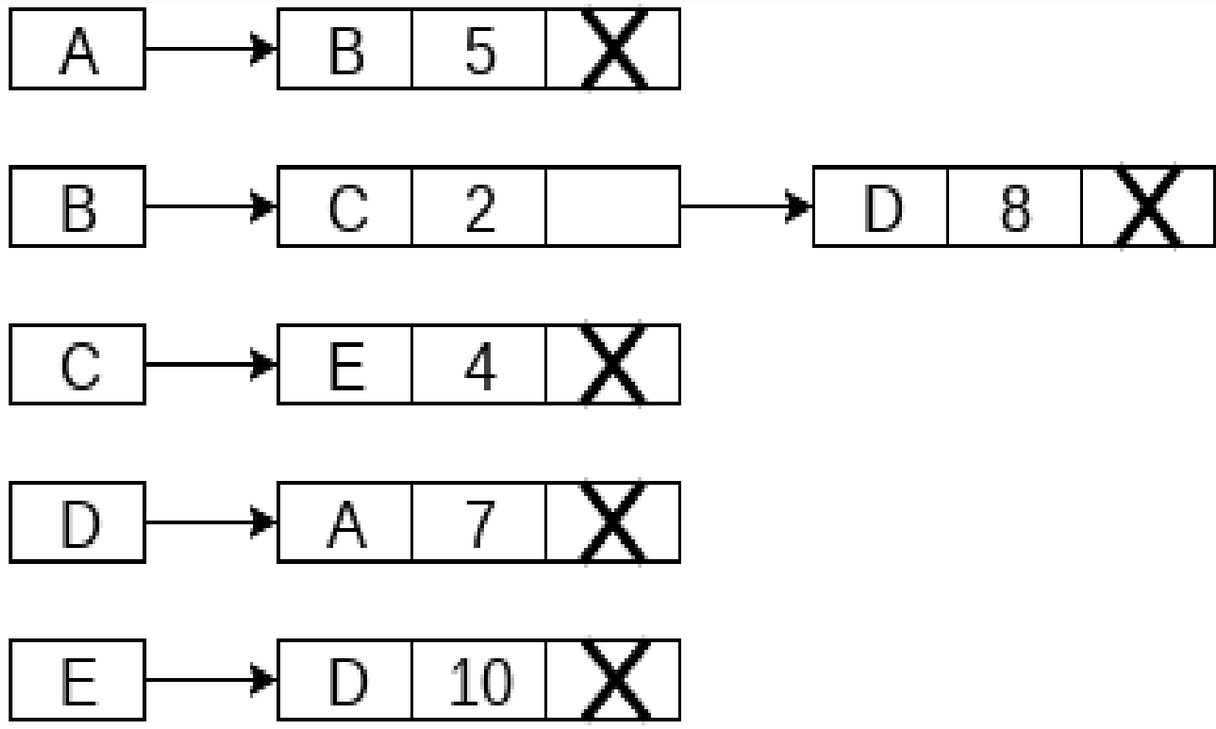
In a directed graph, the sum of lengths of all the adjacency lists is equal to the number of edges present in the graph.

In the case of weighted directed graph, each node contains an extra field that is called the weight of the node. The adjacency list representation of a directed graph is shown in the following figure.

Graph Representation



Weighted Directed Graph



Adjacency List

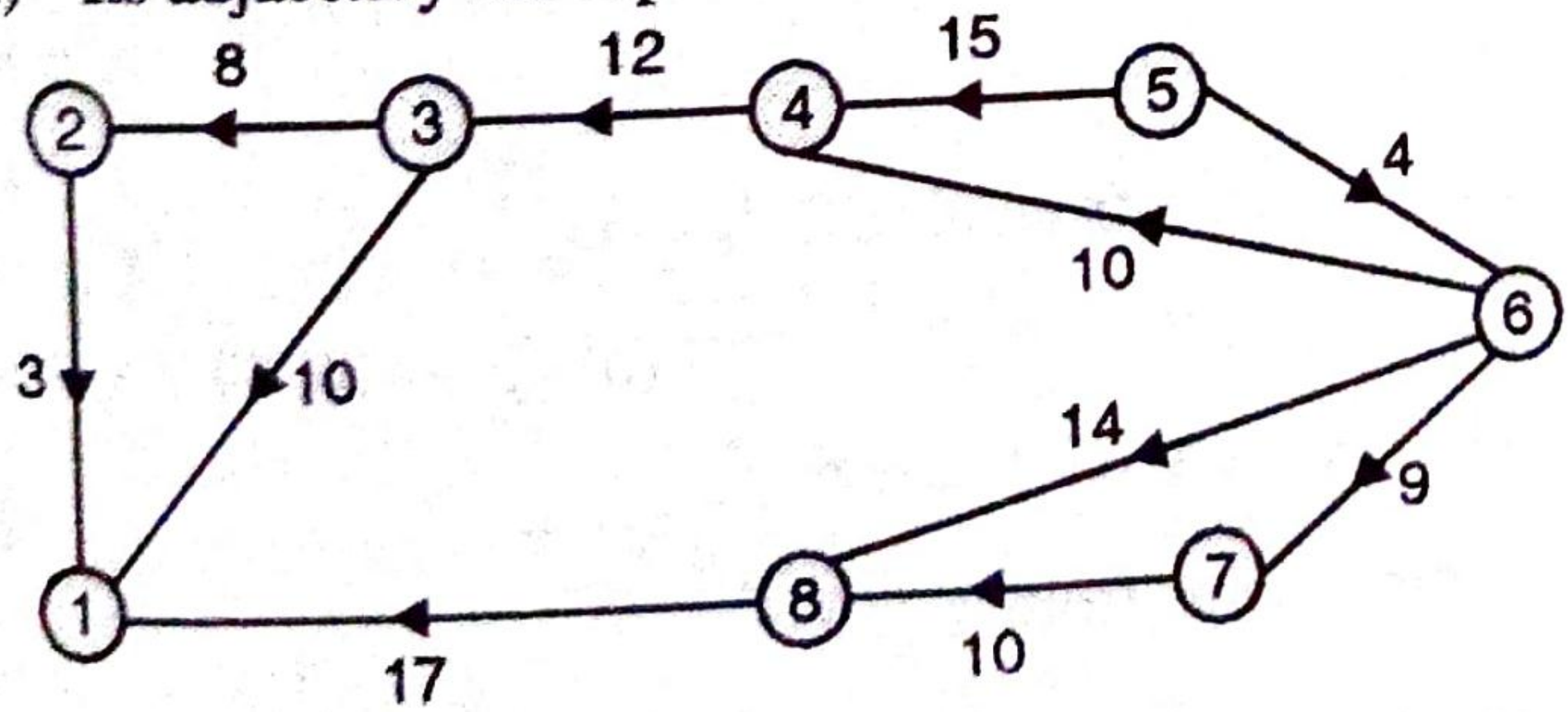
Graph Representation Examples

For the graph shown in Fig. P.3.3.1 obtain :

(i) The in degree and out degree of each vertex,

(ii) Its adjacency matrix

(iii) Its adjacency list representation.

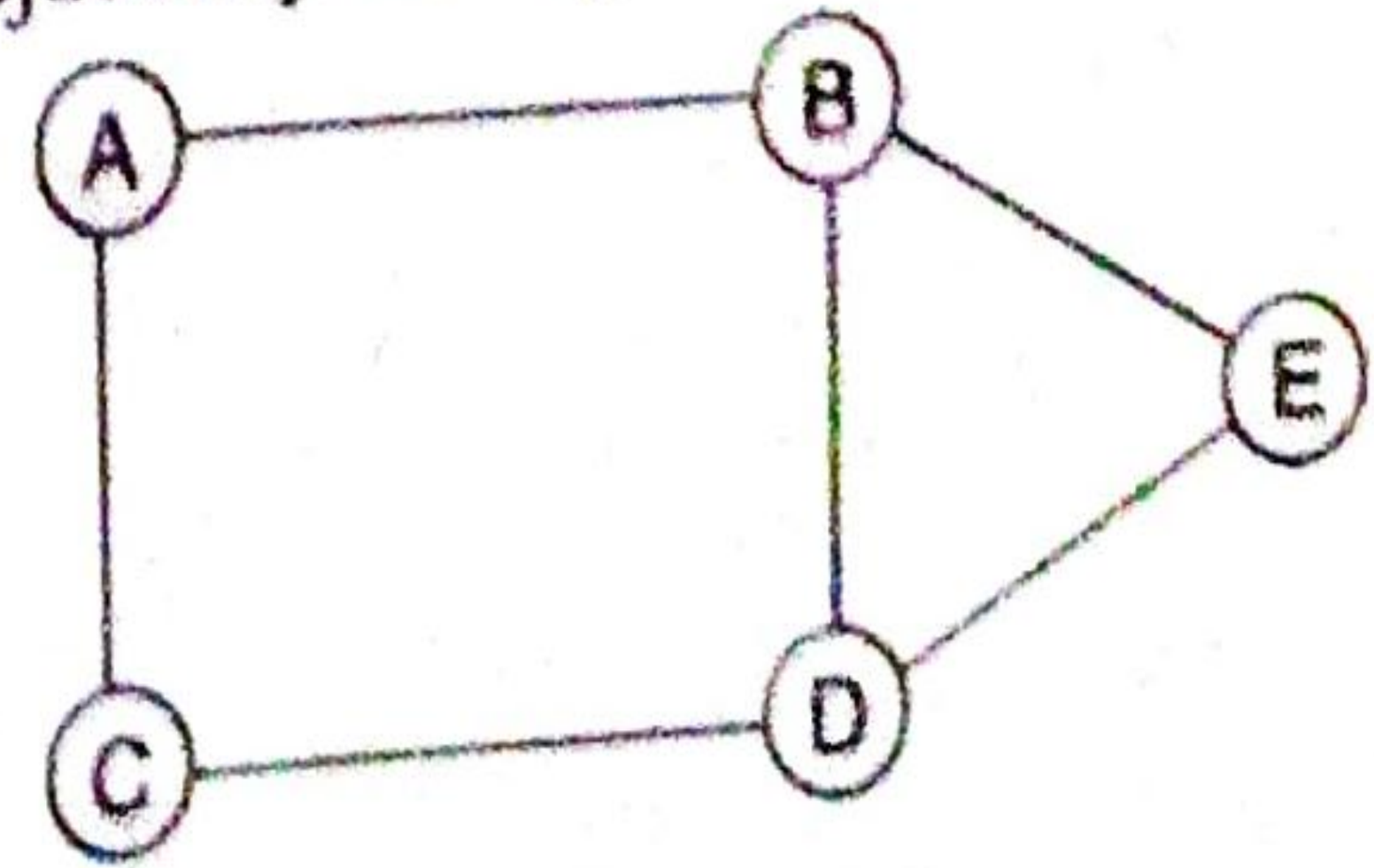


Graph Representation Examples

Consider the graph shown in Fig. P. 3.3.2.

(i) Give adjacency matrix representation.

(ii) Give adjacency list representation of the graph.

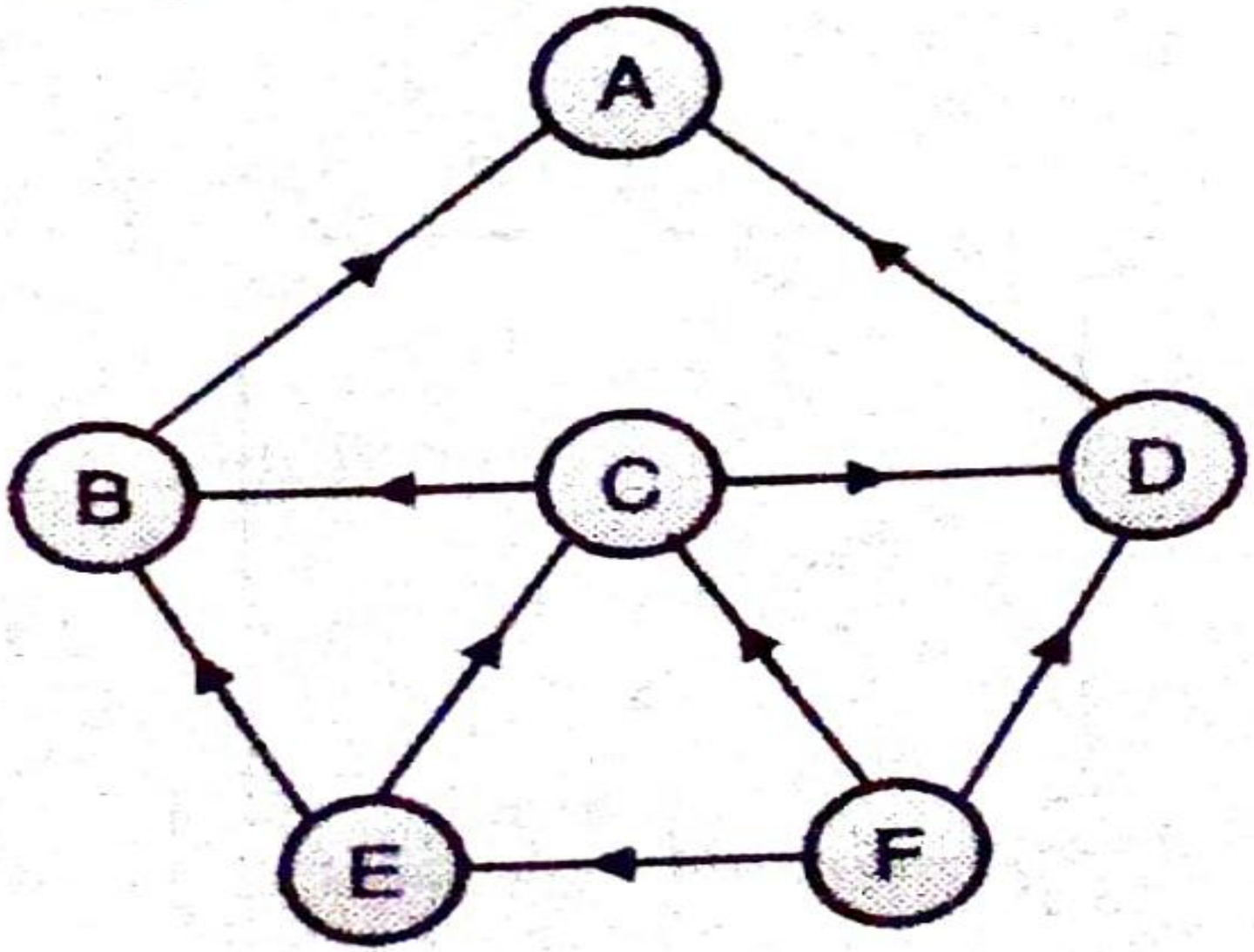


Graph Representation Examples

Consider the graph shown in Fig. P.3.3.5 :

(i) Give adjacency matrix representation.

(ii) Give adjacency list representation of graph.



Adjacency Multi-list

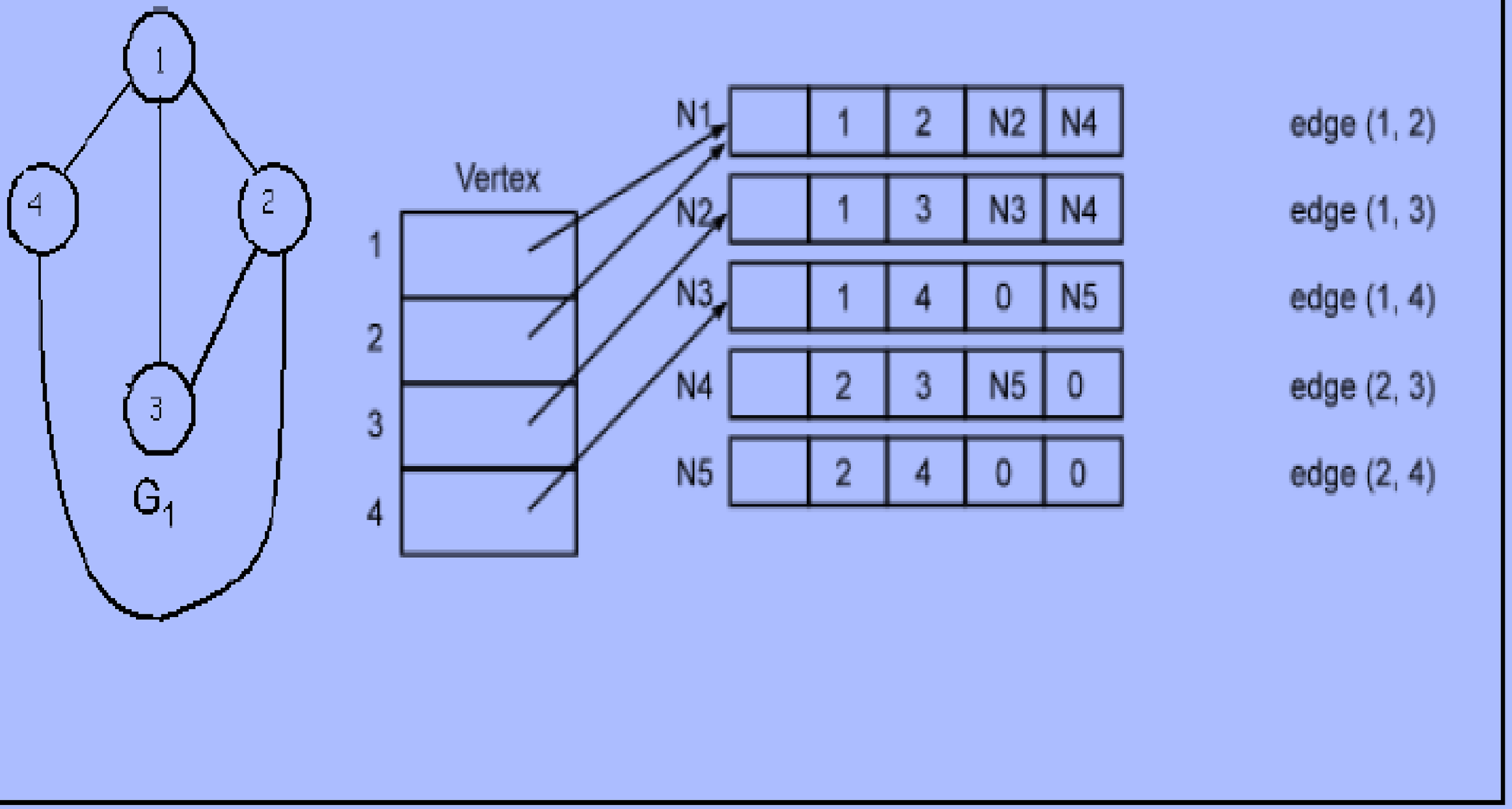
- Multiclass are lists where nodes may be shared among several other lists

Adjacency Multi-list

- The node structure of such a list can be represented as follows :

Visited tag	V_1	V_2	Link1 for V_1	Link2 for V_2

Adjacency Multi-list for Graph

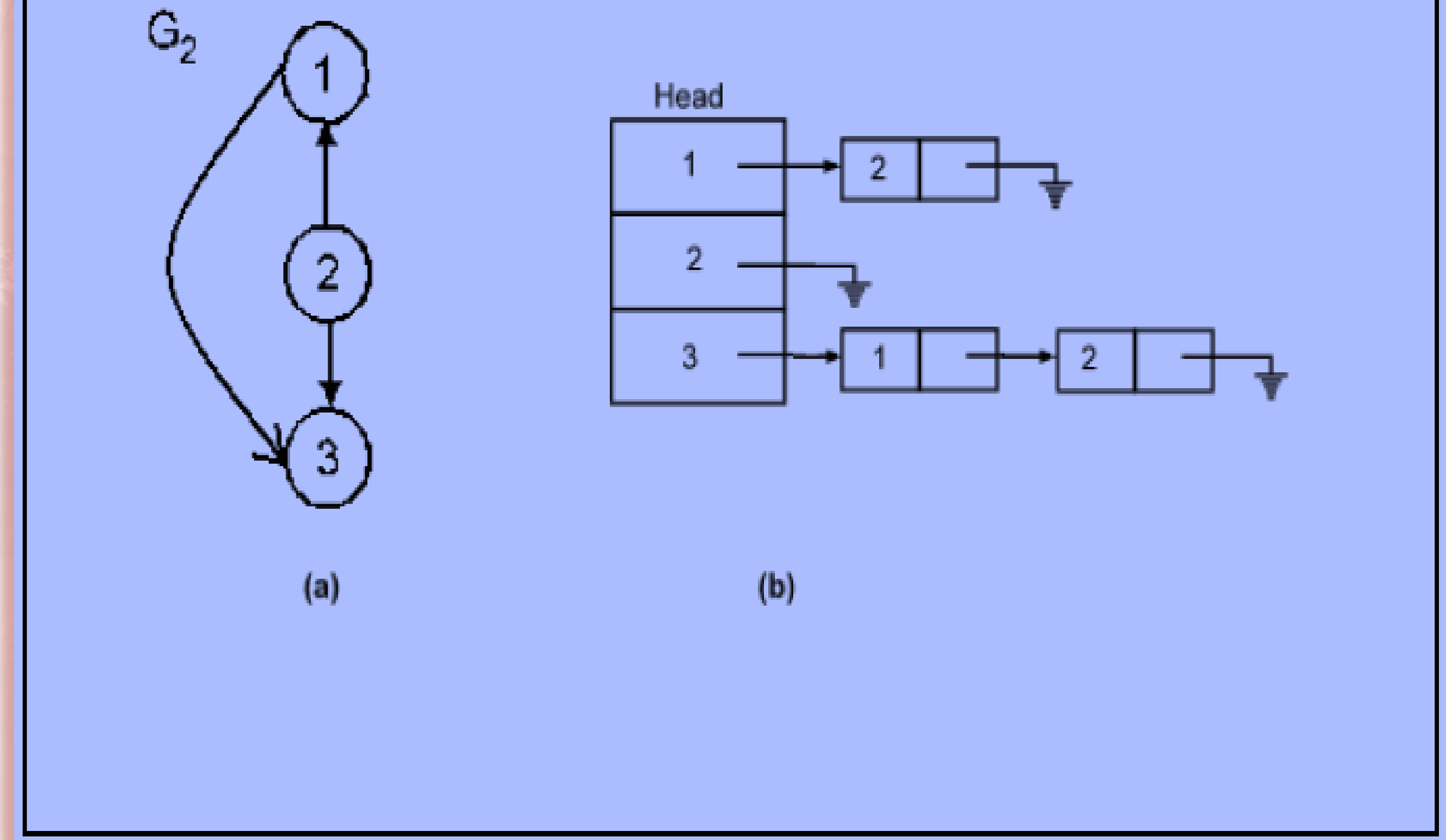


Inverse Adjacency List

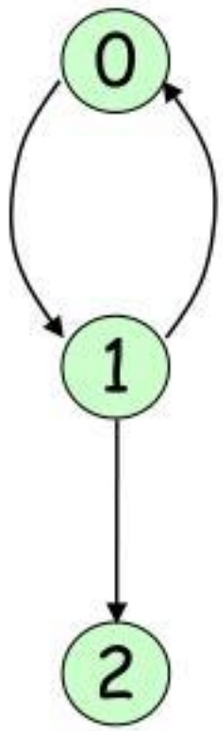
- Inverse adjacency lists is a set of lists that contain one list for vertex

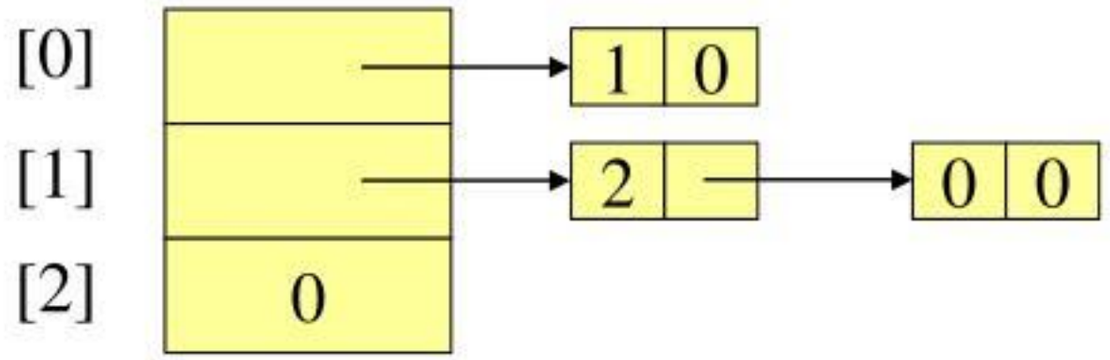
- Each list contains a node per vertex adjacent to the vertex it represents

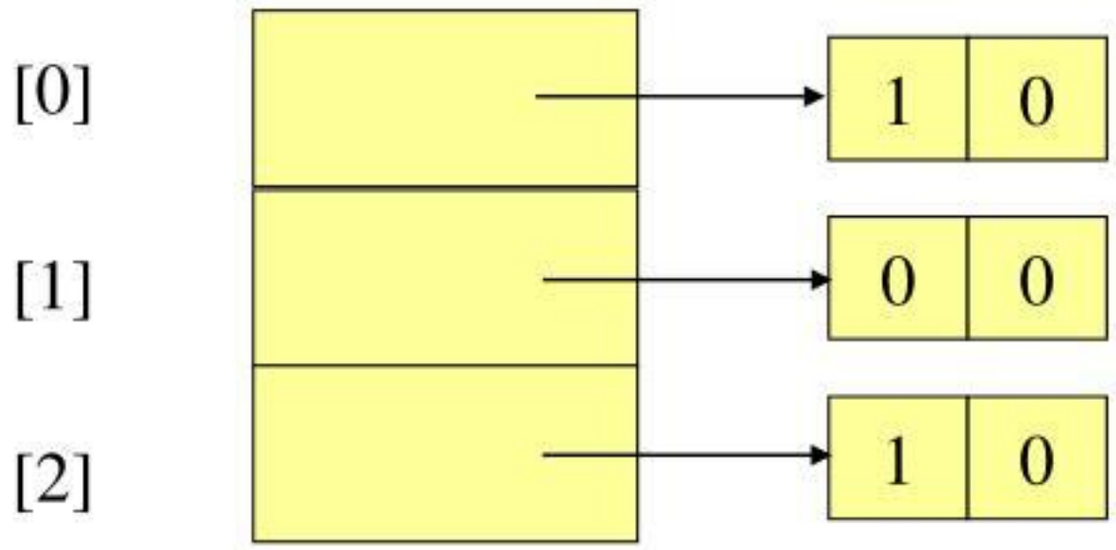
Graph and its Inverse adjacecy list



Inverse Adjacency Lists







Degree of Vertex in Graph

Degree of Vertex in a Directed Graph

In a directed graph, each vertex has an indegree and an outdegree.

Indegree of a Graph

Indegree of vertex V is the number of edges which are coming into the vertex V.

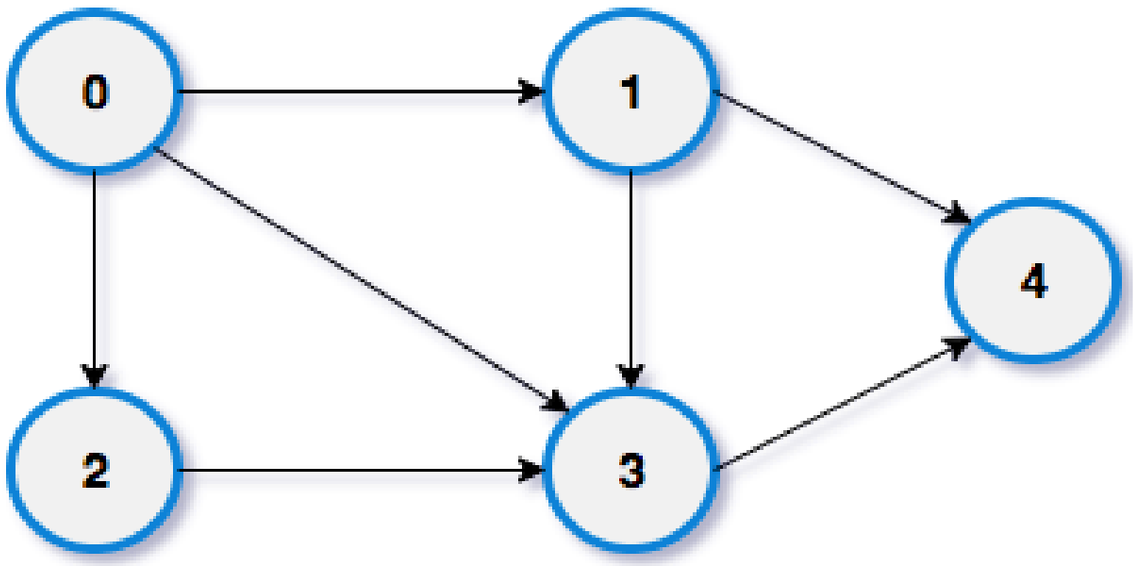
Notation − deg−(V).

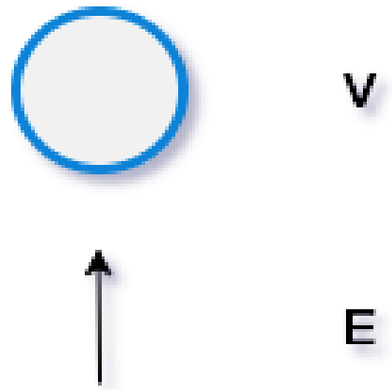
Outdegree of a Graph

Outdegree of vertex V is the number of edges which are going out from the vertex V.

Notation − deg+(V).

Degree of Vertex in Graph





In-degree :

In-degree of a vertex is the number of edges coming to the vertex.

In-degree of vertex 0 = 0

In-degree of vertex 1 = 1

In-degree of vertex 2 = 1

In-degree of vertex 3 = 3

In-degree of vertex 4 = 2

Out-degree

Out-degree of a vertex is the number edges which are coming out from the vertex.

Out-degree of vertex 0 = 3

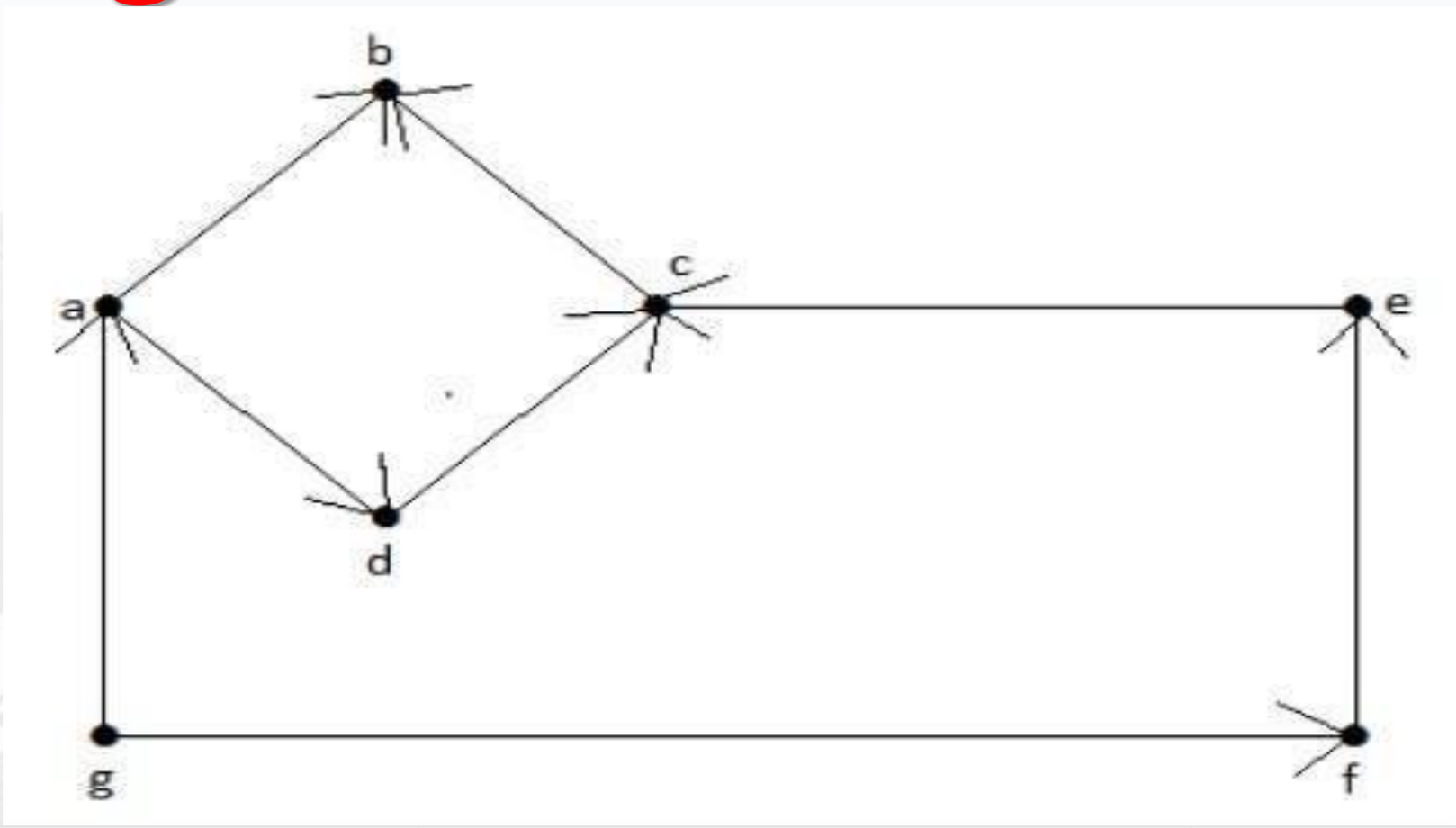
Out-degree of vertex 1 = 2

Out-degree of vertex 2 = 1

Out-degree of vertex 3 = 1

Out-degree of vertex 4 = 0

Degree of Vertex in Graph



Vertex	Indegree	Outdegree
a	1	2
b	2	0
c	2	1
d	1	1
e	1	1
f	1	1
g	0	2

Graph Traversal Algorithms

Traversing the graph means examining all the nodes and vertices of the graph. There are two standard methods by using which, we can traverse the graphs. Lets discuss each one of them in detail.

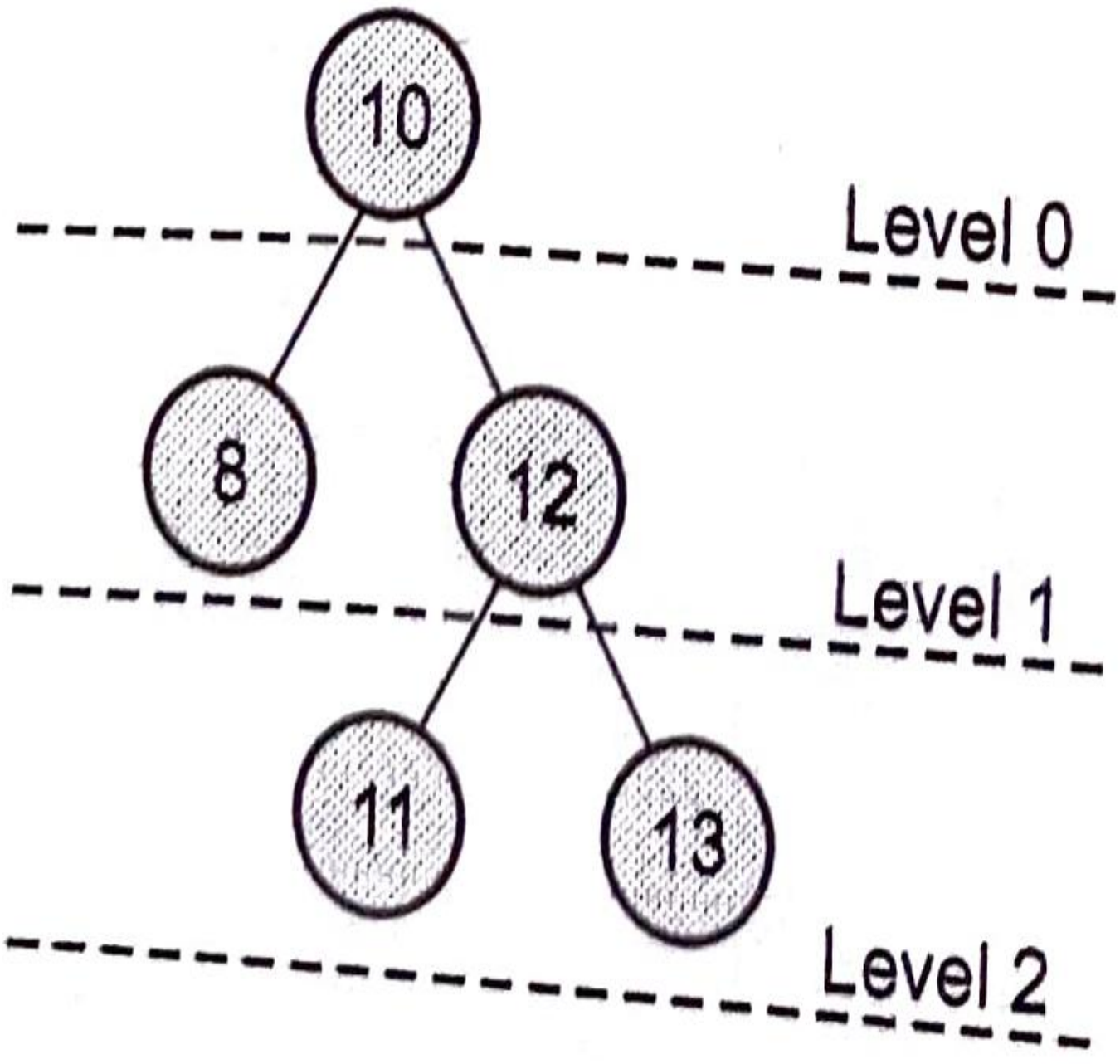
1. Breadth First Search
2. Depth First Search

BFS Algorithms

1. Breadth-first search is a graph traversal algorithm that starts traversing the graph from the root node and explores all the neighboring nodes. Then, it selects the nearest node and explores all the unvisited nodes. While using BFS for traversal, any node in the graph can be considered as the root node.

2. BFS is the most commonly used approach. It is a recursive algorithm to search all the vertices of a tree or graph data structure. BFS puts every vertex of the graph into two categories - visited and non-visited. It selects a single node in a graph and, after that, visits all the nodes adjacent to the selected node

BFS Algorithms



visit 10,

visit 8, 12

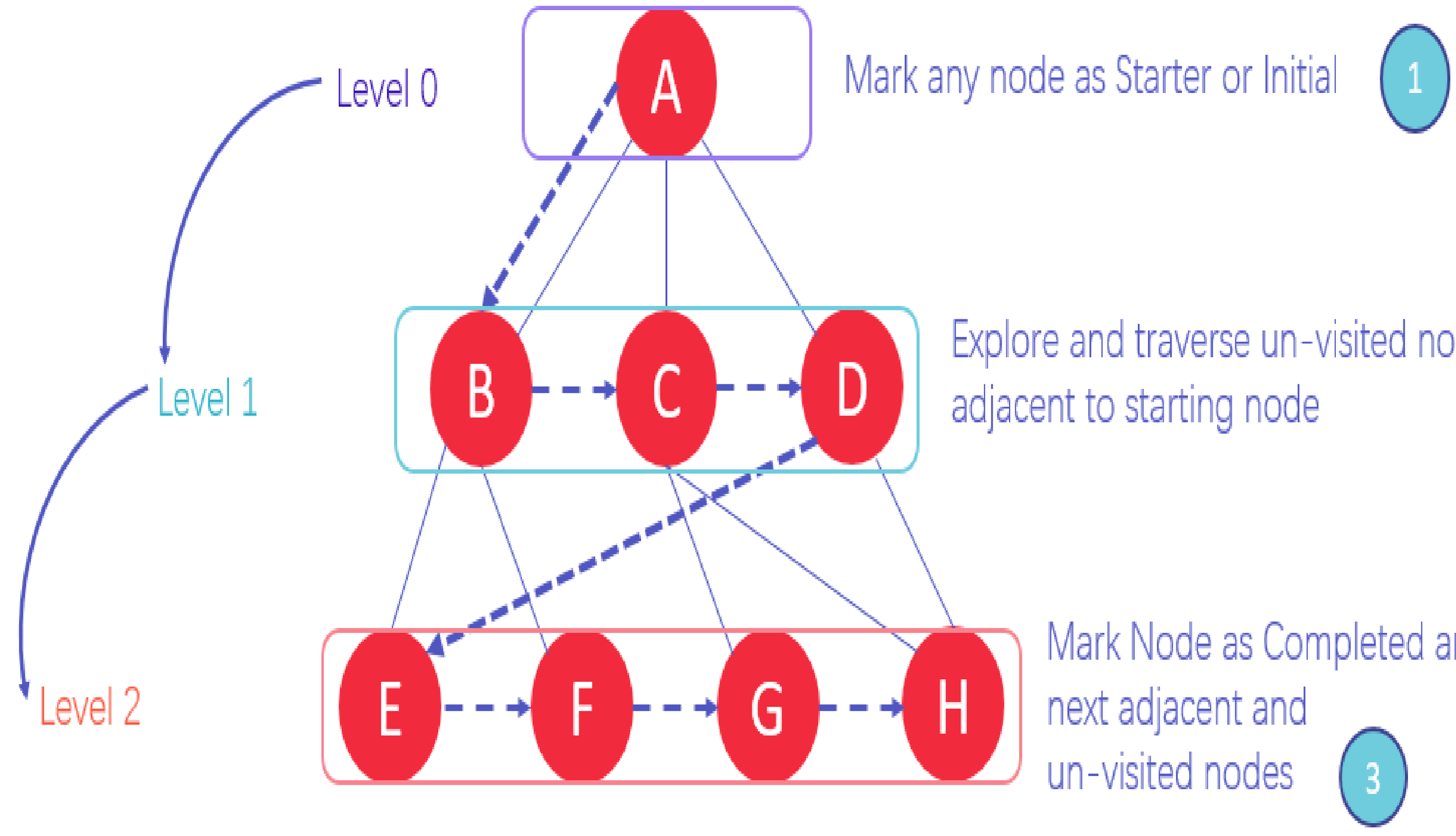
visit 11, 13

∴ The BFS sequence will be

10, 8, 12, 11, 13

BFS Algorithms

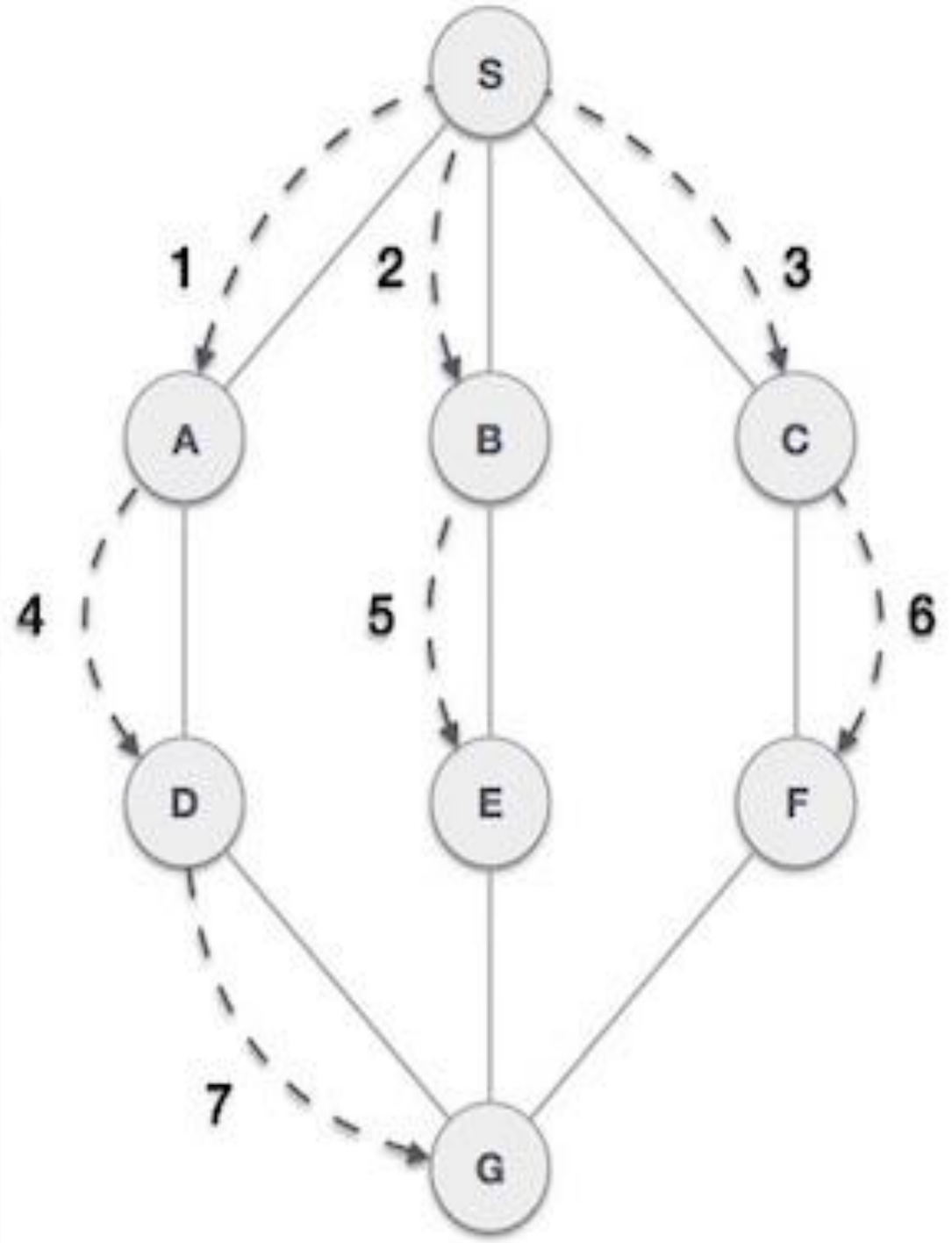
CONCEPT DIAGRAM



BFS Algorithms

As in the example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs the following rules.

- **Rule 1** − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** − If no adjacent vertex is found, remove the first vertex from the queue**.**
- **Rule 3** − Repeat Rule 1 and Rule 2 until the queue is empty.



BFS Algorithms

Step 1: SET STATUS = 1 (ready state)

for each node in G

Step 2: Enqueue the starting node A

and set its STATUS = 2

(waiting state)

Step 3: Repeat Steps 4 and 5 until

QUEUE is empty

Step 4: Dequeue a node N. Process it

and set its STATUS = 3

(processed state).

Step 5: Enqueue all the neighbours of

N that are in the ready state

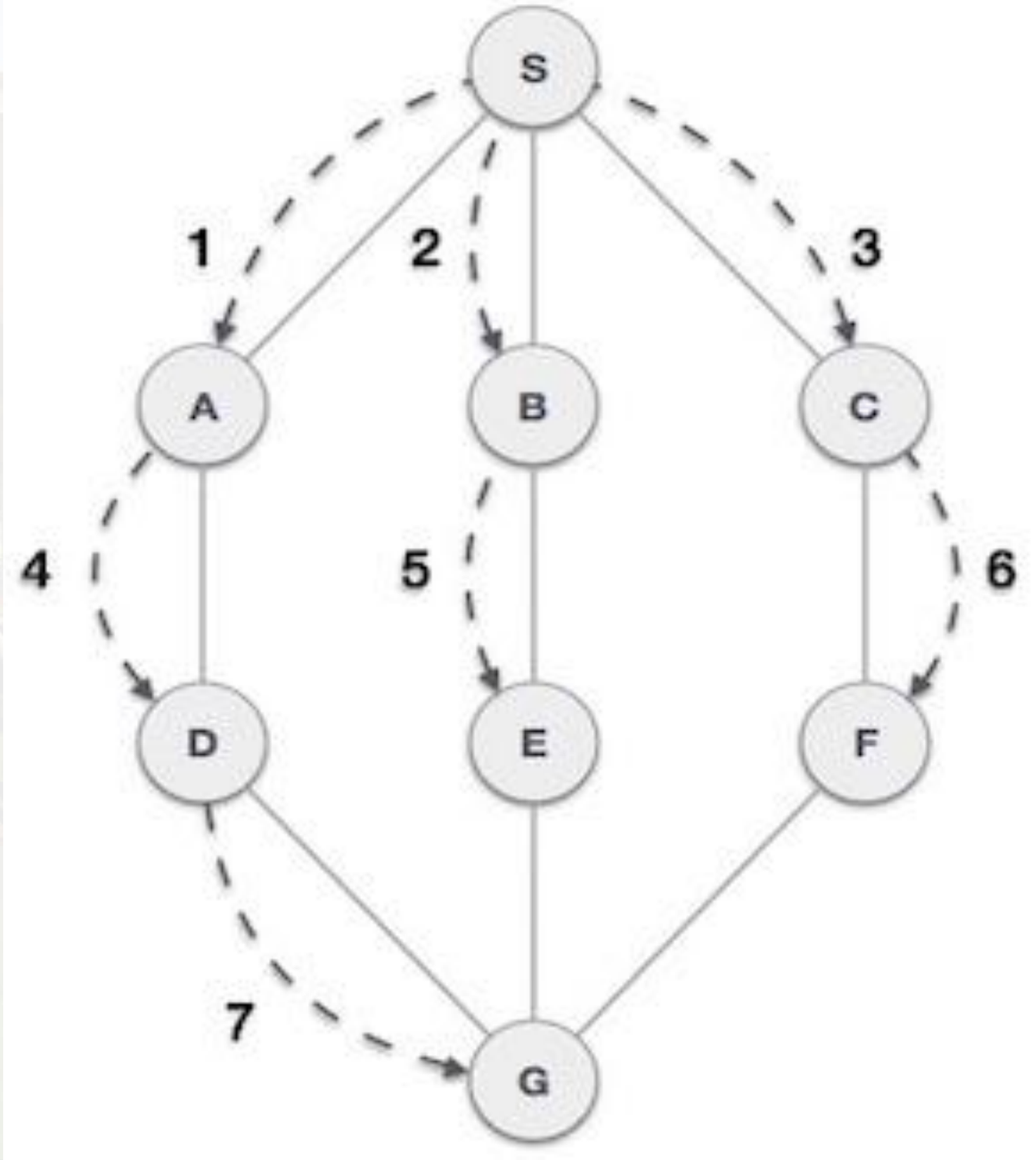
(whose STATUS = 1) and set

their STATUS = 2

(waiting state)

[END OF LOOP]

Step 6: EXIT



BFS Algorithms

Rule 1 − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.

Rule 2 − If no adjacent vertex is found, remove the first vertex from the queue.

Rule 3 − Repeat Rule 1 and Rule 2 until the queue is empty.

BFS Algorithms

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Enqueue the starting node A and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until QUEUE is empty

Step 4: Dequeue a node N. Process it and set its STATUS = 3 (processed state).

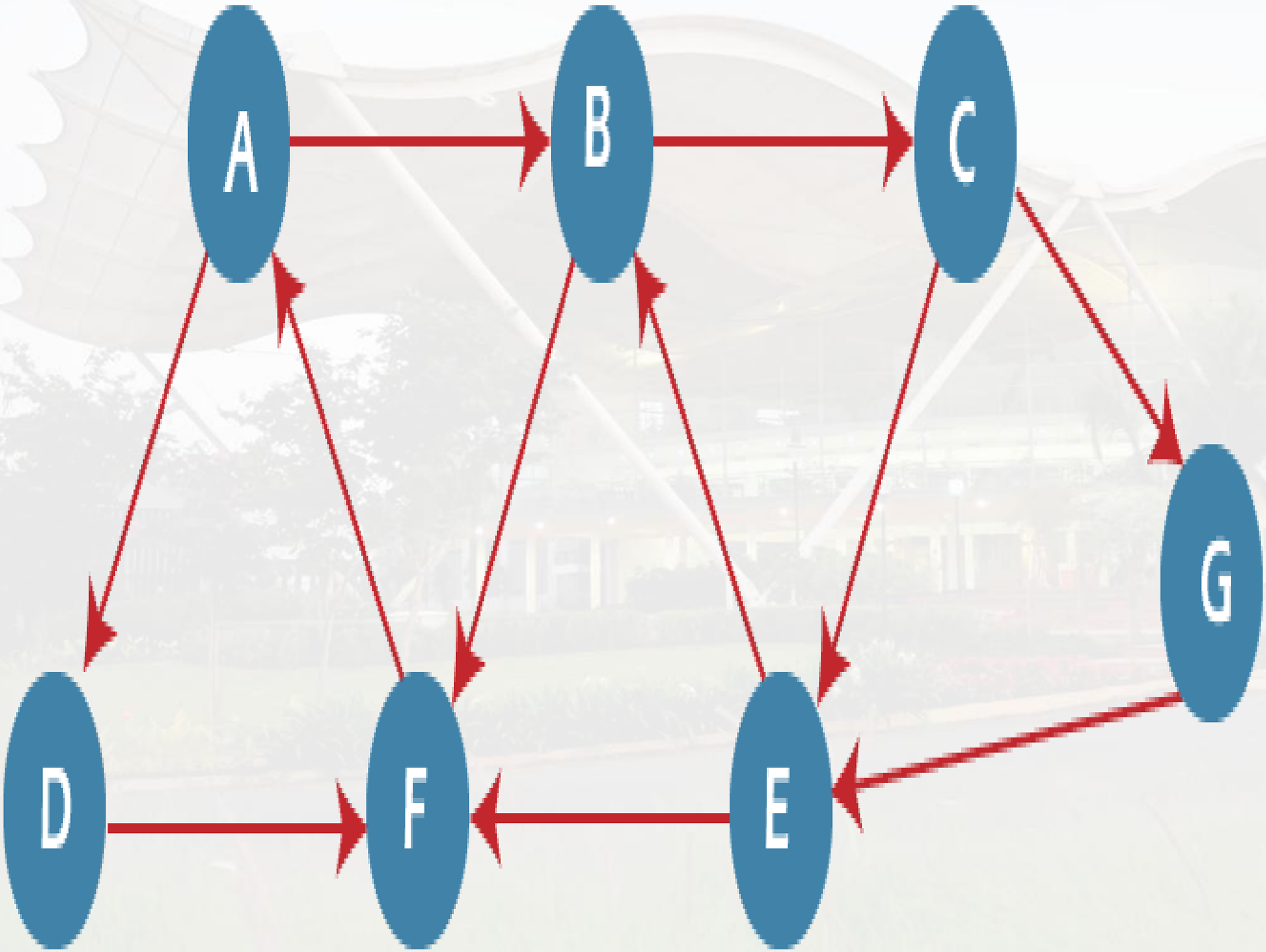
Step 5: Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set

their STATUS = 2

(waiting state)

[END OF LOOP]

Example of BFS Algorithms



Adjacency Lists

A : B, D

B : C, F

C : E, G

G : E

E : B, F

F : A

D : F

Complexity of BFS Algorithms

The time complexity of the BFS algorithm is

represented in the form of O(V + E), where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is O(V).

Advantages and Disadvantages of BFS

- **Advantages :**

1. A BFS will find the shortest path between the starting point and any other reachable node.
2. A depth-first search will not necessarily find the shortest path.

- **Disadvantages :**

A BFS on a binary tree generally requires more memory than a DFS.

Application of BFS Algorithms

1. BFS can be used to find the neighboring locations from a given source location.

2. In a peer-to-peer network, BFS algorithm can be used as a traversal method to find all the neighboring nodes. Most torrent clients, such as BitTorrent, uTorrent, etc. employ this process to find "seeds" and "peers" in the network.

3. BFS is used to determine the shortest path and minimum spanning tree.

4. BFS is also used in Cheney's technique to duplicate the garbage collection.

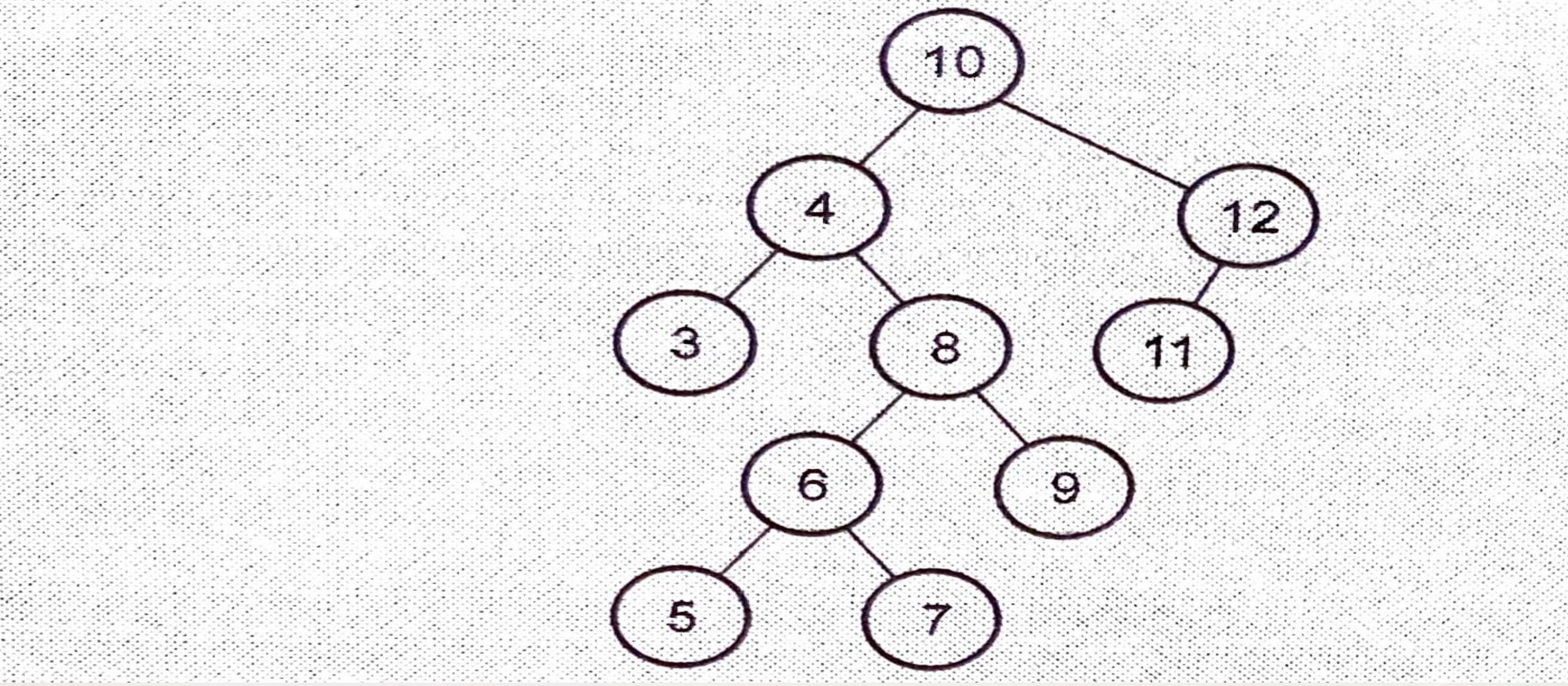
5. It can be used in ford-Fulkerson method to compute the maximum flow in a flow network.

Application of BFS Algorithms

1. To build index by search index
2. For GPS navigation
3. Path finding algorithms
4. Cycle detection in an undirected graph
5. In minimum spanning tree

Application of BFS Algorithms

Consider following tree and find its BFS sequence.



: 7, display it.

BFS sequence : 10, 4, 12, 3, 8, 11, 6, 9, 5, 7

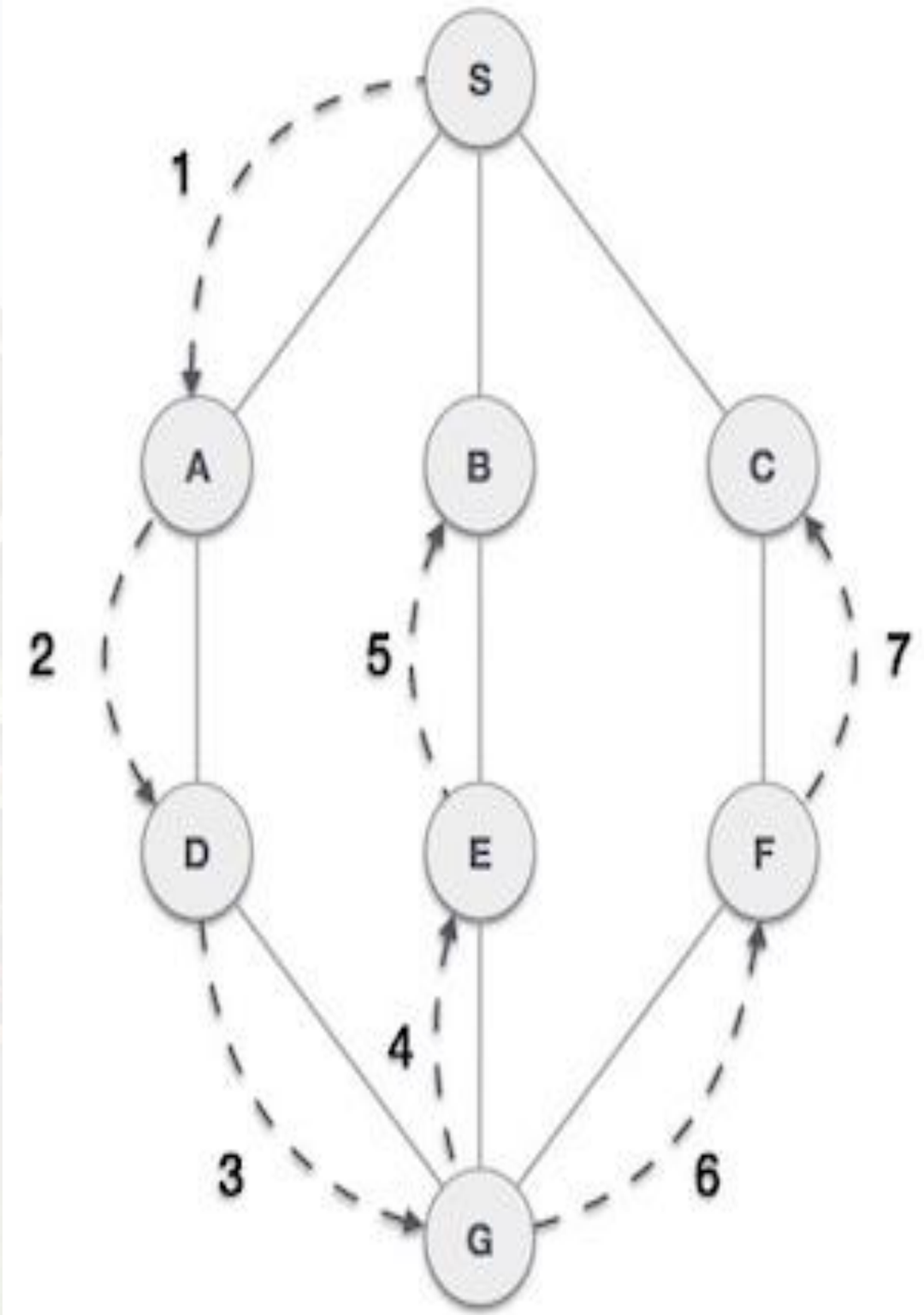
Depth First Search (DFS)

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.

DFS Algorithms

Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



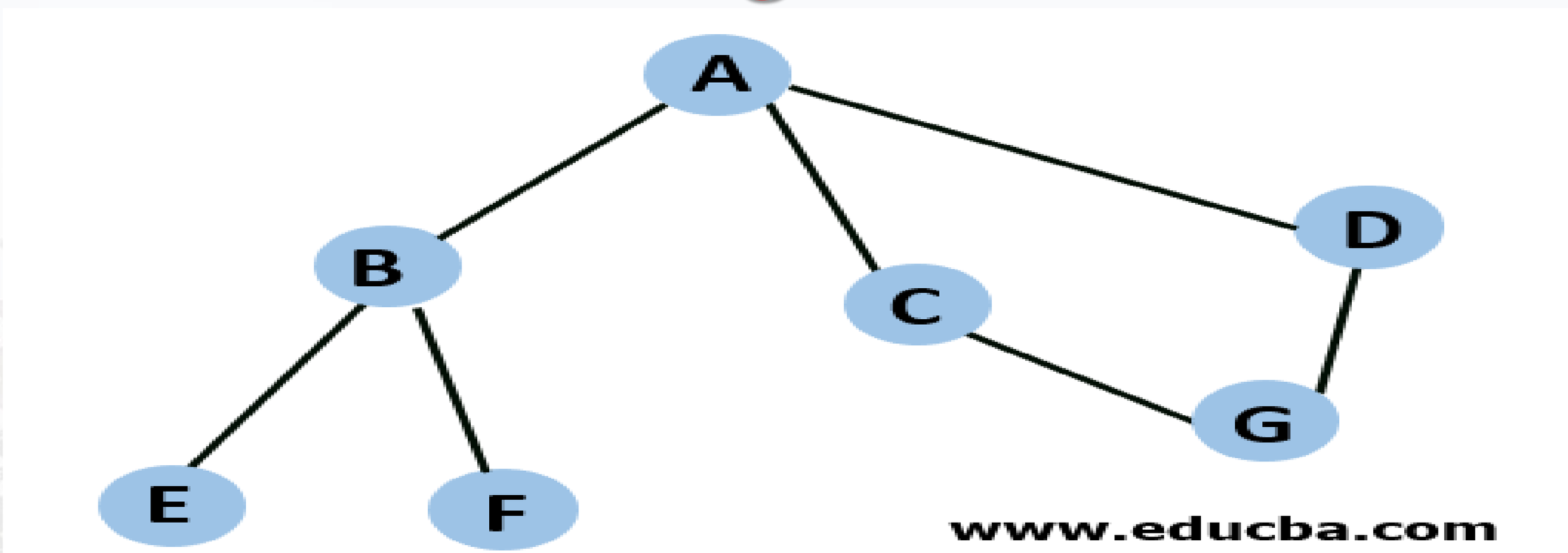
- As in the example given above, DFS algorithm traverses from A to B to C to D first then to E, then to F and lastly to G. It employs the following rules.

- **Rule 1** − Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.

- **Rule 2** − If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)

- **Rule 3** − Repeat Rule 1 and Rule 2 until the stack is empty

DFS Algorithms



Traversing the above graph in BFS fashion would result from A -> B -> E -> F -> C -> G -> D. The algorithm starts from node A and traverses all its child nodes. As soon as it encounters B, it seems that it has further child nodes. So, the child nodes of B are traversed before proceeding to the next child node of A

DFS Algorithms

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Push the starting node A on the stack and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until STACK is empty

Step 4: Pop the top node N. Process it and set its STATUS = 3 (processed state)

Step 5: Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their

STATUS = 2 (waiting state)

[END OF LOOP]

Step 6: EXIT

DFS Algorithms

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

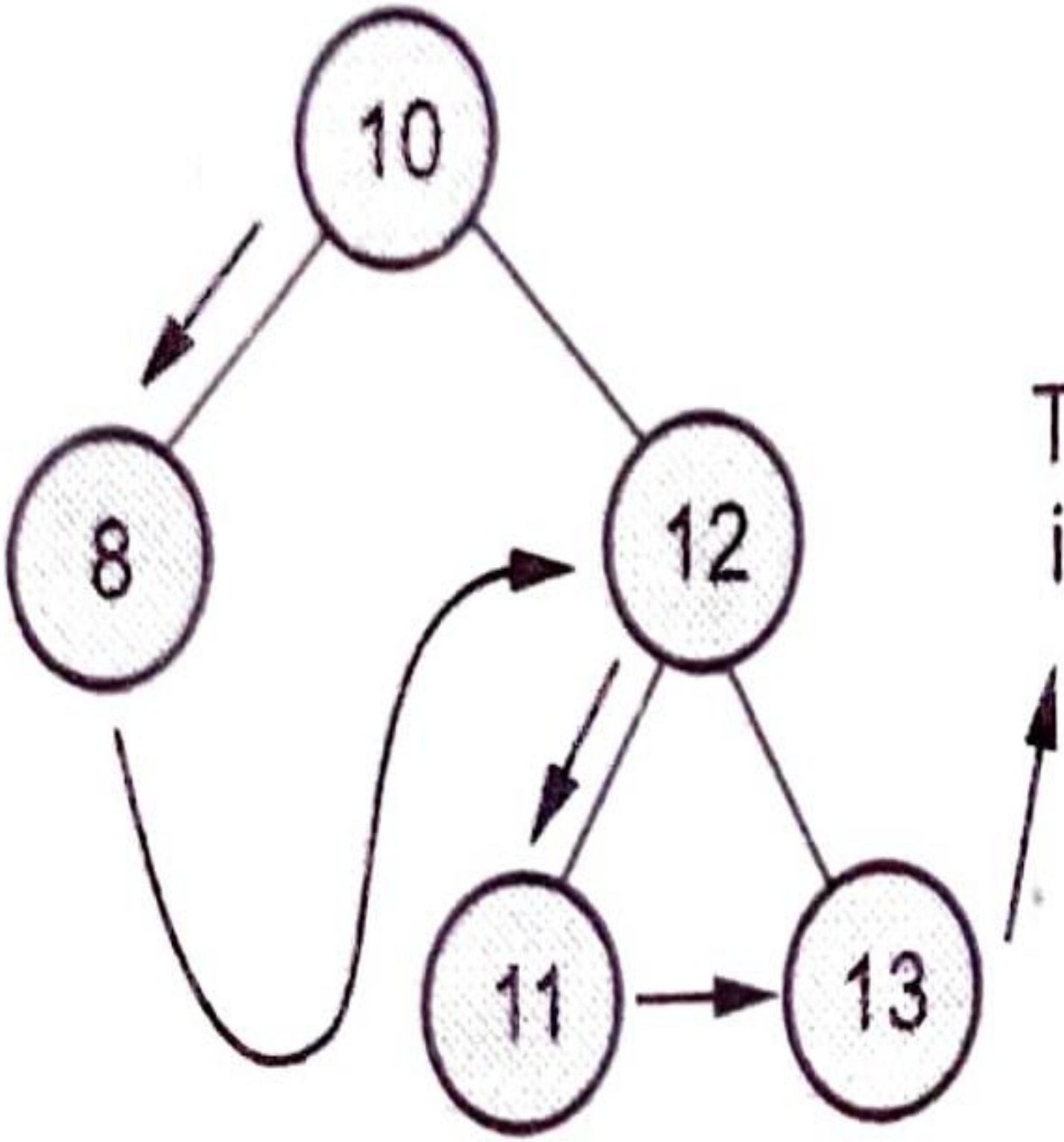
The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.

DFS Algorithms

DFS is an algorithm for finding or traversing graphs or trees in depth-ward direction. The execution of the algorithm begins at the root node and explores each branch before backtracking. It uses a stack data structure to remember, to get the subsequent vertex, and to start a search, whenever a dead-end appears in any iteration.

The full form of DFS is Depth-first search.

DFS Algorithms



The DFS sequence is **10, 8, 12, 11, 13**

DFS Algorithms

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Push the starting node A on the stack and set its STATUS = 2 (waiting state)

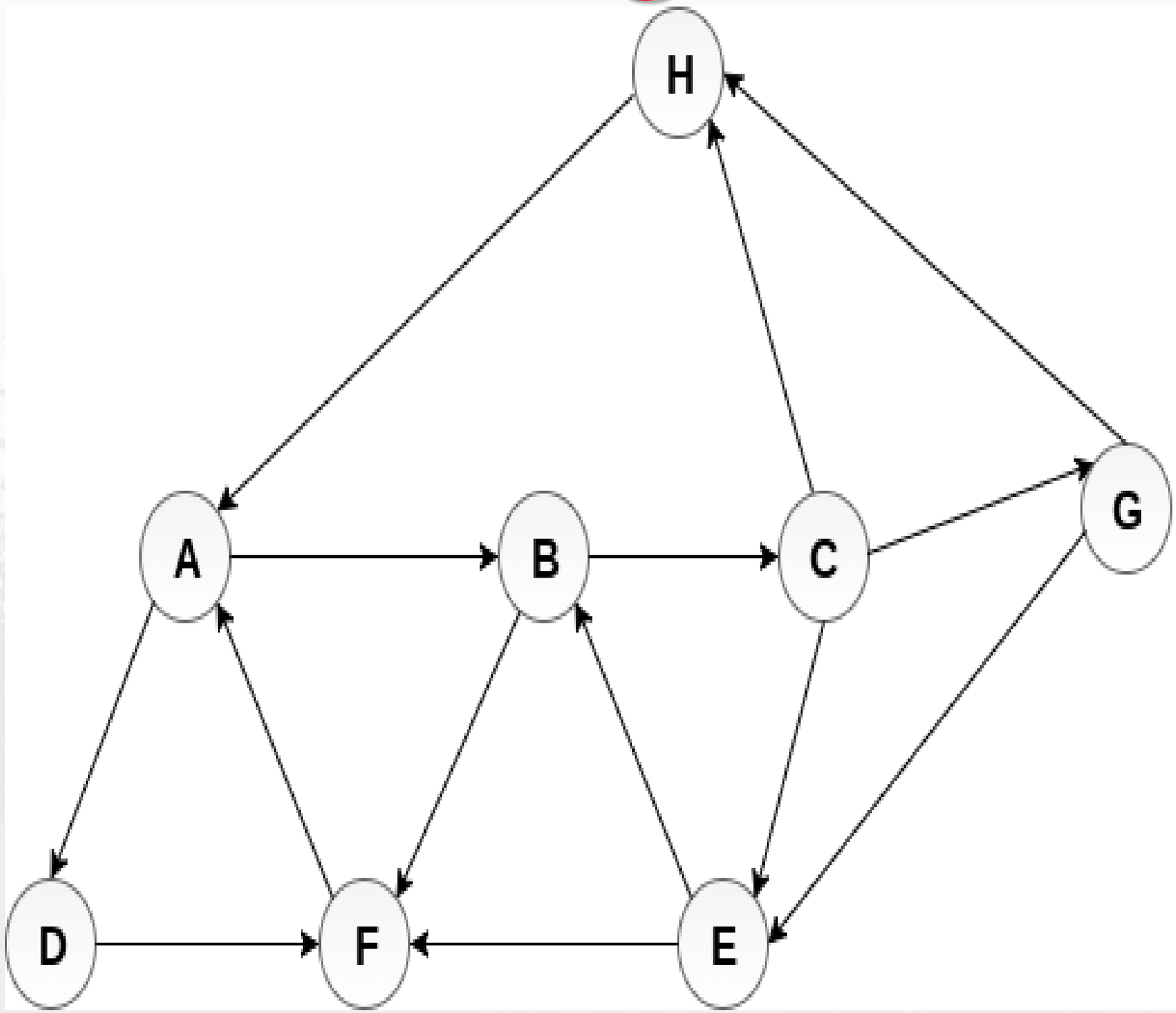
Step 3: Repeat Steps 4 and 5 until STACK is empty

Step 4: Pop the top node N. Process it and set its STATUS = 3 (processed state)

Step 5: Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their
STATUS = 2 (waiting state)
[END OF LOOP]

Step 6: EXIT

DFS Algorithms Example



Adjacency Lists

A : B, D

B : C, F

C : E, G, H

G : E, H

E : B, F

F : A

D : F

H : A

The printing sequence of the graph will be :

H → A → D → F → B → C → G → E

Complexity of DFS Algorithms

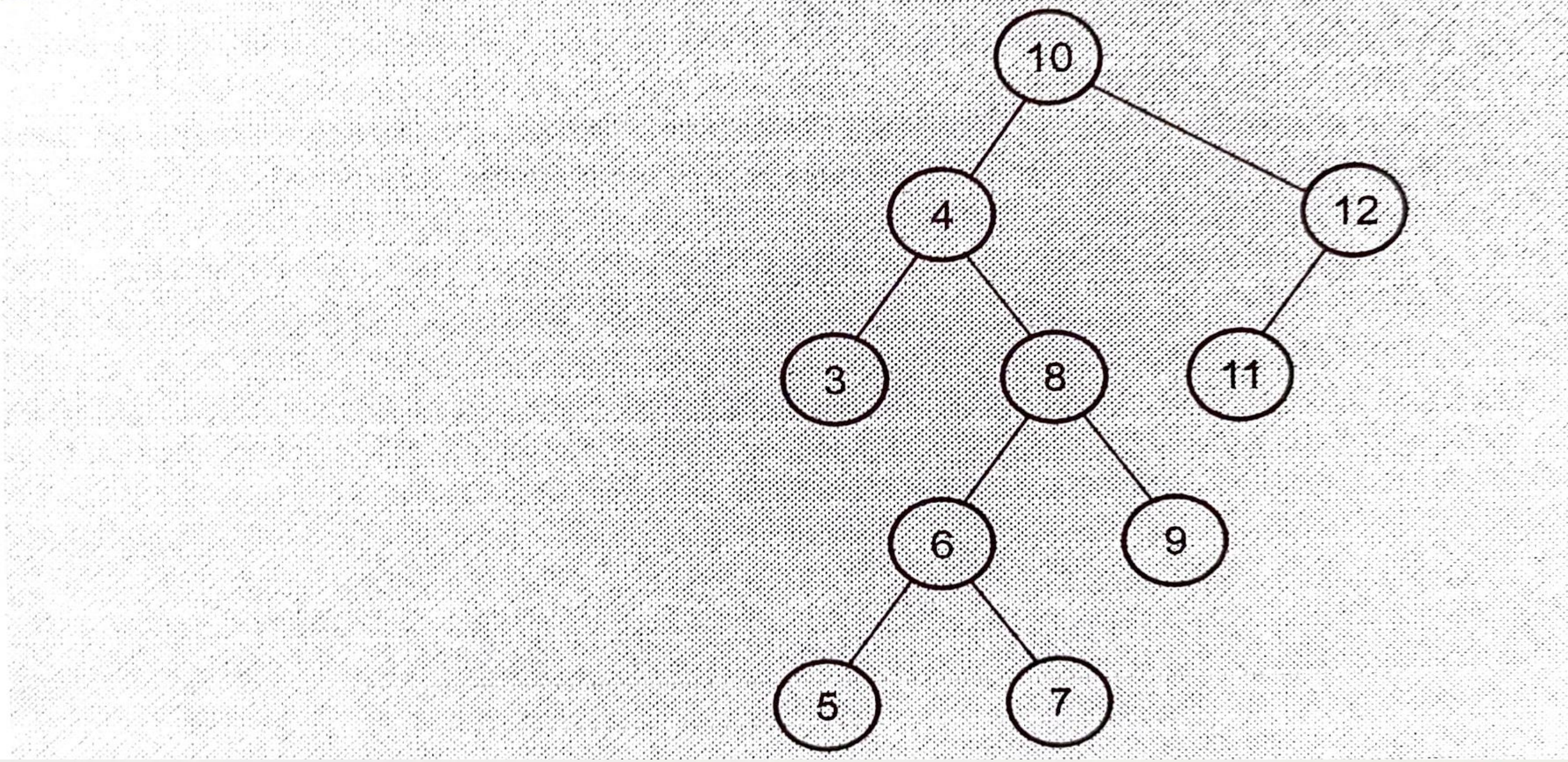
Complexity of Depth First Search

The time complexity of the DFS algorithm is represented in the form of O(V + E), where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is O(V).

Example of DFS Algorithms

Example 2.7.1 *Consider following tree and obtain its DFS sequence.*



10, 4, 3, 8, 6, 5, 7, 9, 12, 11

Advantages and Disadvantages of DFS

- **Advantages :**

1. Depth-first search on a binary tree generally requires less memory than breadth-first.

2. Depth-first search can be easily implemented with recursion..

- **Disadvantages :**

A DFS doesn't necessarily find the shortest path to a node, while breadth-first search does.

Application of DFS Algorithms

1. For finding the path

2. To test if the graph is bipartite

3. For finding the strongly connected components of a graph

4. For detecting cycles in a graph

BFS Vs DFS

sn	BFS	DFS
1	BFS finds the shortest path to the destination.	DFS goes to the bottom of a subtree, then backtracks.
2	The full form of BFS is Breadth-First Search.	The full form of DFS is Depth First Search.
3	It uses a **queue** to keep track of the next location to visit.	It uses a **stack** to keep track of the next location to visit.
4	BFS traverses according to **tree level.**	DFS traverses according to **tree depth.**
5	It is implemented using **FIFO list**.	It is implemented using **LIFO list.**
6	It requires **more memory** as compare to DFS.	It requires **less memory** as compare to BFS.

BFS Vs DFS

sn	BFS	DFS
7	There is no need of backtracking in BFS.	There is a need of backtracking in DFS.
8	You can never be trapped into finite loops.	You can be trapped into infinite loops.
9	If you do not find any goal, you may need to expand many nodes before the solution is found.	If you do not find any goal, the leaf node backtracking may occur.

Applications of Graph

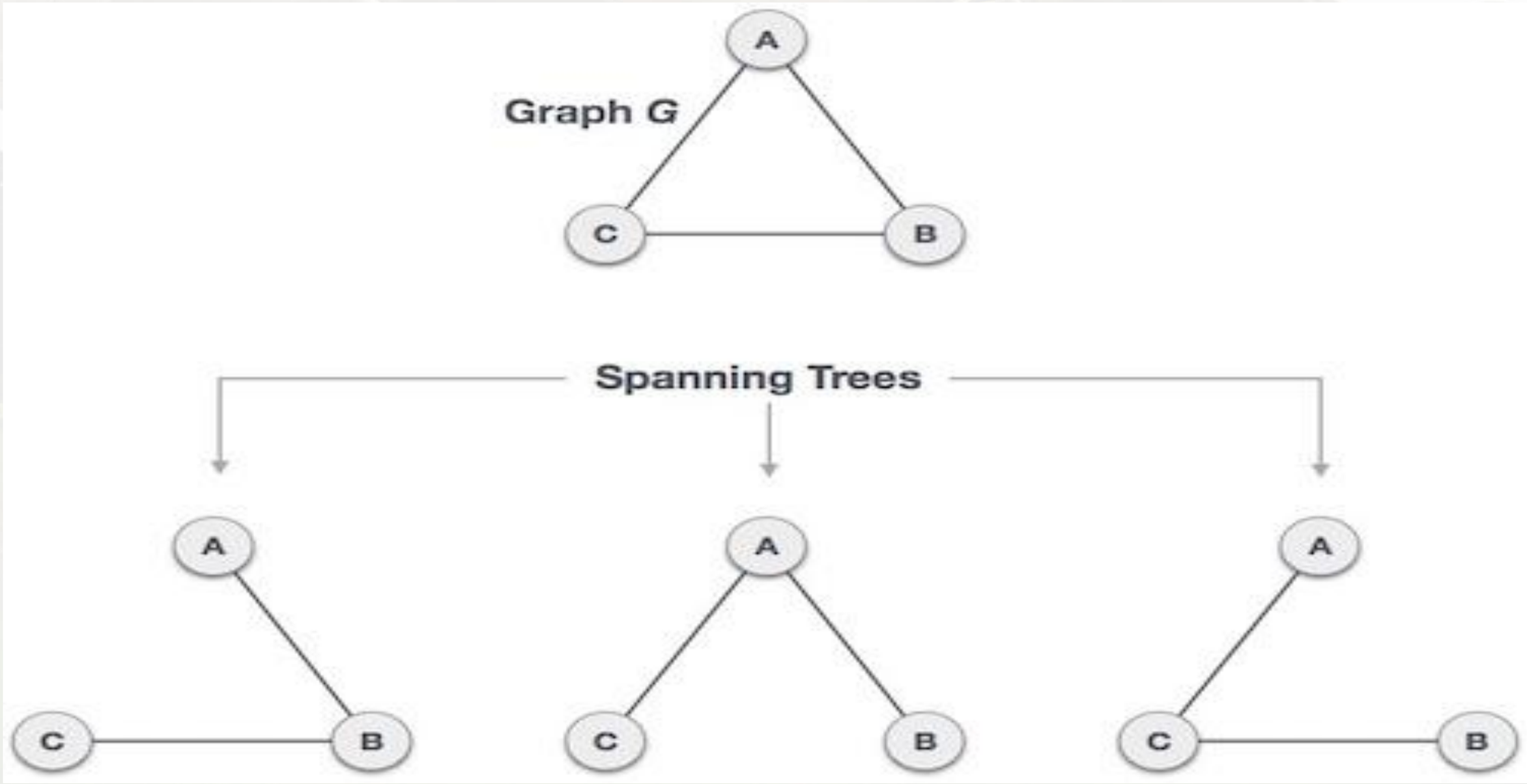
1. In **Computer science** graphs are used to represent the flow of computation.
2. **Google maps** uses graphs for building transportation systems.
3. **In Facebook,** users are considered to be the vertices and if they are friends then there is an edge running between them.
4. Graphs are used to define the flow of computation.
5. Graphs are used to represent networks of communication.
6. Graphs are used to represent data organization.
7. Graph theory is used to find shortest path in road or a network.

Tree Vs Graph

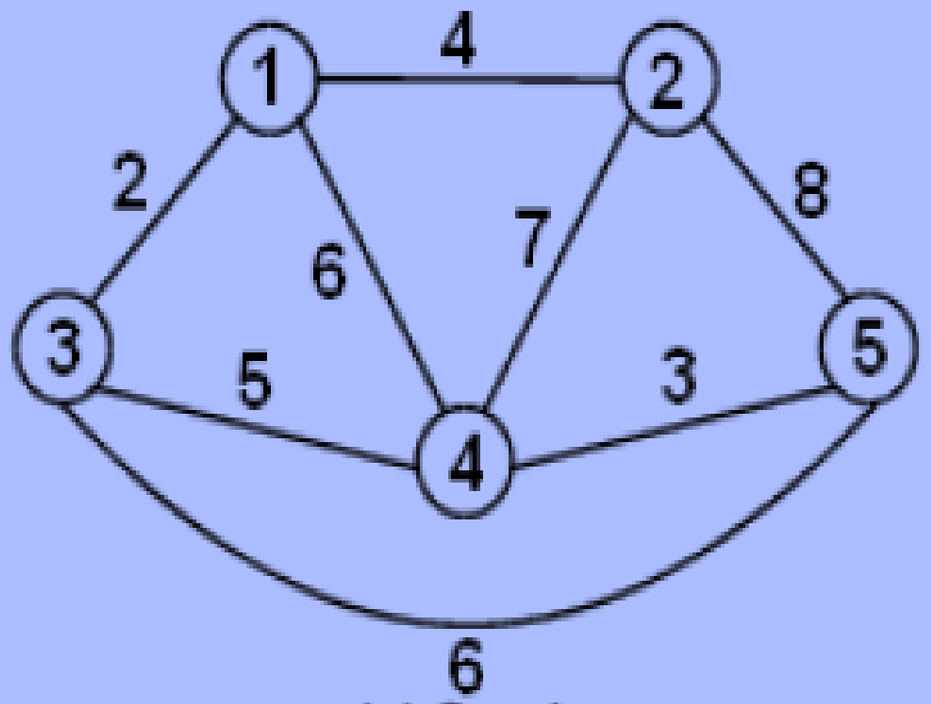
Basis For Comparision	TREE	GRAPH
Path	Only one between Two Vertices	More than one path allowed.
Root node	It Has Exactly one root node	Graph may or may not have root node
Loops	No Loops are Permited	No Loop are Permited if loops exist then we call it Multigraph.
Complexity	Less Complex	More Complex comparetively
Traversal Technique	Pre-order, in-order, post-order, level-order	Breadth First Search, Depth First Search
Number of Edges	N-1	Not Define
Model Type	Hierarchical	Network

Spanning Tree of Graph

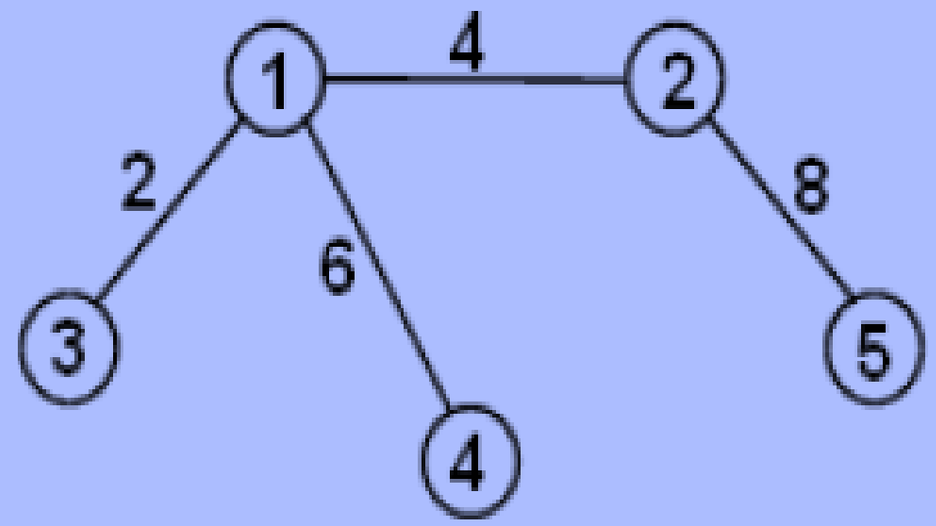
A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..



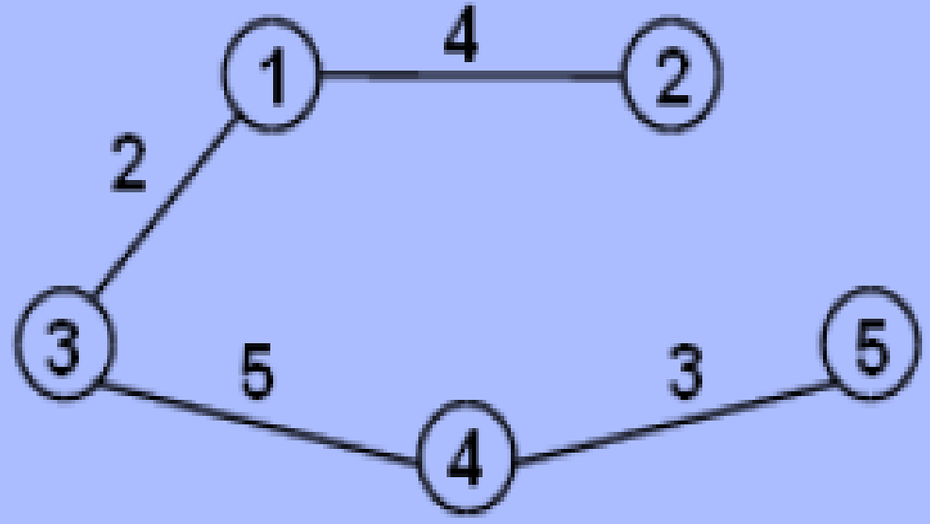
Minimum Spanning Tree



(a) Graph



(b) Spanning Tree



(c) Minimum Spanning Tree

Spanning Tree of Graph

- **Spanning Tree Applications**

1. Computer Network Routing Protocol
2. Cluster Analysis
3. Civil Network Planning

- **Minimum Spanning tree Applications**

1. To find paths in the map
2. To design networks like telecommunication networks, water supply networks, and electrical grids.

GREEDY STRATEGIES

Greedy algorithm :

An algorithm is designed to achieve **optimum solution** for a given problem. In greedy algorithm approach, decisions are made from the given solution domain. As being greedy, **the closest solution that seems to provide an optimum solution is chosen.**

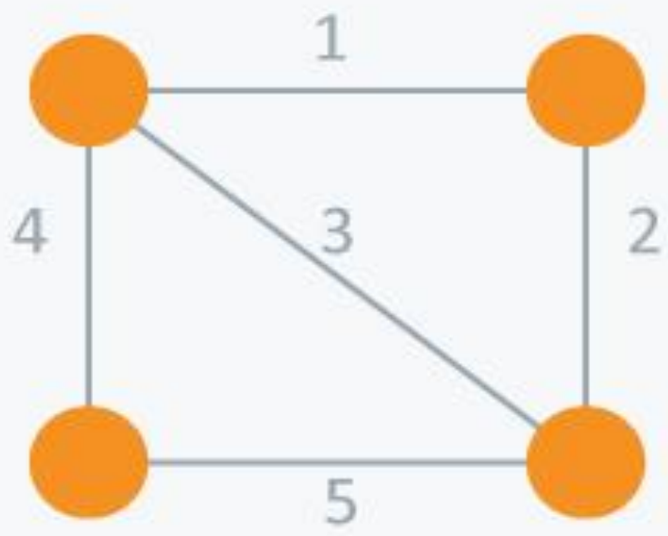
Example of greedy strategy :

1. Travelling Salesman Problem
2. Prim's Minimal Spanning Tree Algorithm
3. Kruskal's Minimal Spanning Tree Algorithm
4. Dijkstra's Minimal Spanning Tree Algorithm
5. Knapsack Problem
6. Job Scheduling Problem

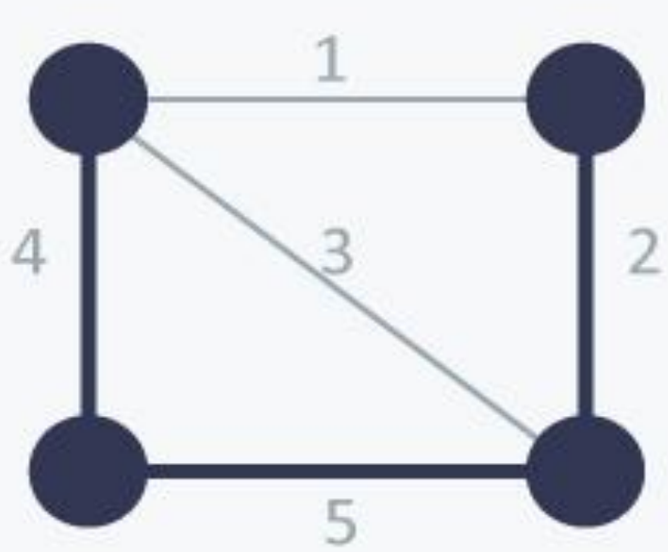
GREEDY STRATEGIES

1. Minimum Spanning tree (Prims or Kruskal's algorithms)

The cost of the spanning tree is the sum of the weights of all the edges in the tree. There can be many spanning trees. Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees. There also can be many minimum spanning trees.

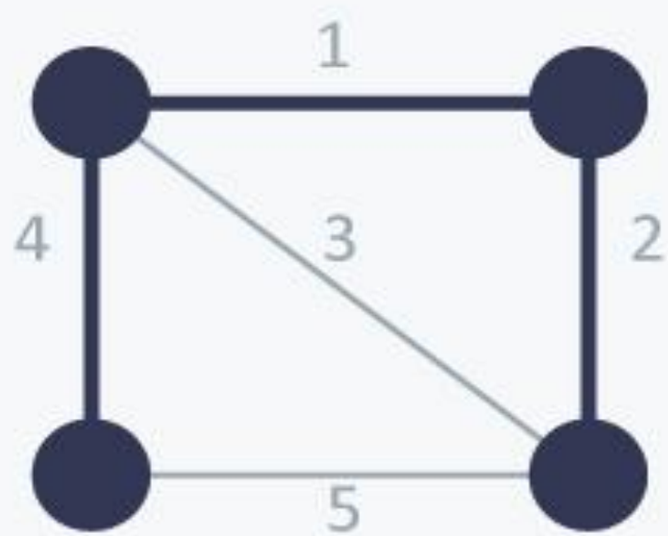


Undirected Graph



Spanning Tree

Cost = 11(=4+5+2)



Minimum Spanning Tree

Cost = 7(=4+1+2)

GREEDY STRATEGIES

1. **Kruskal’s algorithms :**

Kruskal’s Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree.

Algorithm Steps :

1. Sort the graph edges with respect to their weights.
2. Start adding edges to the MST from the edge with the smallest weight until the edge of the largest weight.
3. Only add edges which doesn't form a cycle , edges which connect only disconnected components.

GREEDY STRATEGIES

1. **Kruskal’s algorithms :**

- Kruskal's algorithm is a minimum spanning tree algorithm that takes a graph as input and finds the subset of the edges of that graph which
- form a tree that includes every vertex
- has the minimum sum of weights among all the trees that can be formed from the graph
- **Kruskal's Algorithm Complexity**
- The time complexity Of Kruskal's Algorithm is: O(E log E).

GREEDY STRATEGIES

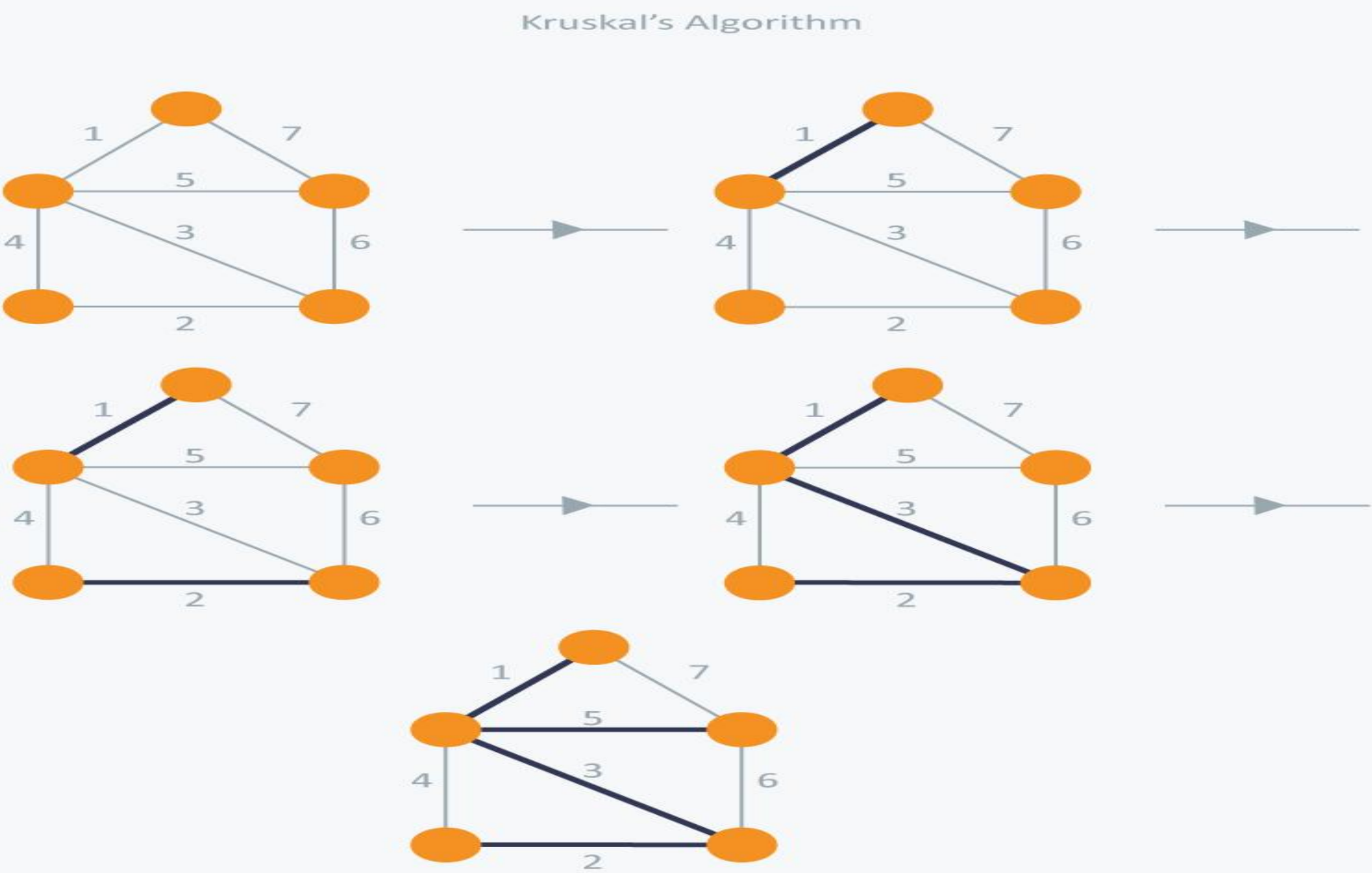
1. Kruskal’s algorithms :

The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
3. Keep adding edges until we reach all vertices.

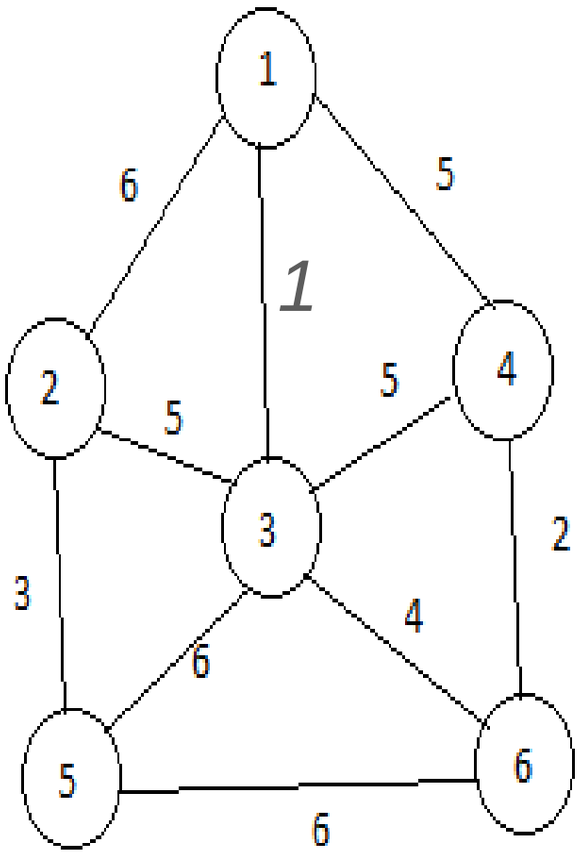
GREEDY STRATEGIES

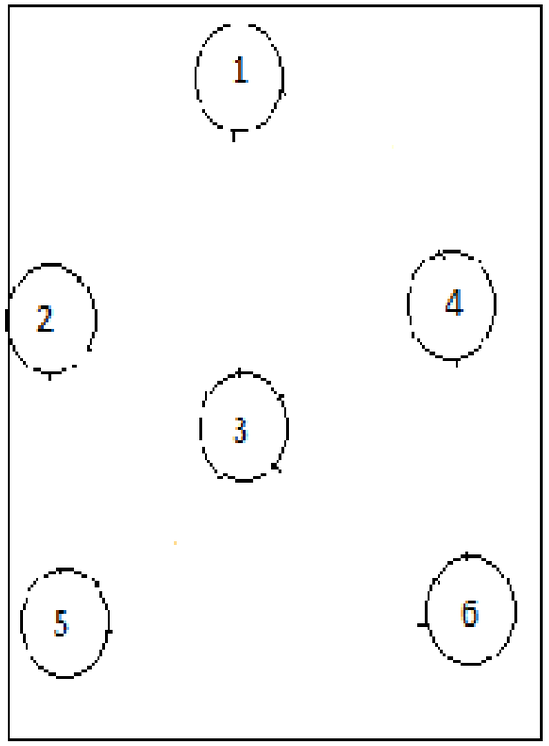
2. Kruskal's algorithms : Example

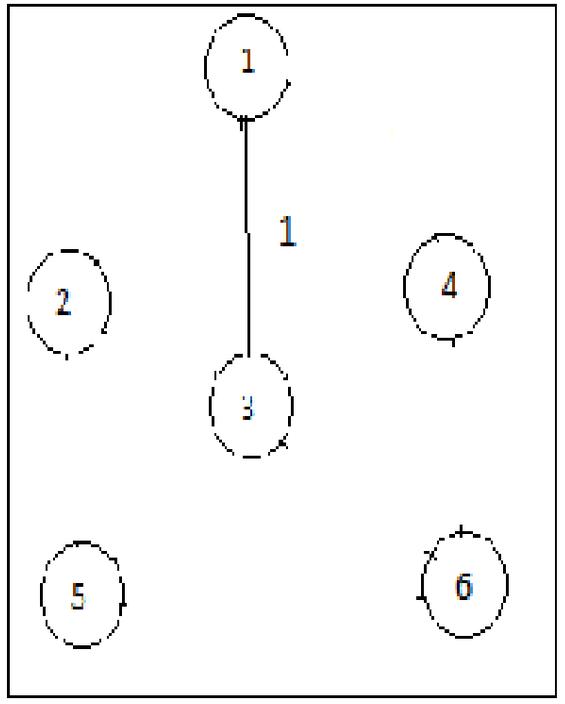


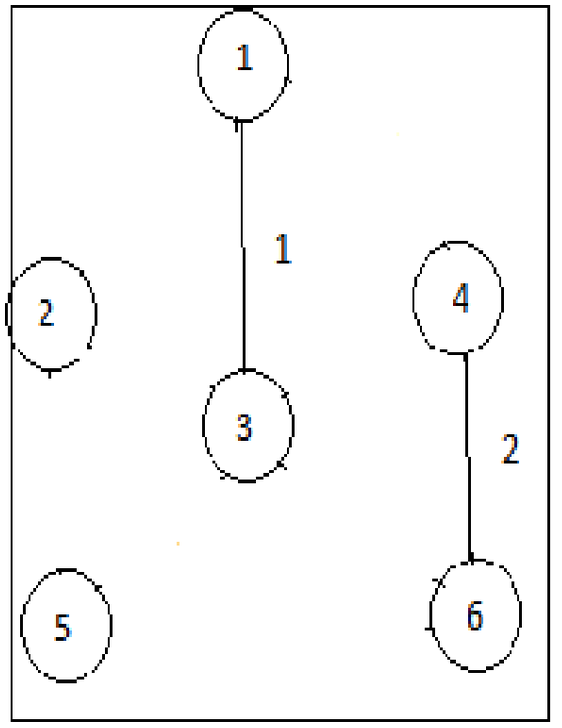
GREEDY STRATEGIES

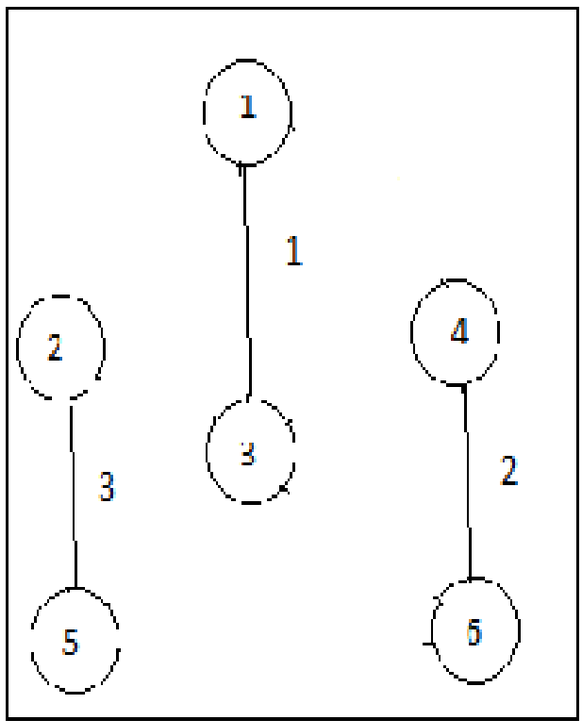
2. Kruskal's algorithms : Example

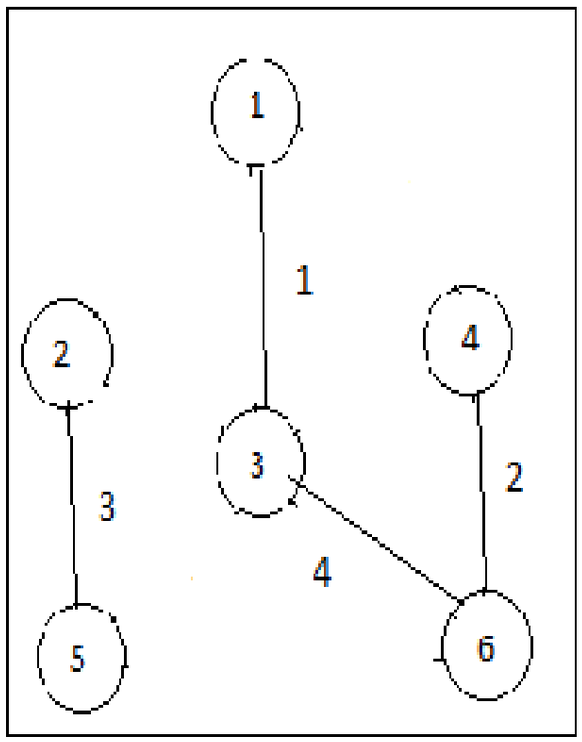


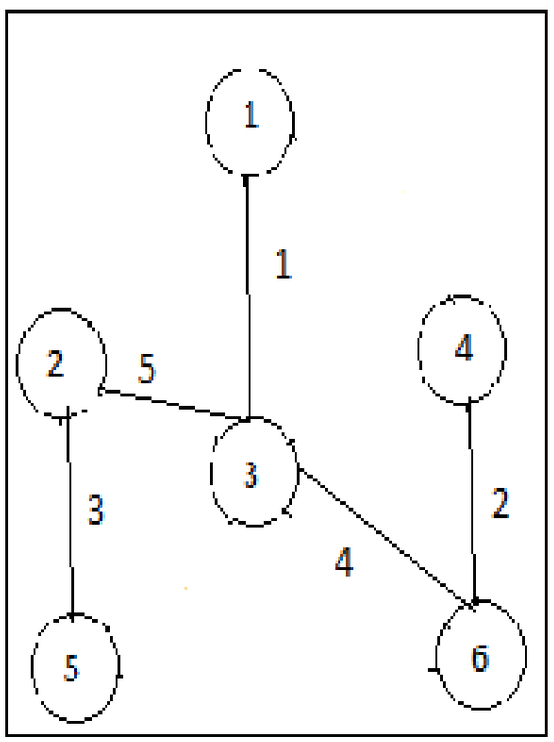






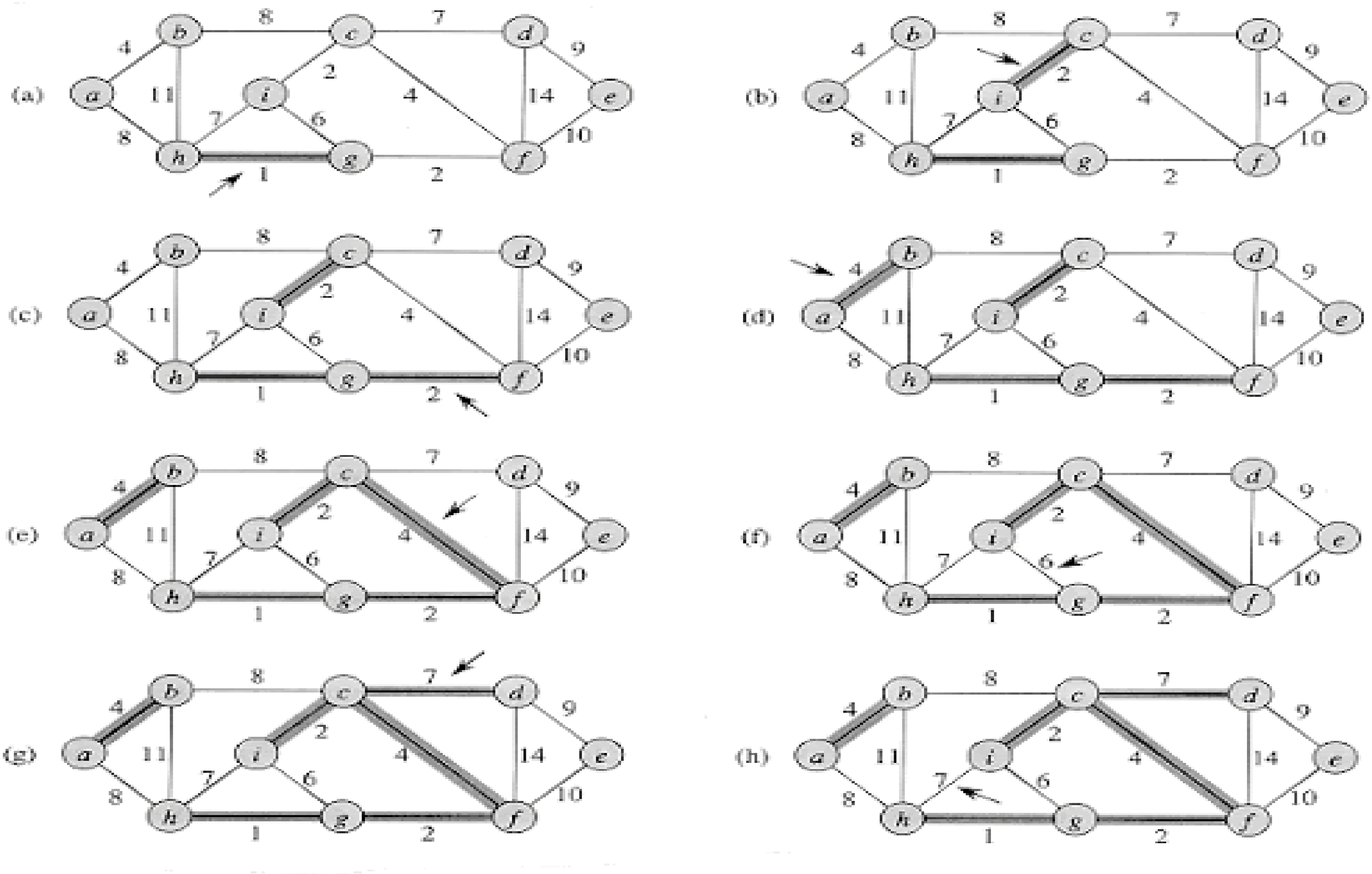






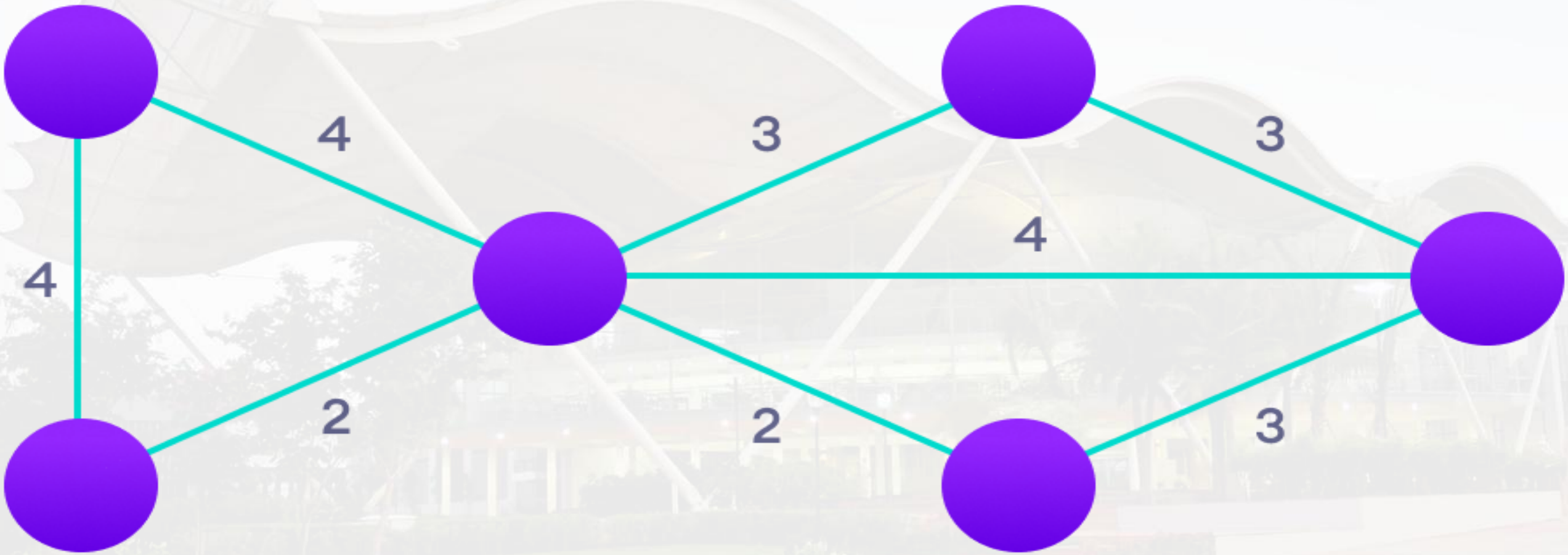
GREEDY STRATEGIES

2. Kruskal's algorithms : Example



GREEDY STRATEGIES

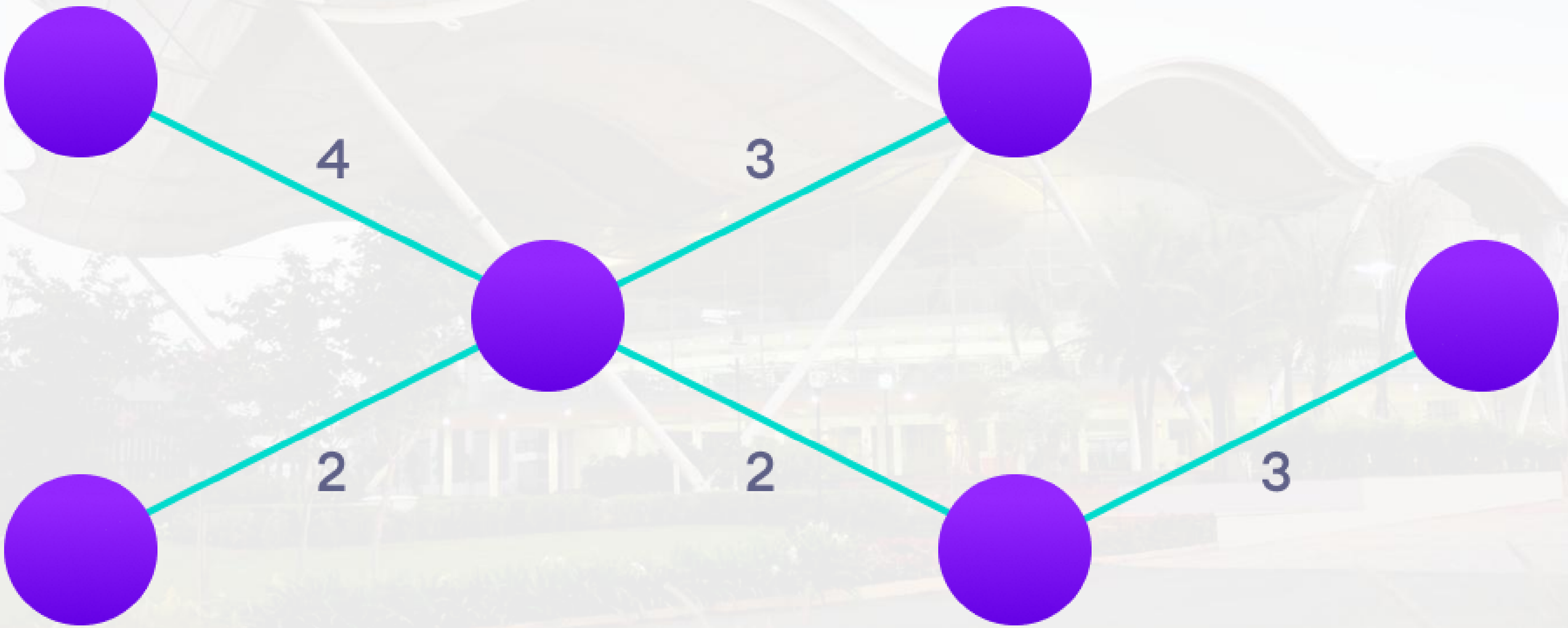
2. **Kruskal Algorithms : Example**



Step: 1

GREEDY STRATEGIES

1. Kruskal Algorithms : Example



Step: 6

GREEDY STRATEGIES

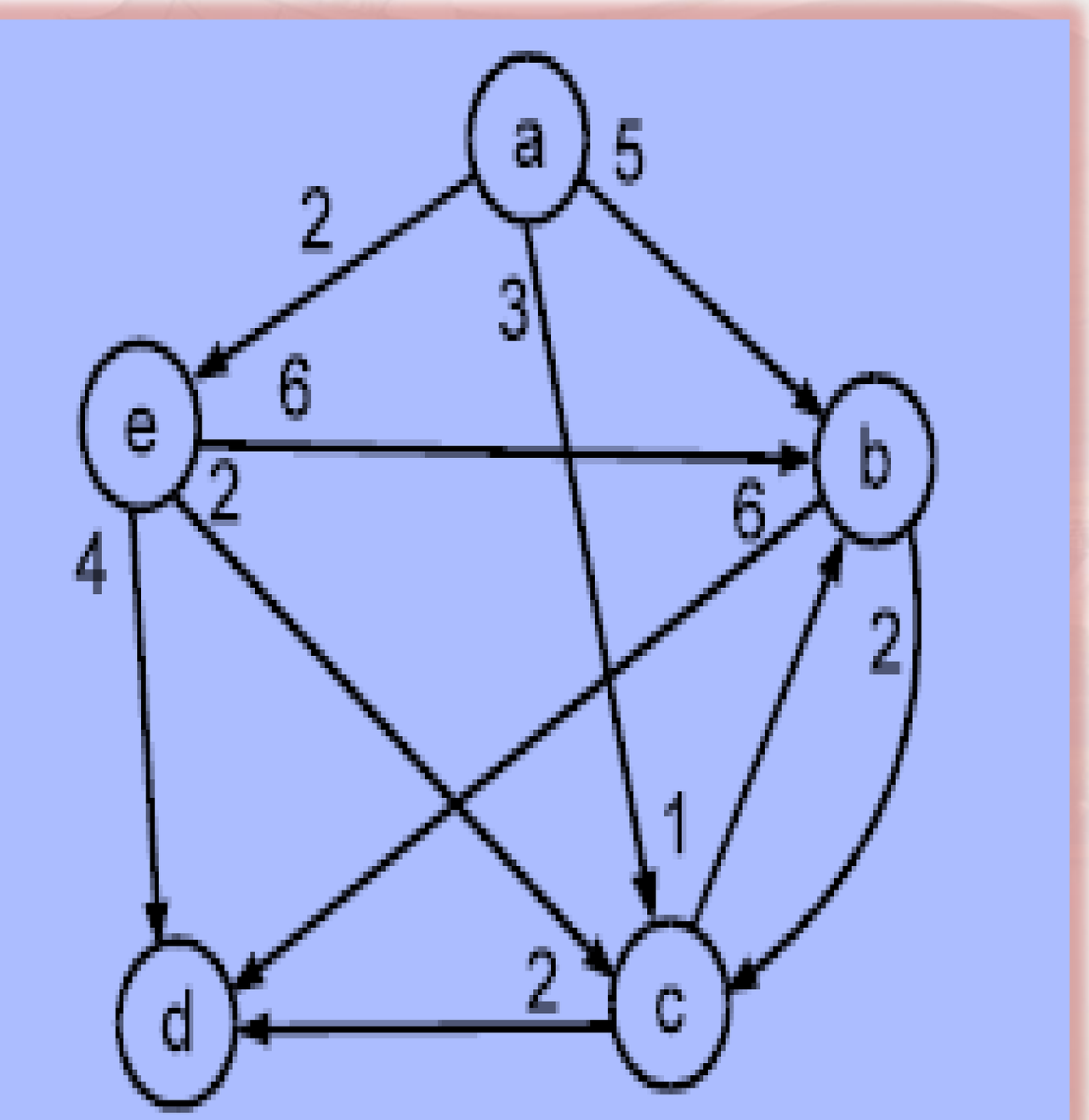
1. Kruskal Algorithms :

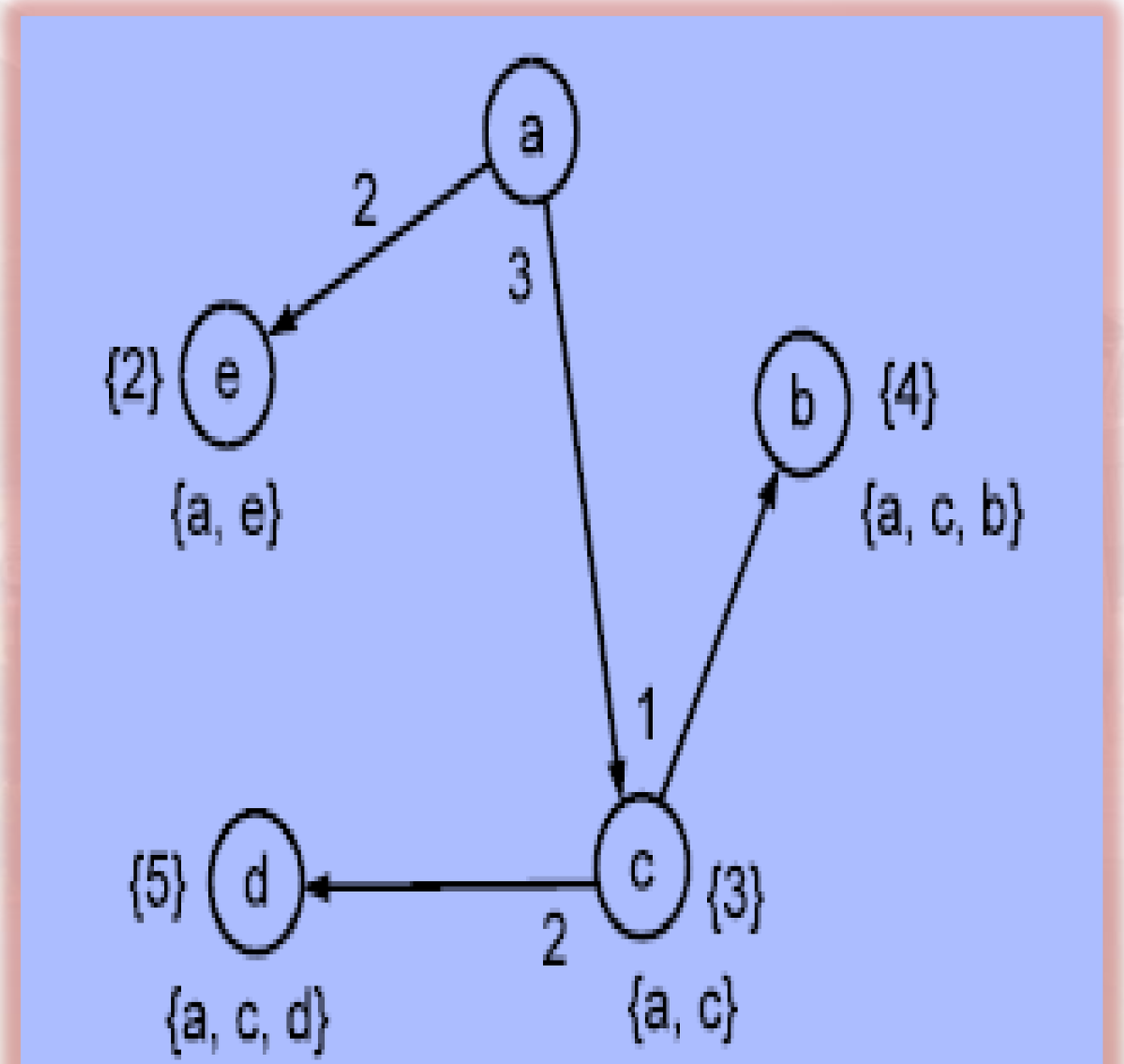
Kruskal's Algorithm Applications

1. In order to layout electrical wiring
2. In computer network (LAN connection)

GREEDY STRATEGIES

1. Kruskal Algorithms :

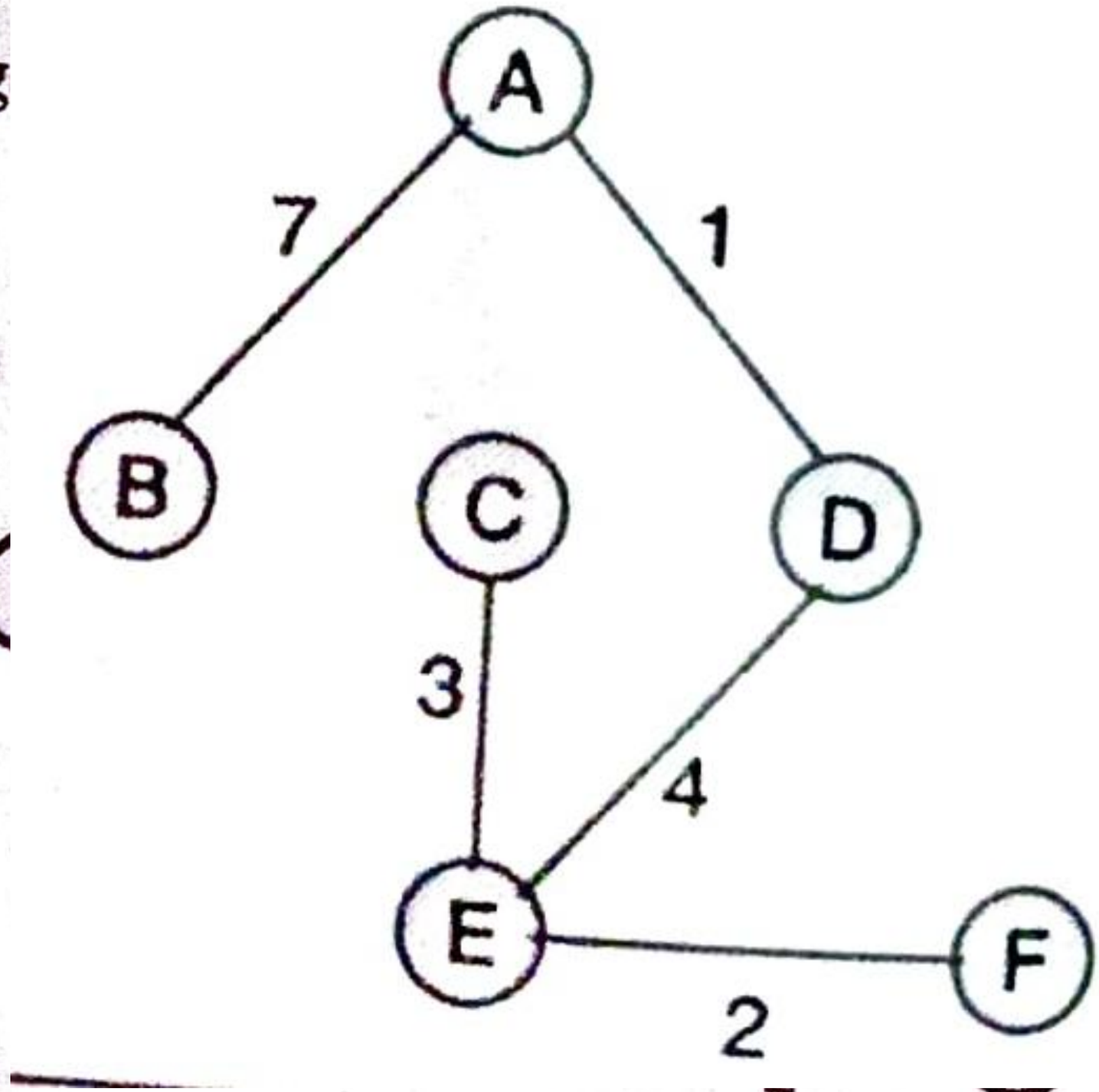




GREEDY STRATEGIES

1. Kruskal Algorithms :

Find the minimum spanning tree for the graph using Kruskal's alg



GREEDY STRATEGIES

2. **Prims algorithm:** Prim’s Algorithm also use Greedy approach to find the minimum spanning tree. In Prim’s Algorithm we grow the spanning tree from a starting position. Unlike an **edge** in Kruskal's, we add **vertex** to the growing spanning tree in Prim's.

Algorithm Steps:

1. Initialize the minimum spanning tree with a vertex chosen at random.

2. Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree.

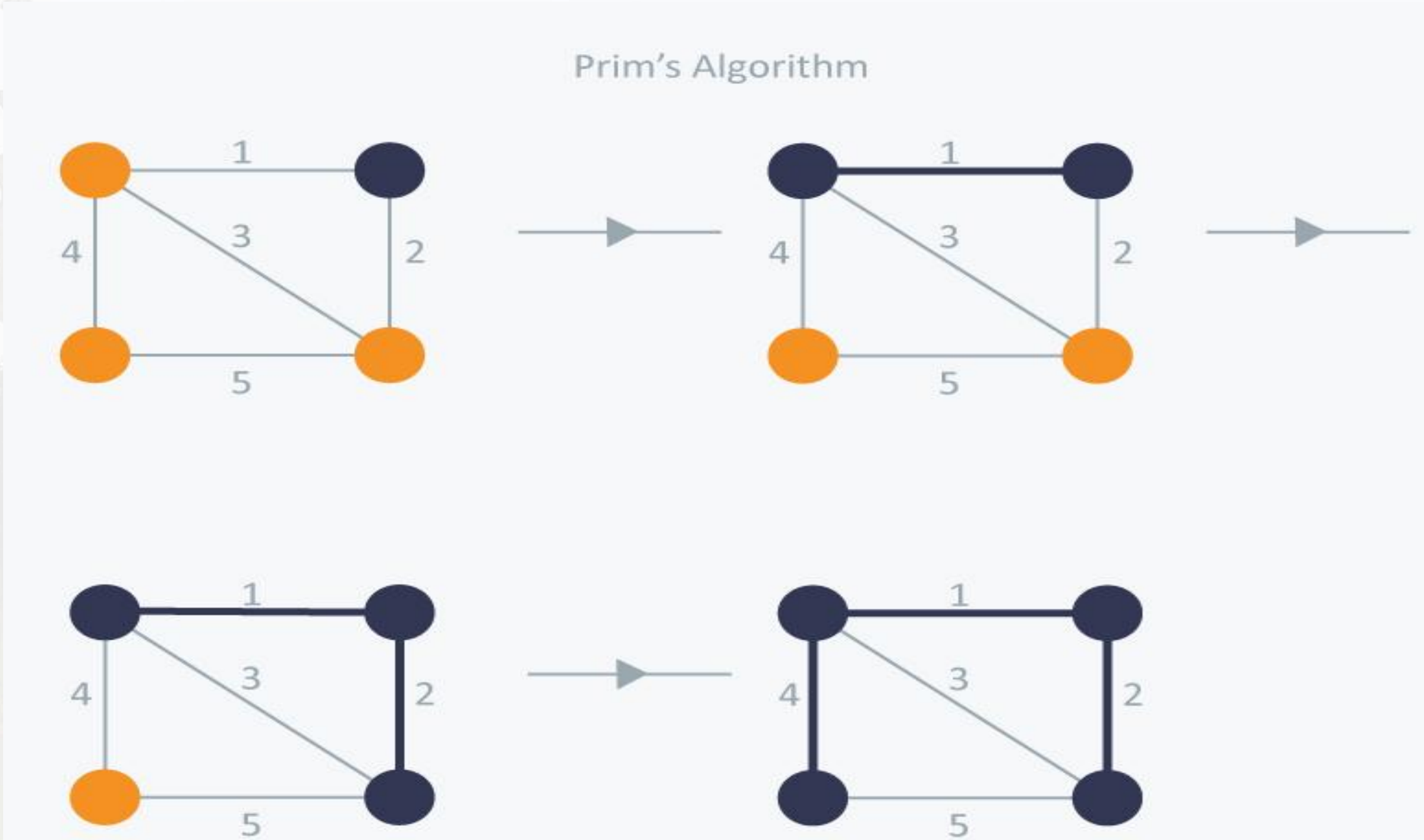
3. Keep repeating **step** 2 until we get a minimum spanning tree.

Prim's Algorithm Complexity

The time complexity of Prim's algorithm is **O(E log V).**

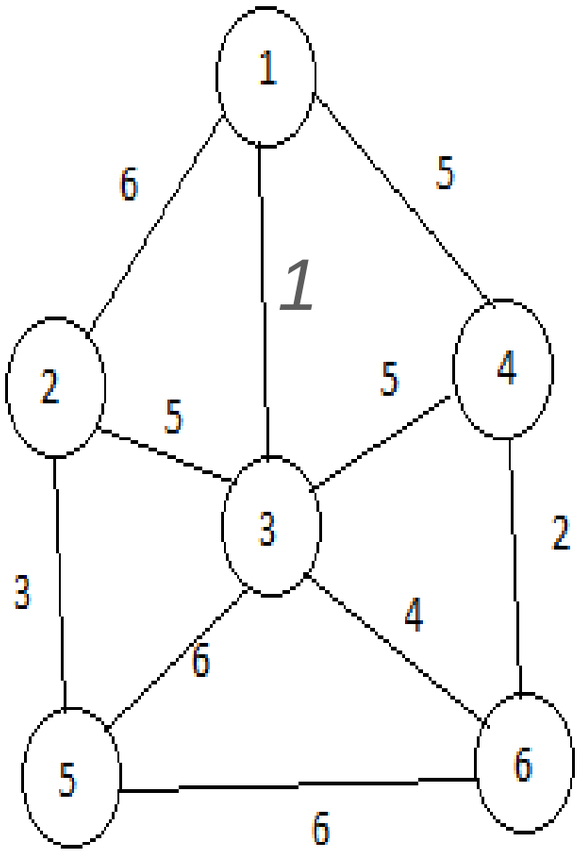
GREEDY STRATEGIES

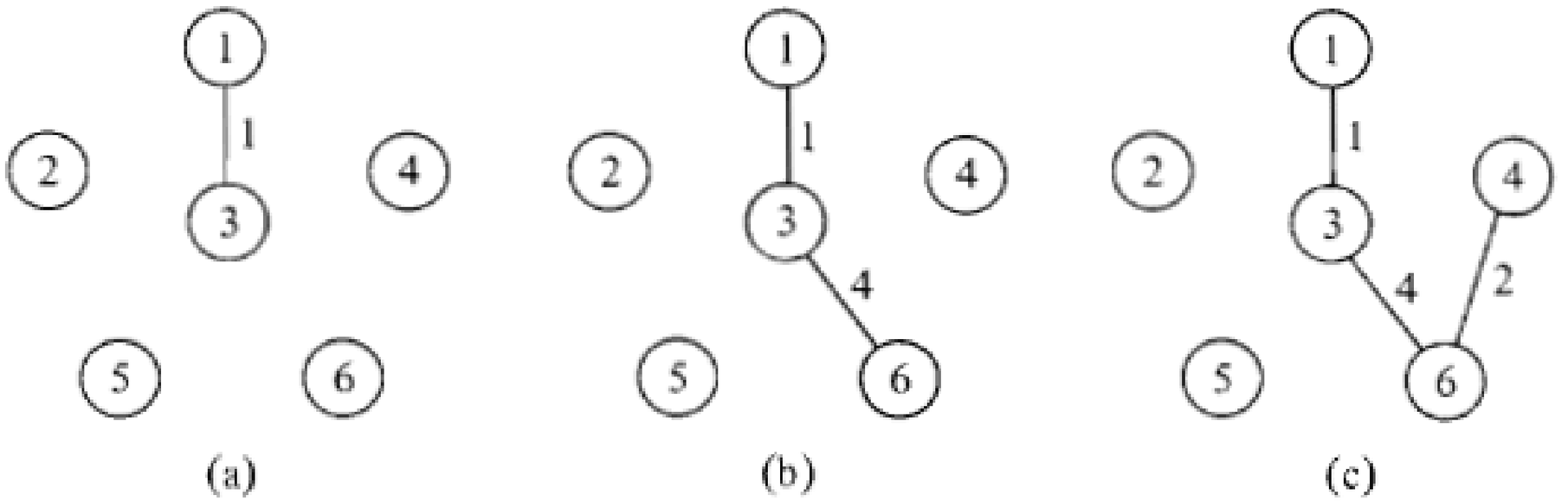
2. Prims algorithm: Example



GREEDY STRATEGIES

1. Prims' algorithms : Example





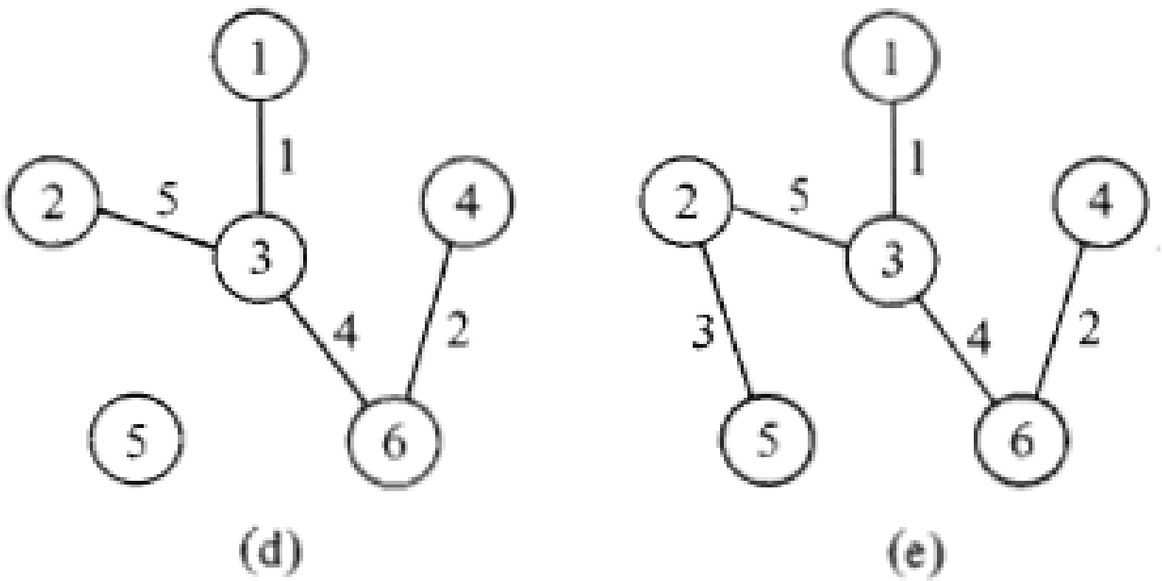
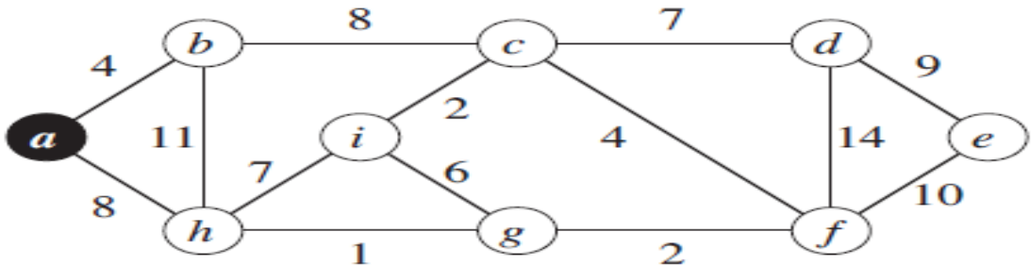


Fig. 7.7. Sequences of edges added by Prim's algorithm.

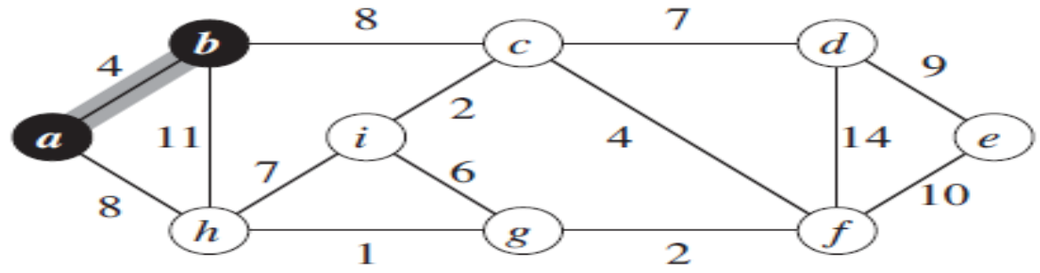
GREEDY STRATEGIES

2. Prims algorithm: Example

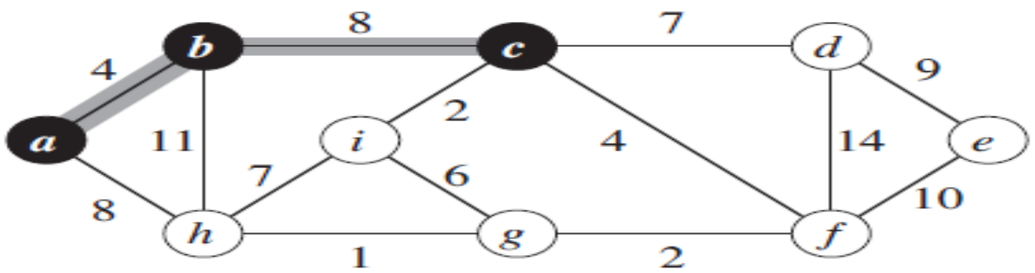
(a)



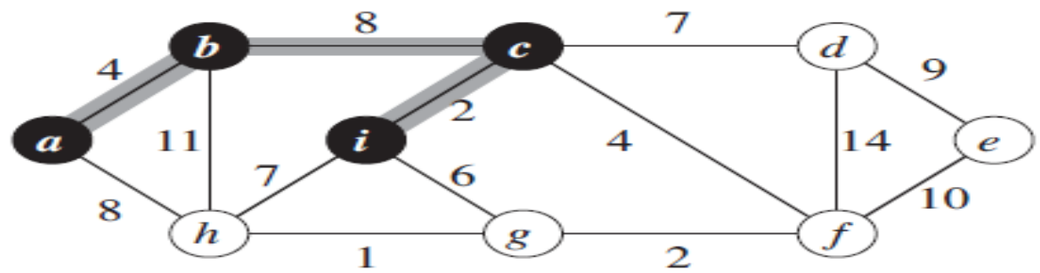
(b)



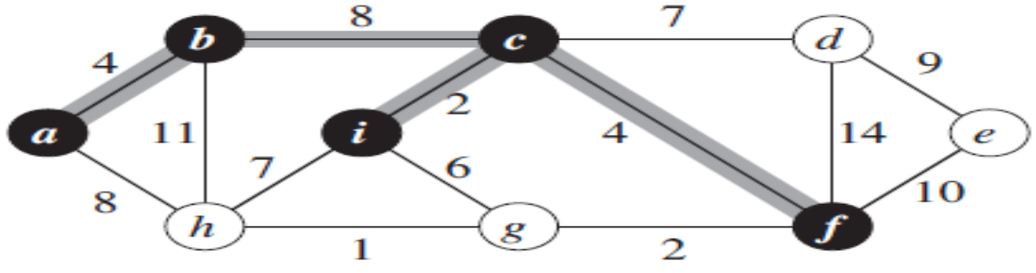
(c)



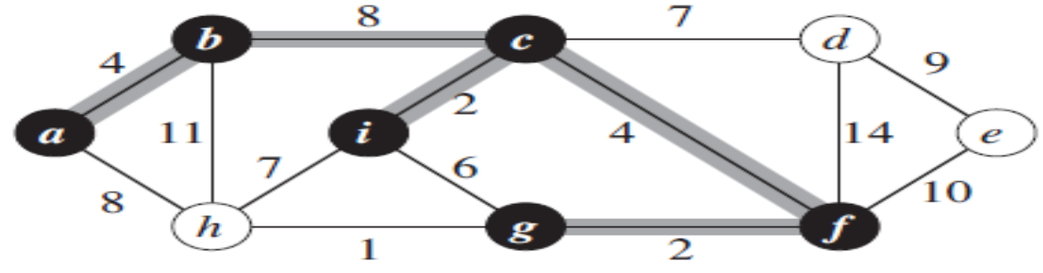
(d)



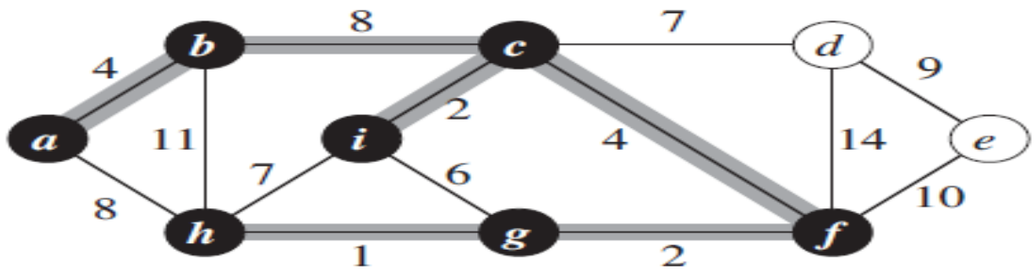
(e)



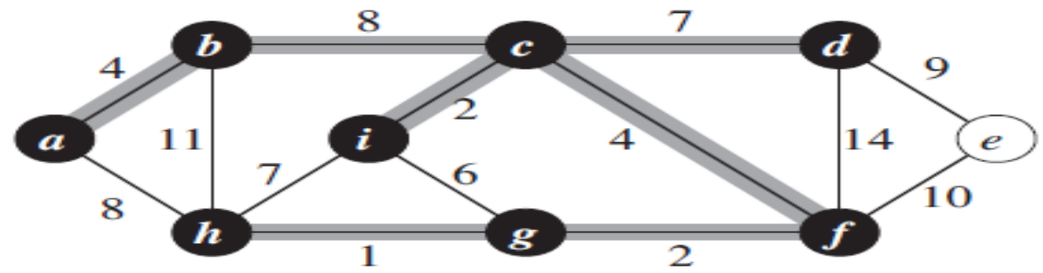
(f)



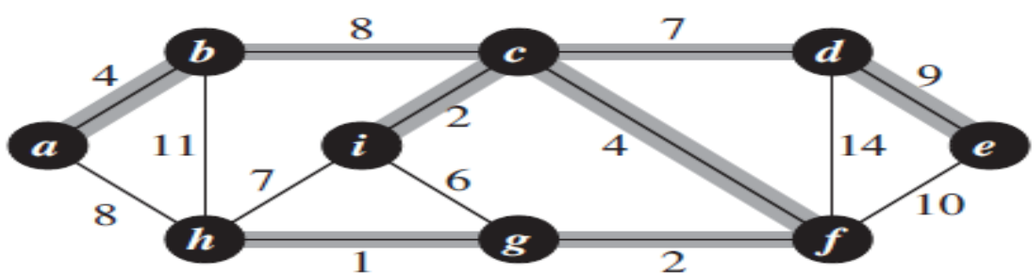
(g)



(h)

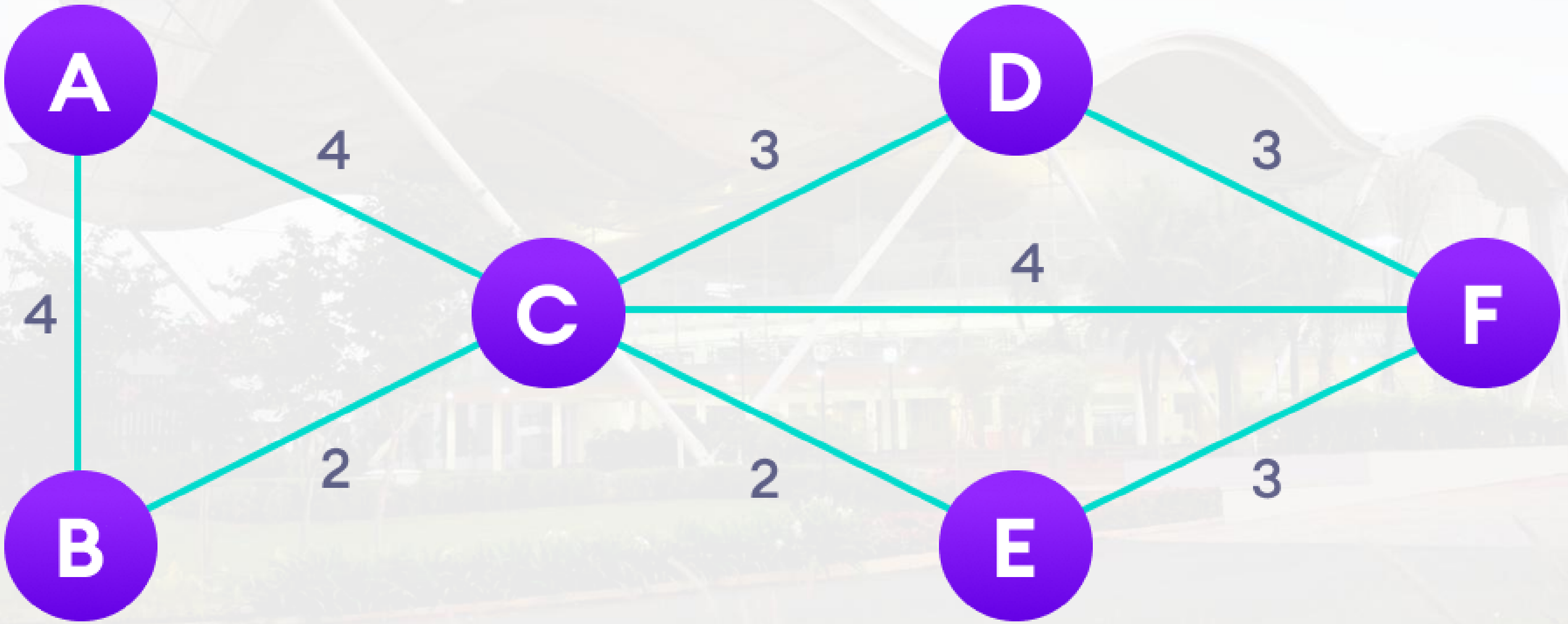


(i)



GREEDY STRATEGIES

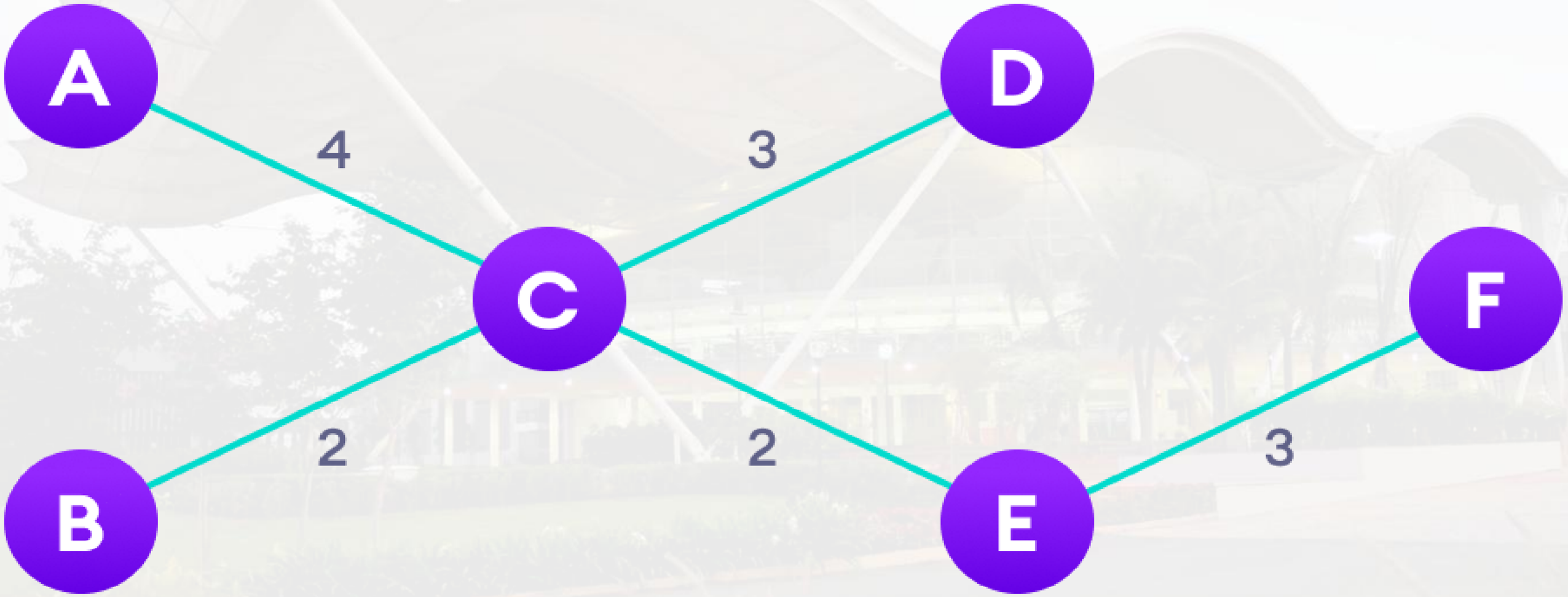
2. **Prims algorithm: Example**



Step: 1

GREEDY STRATEGIES

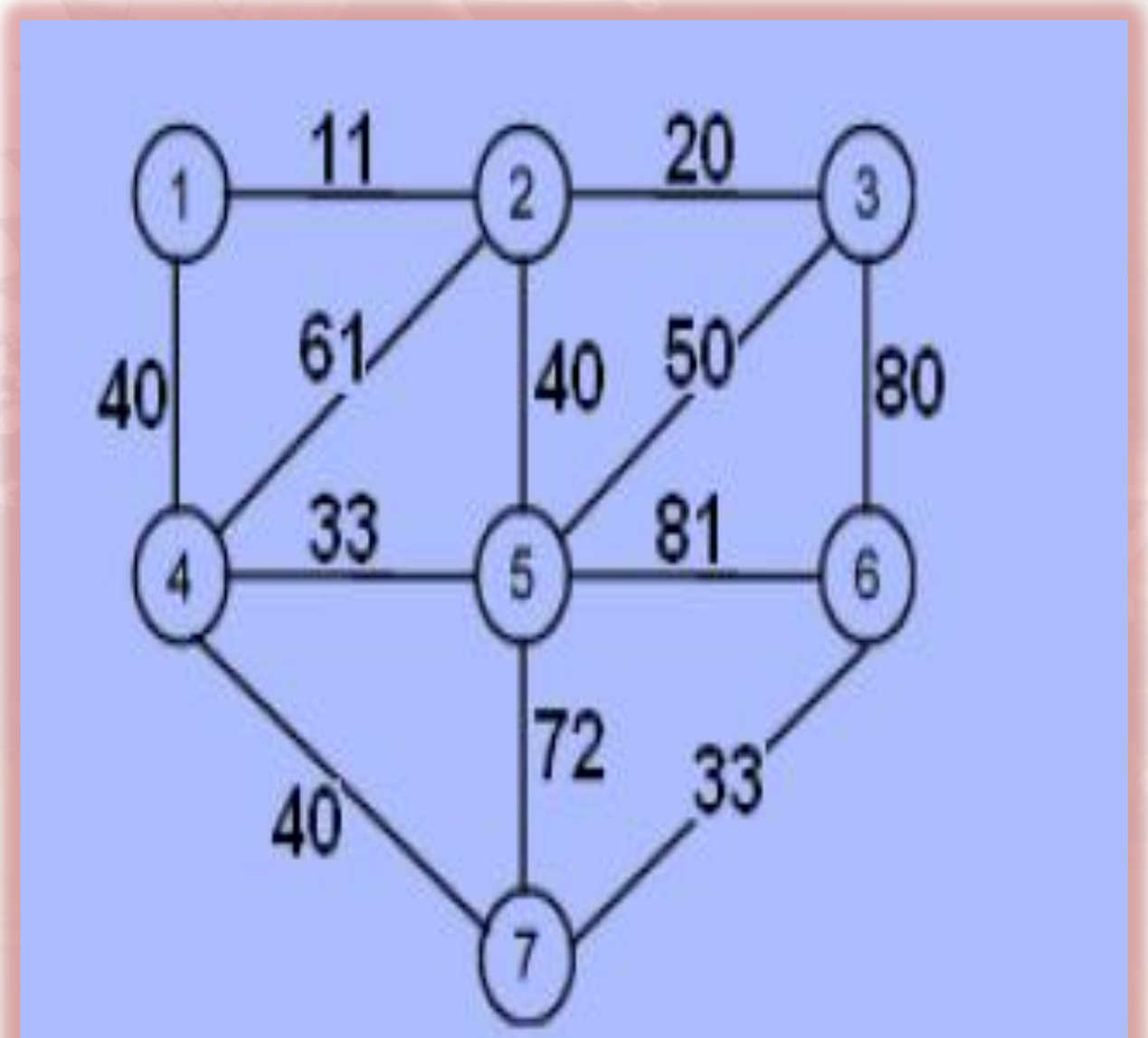
2. **Prims algorithm: Example**

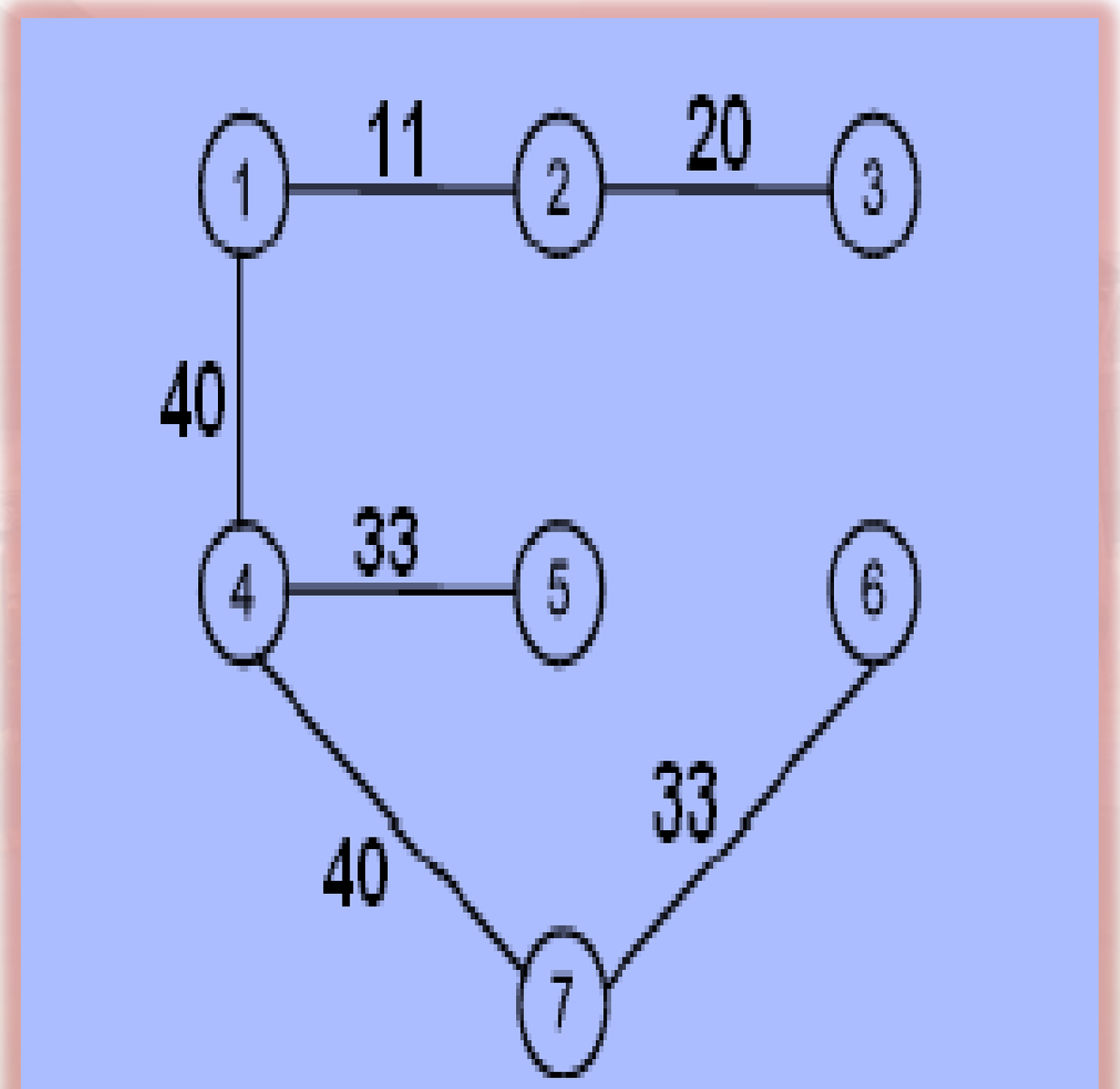


Step: 6

GREEDY STRATEGIES

2. **Prims algorithm: Example**



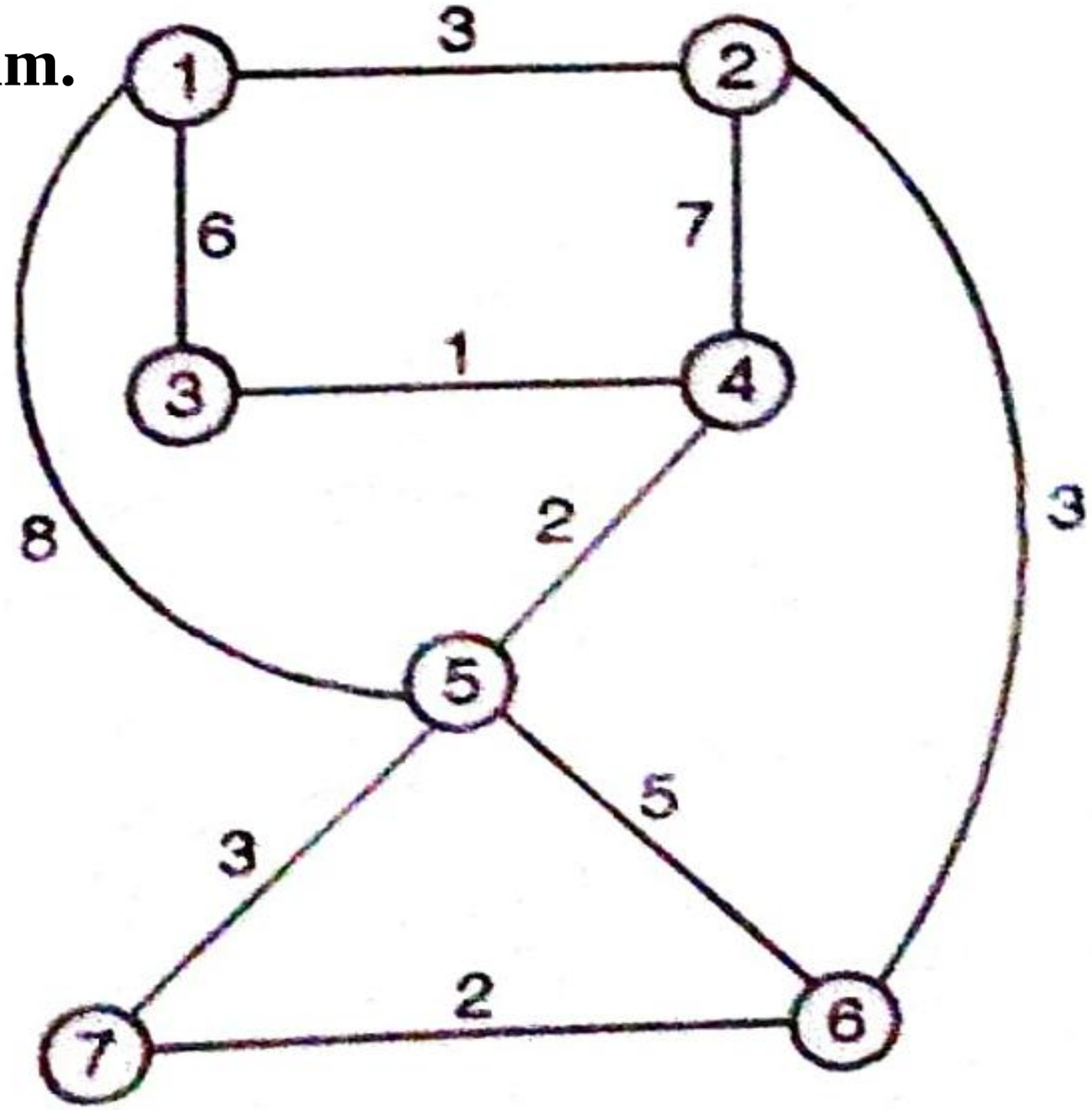


GREEDY STRATEGIES

2. **Prims algorithm: Example**

Find minimum spanning tree for following graph using Prim's algorithm. Show various steps.

Kruska's algorithm.



GREEDY STRATEGIES

2. **Prims algorithm:**

Prim's Algorithm Application

1. Laying cables of electrical wiring
2. In network designed
3. To make protocols in network cycles

Advantages of Disadvantages of Greedy approach

Advantages :

- The algorithm is easier to describe.
- This algorithm can perform better than other algorithms (but, not in all cases).

Disadvantages :

- The greedy algorithm doesn't always produce the optimal solution.

Applications of Greedy Algorithms

- It is used in finding the shortest path.
- It is used to find the minimum spanning tree using the prim's algorithm or the Kruskal's algorithm.
- It is used in a job sequencing with a deadline.
- This algorithm is also used to solve the fractional knapsack problem.

Prim's Vs Kruskal Algorithms

Prims Algorithm	Kruskal Algorithm
It start to build the MST from any of the Node.	It start to build the MST from Minimum weighted vertex in the graph.
Adjencary Matrix , Binary Heap or Fibonacci Heap is used in Prims algorithm	Disjoint Set is used in Kruskal Algorithm.
Prims Algorithm run faster in dense graphs	Kruskal Algorithm run faster in sparse graphs
Time Complexity is O(EV log V) with binay heap and O(E+V log V) with fibonacci heap.	Time Complexity is **O(E log V)**
The next Node included must be connected with the node we traverse	The next edge include may or may not be connected but should not form the cycle.
It traverse the one node saveral time in order to get it minimum distance	It travese the edge only once and based on cycle it will either reject it or accept it,
Greedy Algorithm	Greedy Algorithm

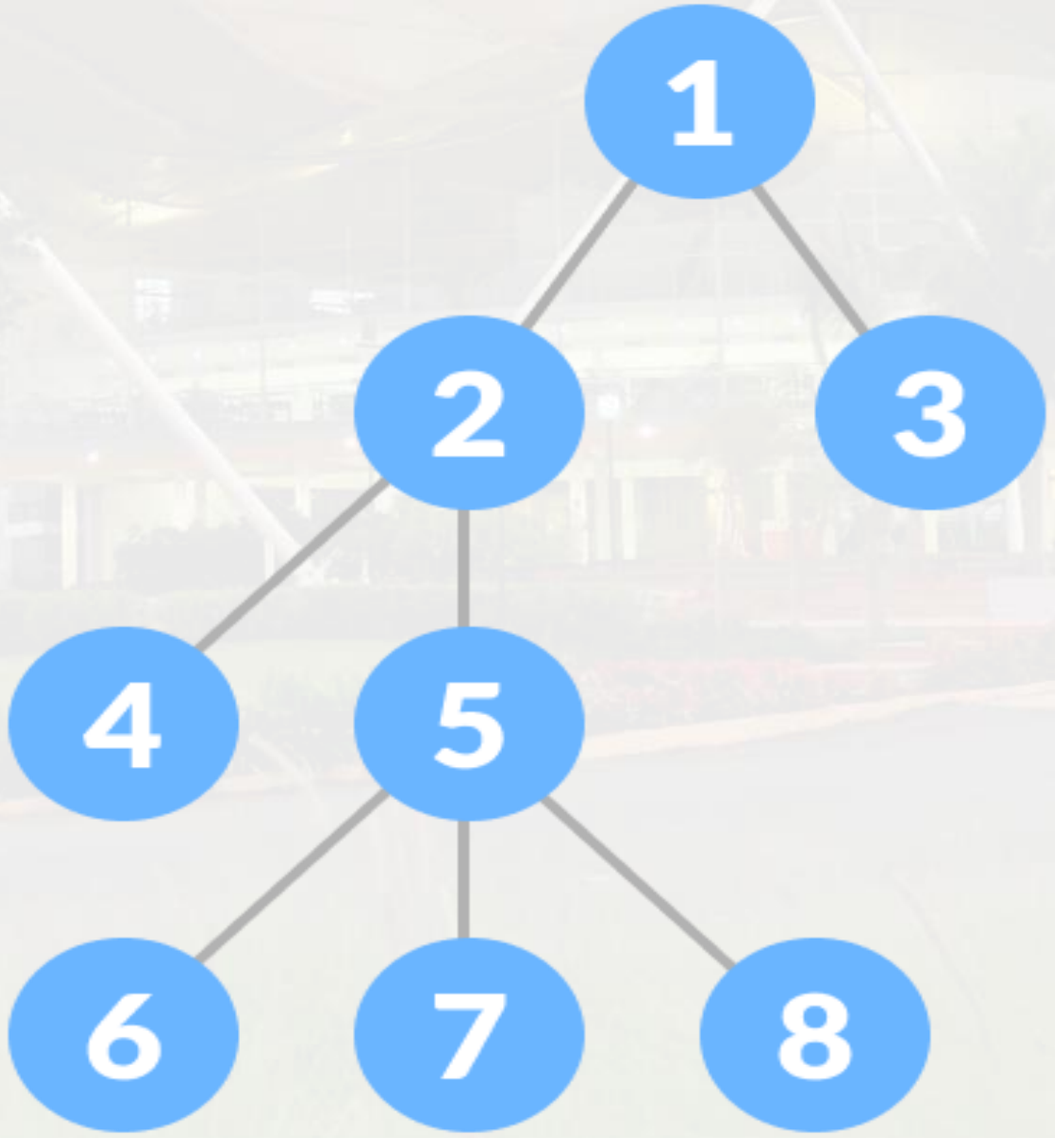
References

https://www.geeksforgeeks.org/introduction-to-dijkstras-shortest-path-algorithm/

TREE

TERMINOLOGIES IN TREE

- "A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges."



Why Tree Data Structure?

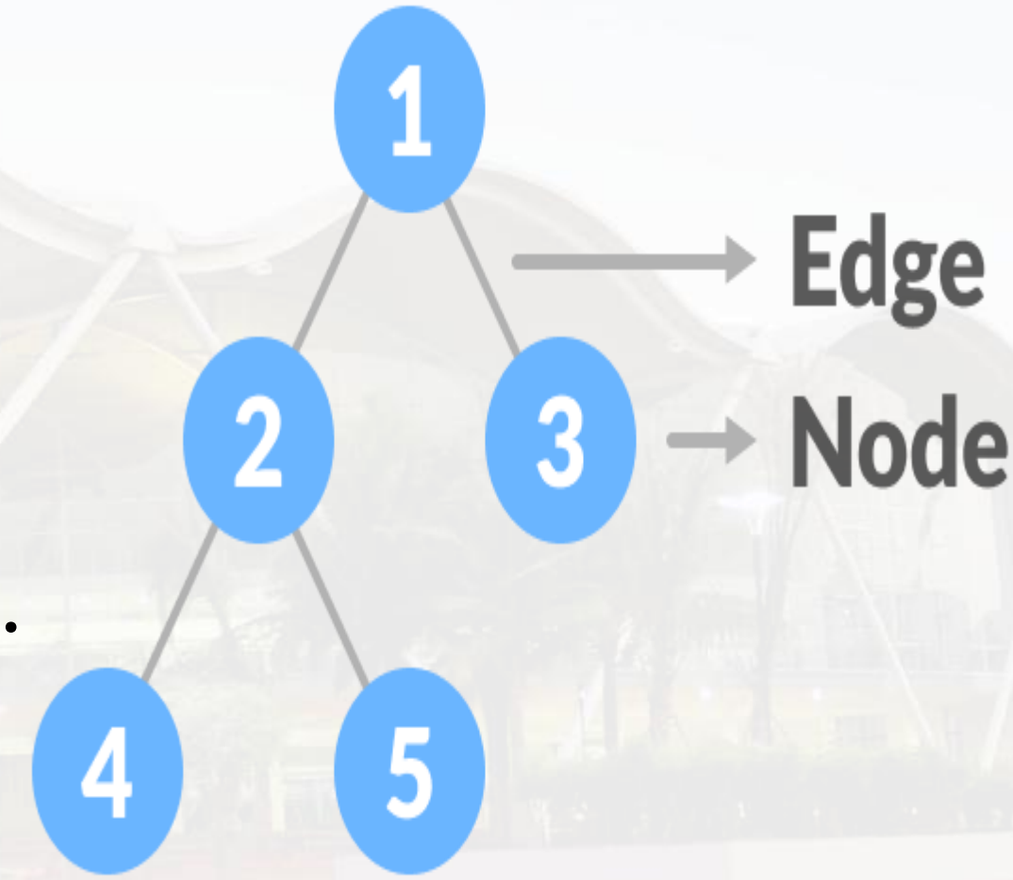
Other data structures such as arrays, linked list, stack, and queue are linear data structures that store data sequentially. In order to perform any operation in a linear data structure, the time complexity increases with the increase in the data size. But, it is not acceptable in today's computational world.

Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.

Terminologies of Tree

- **Root :**

It is the topmost node of a tree.



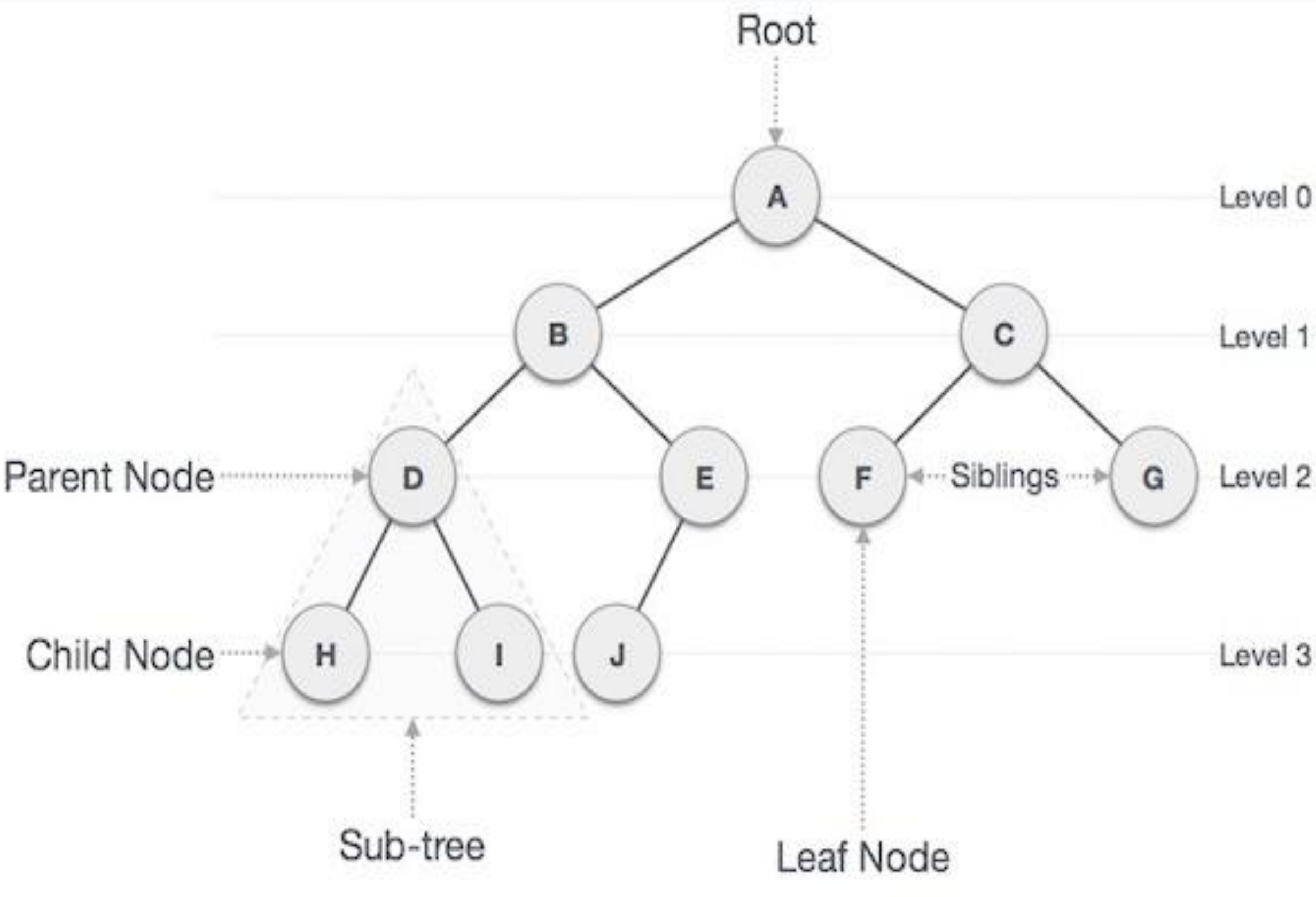
- **Edge :**

It is the link between any two nodes.

- **Node :**

A node is an entity that contains a key or value and pointers to its child nodes.

Terminologies of Tree



Terminologies of Tree

- **Child node:** If the node is a descendant of any node, then the node is known as a child node.

- **Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.

- **Sibling:** The nodes that have the same parent are known as siblings.

Terminologies of Tree

- **Leaf** – The node which does not have any child node is called the leaf node.

- **Subtree** – Subtree represents the descendants of a node.

- **Path** – Path refers to the sequence of nodes along the edges of a tree

Terminologies of Tree

- **Traversing –** Traversing means passing through nodes in a specific order.

- **Levels –** Level of a node represents the generation of a node. If the root node is at level 0, then its next child node is at level 1, its grandchild is at level 2, and so on.

- **keys –** Key represents a value of a node based on which a search operation is to be carried out for a node.

Terminologies of Tree

Leaf/External node: Node with no children.

Internal node:

The node having at least a child node is called an **internal node**

- **Height of a Node :**

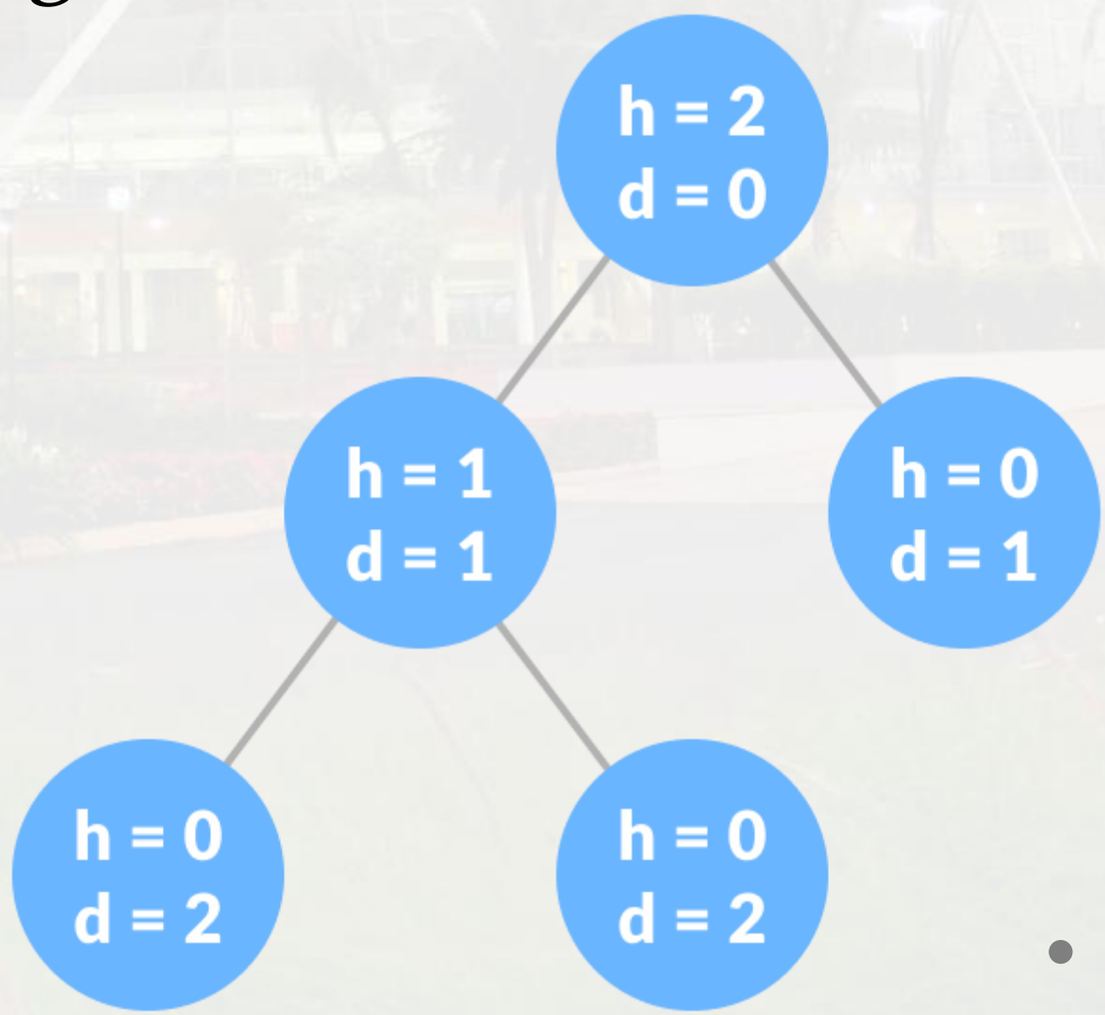
The height of a node is the number of edges from the node to the deepest leaf (ie. the longest path from the node to a leaf node).

Terminologies of Tree

The depth of a node is the number of edges from the root to the node.

- **Height of a Tree :**

The height of a Tree is the height of the root node or the depth of the deepest node.



Terminologies of Tree

- **Degree of a Node :**

The degree of a node is the total number of branches of that node.

- **Forest :**

A collection of disjoint trees is called a forest.

You can create a forest by cutting the root of a tree.



Applications of Tree

- Binary Search Tree (BST) is used to check whether elements present or not.
- Heap is a type of tree that is used to heap sort.
- Tries are the modified version of the tree used in modem routing to information of the router.
- The widespread database uses B-tree.
- Compilers use syntax trees to check every syntax of a program.

Advantages of Tree

1. Trees reflect structural relationships in the data.
2. Trees are used to represent hierarchies.
3. Trees provide an efficient insertion and searching.
4. Trees are very flexible data, allowing to move subtrees around with minimum effort.

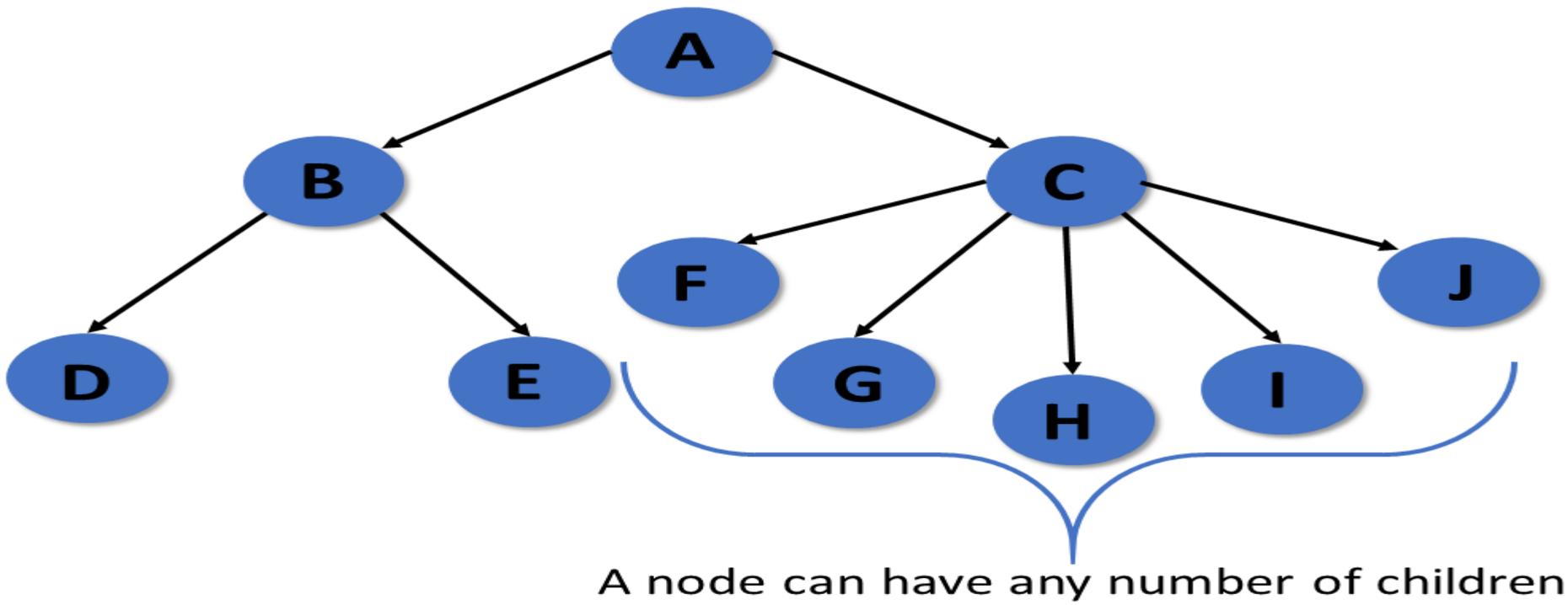
Types of Tree

- General Tree
- Binary Tree
- Binary Search Tree
- AVL Tree
- B-Tree

General Tree

- **General Tree**
- The general tree is the type of tree where there are no constraints on the hierarchical structure.

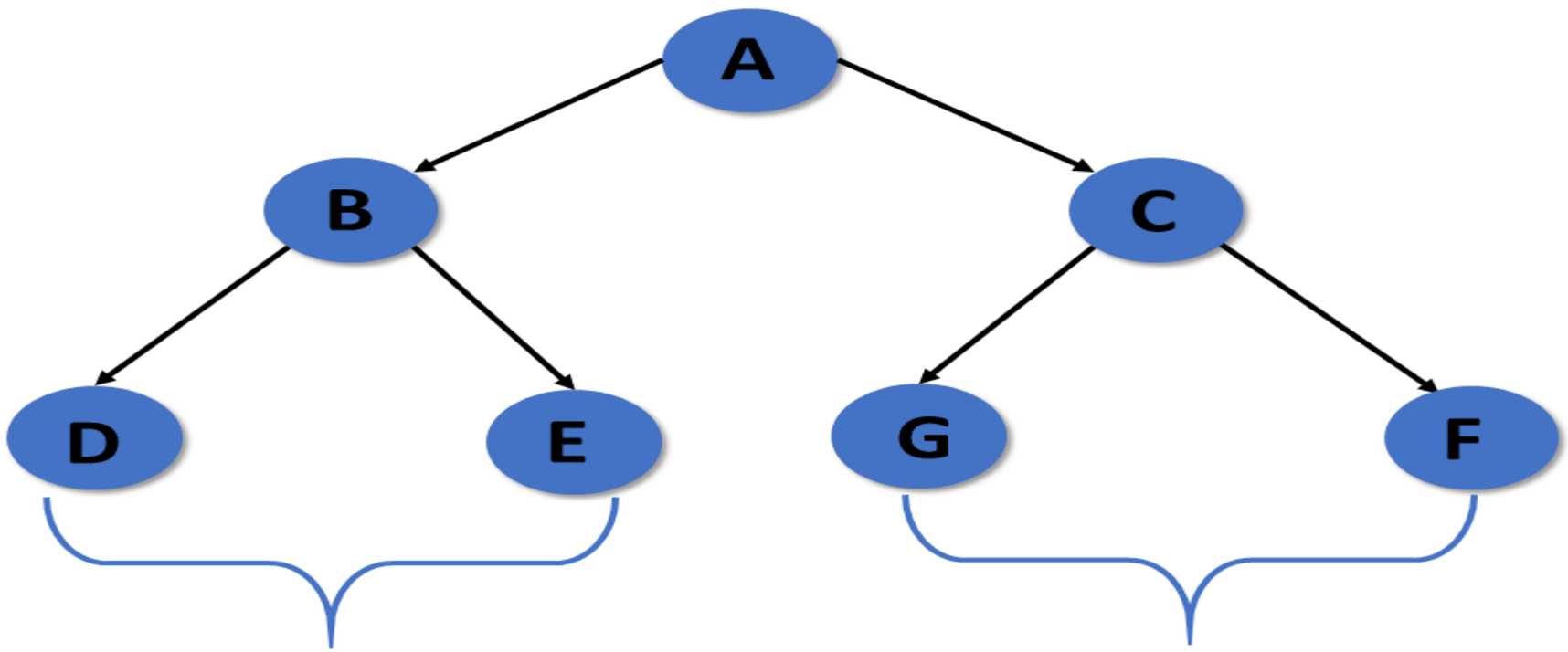
- **Properties**
- The general tree follows all properties of the tree data structure.
- A node can have any number of nodes.



Binary Tree

- **A binary tree has the following properties:**

- **Properties**
- Follows all properties of the tree data structure.
- Binary trees can have at most two child nodes.
- These two children are called the left child and the right child.



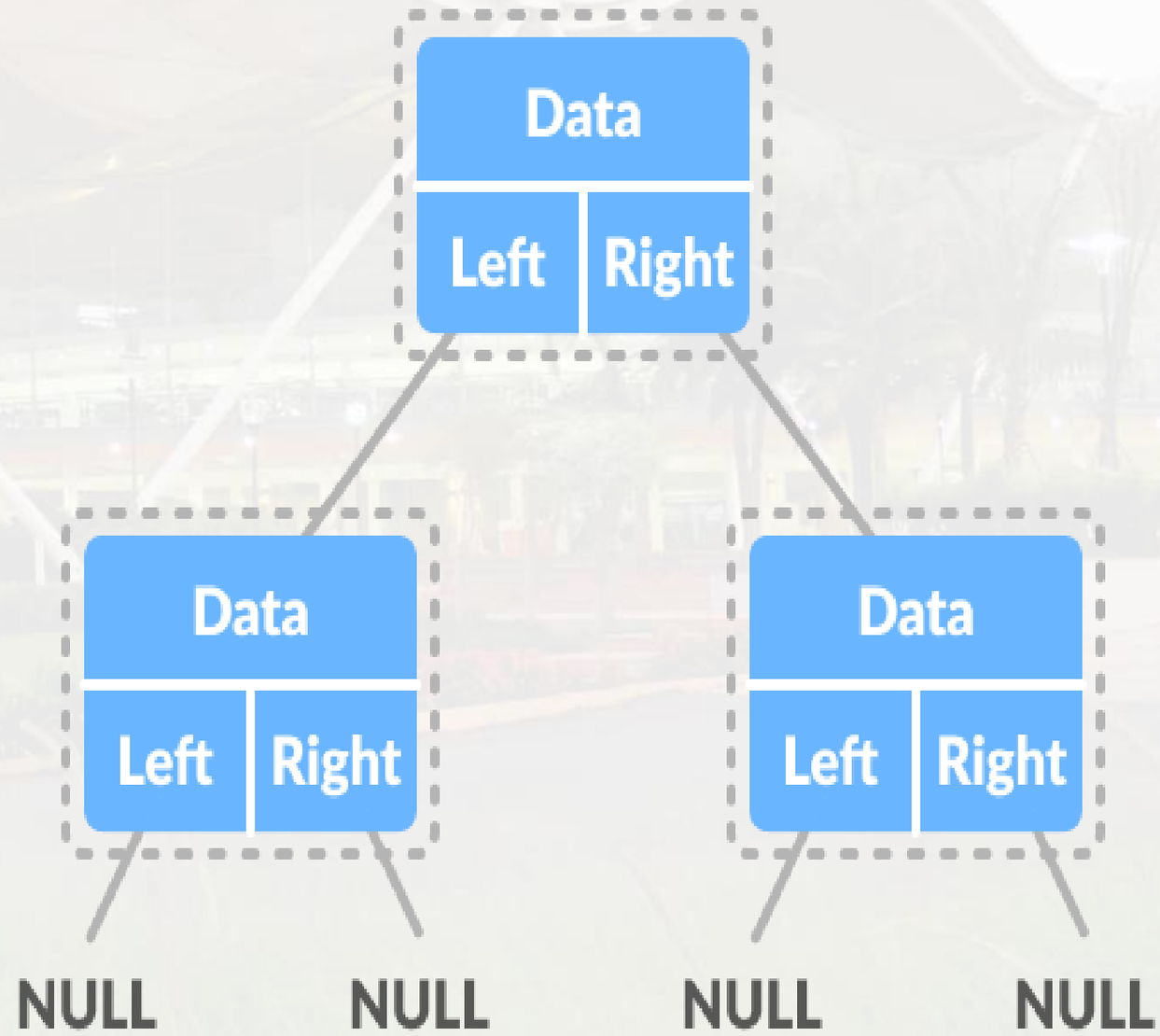
Binary Tree

- **A binary tree has the following properties:**
- At each level of i, the maximum number of nodes is 2i.
- The height of the tree is defined as the longest path from the root node to the leaf node. The tree which is shown above has a height equal to 3. Therefore, the maximum number of nodes at height 3 is equal to (1+2+4+8) = 15. In general, the maximum number of nodes possible at height h is (20 + 21 + 22+....2h) = 2h+1 -1.
- The minimum number of nodes possible at height h is equal to h+1.
- If the number of nodes is minimum, then the height of the tree would be maximum. Conversely, if the number of nodes is maximum, then the height of the tree would be minimum

Binary Tree Representation

A node of a binary tree is represented by a structure containing a data part and two pointers to other structures of the same type.

```
struct node
{
 int data;
 struct node *left;
 struct node *right;
};
```



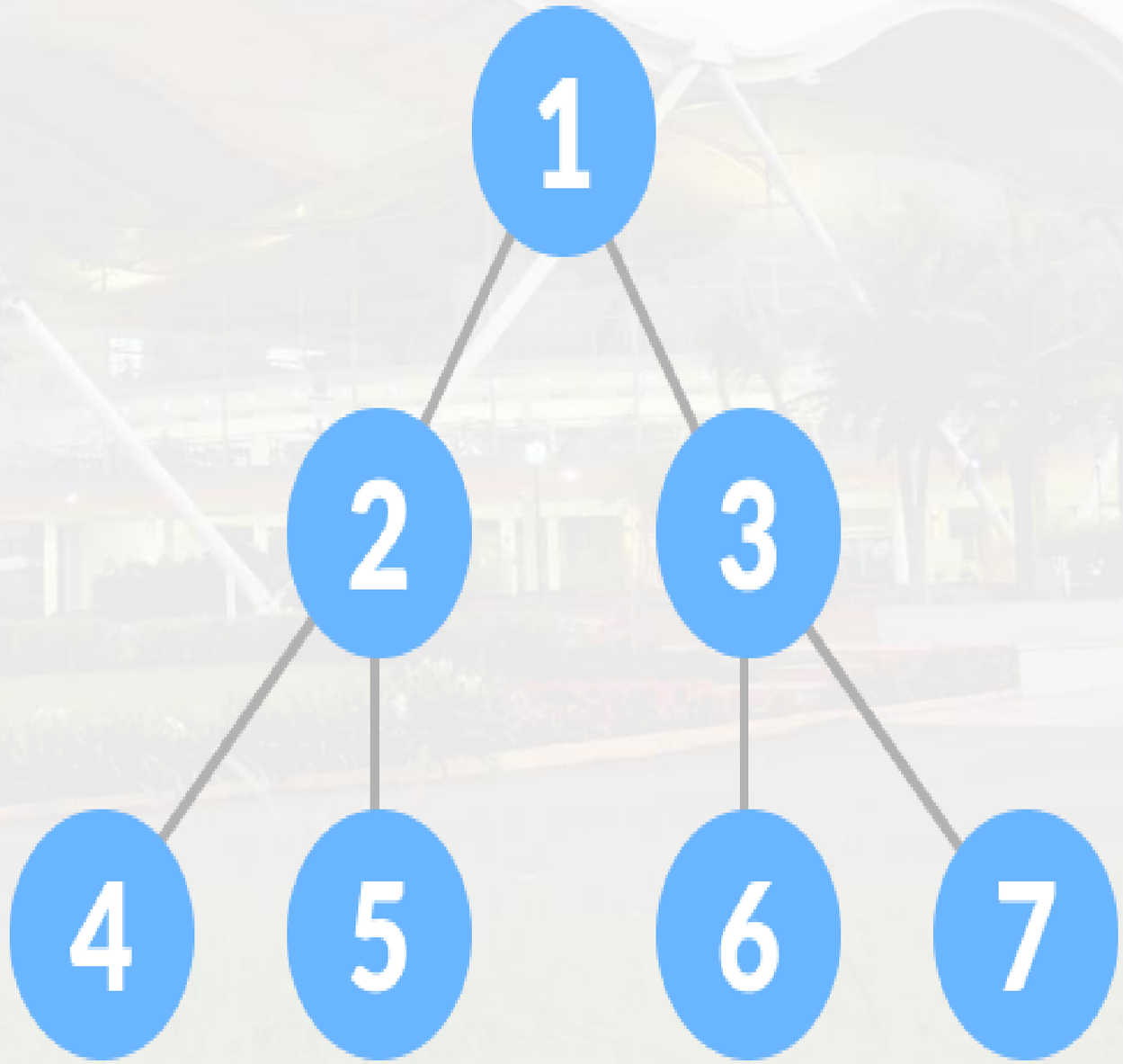
Types of Binary Tree

- **Full Binary tree:** It is a special type of binary tree. In this tree data structure, every parent node or an internal node has either two children or no child nodes.



Types of Binary Tree

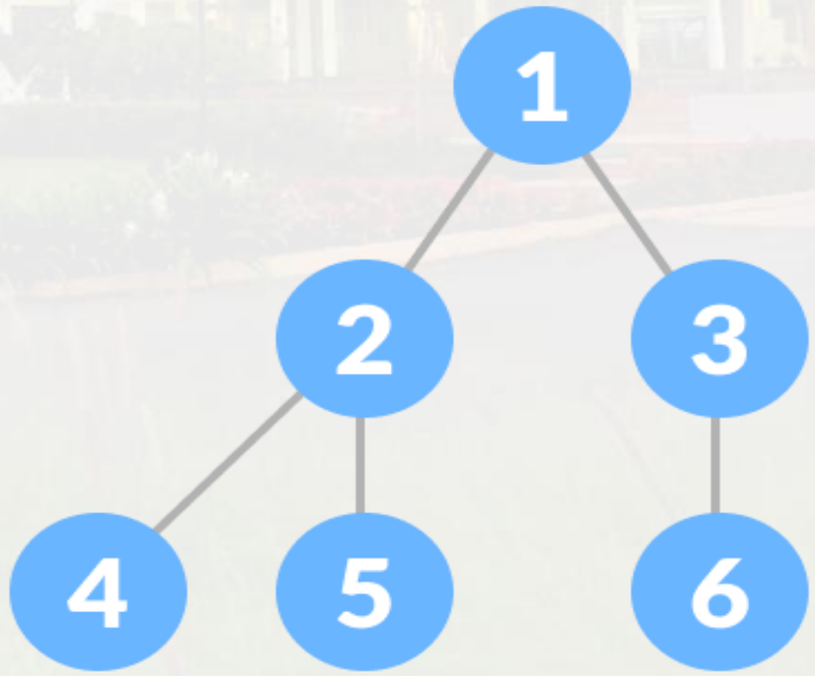
- **Perfect binary tree:** In this type of tree data structure, every internal node has exactly two child nodes and all the leaf nodes are at the same level.
- .



Binary Tree

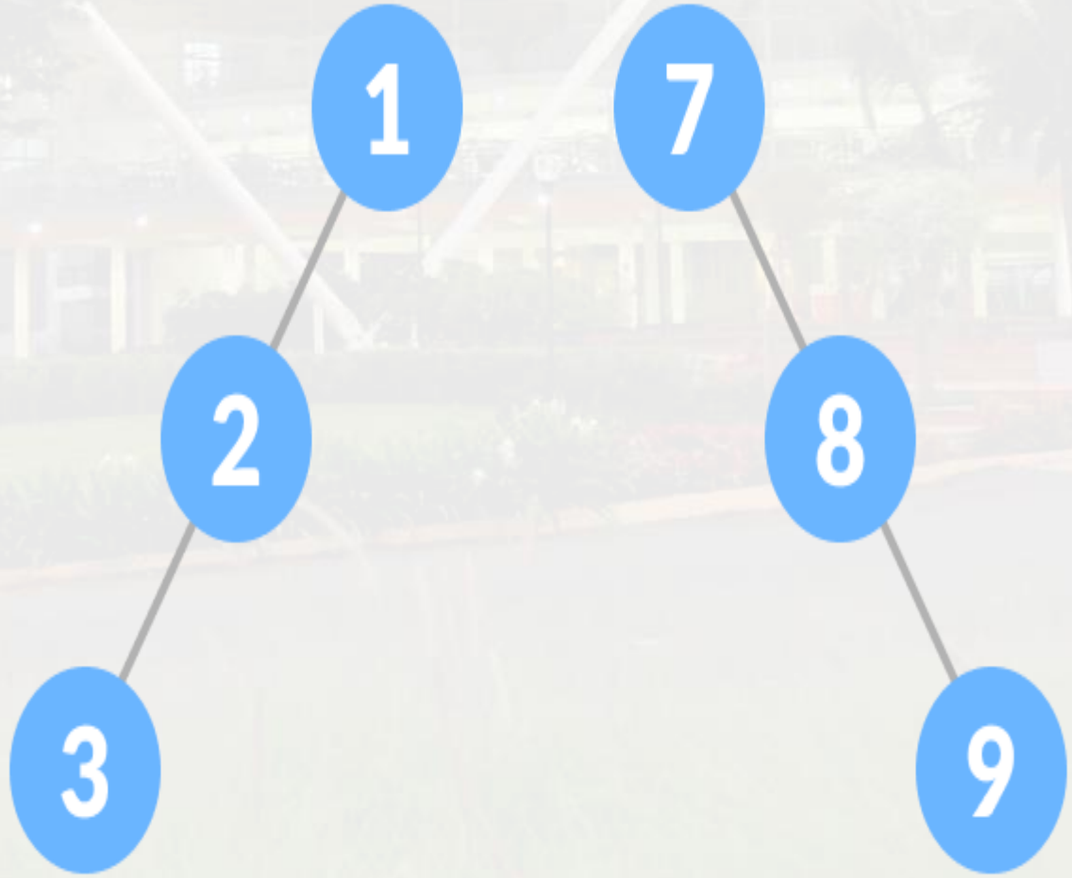
- **Complete binary tree:** It resembles that of the full binary tree with a few differences.

1. Every level is completely filled.
2. The leaf nodes lean towards the left of the tree.
3. It is not a requirement for the last leaf node to have the right sibling, i.e. a complete binary tree doesn't have to be a full binary tree.



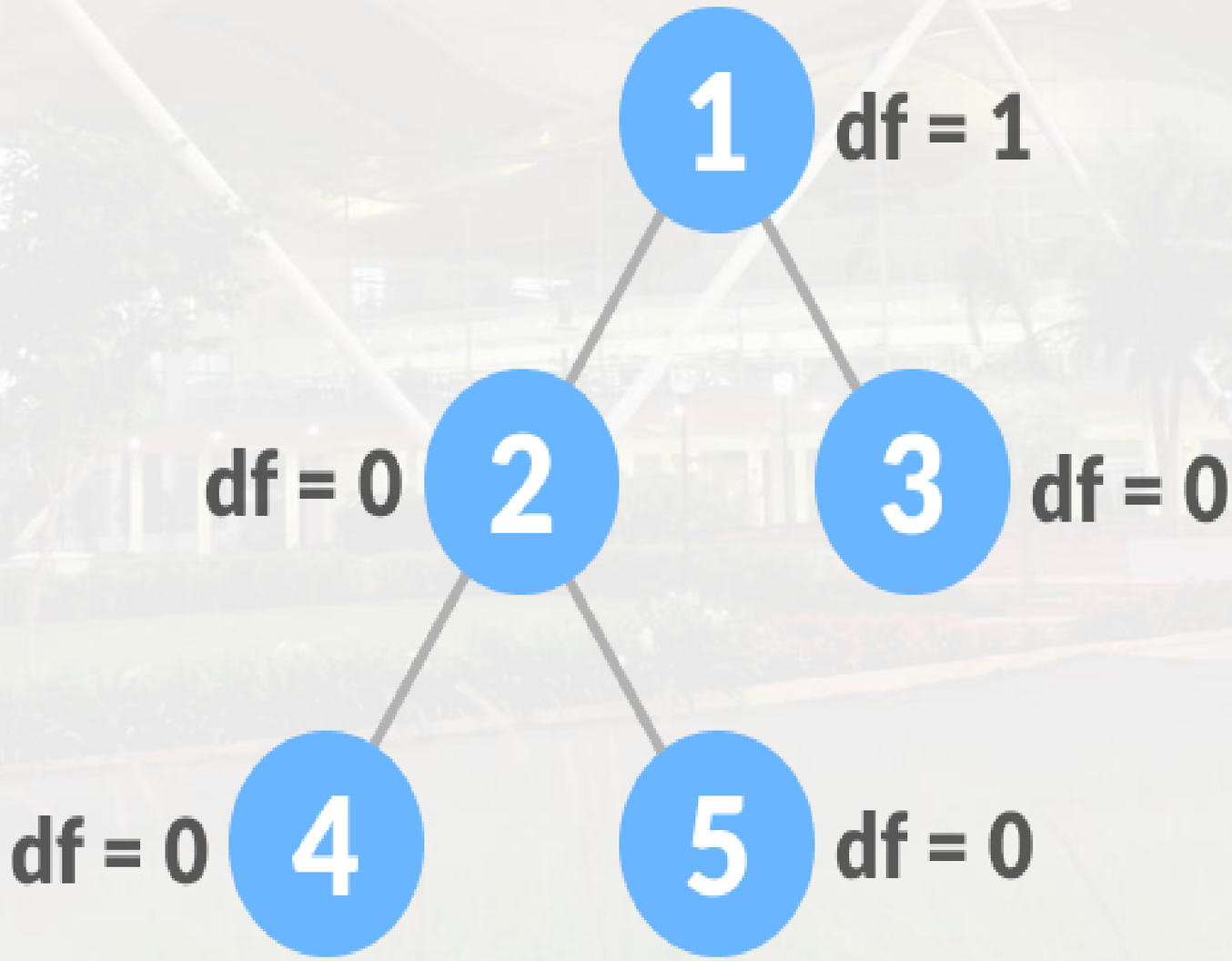
Binary Tree

- **Skewed binary tree:** It is a pathological or degenerate tree where the tree is dominated by either the left nodes or the right nodes. Therefore, there are two types of skewed binary trees, i.e. left-skewed or the right-skewed binary tree.



Binary Tree

- **Balanced binary tree:** The difference between the height of the left and right sub tree for each node is either 0 or 1.



Static Binary Tree Representation :

- **Array Representation :**
- Binary tree using array represents a node which is numbered sequentially level by level from left to right. Even empty nodes are numbered.

- Array index is a value in tree nodes and array value gives to the parent node of that particular index or node. Value of the root node index is always -1 as there is no parent for root. When the data item of the tree is sorted in an array, the number appearing against the node will work as indexes of the node in an array.

- Location number of an array is used to store the size of the tree. The first index of an array that is 'o', stores the total number of nodes. All nodes are numbered from left to right level by level from top to bottom. In a tree, each

Binary Tree Representation using Array

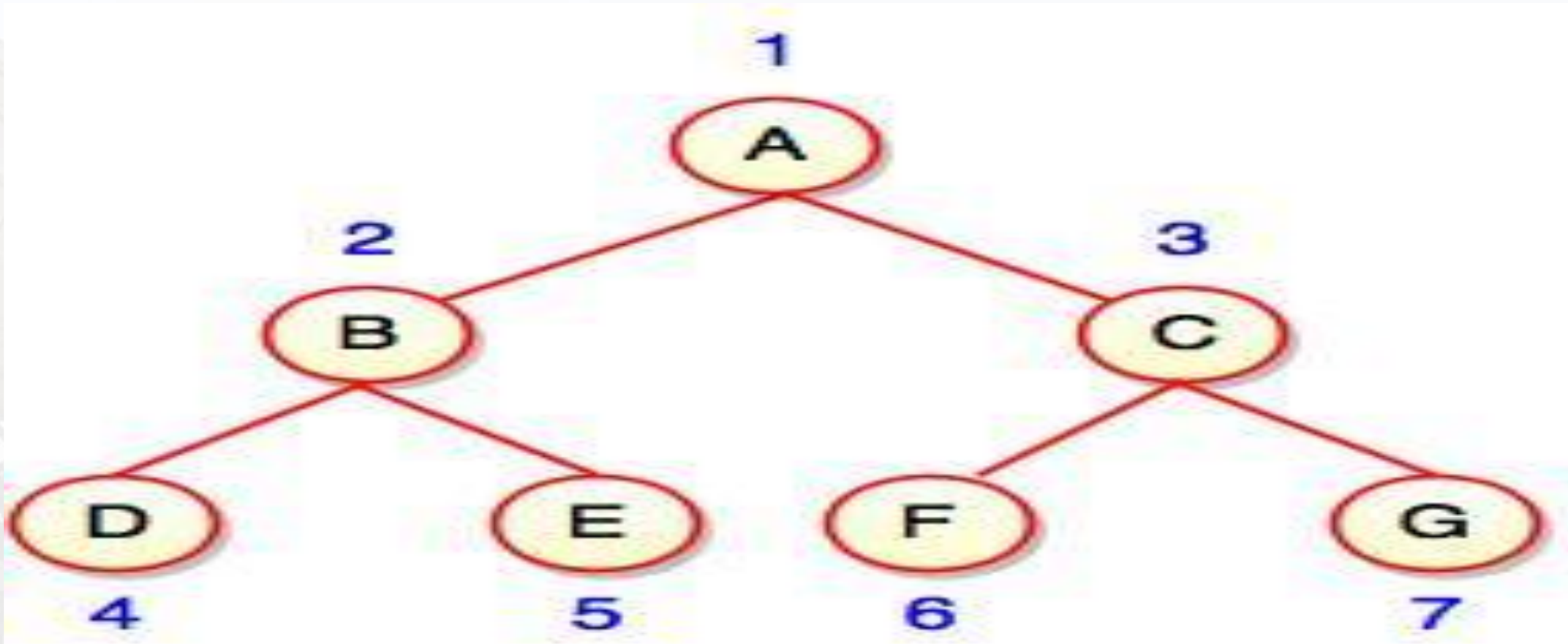


Fig. Binary Tree using Array



Fig. Location Number of an Array in a Tree

Dynamic -Binary Tree Representation

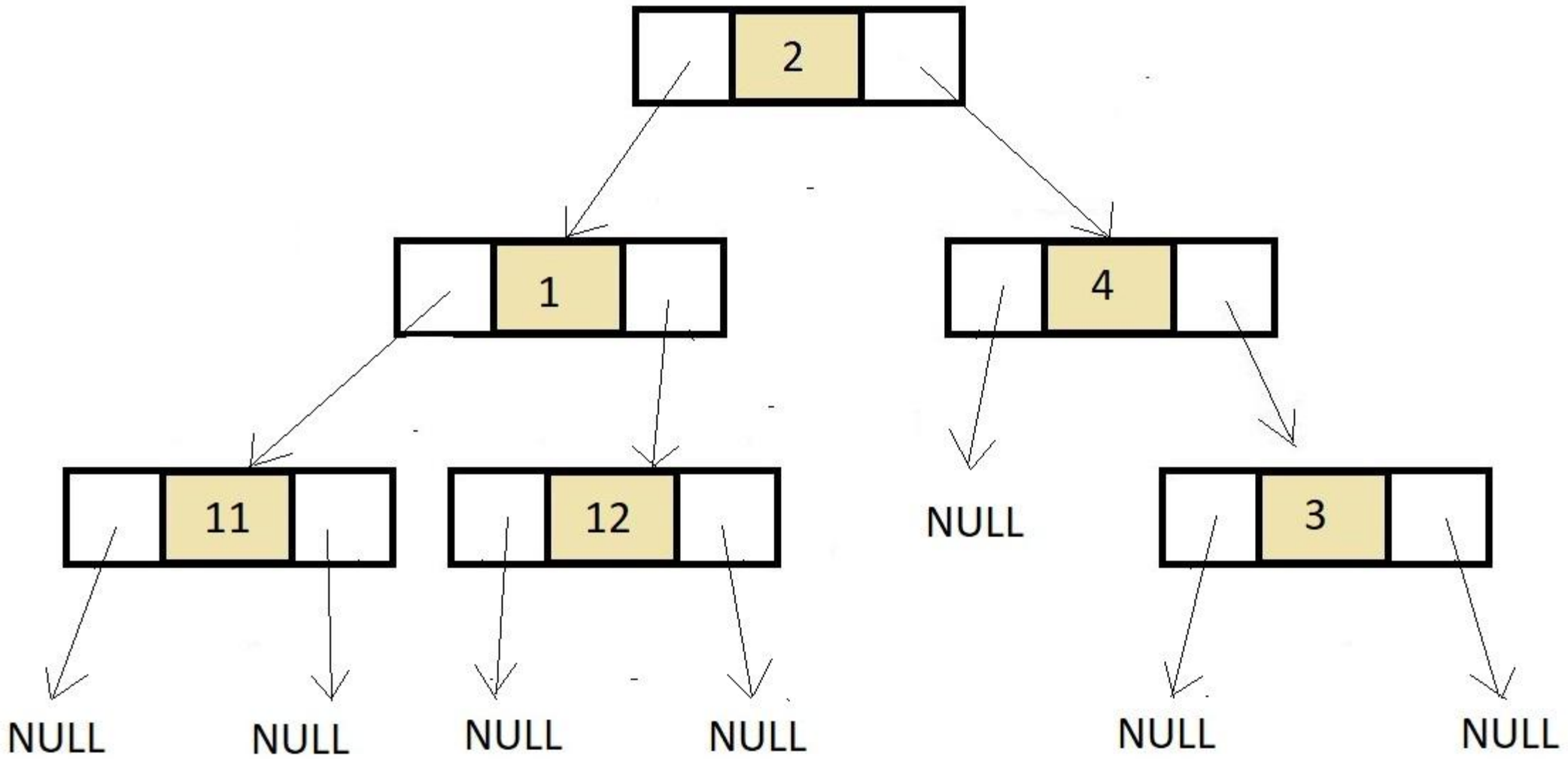
Linked representation :

Binary trees in linked representation are stored in the memory as linked lists. These lists have nodes that aren't stored at adjacent or neighboring memory locations and are linked to each other through the parent-child relationship associated with trees.

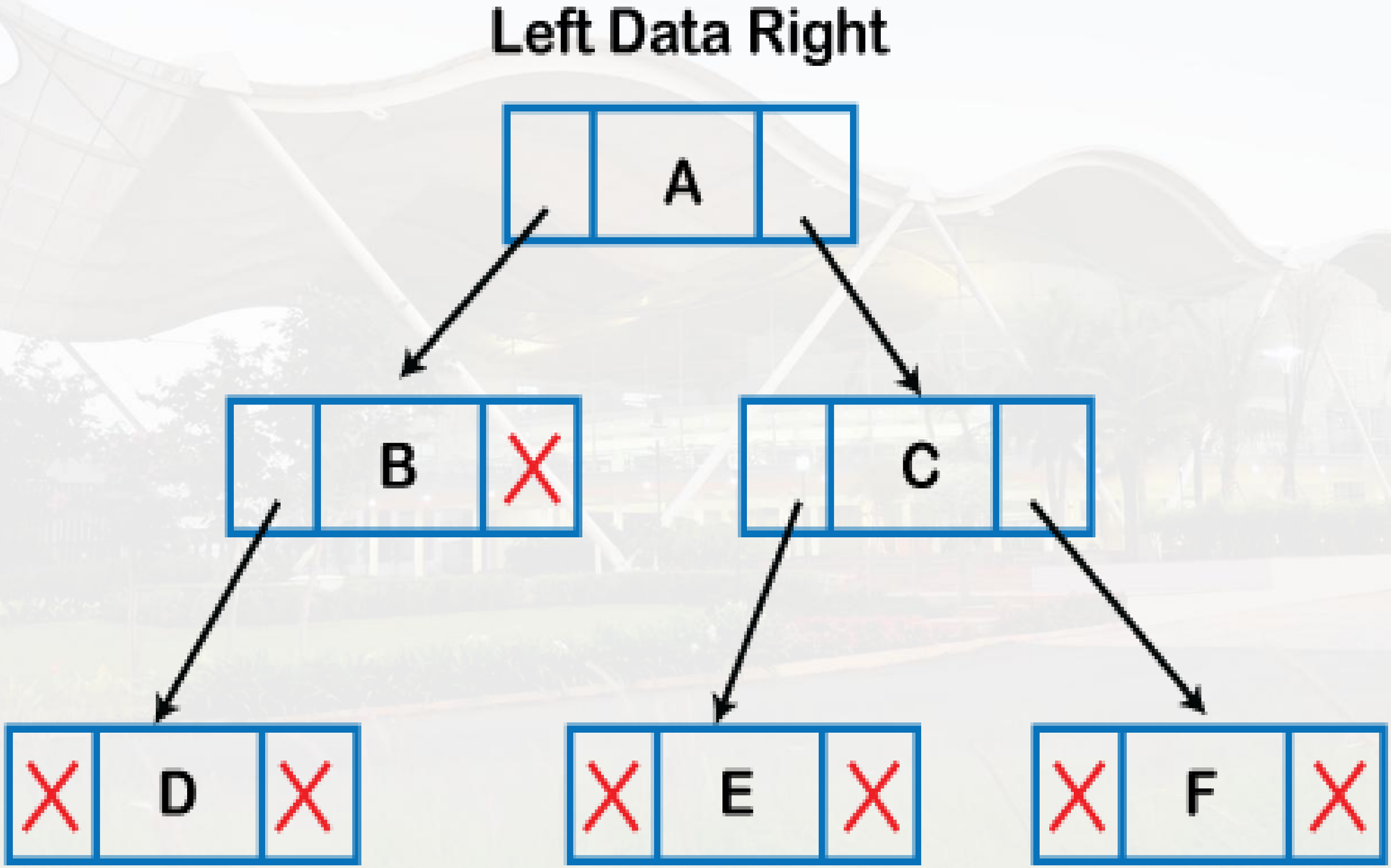
In this representation, each node has three different parts –

- pointer that points towards the right node,
- pointer that points towards the left node,
- data element.

Binary Tree Representation



Binary Tree Representation



Binary Tree Representation

Linked representation :

This is the more common representation. All binary trees consist of a root pointer that points in the direction of the root node. When you see a root node pointing towards null or you should know that you are dealing with an empty binary tree. The right and left pointers store the address of the right and left children of the tree

Operations of Binary Tree

Searching: For searching element 2, we have to traverse all elements (assuming we do breadth first traversal). Therefore, searching in binary tree has worst case complexity of O(n).

Insertion: For inserting element as left child of 2, we have to traverse all elements. Therefore, insertion in binary tree has worst case complexity of O(n).

Deletion: For deletion of element 2, we have to traverse all elements to find 2 (assuming we do breadth first traversal). Therefore, deletion in binary tree has worst case complexity of O(n).

Binary Tree Examples

Example 1: Consider the example of Perfect binary tree

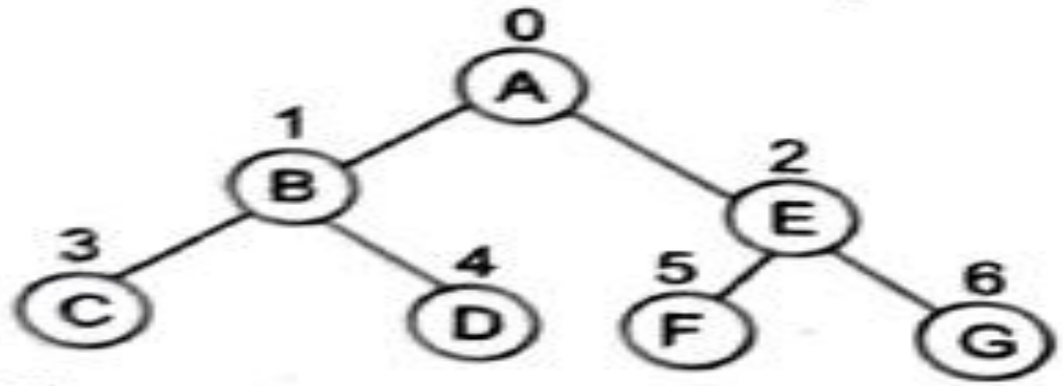


Fig. 6.13: Perfect binary tree

In Fig. 6.13,

Number of levels = 3 (0 to 2) and height = 2

Therefore, we need the array of size $2^{2+1} - 1$ is $2^3 - 1 = 7$.

The representation of the above binary tree using array is as followed:

1	2	3	4	5	6	7
A	B	C	D	E	F	G

We will number each node of tree starting from the root. Nodes on same level will be numbered left to right.

Example 2: Consider the example of

Binary Tree Examples

Example 2: Consider the example of Complete binary tree.

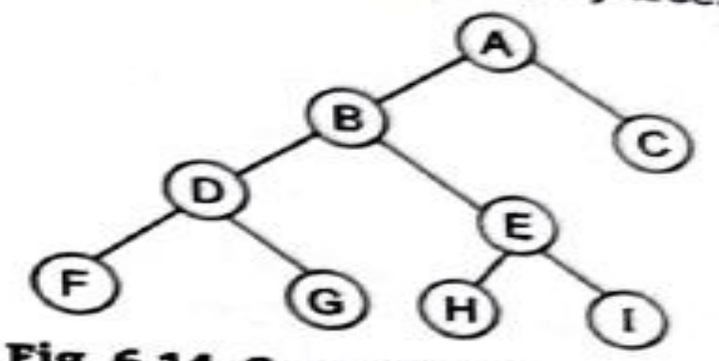


Fig. 6.14: Complete binary tree

Here, depth = 4 (level), therefore we need the array of size $2^4 - 1 = 15$. The representation of the above binary tree using array is as follows:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	B	C	D	E	–	–	F	G	H	I	–	–	–	–

level 0 1 2 3

We can apply the above rules to find array representation.

1. Parent of node E (node 5) $= \frac{i}{2} = \frac{5}{2} = 2$ i.e. B.

   Hence, node B is at position 2 in the array.
2. Left child (i) = 2i.

   For example, left child of E = $2 \times 5 = 10$ i.e. H.

   Since, E is the 5th node of tree.
3. Right child (i) = 2i + 1

For example: Right child of D = $2 \times 4 + 1 = 8 + 1 = 9$ i.e. G.

Since, D is the 4th node of tree and G is the 9th element of an array.

Advantages and Disadvantages

Advantages of linked representation of Binary Tree:

1. Efficient use of memory than that of sequential representation.
2. Insertion and deletion operations are more efficient.
3. Use for dynamic memory allocation.

Disadvantages of linked representation of Binary Tree:

1. If we want to access a particular node, we have to traverse from root to that node; there is no direct access to any node.
2. Since, two additional pointer are present (left child and right child), the memory needed per node is more than that of sequential representation.

Binary Search Tree

- **Binary search tree is a data structure that quickly allows us to maintain a sorted list of numbers.**

1. It is called a binary tree because each tree node has a maximum of two children.
2. It is called a search tree because it can be used to search for the presence of a number in O(log(n)) time

Binary Search Tree

- **Properties**
- Follows all properties of the tree data structure.

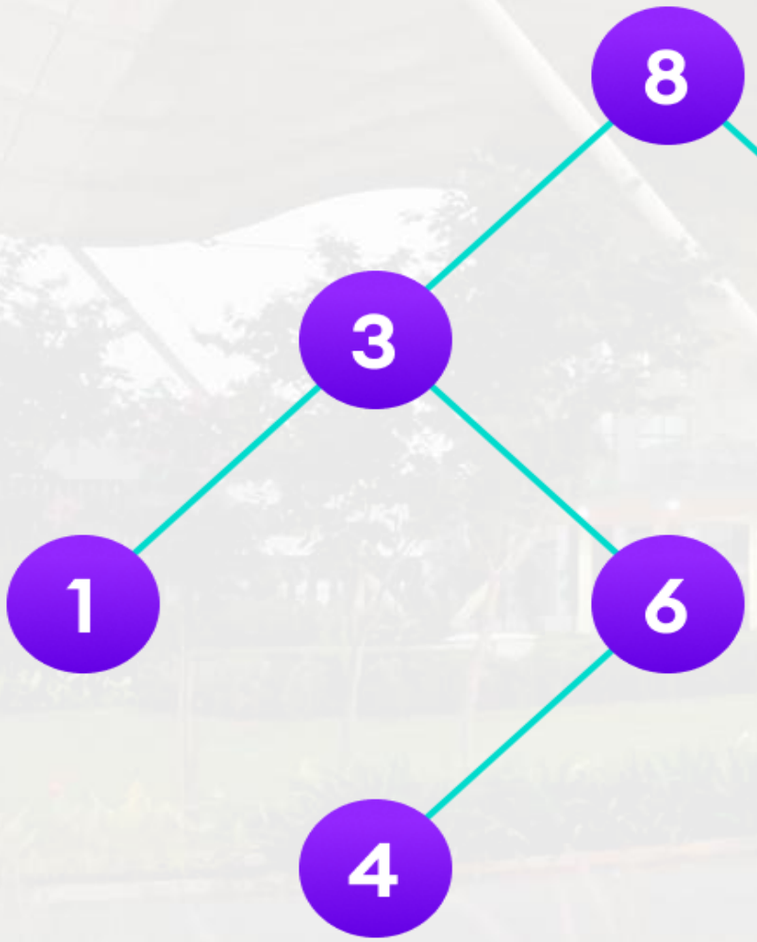
1. The value of the key of the left sub-tree is less than the value of its parent (root) node's key.

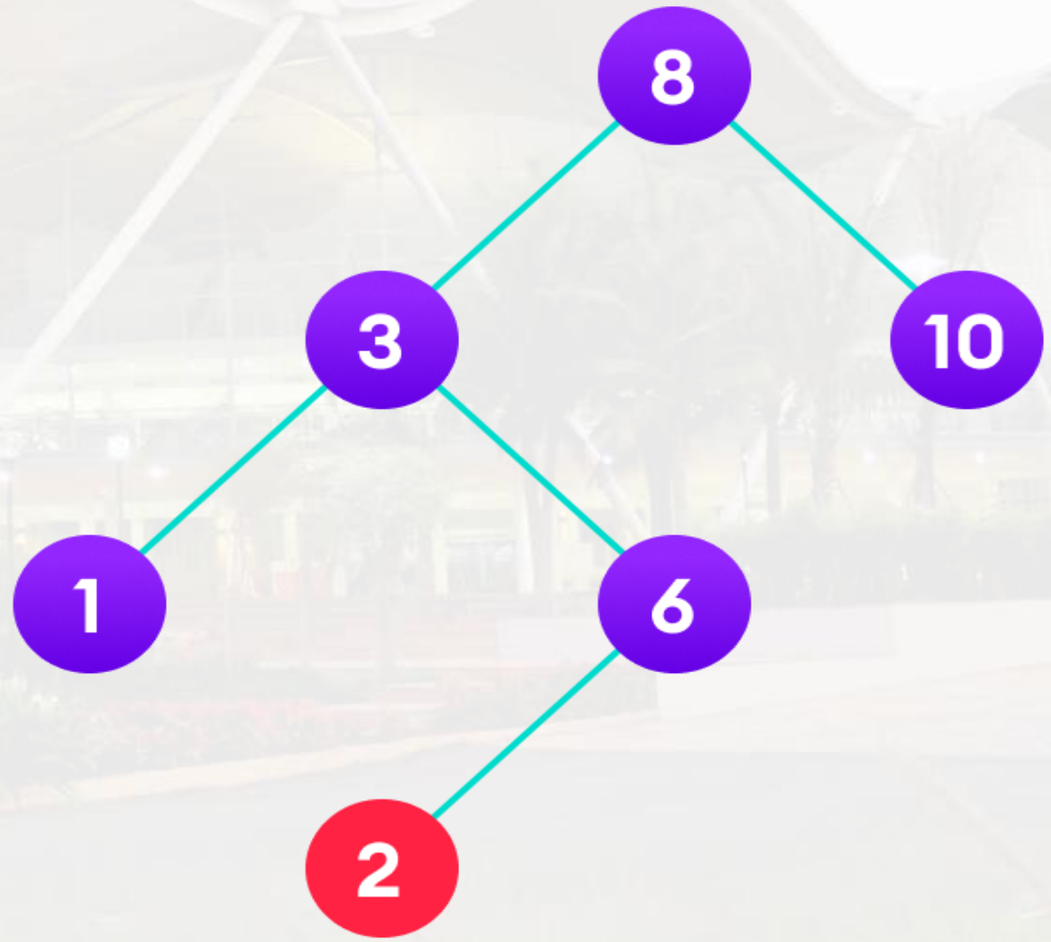
2. The value of the key of the right sub-tree is greater than or equal to the value of its parent (root) node's key.

3. Both subtrees of each node are also BSTs i.e. they have the above two properties

Binary Search Tree

- The binary tree on the right isn't a binary search tree because the right subtree of the node "3" contains a value smaller than it.



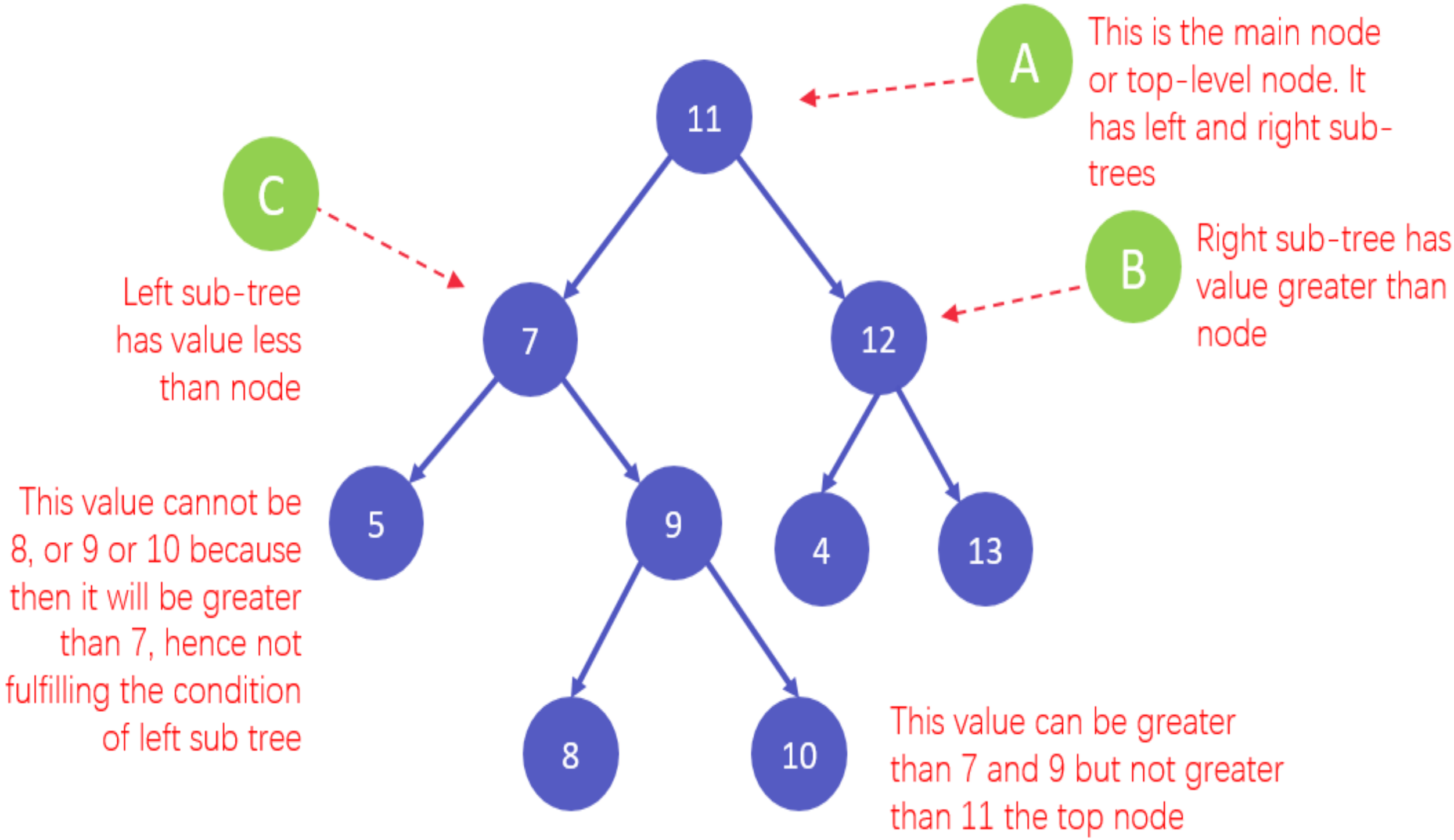


Operation of Binary Search Tree

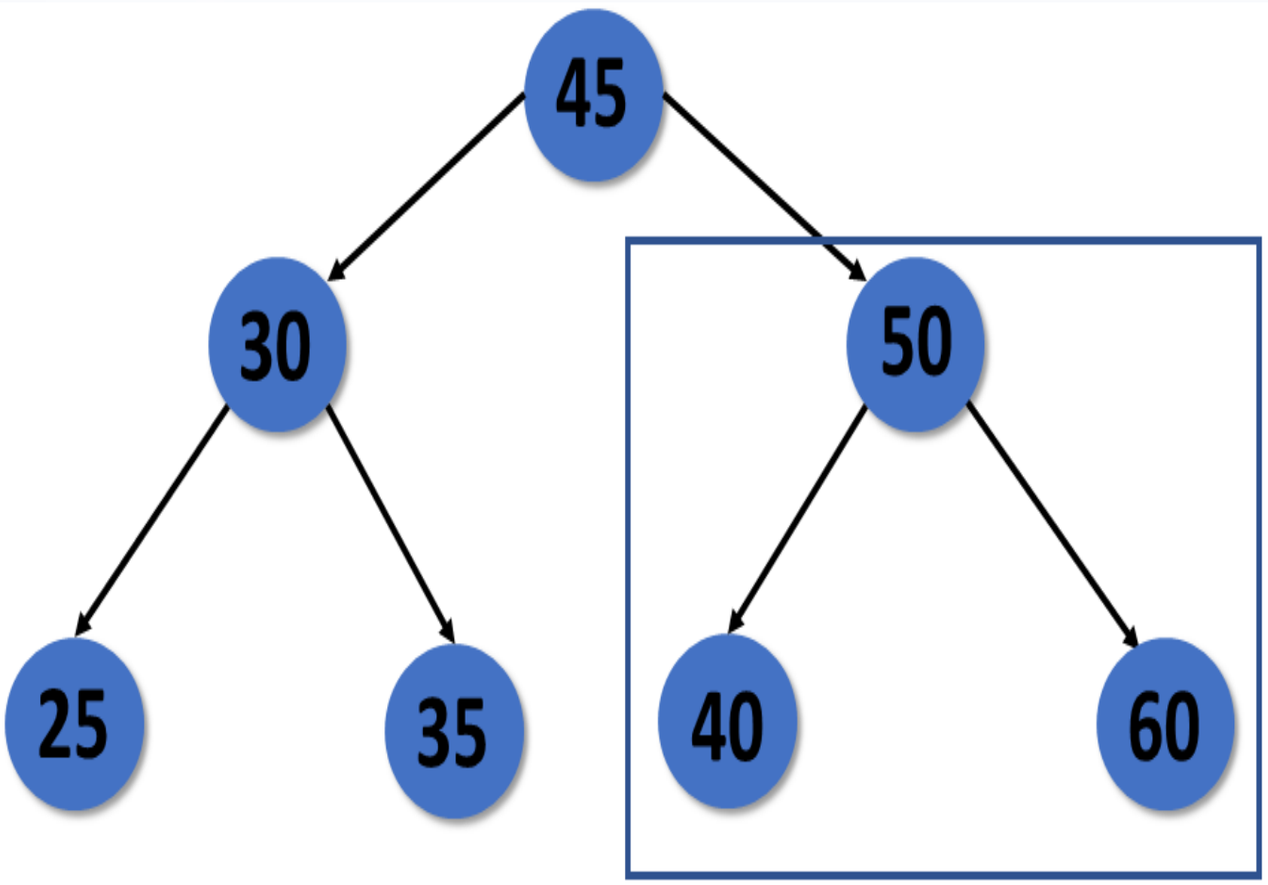
- **Insert** – Inserts an element in a tree/create a tree.
- **Search** – Searches an element in a tree.
- **Preorder Traversal** – Traverses a tree in a pre-order manner.
- **Inorder Traversal** – Traverses a tree in an in-order manner.
- **Postorder Traversal** – Traverses a tree in a post-order manner.

Binary Search Tree

CONCEPT DIAGRAM



Binary Search Tree



Left node value<= root node <= right node value

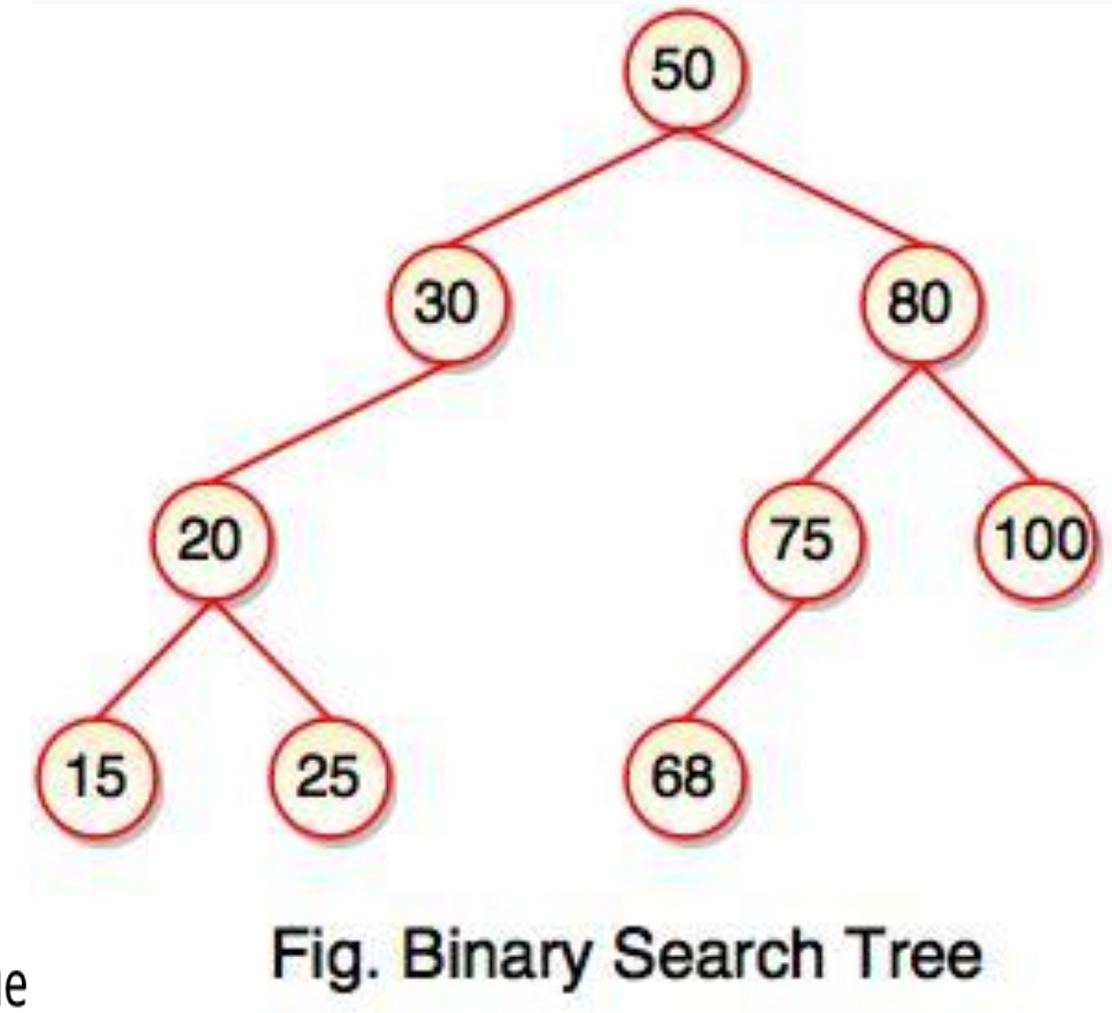


Fig. Binary Search Tree

Binary Search Tree

Create the binary search tree using the following data elements.

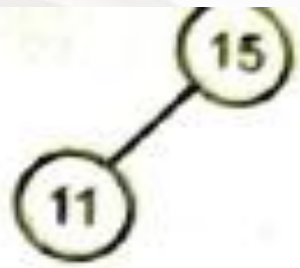
15, 11, 13, 8, 9, 17, 16, 18

1.



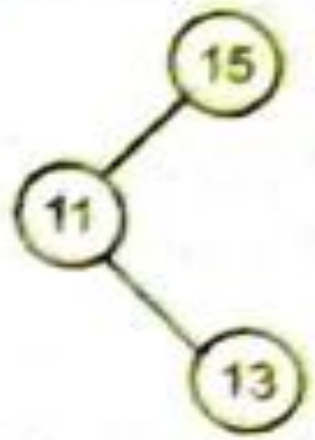
Key = 15

2.



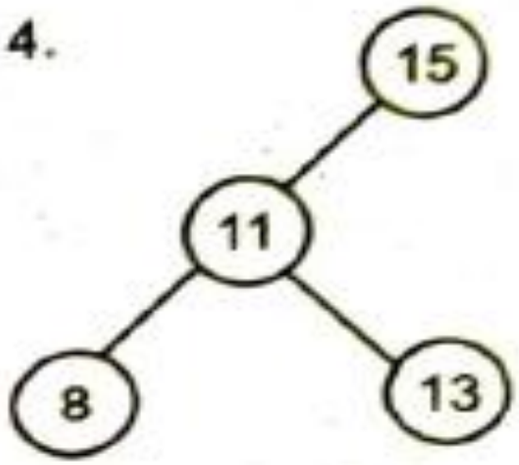
Key < 15
Key = 11

3.



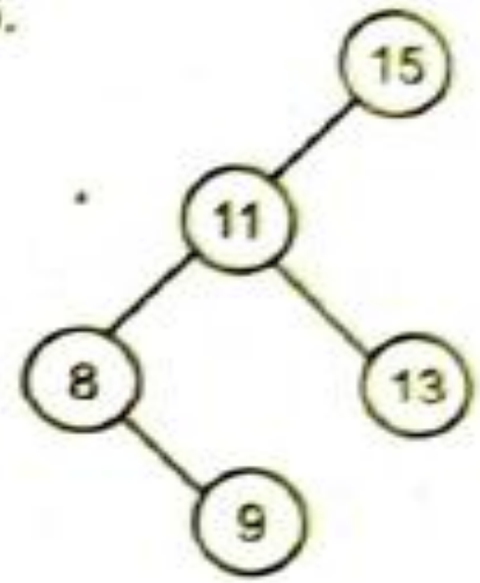
Key = 13

4.



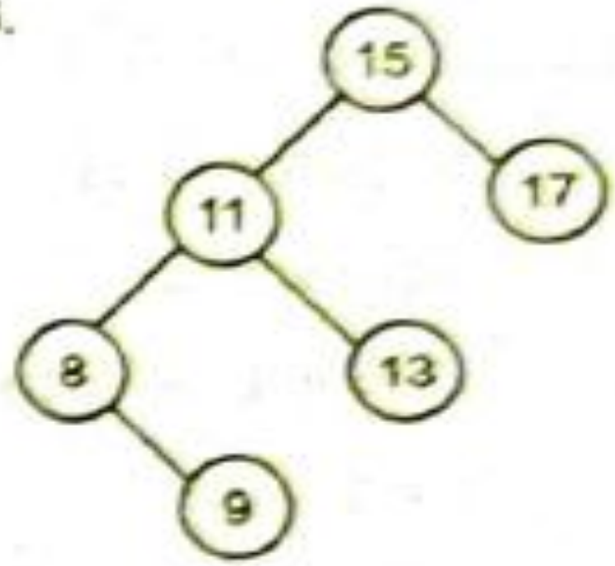
Key = 8

5.



Key = 9

6.



Key = 17

Binary Search Tree

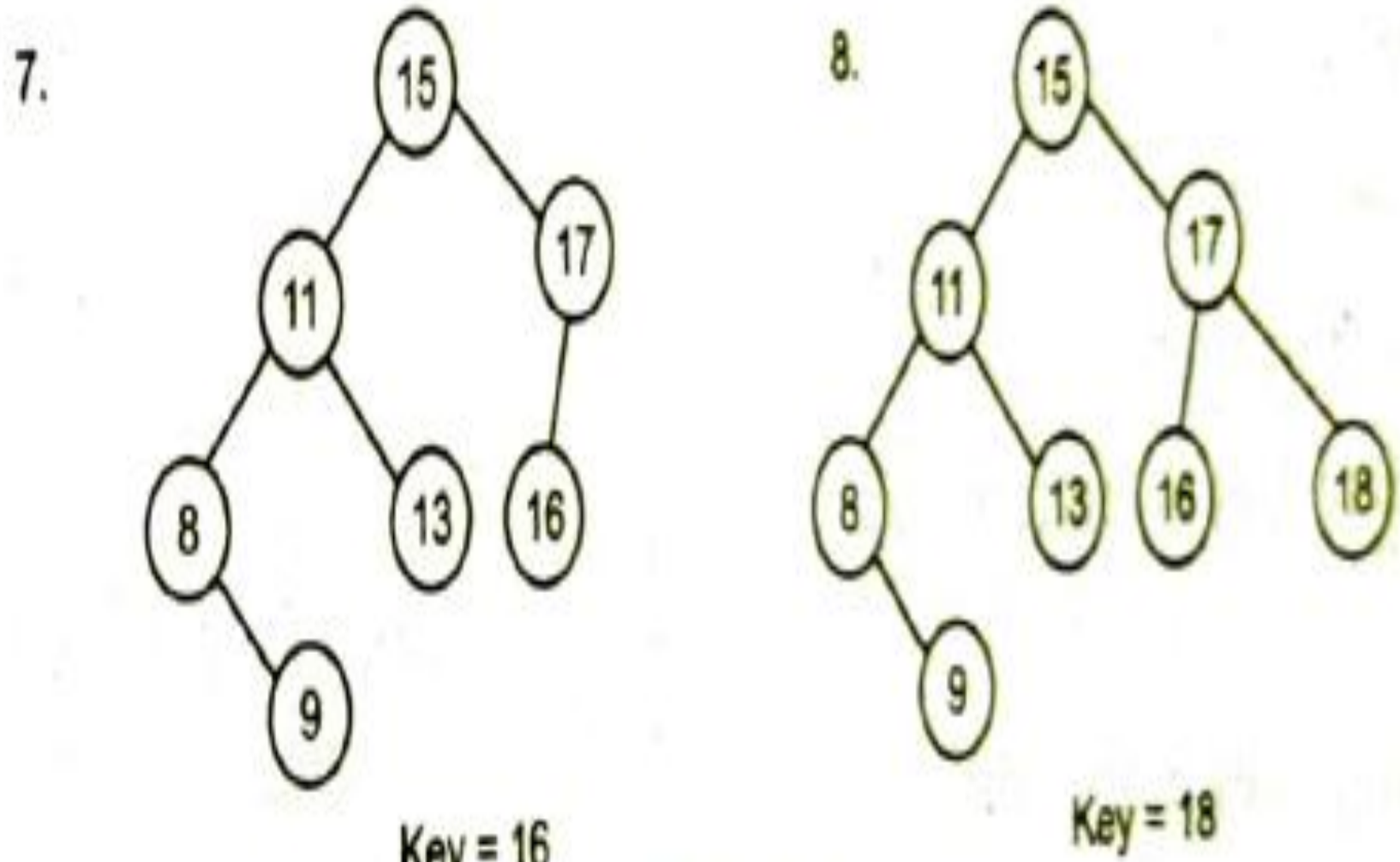


Fig. 6.23

- Here, all values in the left subtrees are less than the root and all values in the right subtrees are greater than the root.

Binary Search Tree

Create the binary search tree using the following data elements.

43, 10, 79, 90, 12, 54, 11, 9, 50

Binary search Tree Creation

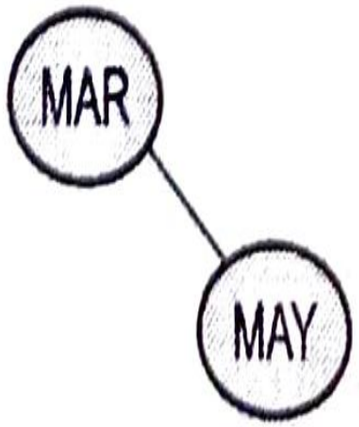
Binary Search Tree

Construct Binary Search Tree(BST) for the following : MAR, MAY, NOV, AUG, APR, JAN, DEC, JUL, FEB, JUN, OCT, SEPT.

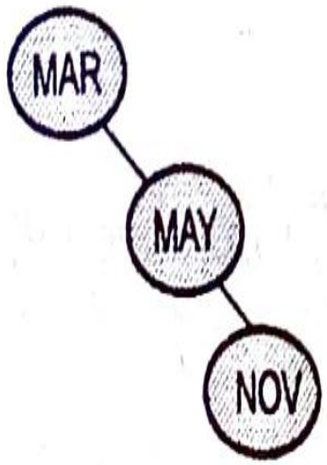
1.



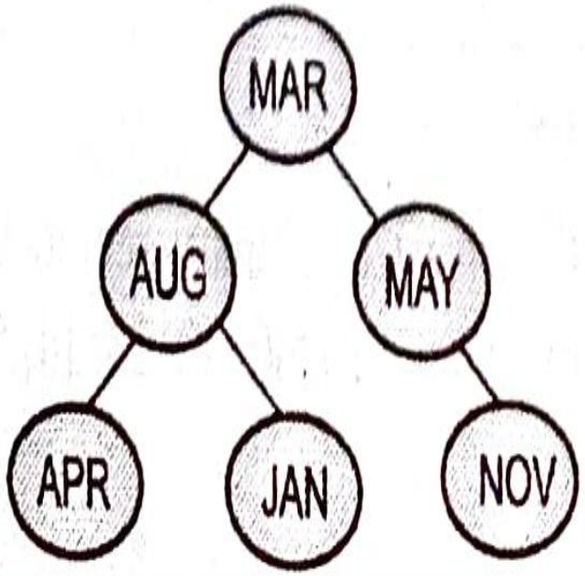
2.



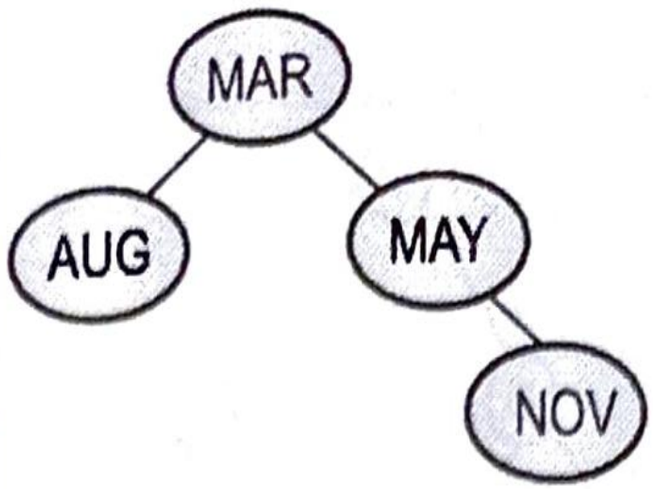
3.



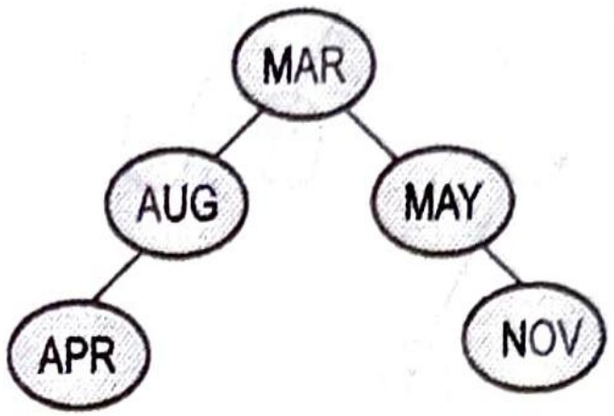
6.



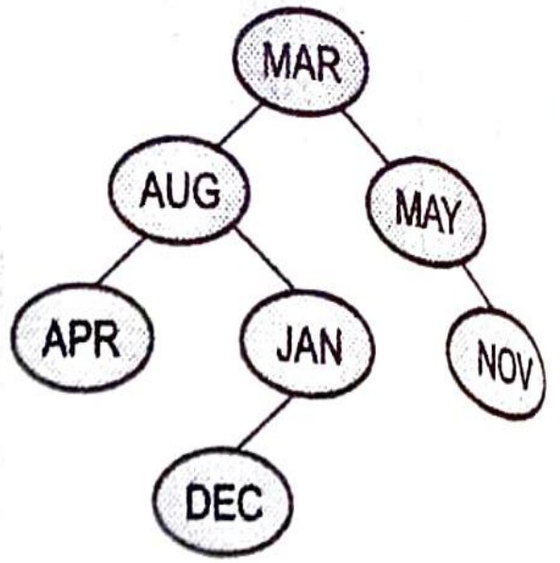
4.



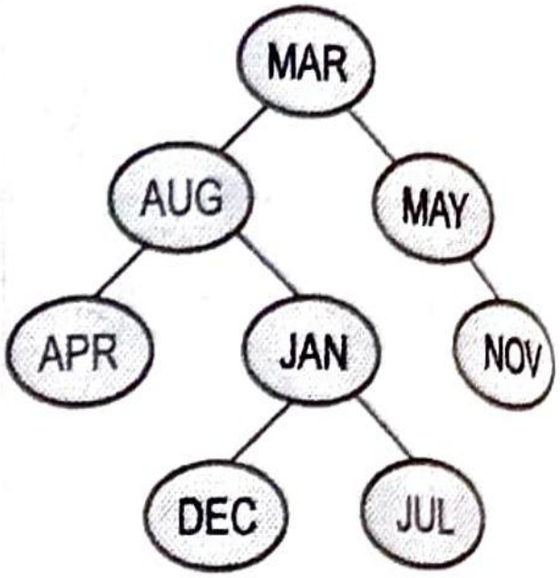
5.



7.



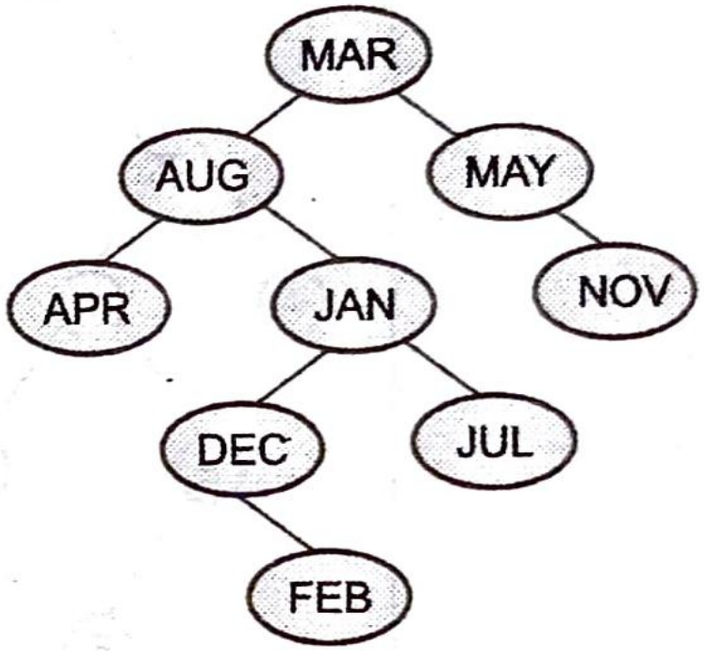
8.



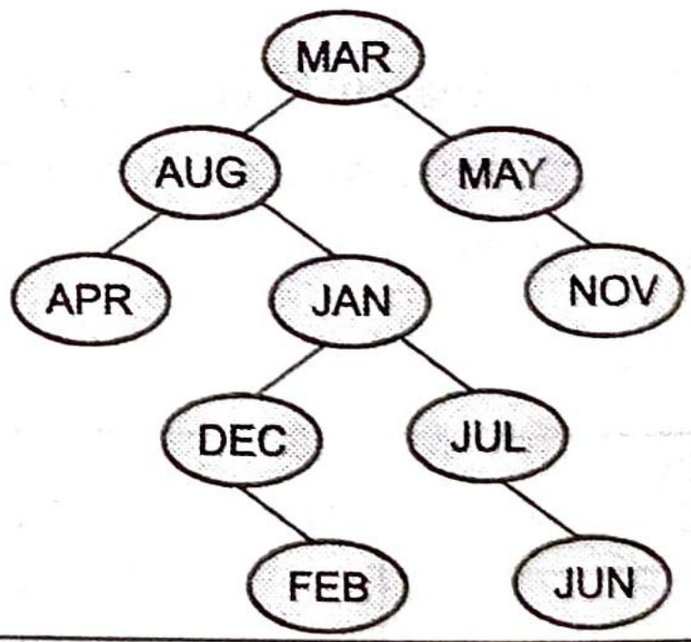
Binary Search Tree

Construct Binary Search Tree(BST) for the following : MAR, MAY, NOV, AUG, APR, JAN, DEC, JUL, FEB, JUN, OCT, SEPT.

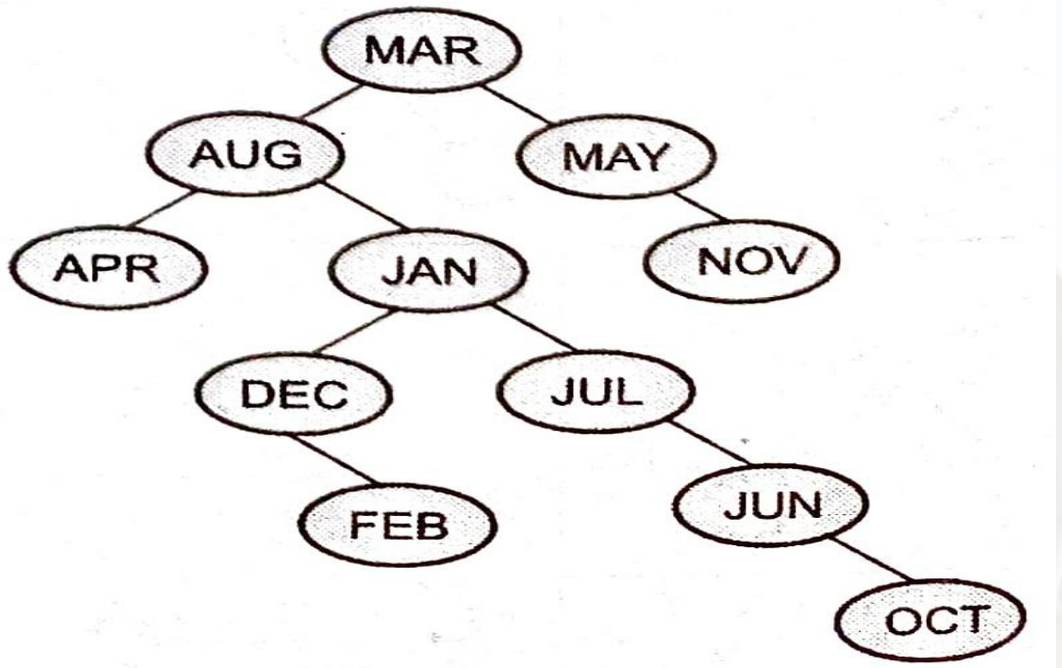
9.



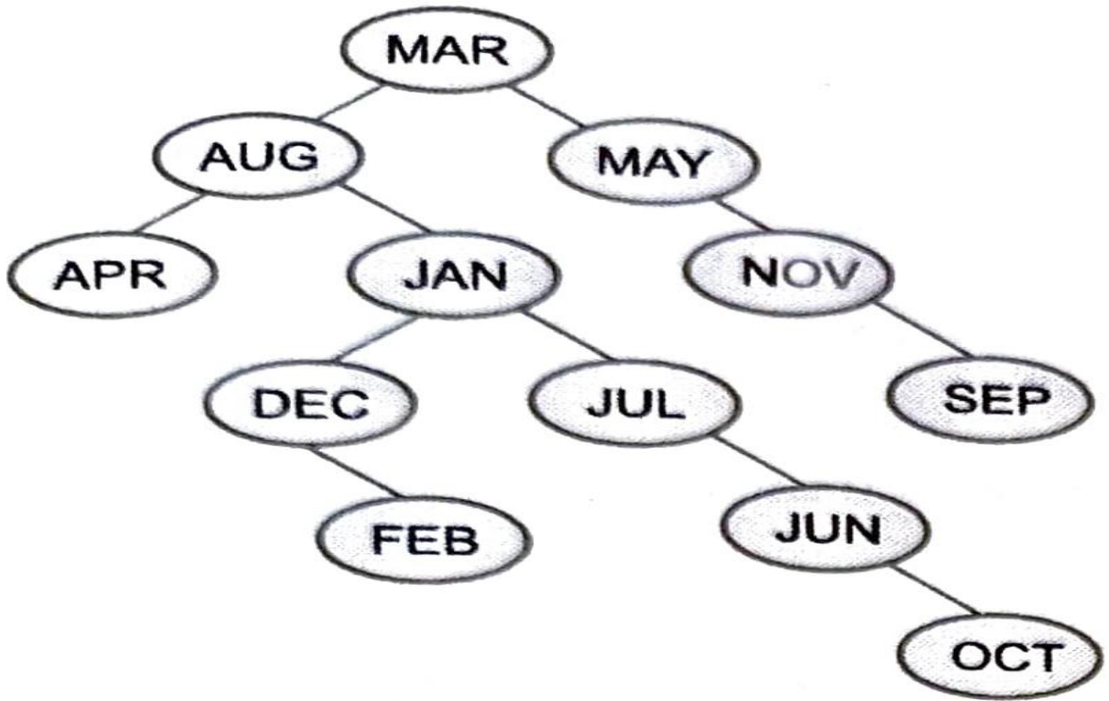
10.



11.



12.



Operation of Binary Search Tree

1. **Create:** creates an empty tree.

2. **Insert:** insert a node in the tree.

3. **Search:** Searches for a node in the tree.

4. **Delete:** deletes a node from the tree.

5. **Inorder:** in-order traversal of the tree.

6. **Preorder:** pre-order traversal of the tree.

7. **Postorder:** post-order traversal of the tree

Operation of Binary Search Tree

Search Operation

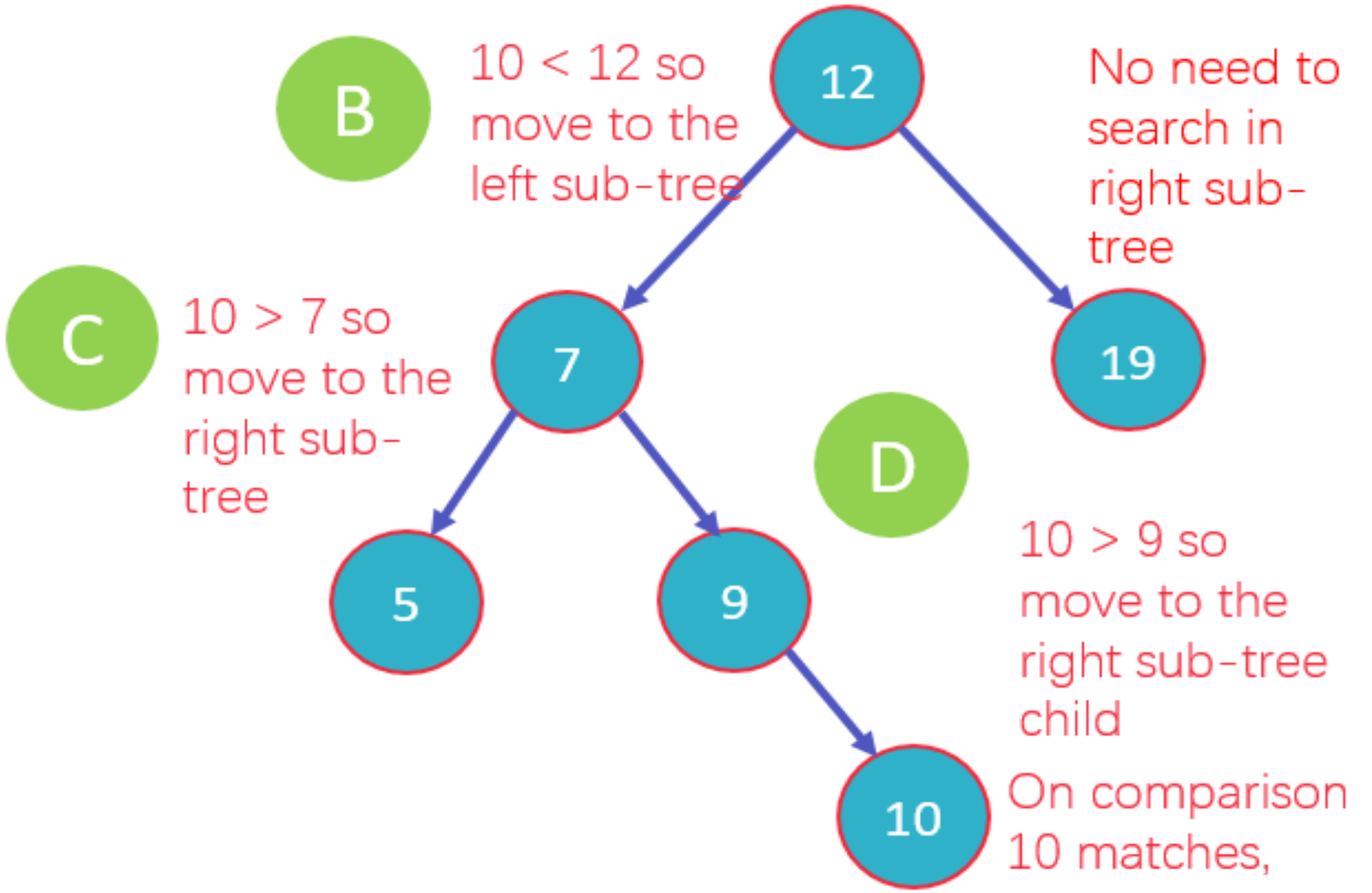
A — Elements to be searched in the tree 10

B — 10 < 12 so move to the left sub-tree

C — 10 > 7 so move to the right sub-tree

D — 10 > 9 so move to the right sub-tree child

E — On comparison 10 matches, return the value



Operation of Binary Search Tree

1. Search :

Algorithm:

If root == NULL

    return NULL;

If number == root->data

    return root->data;

If number < root->data

    return search(root->left)

If number > root->data

    return search(root->right)

Operation of Binary Search Tree

Insert Operation

A

Elements to be inserted in the tree from left to right:
12, 7, 9, 19, 5, 10

B

Insert 12 as root node and compare 7 and 9 values for inserting to right or left-sub tree

Root

12

7 < 12, so add to left

7

9 < 12 & 9 > 7 so add to right

9

C

Compare 19, 5 and 10 with 12 and other nodes, and build the tree accordingly

12

7

19 > 12 & 19 > 7 so add to right

19

5

5 < 12 & 5 < 7 so add to left

9

10 < 12 & 10 > 7 & 10 > 9 so add to right

10

Operation of Binary Search Tree

2. Insert :

Algorithm:

```
If node == NULL
    return createNode(data)
if (data < node->data)
    node->left  = insert(node->left, data);
else if (data > node->data)
    node->right = insert(node->right, data);
return node;
```

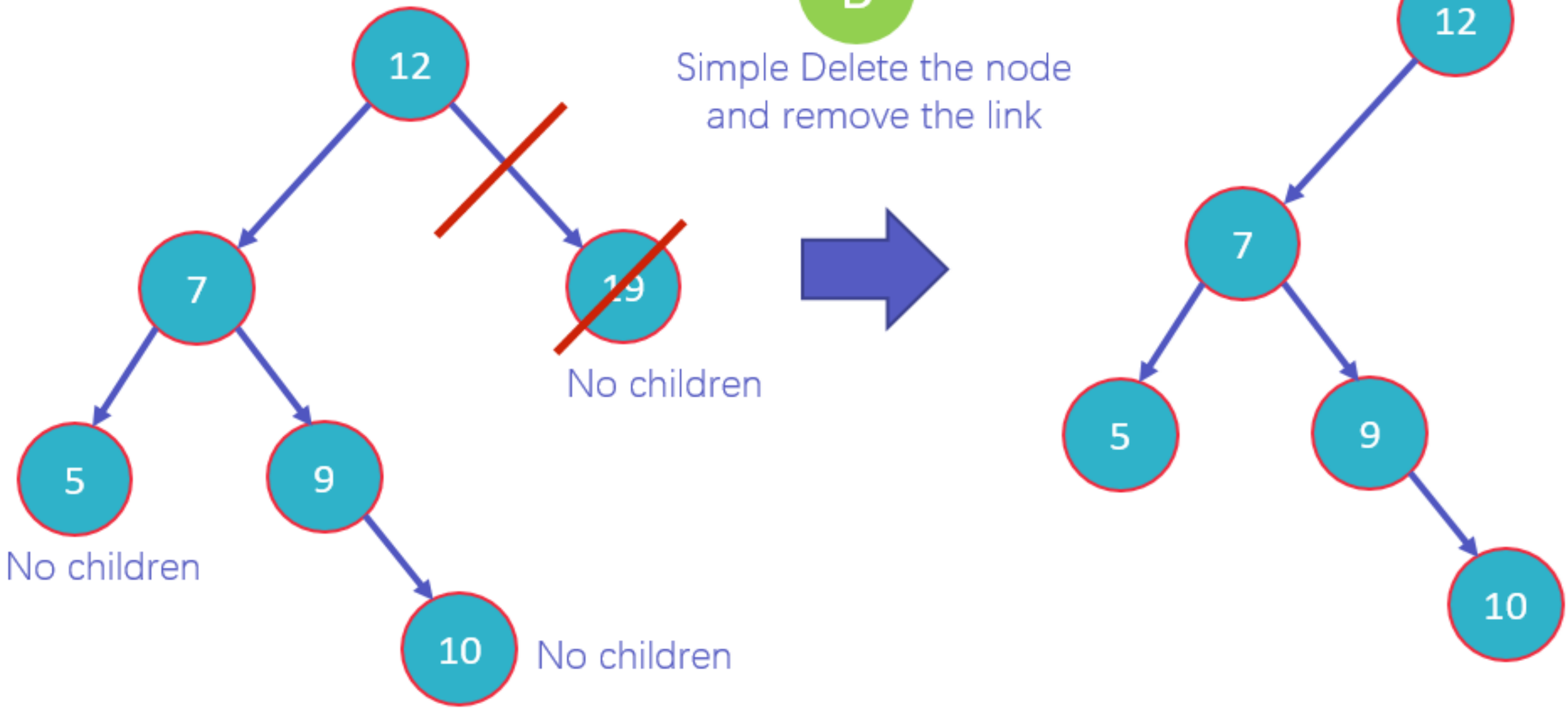
Operation of Binary Search Tree

Delete Operation – Case 1

A — Node to be deleted has 0 children

B — Simple Delete the node and remove the link

C — Result



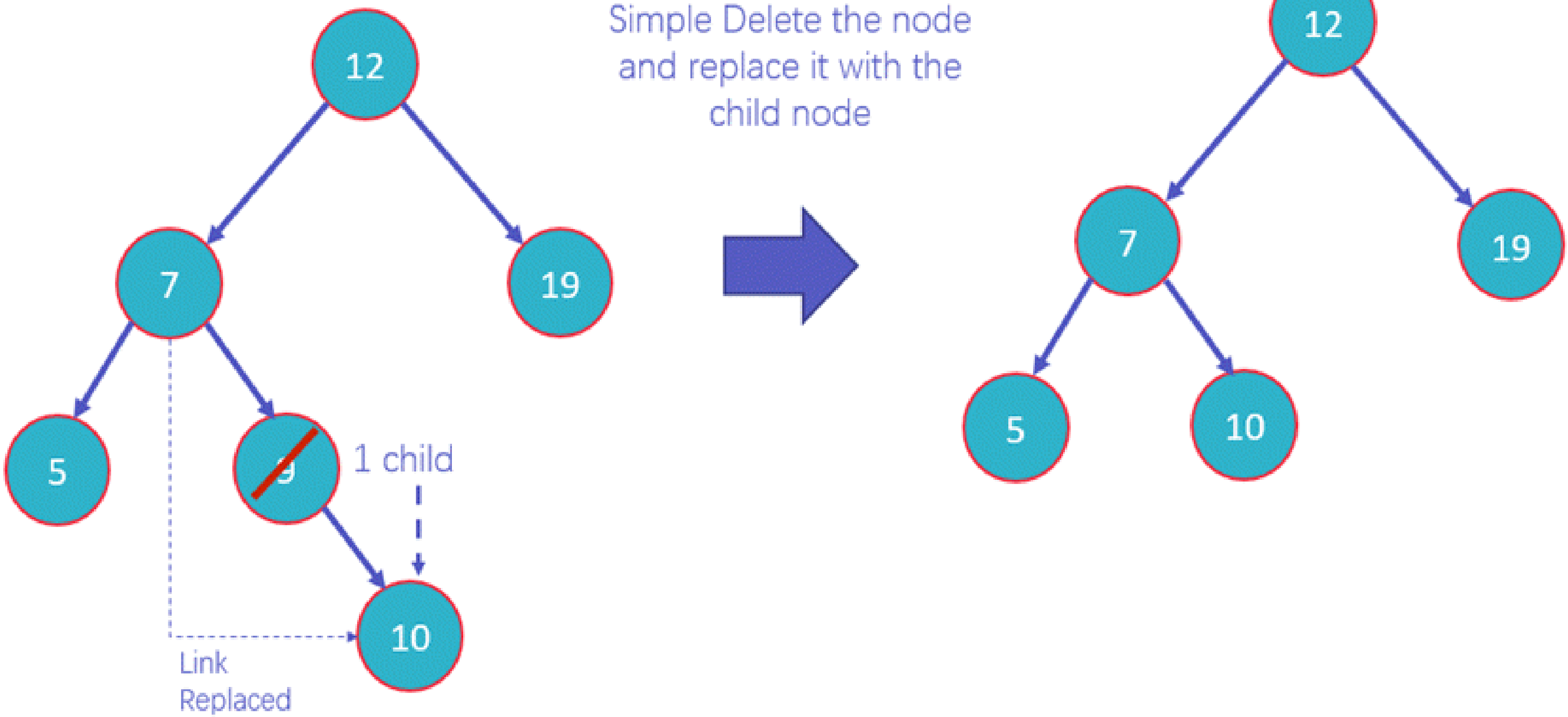
Operation of Binary Search Tree

Delete Operation – Case 2

A — Node to be deleted has 1 child

B — Simple Delete the node and replace it with the child node

C — Result



Operation of Binary Search Tree

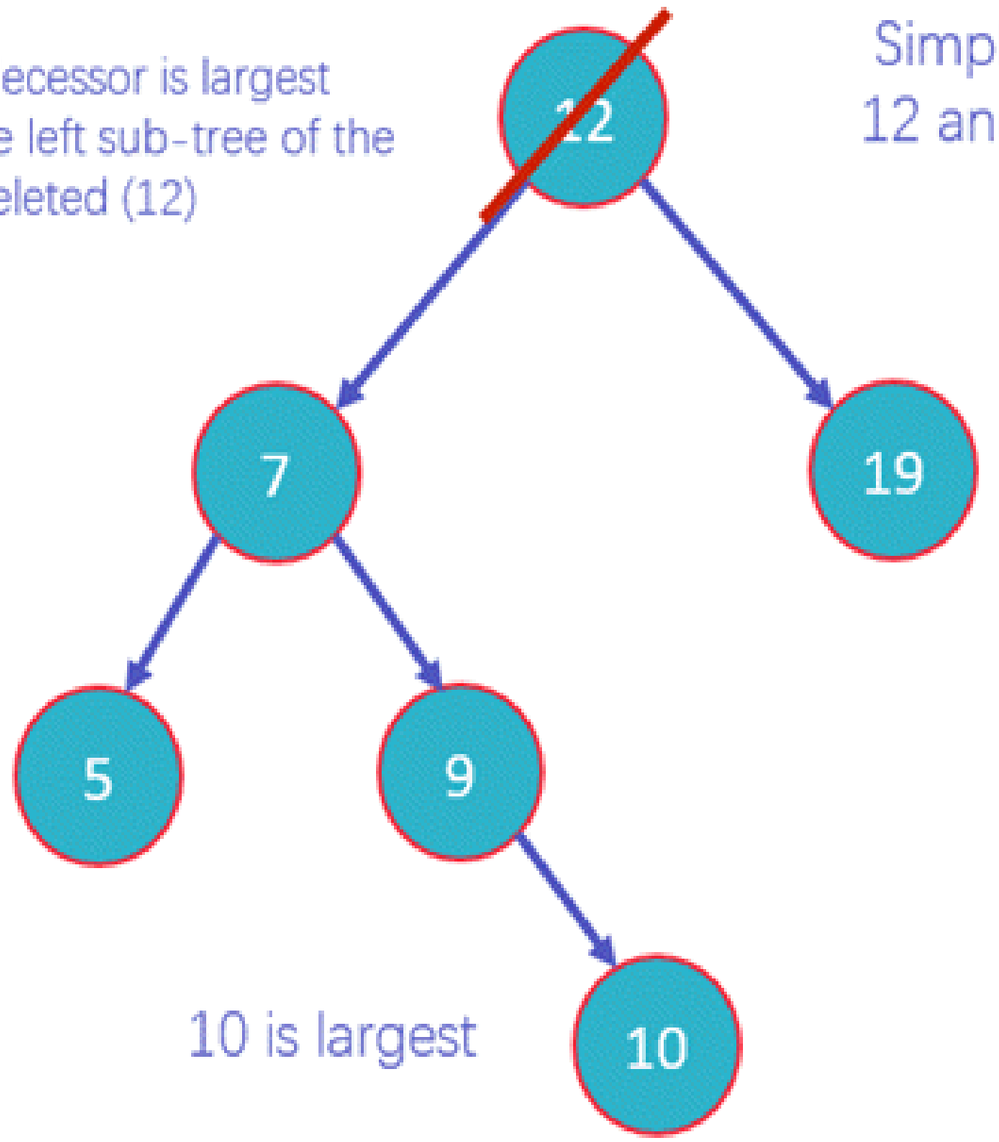
Delete Operation – Case 3 (a)

A

Node to be deleted has 2 child
Replace Situation: In Order Predecessor

In Order predecessor is largest element in the left sub-tree of the node to be deleted (12)

B

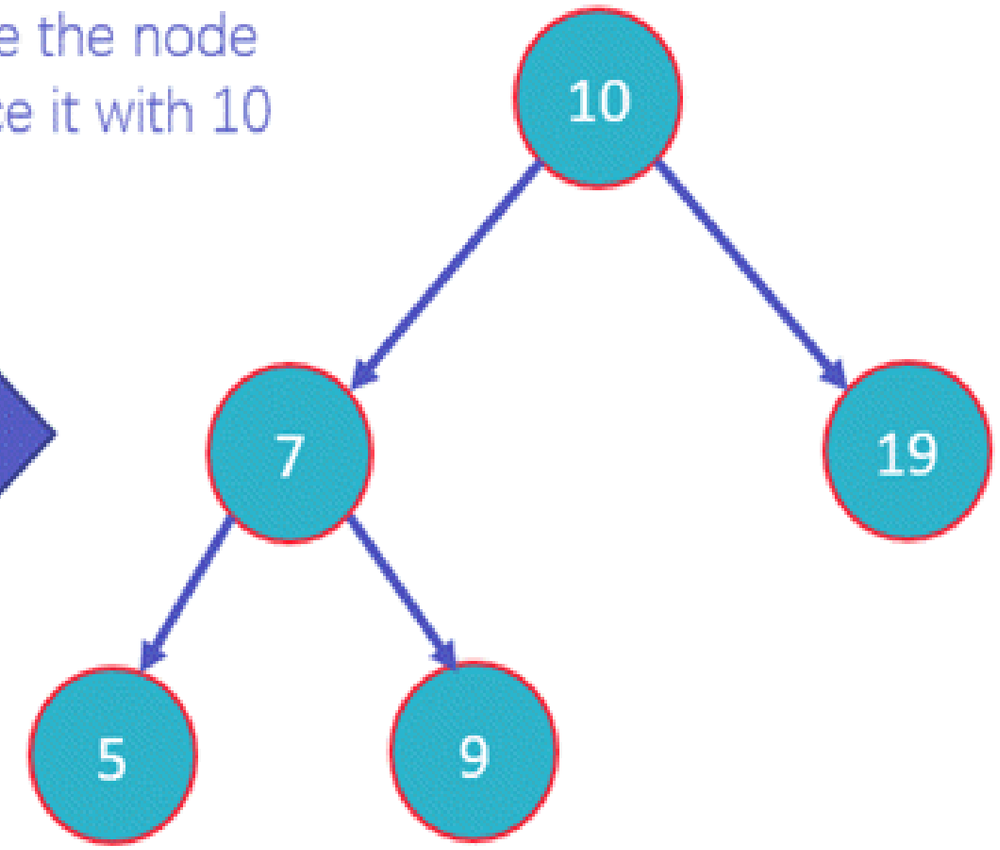


C

Simple Delete the node 12 and replace it with 10

D

Result



Operation of Binary Search Tree

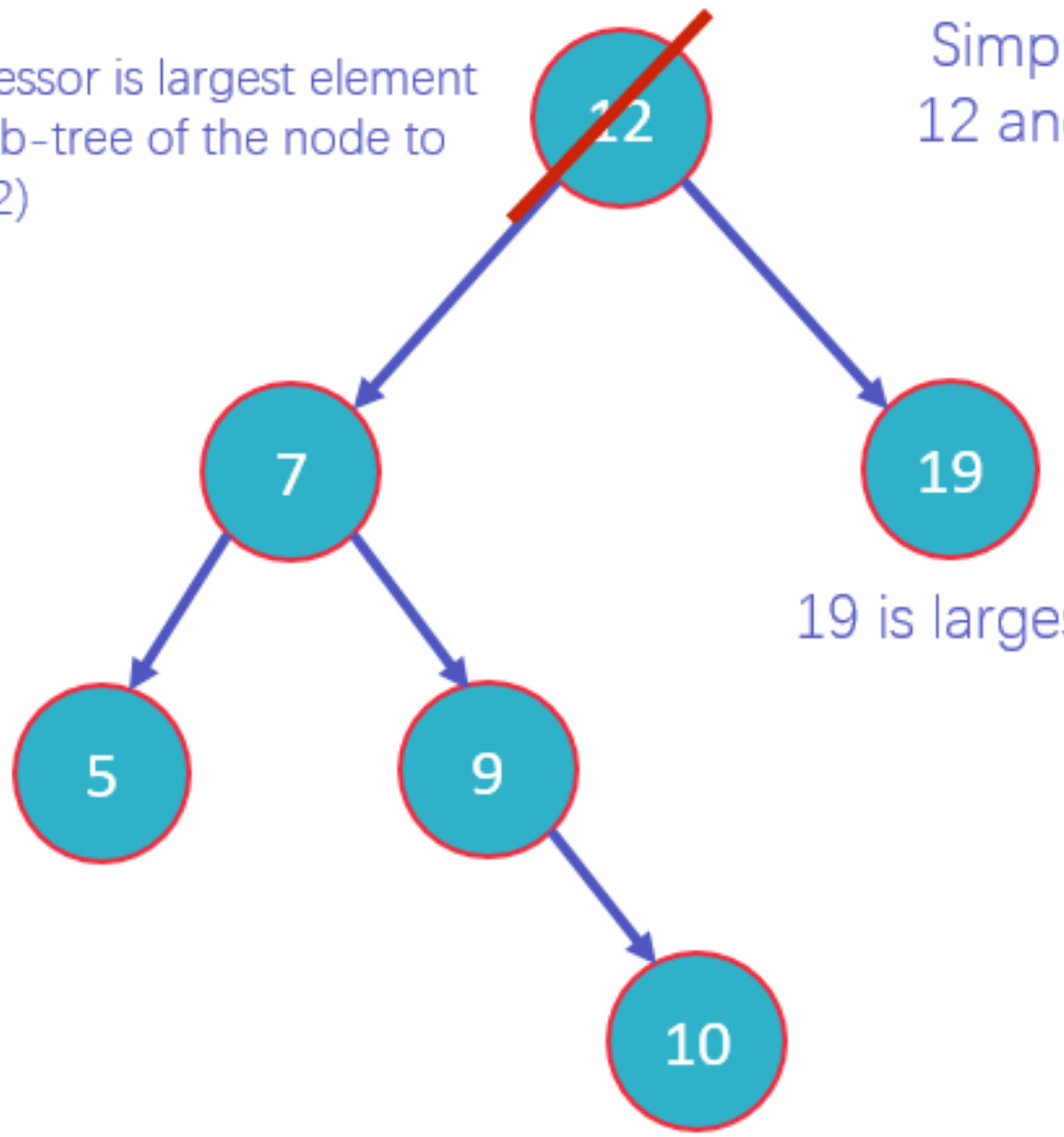
Delete Operation – Case 3 (b)

A

Node to be deleted has 2 child
Replace Situation: In Order Successor

In Order successor is largest element in the right sub-tree of the node to be deleted (12)

B



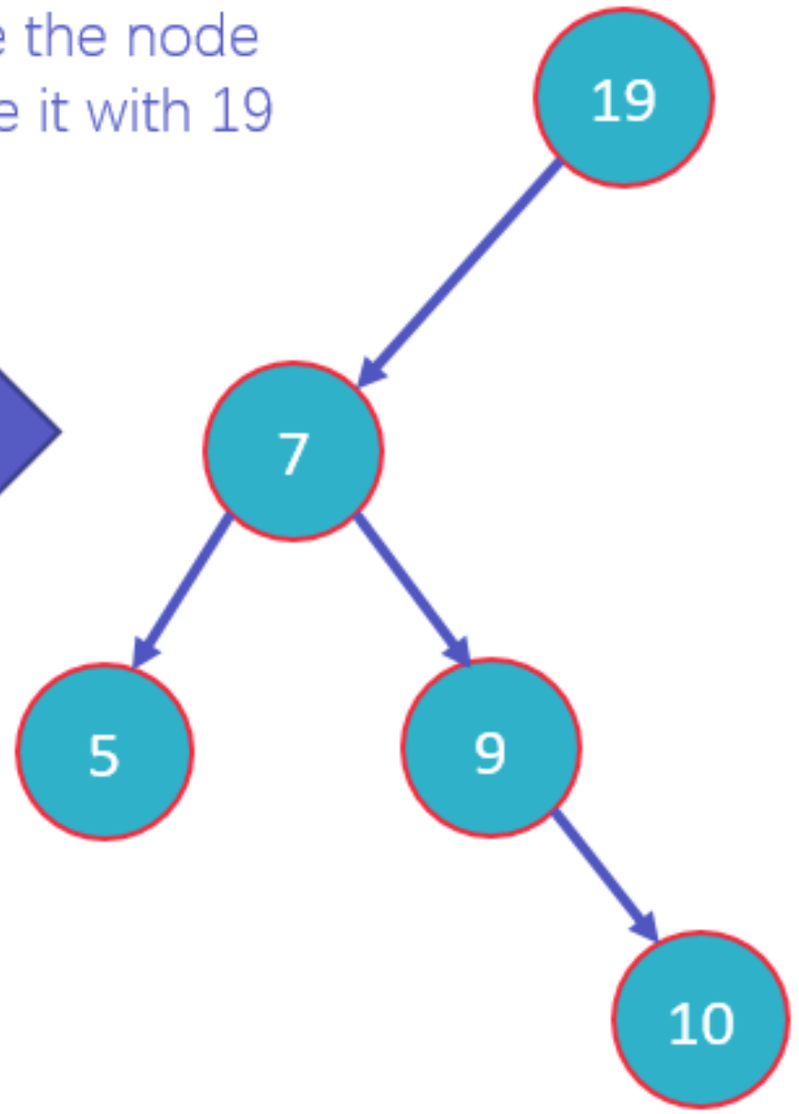
C

Simple Delete the node 12 and replace it with 19

19 is largest

D

Result



Operation of Binary Search Tree

3. Delete :

Algorithm:

Case 1: Deleting a leaf node

We use the following steps to delete a leaf node from BST...

Step 1 - Find the node to be deleted using search operation

Step 2 - Delete the node using free function (If it is a leaf) and terminate the function.

Operation of Binary Search Tree

3. Delete :

Algorithm:

Case 2: Deleting a node with one child

We use the following steps to delete a node with one child from BST...

Step 1 - Find the node to be deleted using search operation

Step 2 - If it has only one child then create a link between its parent node and child node.

Step 3 - Delete the node using free function and terminate the function.

Operation of Binary Search Tree

3. Delete :

Algorithm:

Case 3: Deleting a node with two children

We use the following steps to delete a node with two children from BST...

Step 1 - Find the node to be deleted using search operation

Step 2 - If it has two children, then find the largest node in its left subtree (OR) the smallest node in its right subtree.

Step 3 - Swap both deleting node and node which is found in the above step.

Step 4 - Then check whether deleting node came to case 1 or case 2 or else goto step 2

Operation of Binary Search Tree

3. Delete :

Algorithm:

Case 3: Deleting a node with two children

We use the following steps to delete a node with two children from BST...

Step 5 - If it comes to case 1, then delete using case 1 logic.

Step 6- If it comes to case 2, then delete using case 2 logic.

Step 7 - Repeat the same process until the node is deleted from the tree.

Convert a Generic Tree (N-array Tree) to Binary Tree

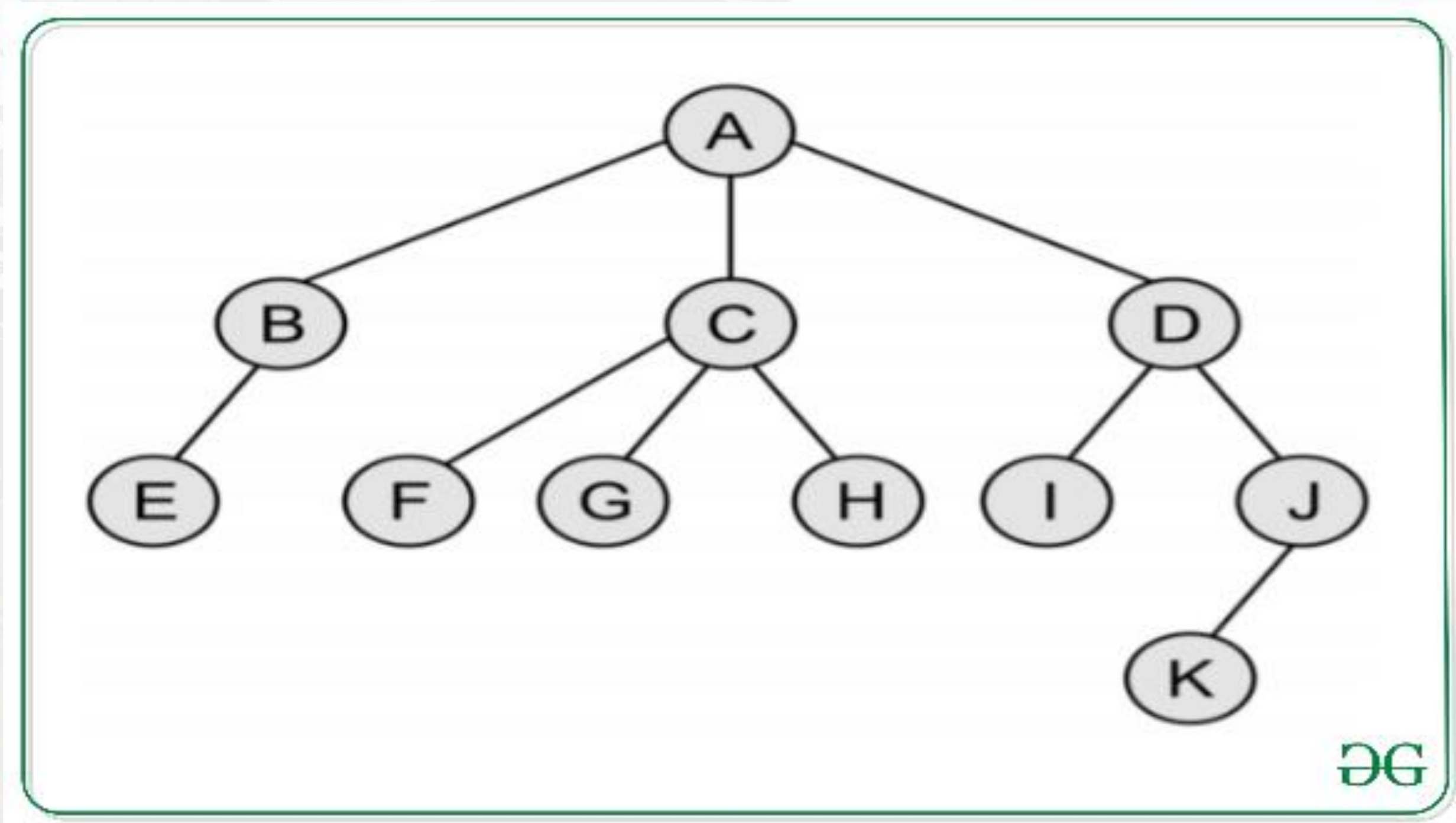
Following are the rules to convert a Generic(N-array tree) to binary tree :

1. The root of the Binary Tree is the Root of the Generic Tree.
2. The left child of a node in the Generic Tree is the Left child of that node in the Binary Tree.
3. The right sibling of any node in the Generic Tree is the Right child of that node in the Binary Tree.

Convert a Generic Tree(N-array Tree) to Binary Tree

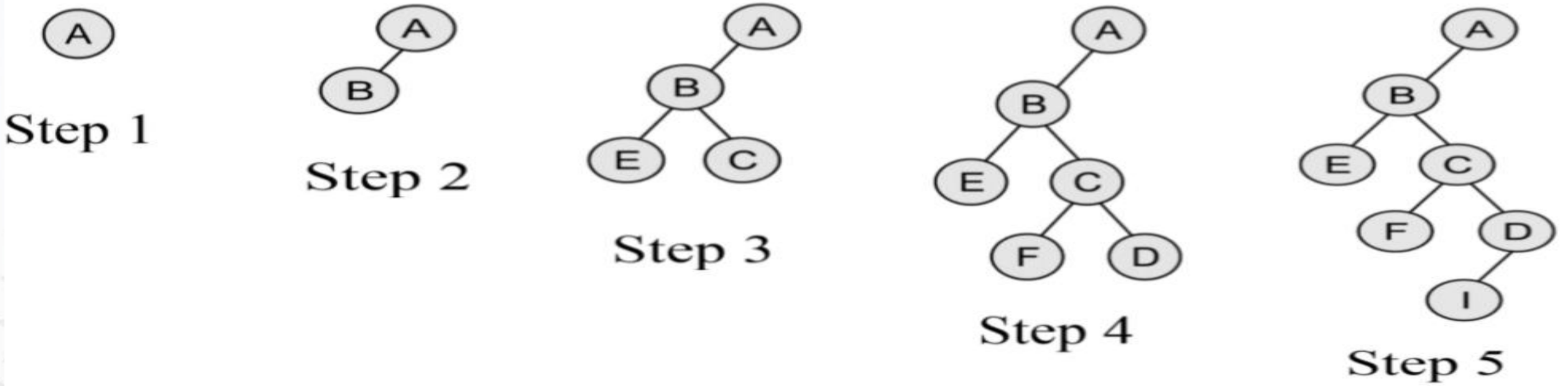
Examples:

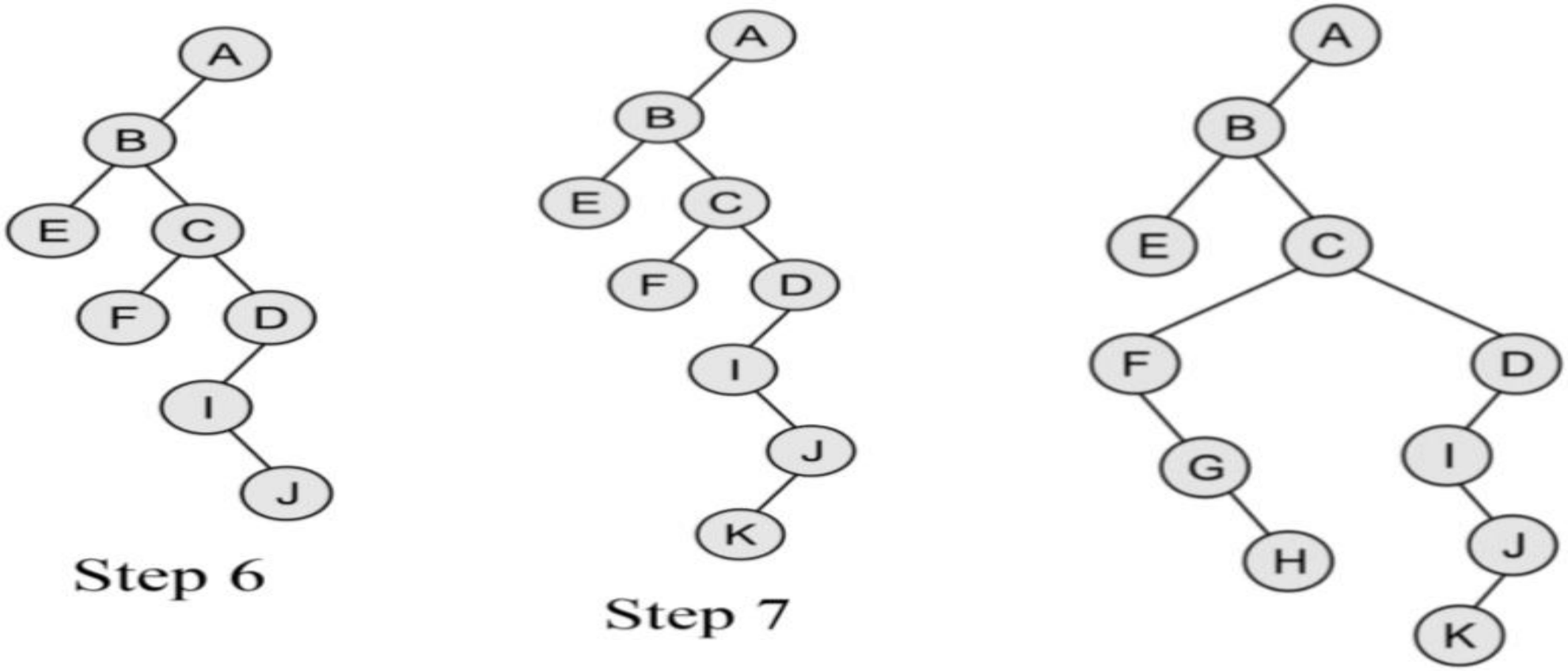
Convert the following Generic Tree to Binary Tree:



Convert a Generic Tree(N-array Tree) to Binary Tree

Below is the Binary Tree of the above Generic Tree:





Convert a Generic Tree(N-array Tree) to Binary Tree

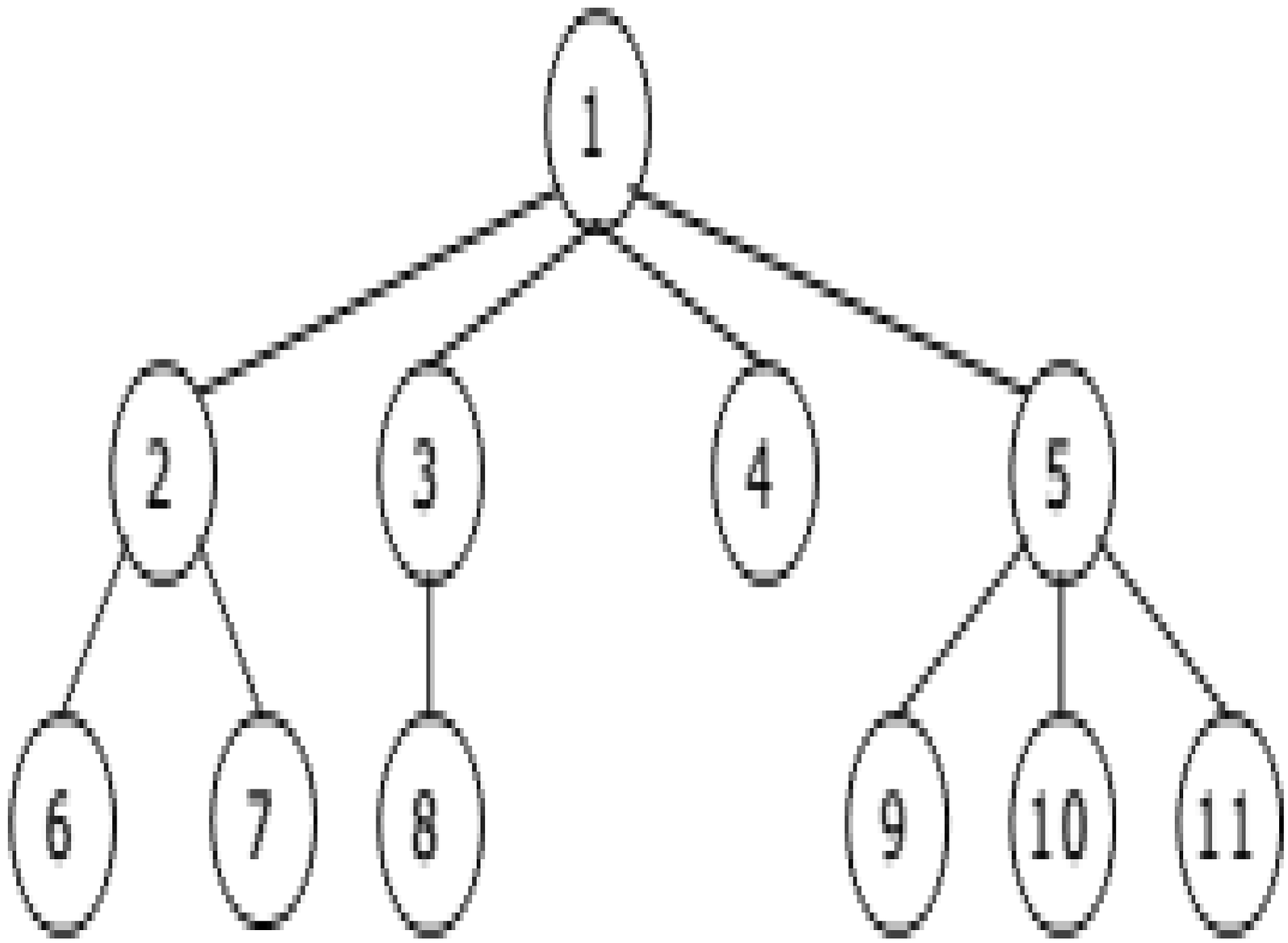
Note: If the parent node has only the right child in the general tree then it becomes the rightmost child node of the last node following the parent node in the binary tree.

In the above example, if node B has the right child node L then in binary tree representation L would be the right child of node D.

Convert a Generic Tree(N-array Tree) to Binary Tree

Examples:

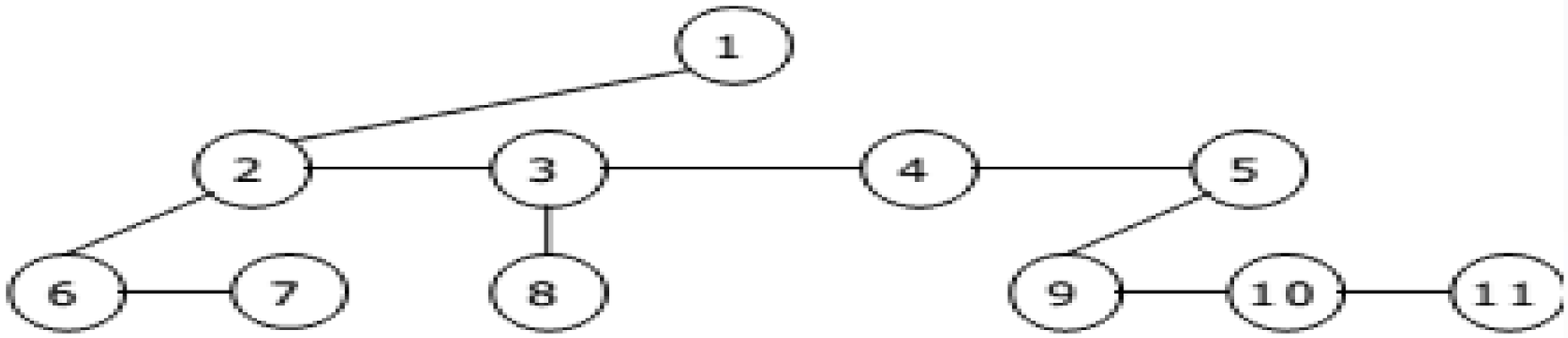
Convert the following Generic Tree to Binary Tree:



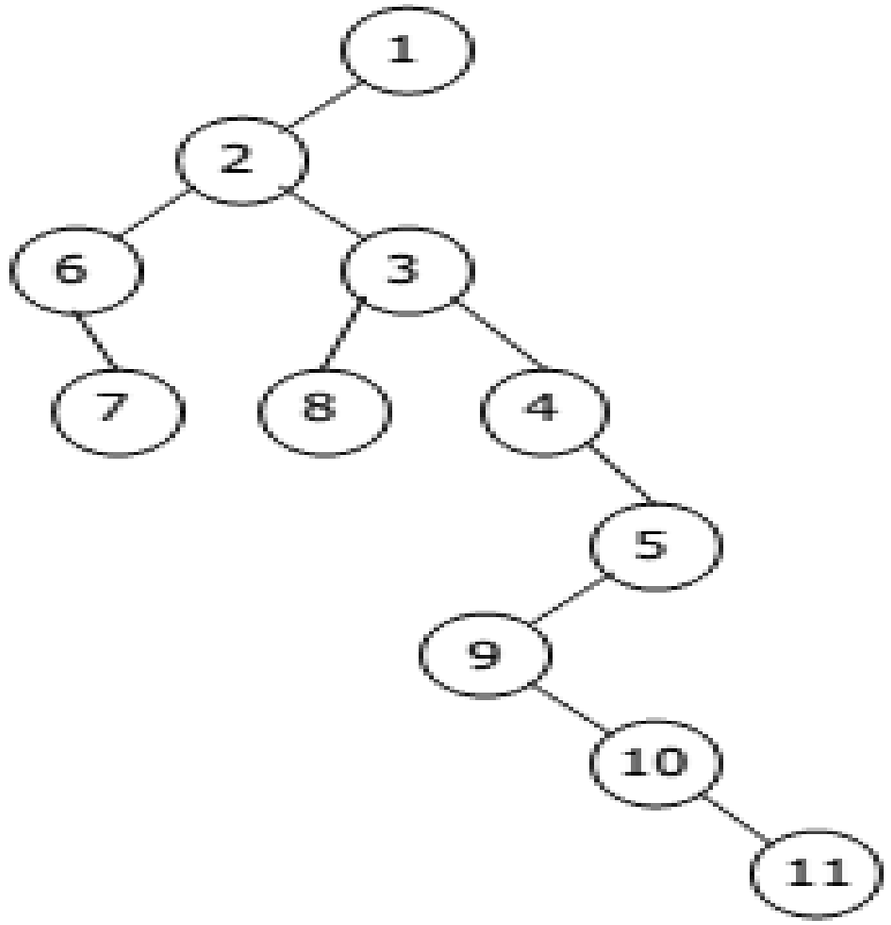
Convert a Generic Tree(N-array Tree) to Binary Tree

Solution:

Stage 1 tree by using the above mentioned procedure is as follows:



Stage 2 tree by using the above mentioned procedure is as follows:

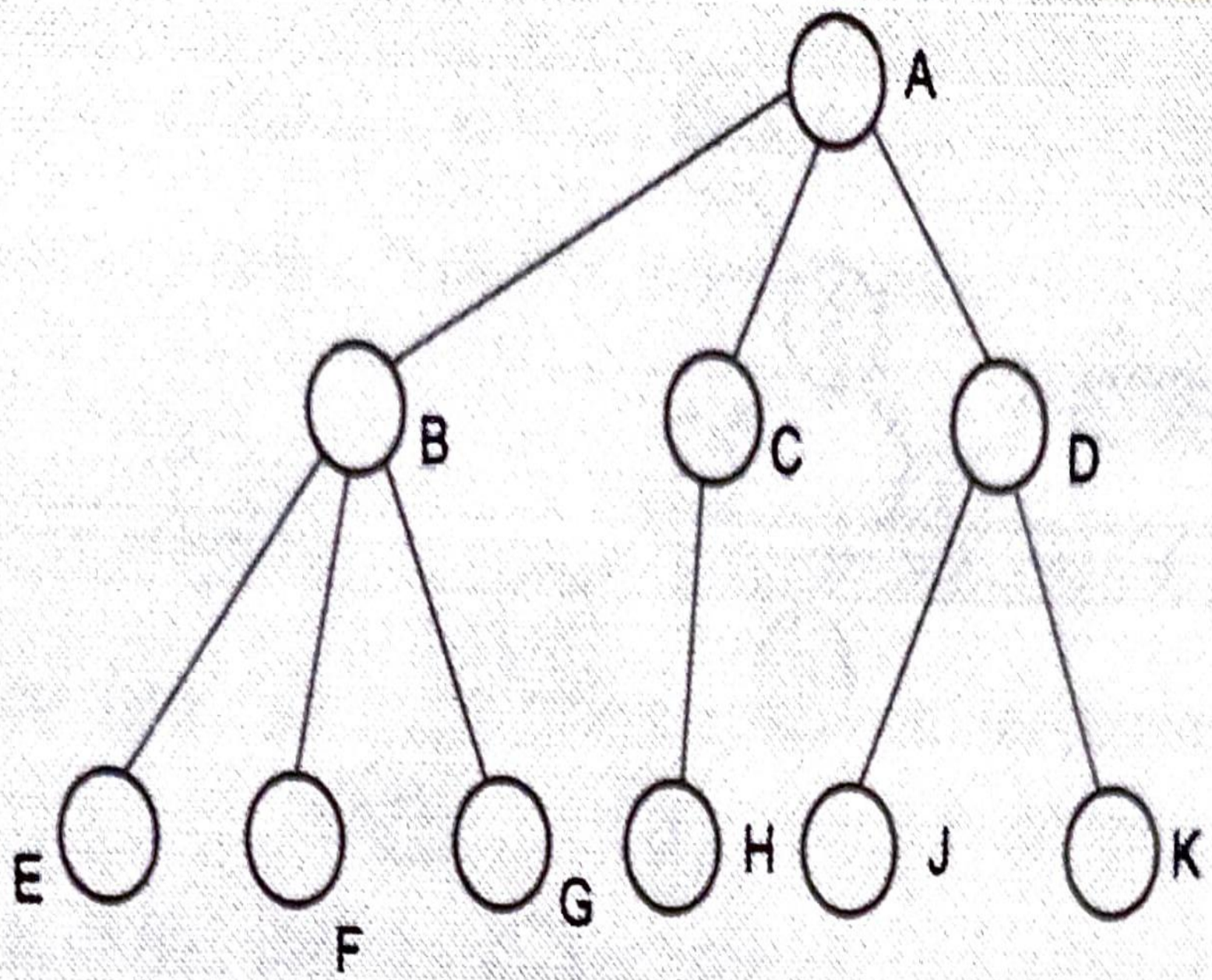


Convert a Generic Tree(N-array Tree) to Binary Tree

Examples:

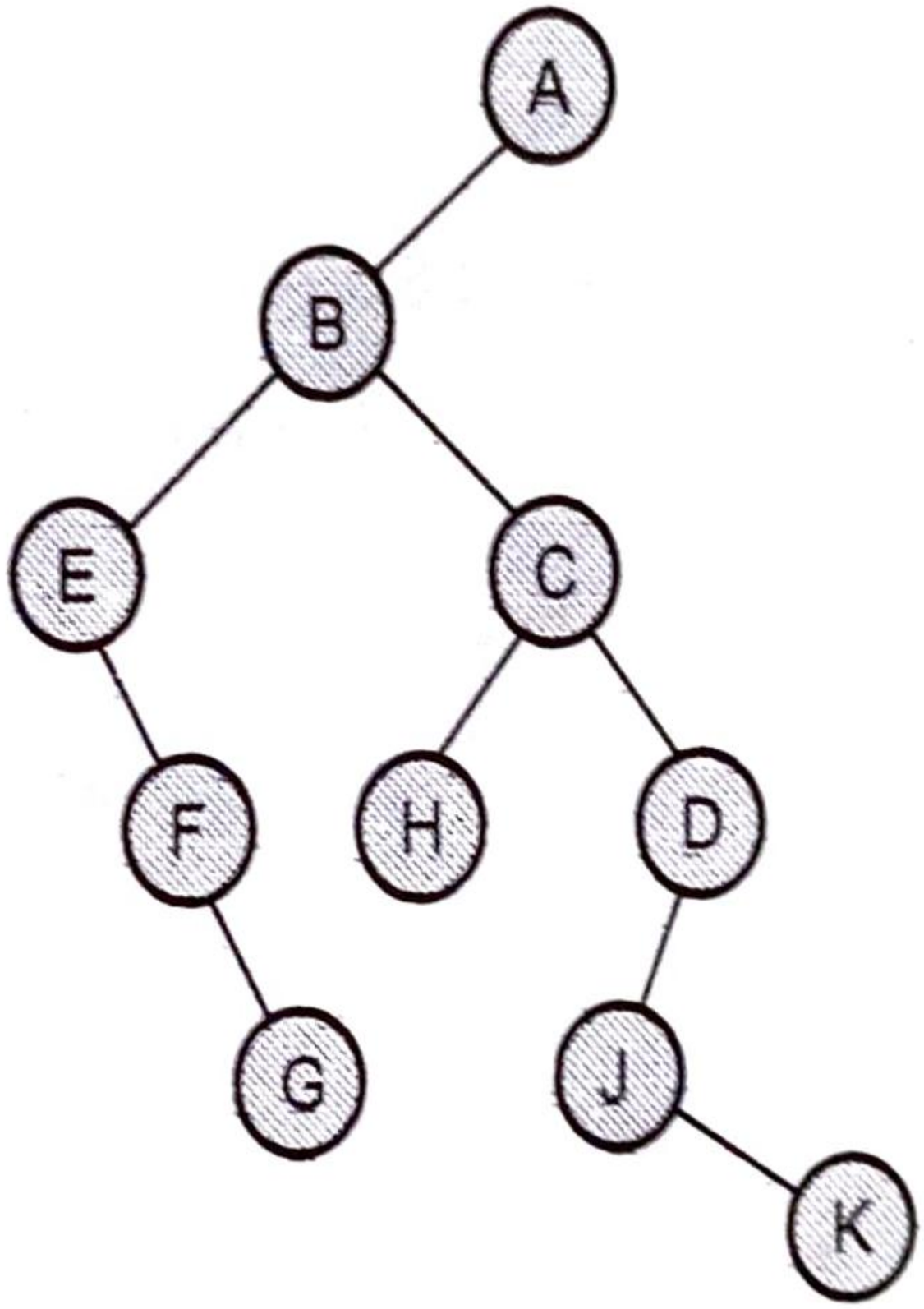
Convert the following Generic Tree to Binary Tree:

Convert following generalized tree into a inary tree. **SPPU : Dec.-13, Marks 3**



Convert a Generic Tree(N-array Tree) to Binary Tree

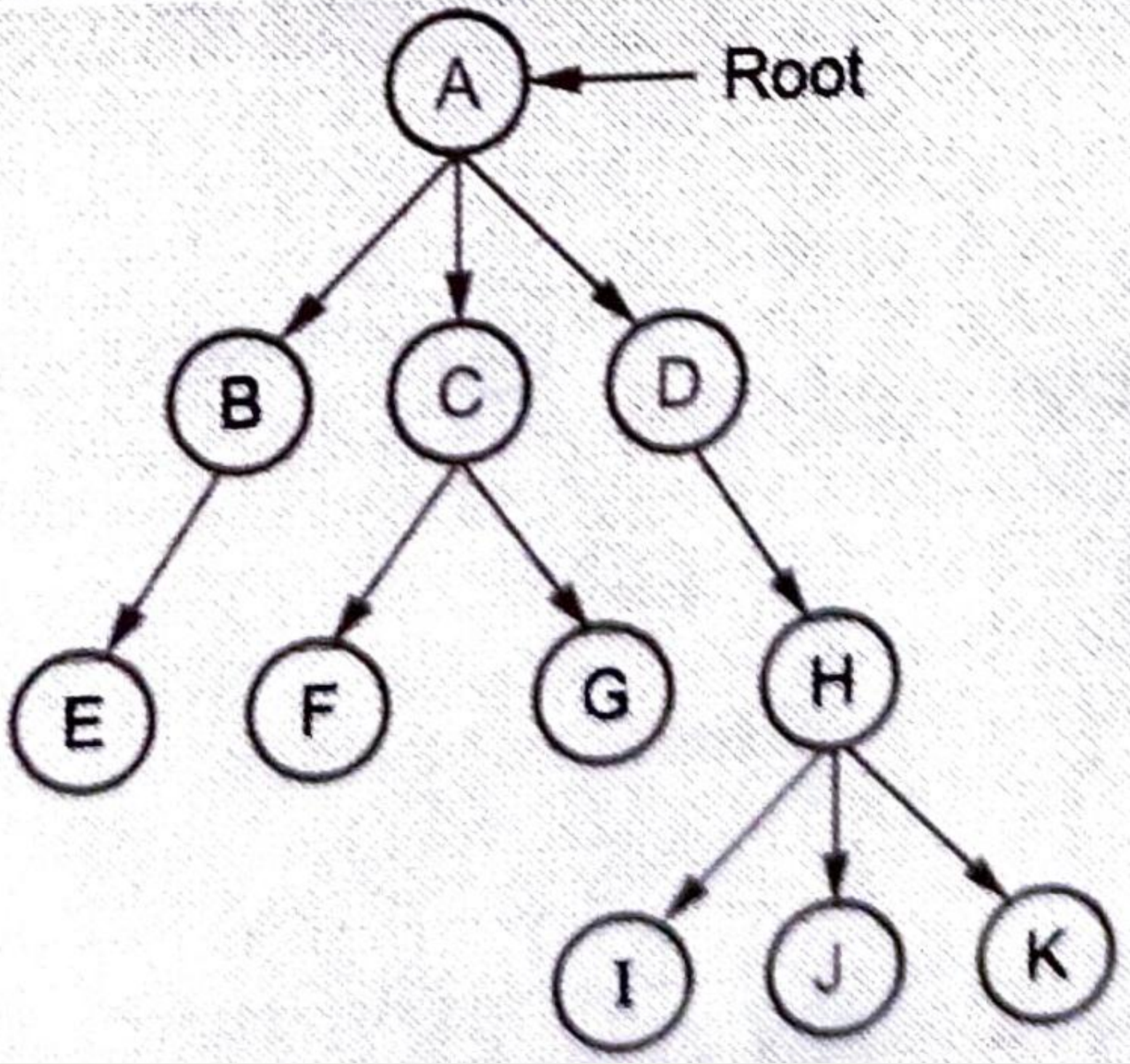
Solution :



Convert a Generic Tree(N-array Tree) to Binary Tree

Examples:

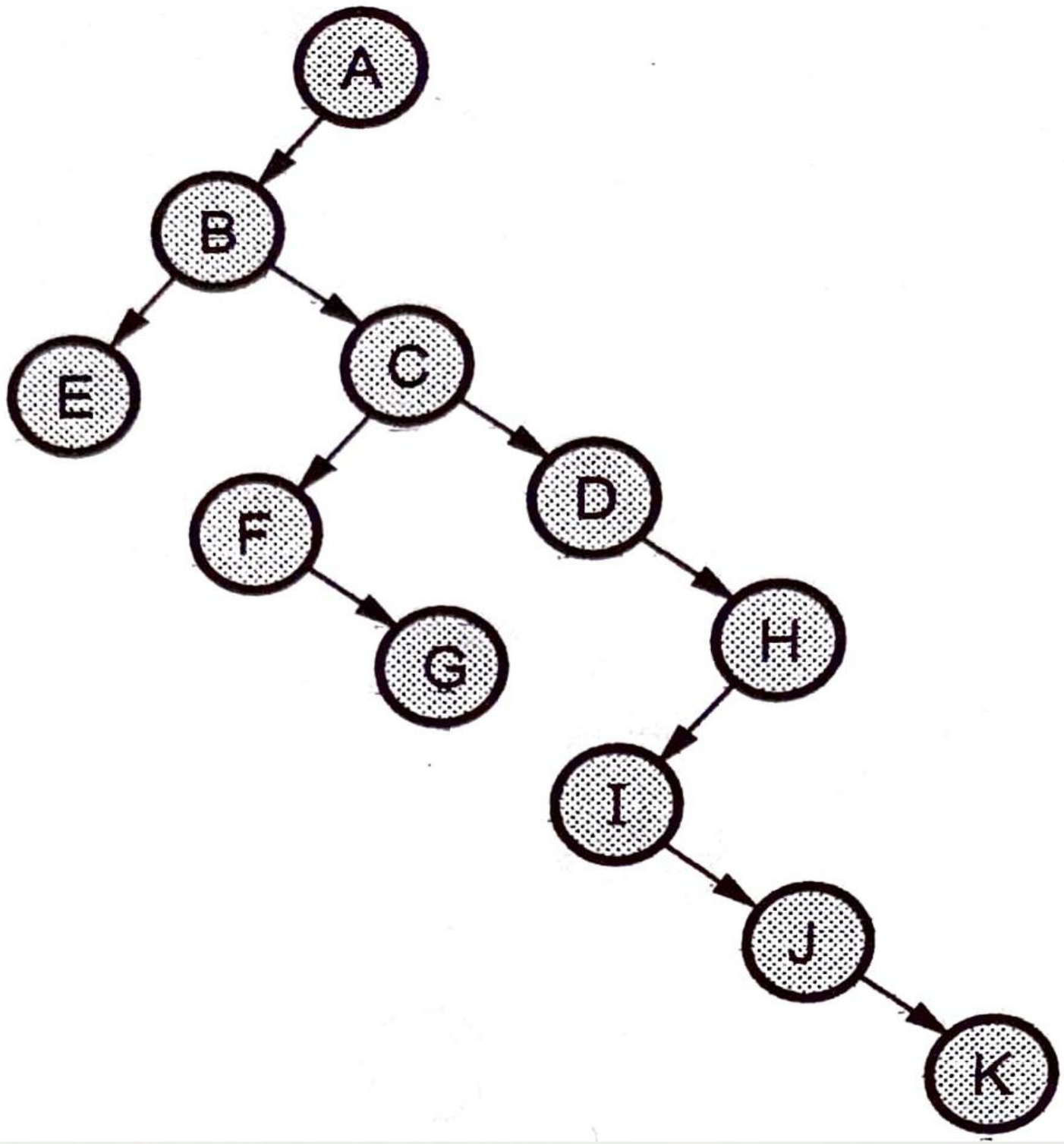
Convert the following tree into Binary tree.



Convert a Generic Tree(N-array Tree) to Binary Tree

Examples:

The binary tree will be -

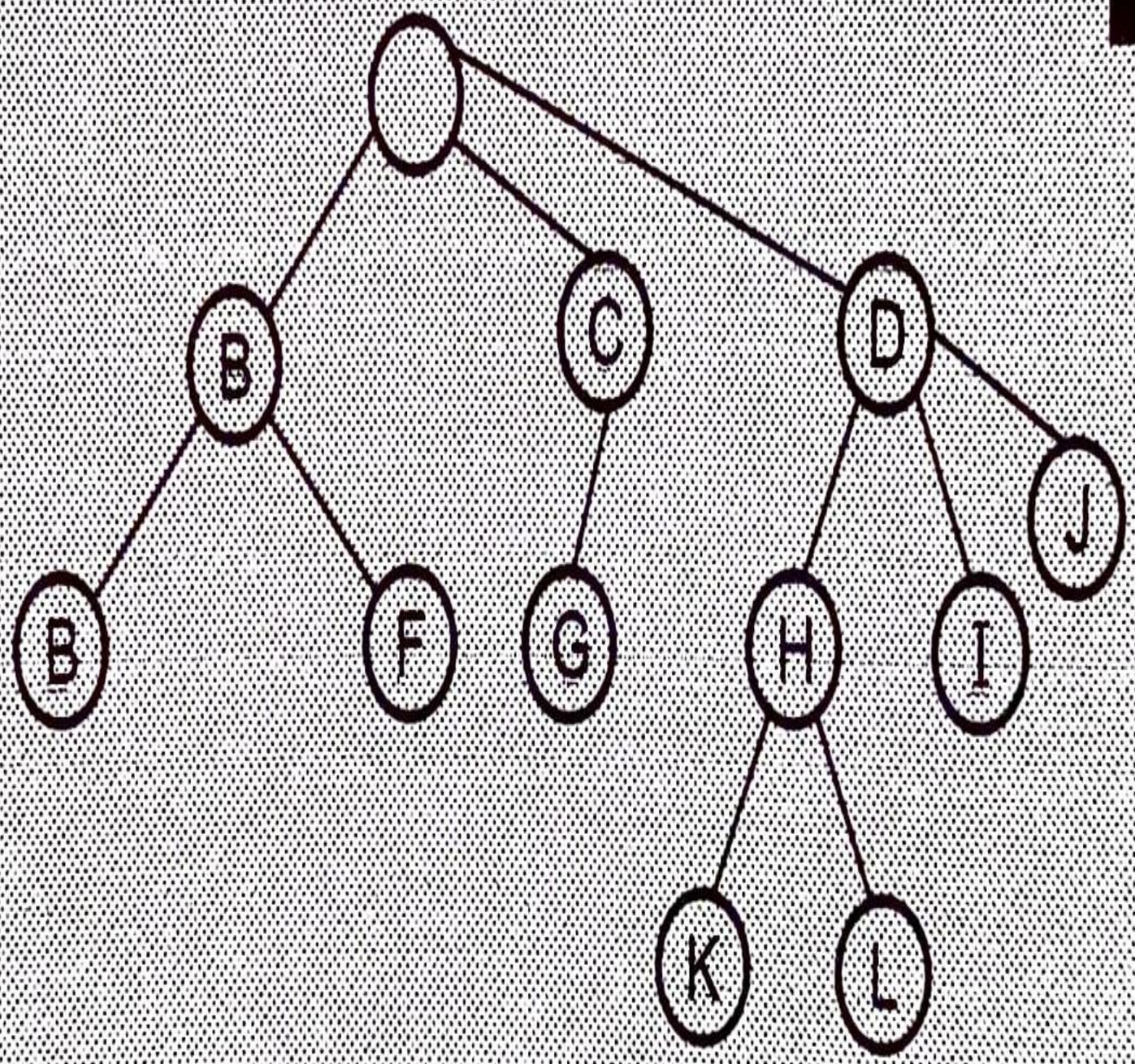


Convert a Generic Tree(N-array Tree) to Binary Tree

Examples:

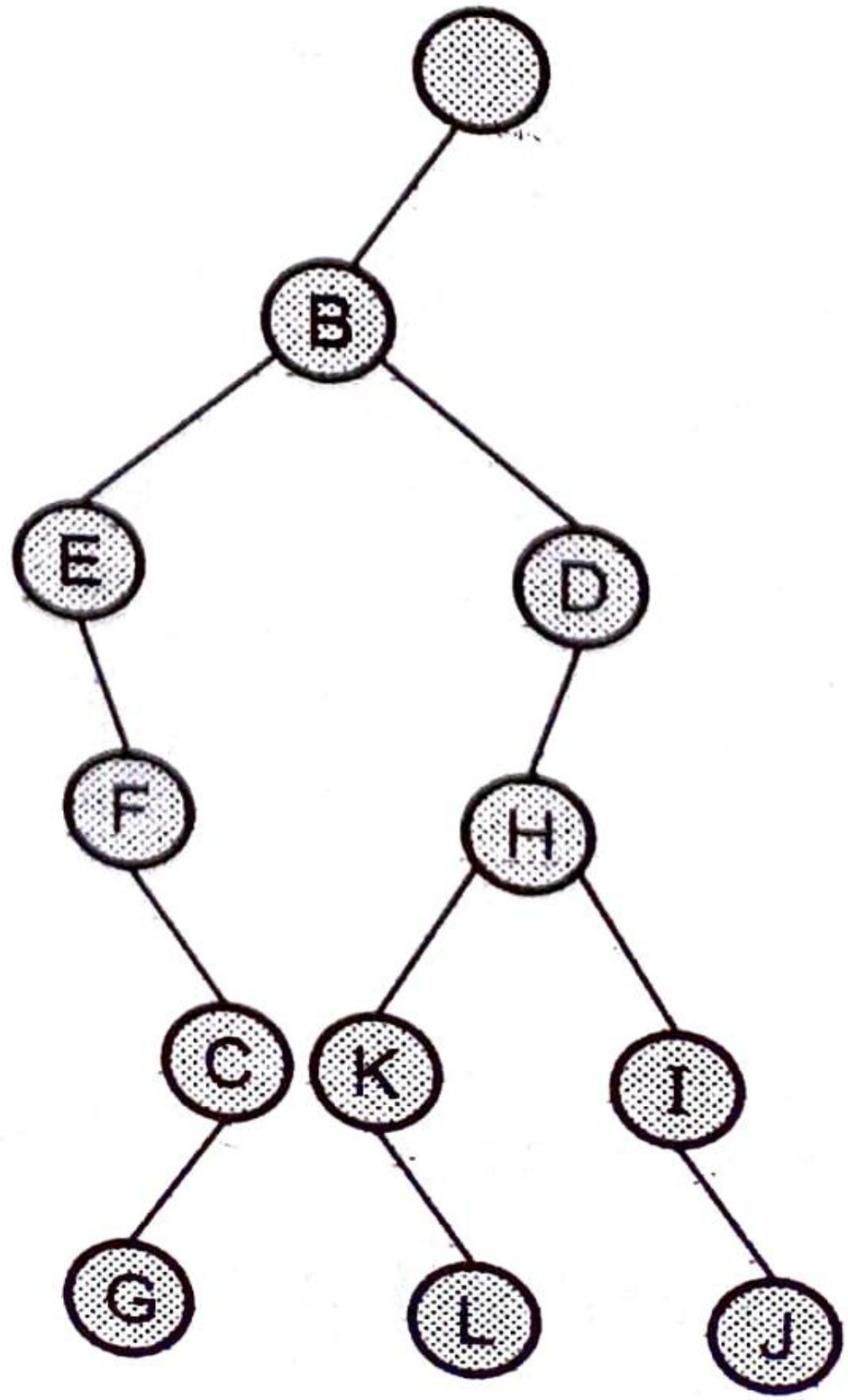
Convert the given general tree to its equivalent binary tree.

SPPU : May-19, Marks 6



Convert a Generic Tree(N-array Tree) to Binary Tree

Solution : Binary tree is



Tree Traversal

Traversal of the tree in data structures is a process of visiting each node and prints their value. There are three ways to traverse tree data structure.

1. Pre-order Traversal
2. In-Order Traversal
3. Post-order Traversal

Tree Traversal

Tree traversal means traversing or visiting each node of a tree. Linear data structures like Stack, Queue, linked list have only one way for traversing, whereas the tree has various ways to traverse or visit each node. The following are the three different ways of traversal:

1. Inorder traversal
2. Preorder traversal
3. Postorder traversal

Inorder Traversal

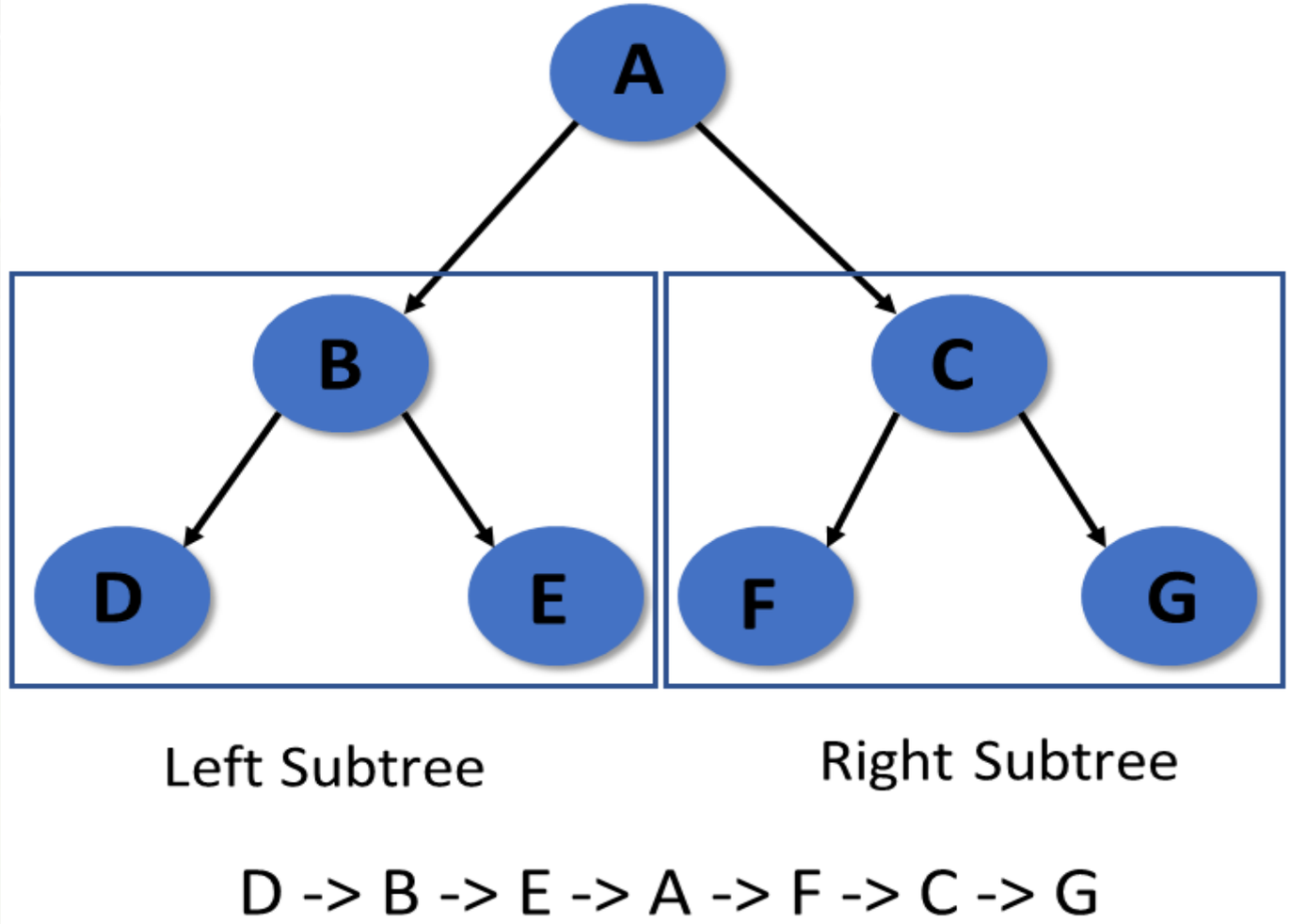
In the in-order traversal, the left subtree is visited first, then the root, and later the right subtree.

Algorithm:

Step 1- Recursively traverse the left subtree

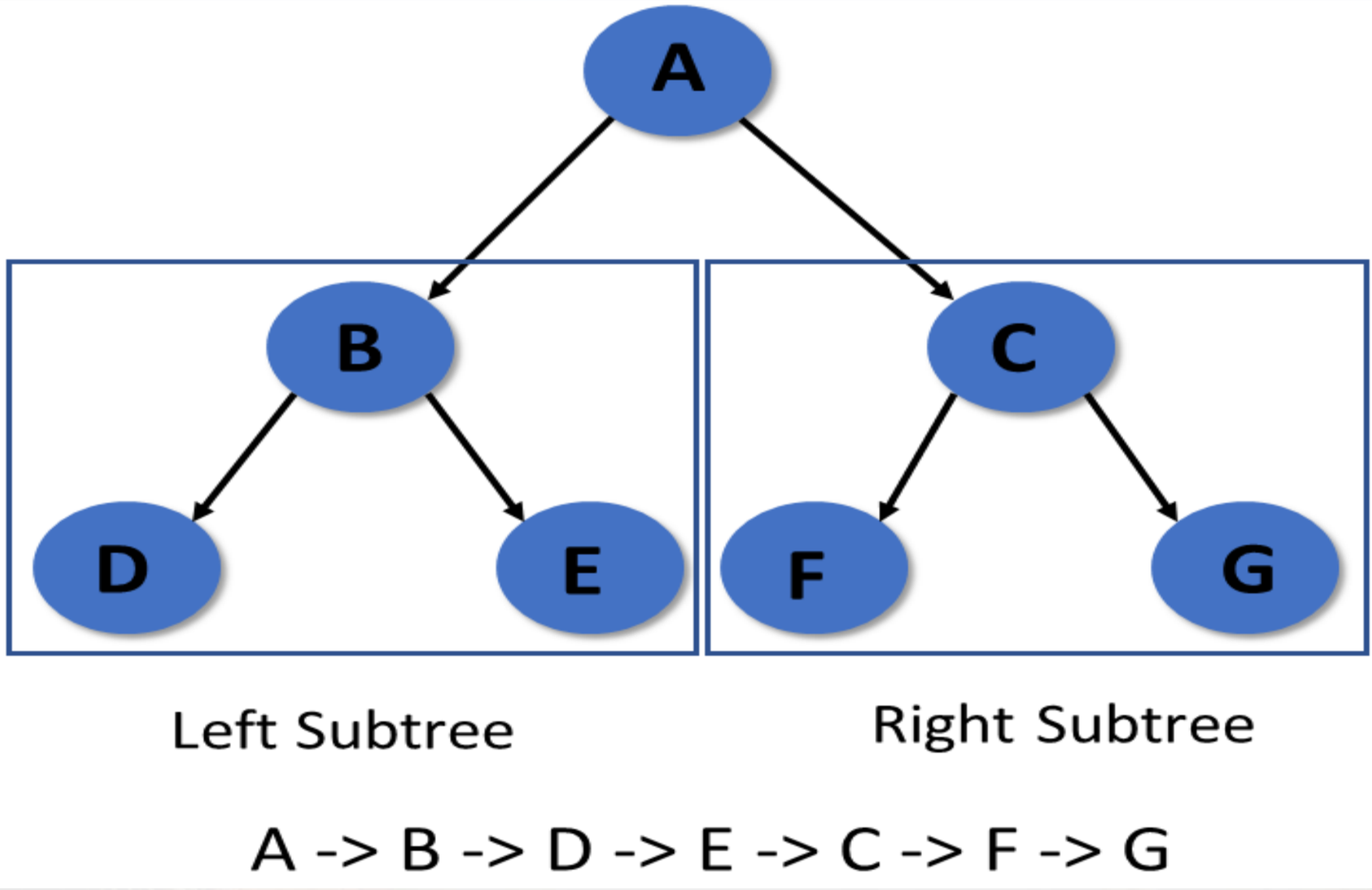
Step 2- Visit root node

Step 3- Recursively traverse right



Pre-order Traversal

In pre-order traversal, it visits the root node first, then the left subtree, and lastly right subtree.



Algorithm:

Step 1- Visit root node

Step 2- Recursively traverse the left subtree

Step 3- Recursively traverse right subtree

Post-order Traversal

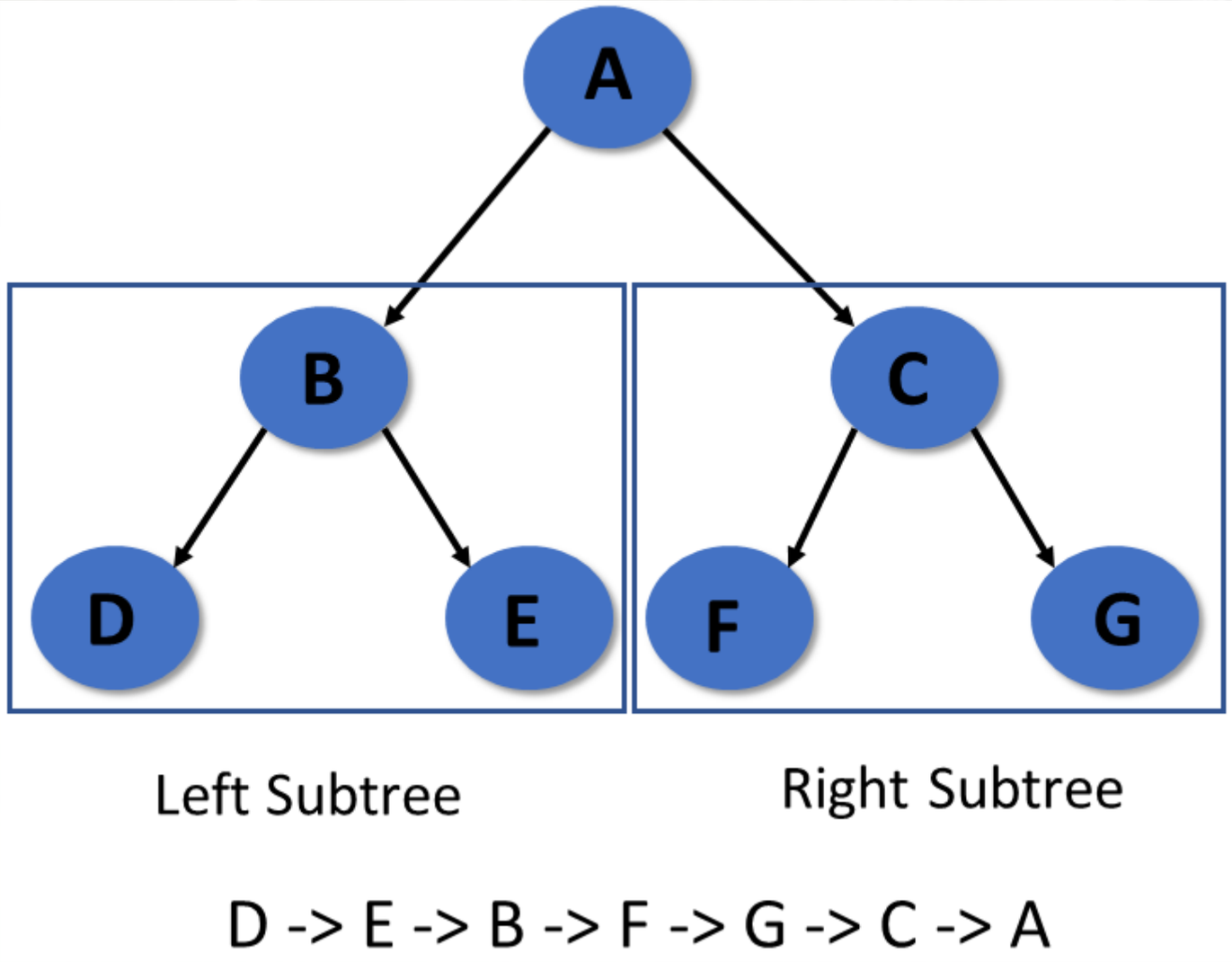
It visits the left subtree first in post-order traversal, then the right subtree, and finally the root node.

Algorithm:

Step 1- Recursively traverse the left subtree

Step 2- Recursively traverse right

Step 3- Visit root node



Tree Traversal Example

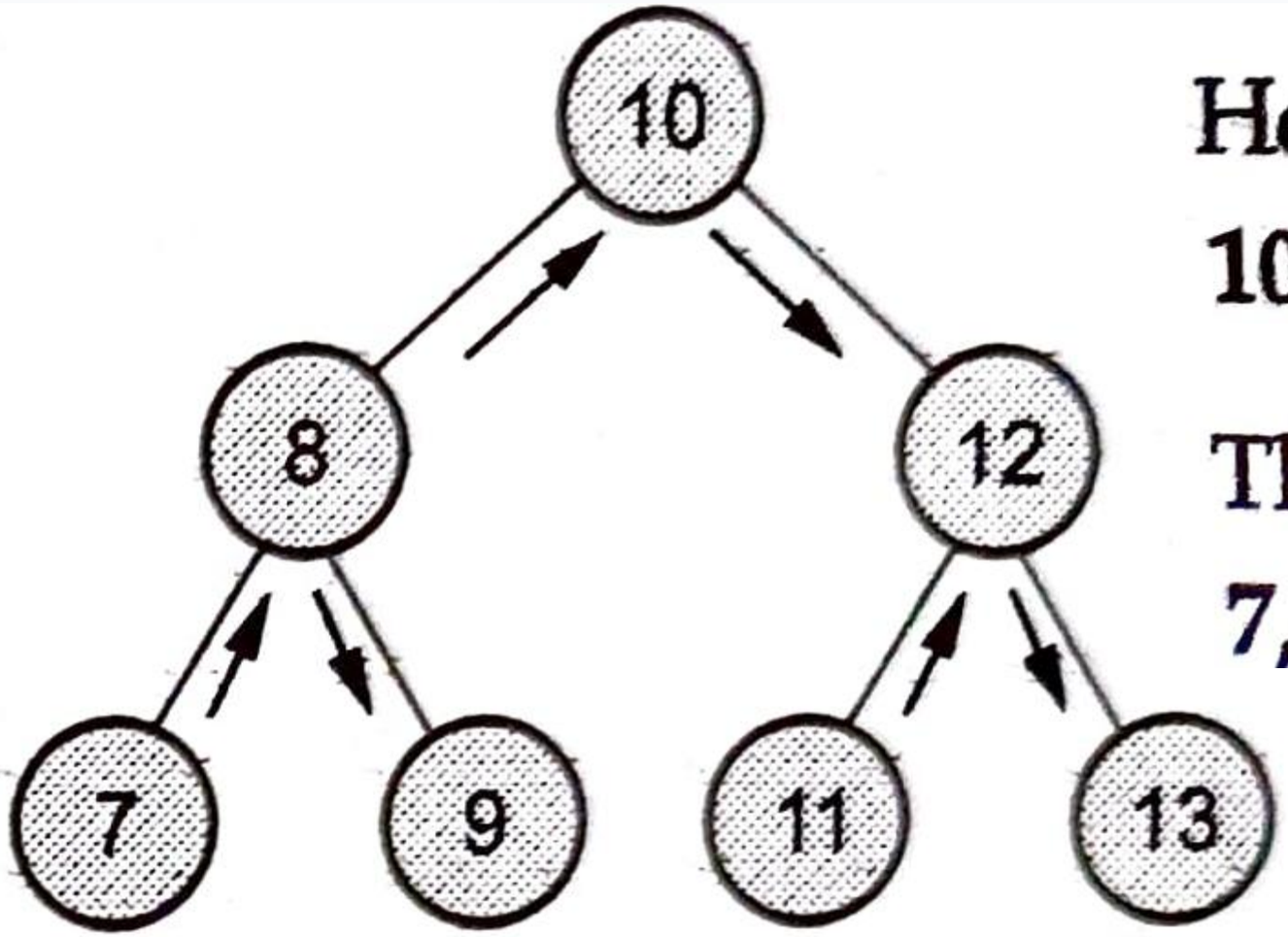


Fig. 2.6.1 Binary tree

Hence, Preorder traversal is

10, 8, 7, 9, 12, 11, 13.

The Postorder sequence is

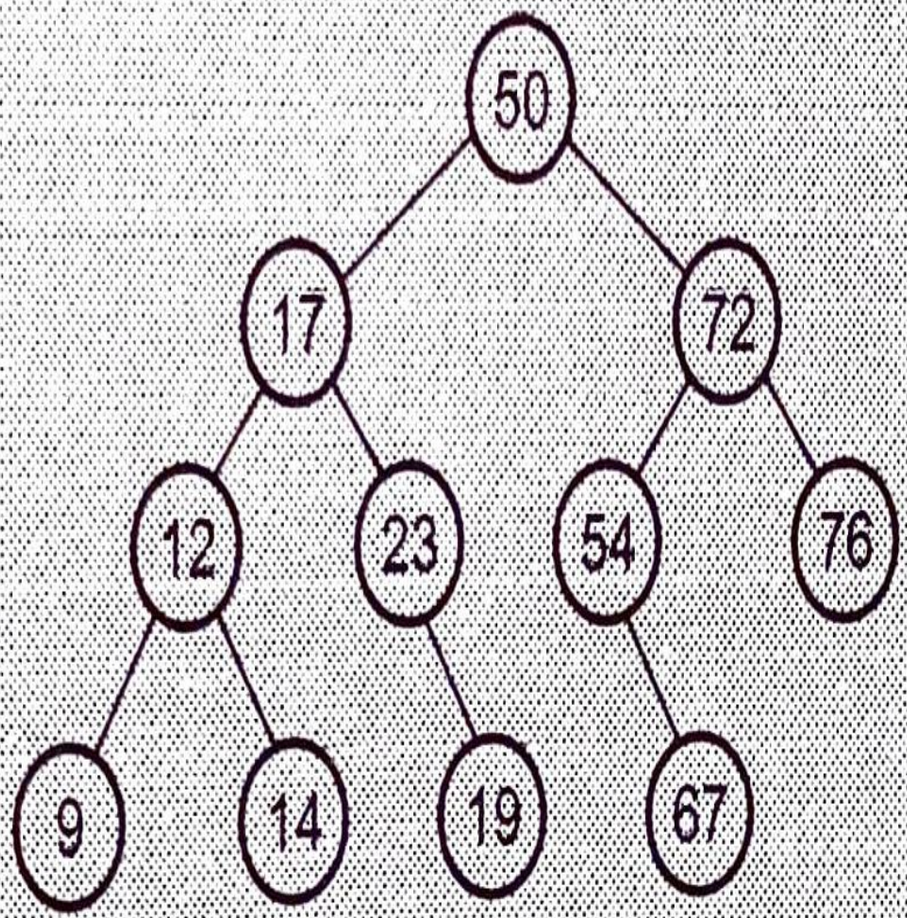
7, 9, 8, 11, 13, 12, 10.

Inorder sequence is 7, 8, 9, 10, 11, 12, 13

Tree Traversal Example

Example 2.6.1 *Consider the following tree given in the problem. Show a Postorder, Preorder and In order Traversal of tree.*

SPPU : May-14, Marks 3



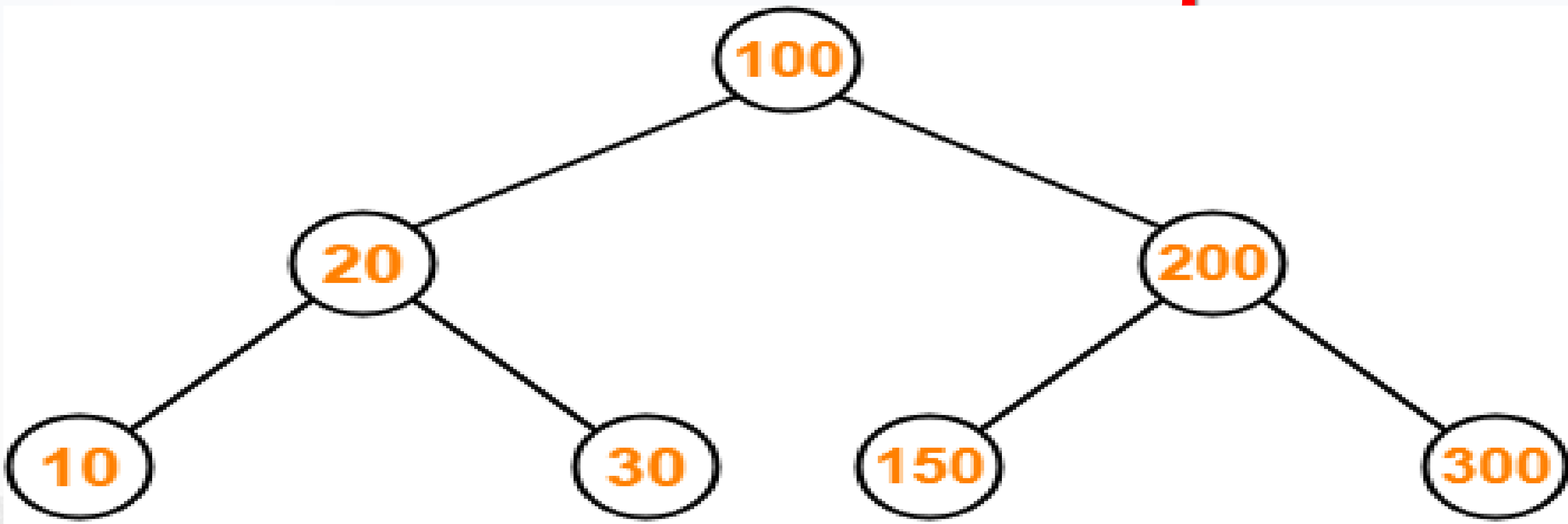
SPECIMEN CO

Solution : Preorder Sequence : 50, 17, 12, 9, 14, 23, 19, 72, 54, 67, 76.

Inorder Sequence : 9, 12, 14, 17, 23, 19, 50, 54, 67, 72, 76.

Postorder Sequence : 9, 14, 12, 19, 23, 17, 67, 54, 76, 72, 50.

Tree Traversal Example



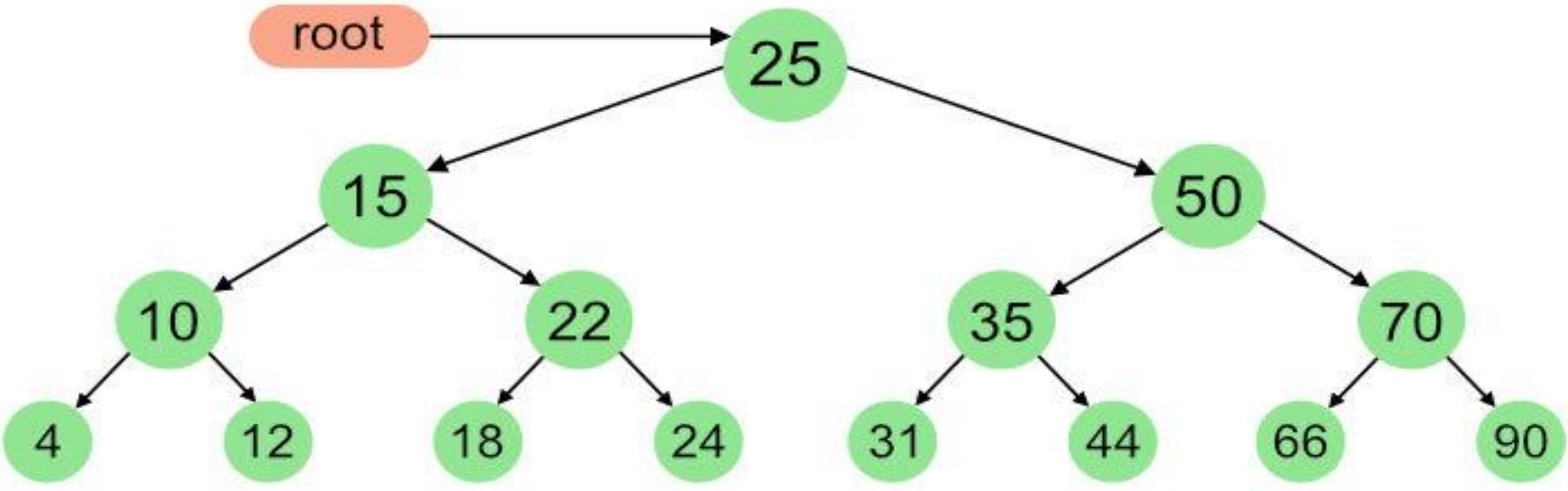
Binary Search Tree

Preorder Traversal - 100 , 20 , 10 , 30 , 200 , 150 , 300

Inorder Traversal- 10 , 20 , 30 , 100 , 150 , 200 , 300

Postorder Traversal : 10 , 30 , 20 , 150 , 300 , 200 , 100

Tree Traversal Example



InOrder(root) visits nodes in the following order:

4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25

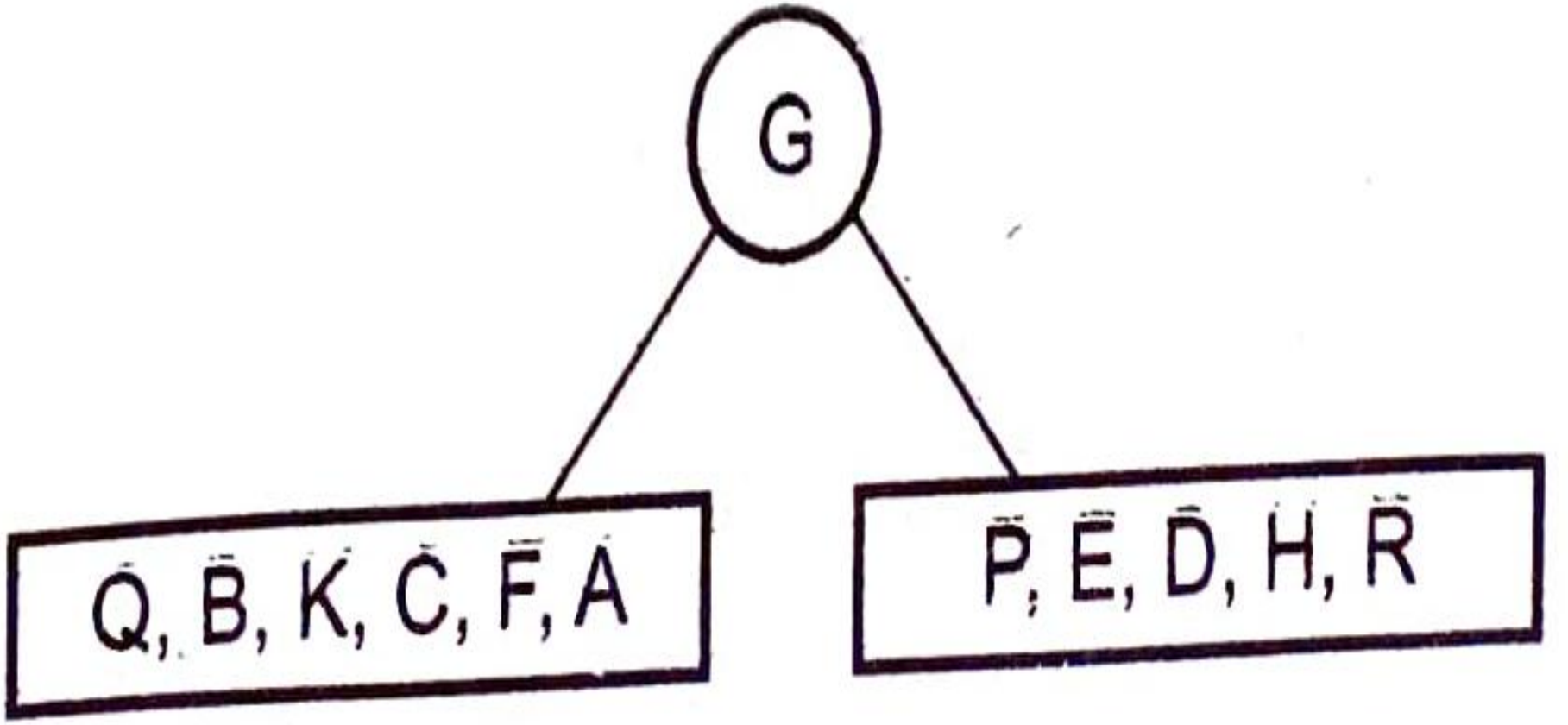
Tree Traversal Example

Example 2.6.2 *From the given traversals construct the binary tree.*

Pre-order : G, B, Q, A, C, K, F, P, D, E, R, H

In-order : Q, B, K, C, F, A, G, P, E, D, H, R

SPPU : Dec.-17, Marks 4



Tree Traversal Example

Example 2.6.2 *From the given traversals construct the binary tree.*

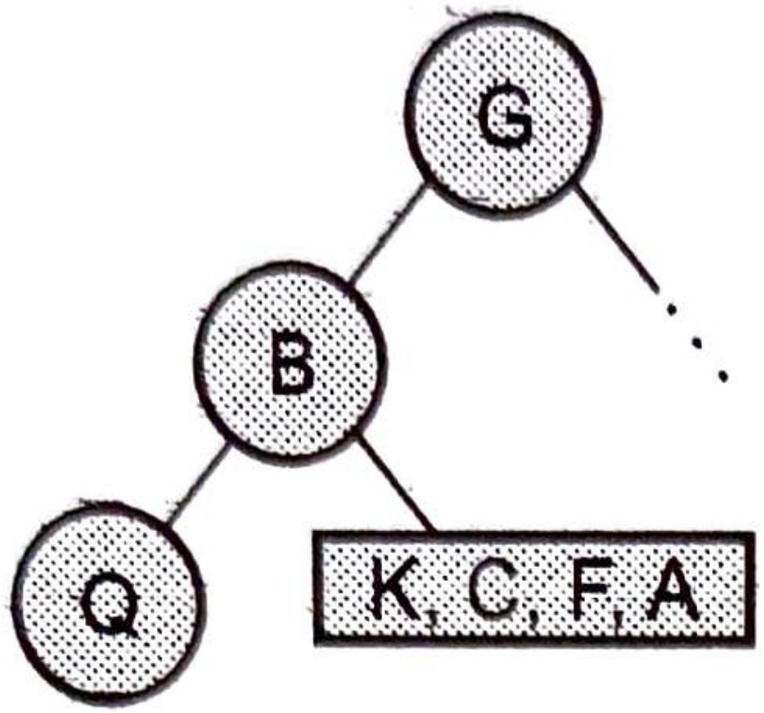
Pre-order : G, B, Q, A, C, K, F, P, D, E, R, H

In-order : Q, B, K, C, F, A, G, P, E, D, H, R

SPPU : Dec.-17, Marks 4

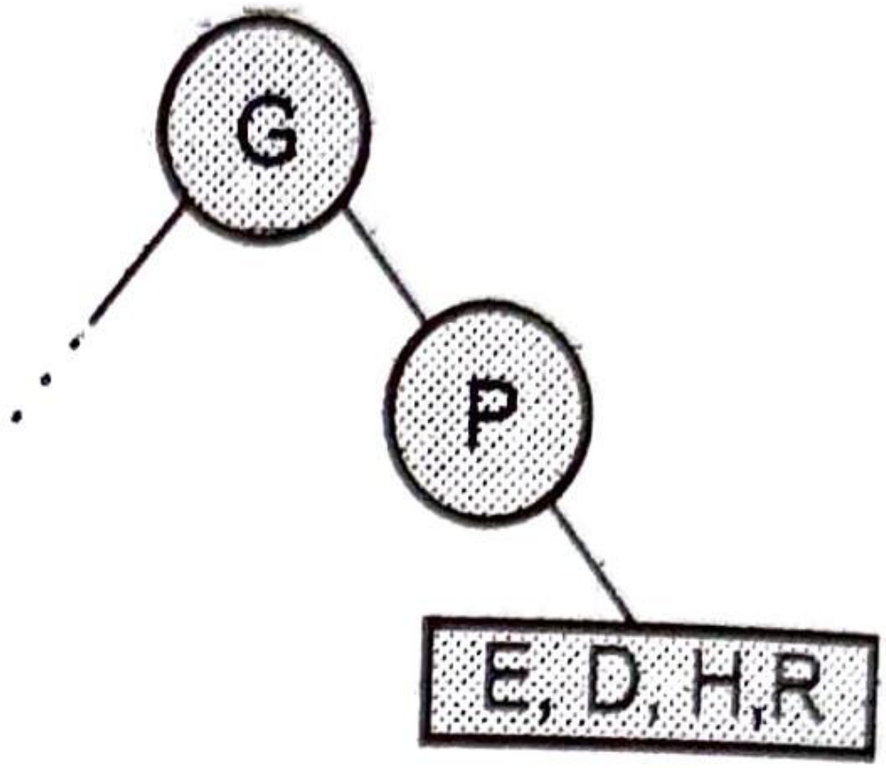
Preorder : **B** , Q, A, C, K, F,

Inorder : Q, **B** , K, C, F, A



Preorder : **P** , D, E, R, H,

Inorder : **P** , E, D, H, R,



Tree Traversal Example

Example 2.6.2 *From the given traversals construct the binary tree.*

Pre-order : G, B, Q, A, C, K, F, P, D, E, R, H

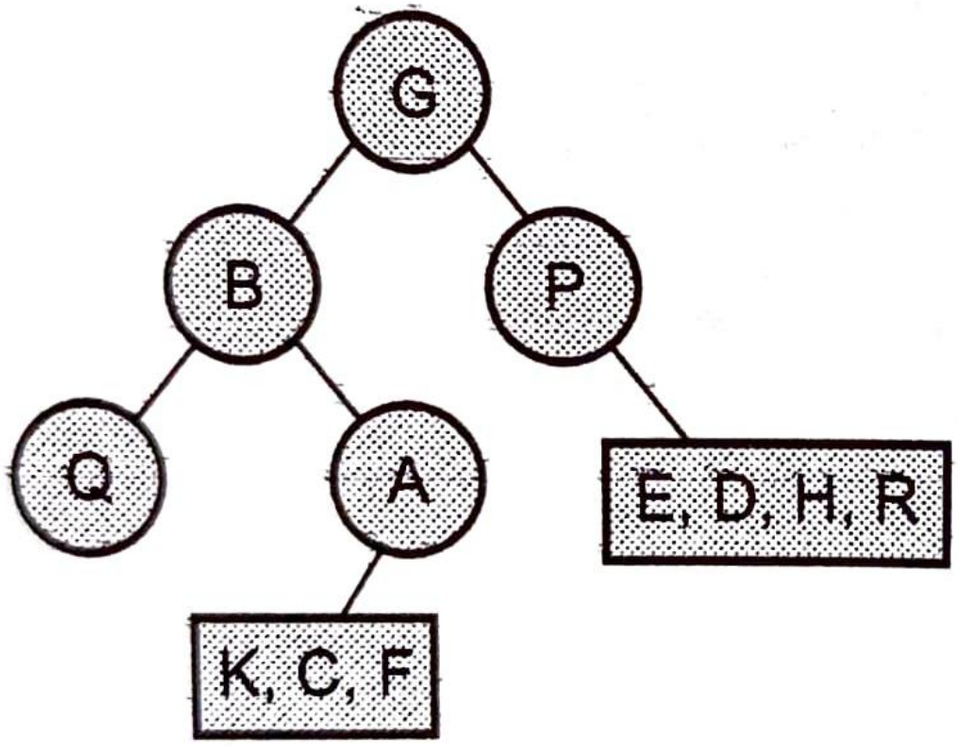
In-order : Q, B, K, C, F, A, G, P, E, D, H, R

SPPU : Dec.-17, Marks 4

Step 3 :

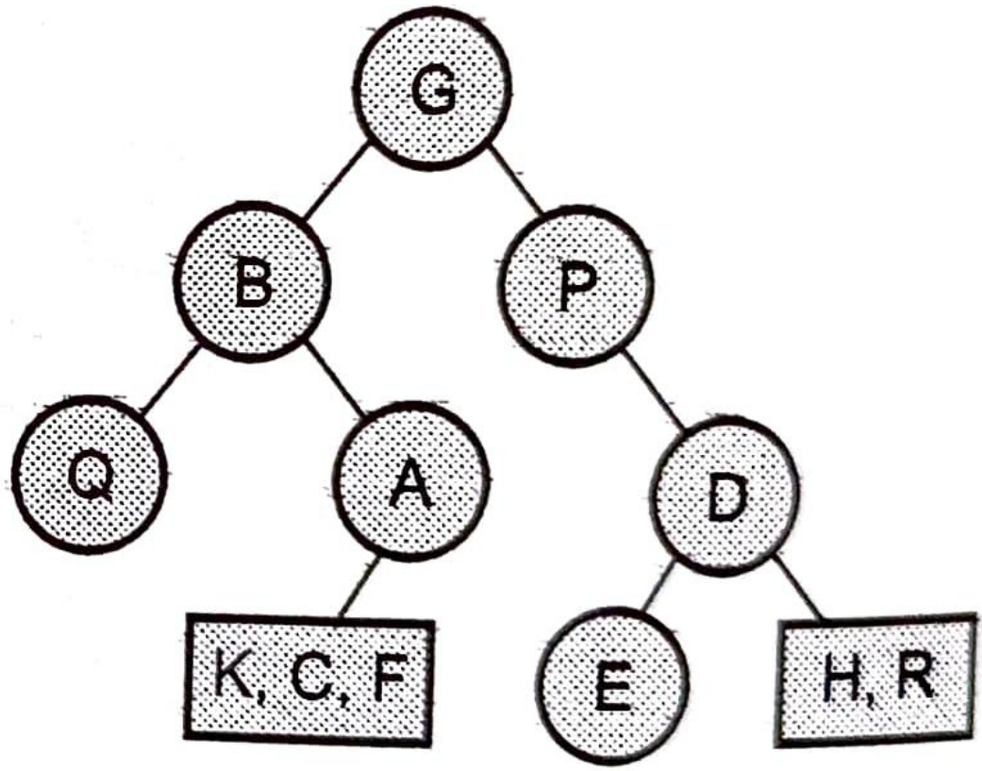
Preorder : A , C, K, F

Inorder : K, C, F, A



Preorder : D , E, R, H

Inorder : E , D , H, R



Tree Traversal Example

Example 2.6.2 *From the given traversals construct the binary tree.*

Pre-order : G, B, Q, A, C, K, F, P, D, E, R, H

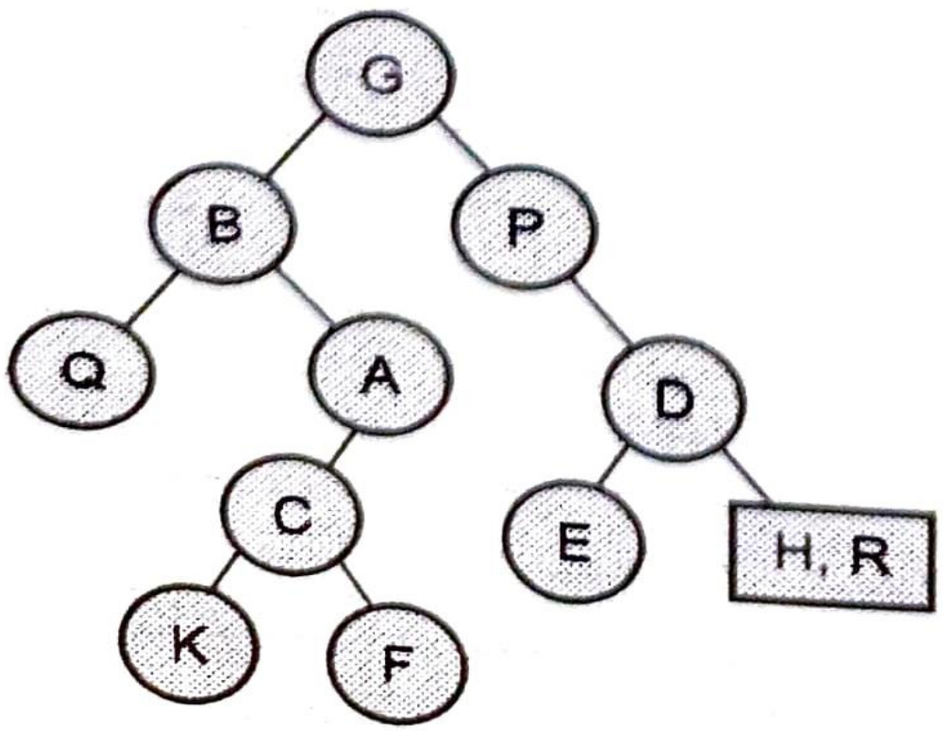
In-order : Q, B, K, C, F, A, G, P, E, D, H, R

SPPU : Dec.-17, Marks 4

Step 4 :

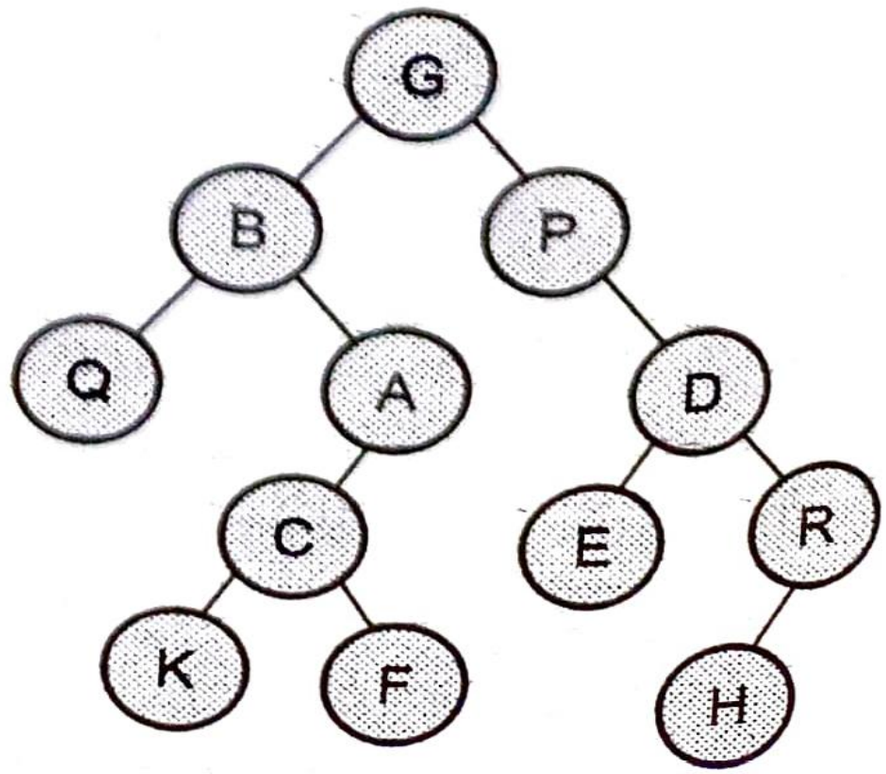
Preorder : [C] , K, F,

Inorder : K, [C] , F,



Preorder : [R] , H,

Inorder : H, [R]



Tree Traversal Example

Construct a binary tree from given two traversals :

Inorder traversal : 1 2 3 14 7 10 11 40 30.

Postorder traversal : 1 3 2 7 10 40 30 11 14

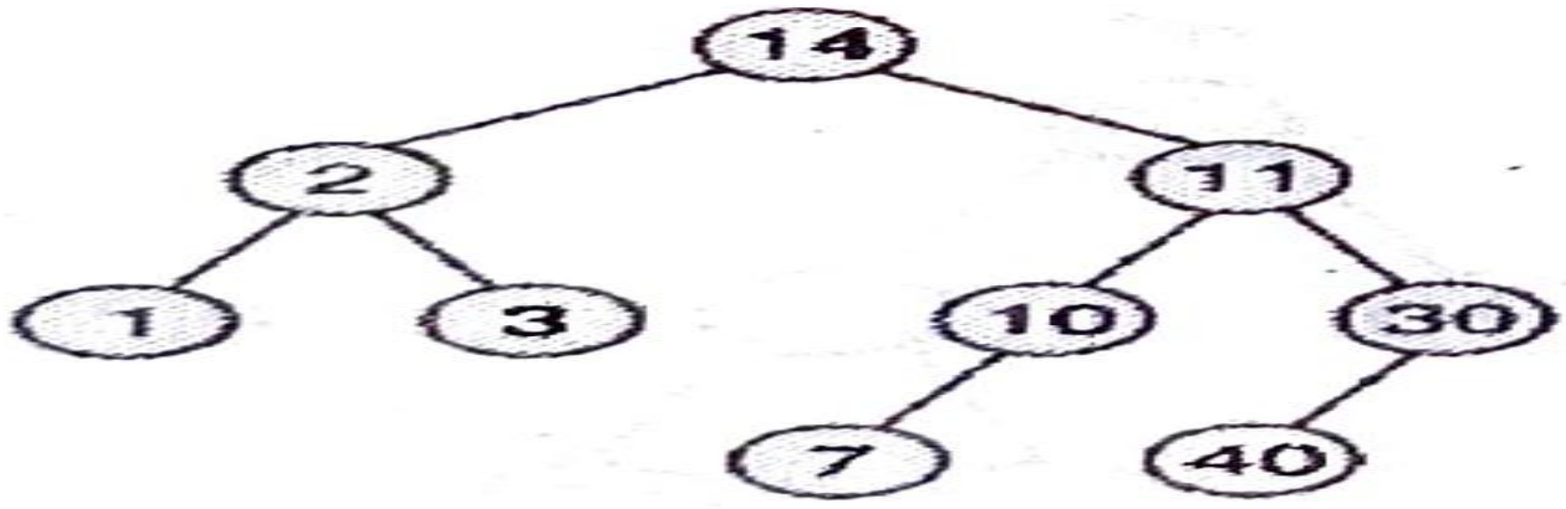
Step 1 :



Step 2 :



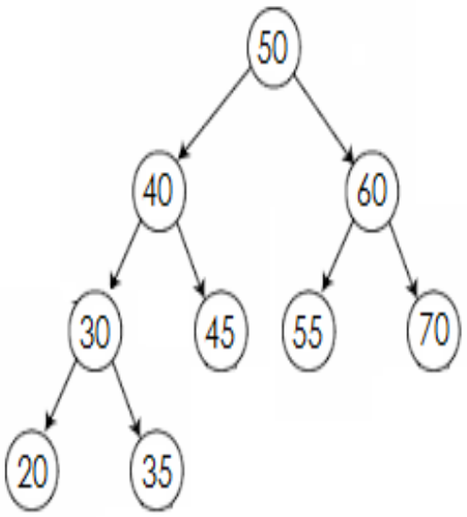
Step 3 :



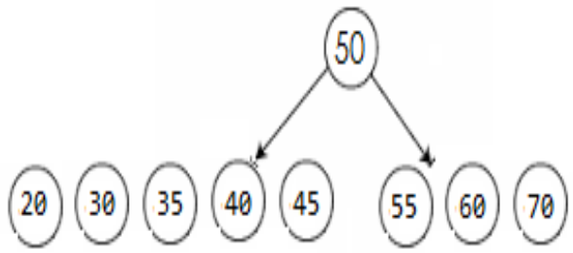
Tree Traversal Example

```
int inOrder[] =   {20, 30, 35, 40, 45, 50, 55, 60, 70};
int postOrder[] = {20, 35, 30, 45, 40, 55, 70, 60, 50};
```

Binary Tree

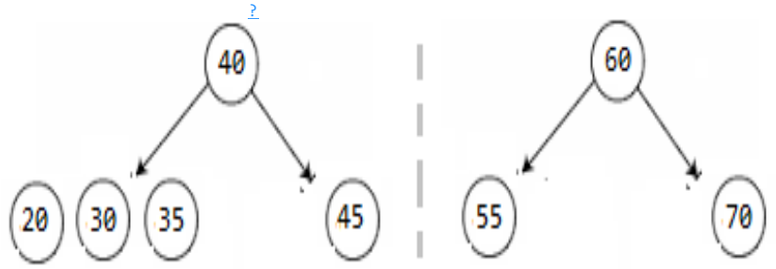



```
int inOrder[]   = {20, 30, 35, 40, 45, 50, 55, 60, 70};
int postOrder[] = {20, 35, 30, 45, 40, 55, 70, 60, 50};
```

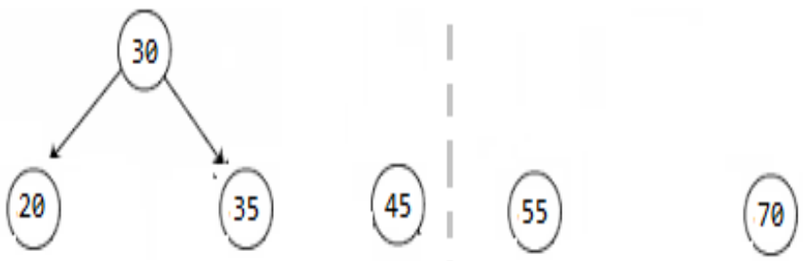


```
int inOrder[]   = {20, 30, 35, 40, 45,   ✗, 55, 60, 70};
int postOrder[] = {20, 35, 30, 45, 40,   55, 70, 60, ✗};
```

Question:
Two traversals are given as input,



```
int inOrder[] =   {20, 30, 35,   ✗, 45,   ✗, 55,   ✗, 70};
int postOrder[] = {20, 35, 30,   45, ✗,   55,   70, ✗, ✗};
```



```
int inOrder[]   = 20   ✗, 35
int postOrder[] = 20   35, ✗
```



Tree Traversal Example

Example Input:

Postorder = [10, 18, 9, 22, 4]

Inorder = [10, 4, 18, 22, 9]

Example :

preorder = [1,2,4,5,3,6,7],

postorder = [4,5,2,6,7,3,1]

Inorder Traversal (recursive version)

```
void inorder(tree_pointer ptr)
/* inorder tree traversal */
{
      if (ptr) {
          inorder(ptr->left_child);
          printf("%d", ptr->data);
          indorder(ptr->right_child);
      }
}
```

A / B * C * D + E

Preorder Traversal (recursive version)

```
void preorder(tree_pointer ptr)
/* preorder tree traversal */
{
     if (ptr) {
          printf("%d", ptr->data);
          preorder(ptr->left_child);
          predorder(ptr->right_child);
     }
}
```

+ * * / A B C D E

Postorder Traversal (recursive version)

```
void postorder(tree_pointer ptr)
/* postorder tree traversal */
{
     if (ptr) {
          postorder(ptr->left_child);
          postdorder(ptr->right_child);
          printf("%d", ptr->data);
     }
}
```

A B / C * D * E +

THANK YOU!!!

My Blog : https://anandgharu.wordpress.com/

Email : gharu.anand@gmail.com